GnssProcessor: Mathematical Reference

Revision date: September 28, 2026

Contents

1 Introduction **6**

I Foundations **7**

2 Notation and conventions **7**

3 State vector layout **9**
3.1 Schematic layout . . . 10
3.2 Component-by-component description . . . 10
3.3 Dynamic indexing . . . 14

4 Observation equations **15**
4.1 Code (pseudorange) observation . . . 16
4.2 Carrier-phase observation . . . 16
4.3 Geometric range and linearisation . . . 16
4.4 Stochastic model . . . 17

5 Differencing transformations **17**
5.1 Between-satellite single difference (BSSD) . . . 17
5.2 Between-receiver single difference (BRSD) . . . 19
5.3 Double difference (DD) . . . 19
5.4 S-transform structure . . . 20

II Reference systems and measurement models **21**

6 Coordinate Frames and Time Systems **21**
6.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames . . . 21
6.2 Station reference points: marker, ARP, and phase centre . . . 23
6.3 GNSS time scales and the GPS reference epoch . . . 24
6.4 Leap seconds and per-system dispatch . . . 24
6.5 GLONASS time-of-day to full GPS week . . . 25
6.6 BeiDou time: the 14 s offset and independent week base . . . 26
6.7 Legacy ECI→ECEF: the equinox-based chain . . . 26
6.8 CIO-based ITRS↔GCRS (IAU 2000A/2006) . . . 28
6.9 Earth-orientation parameters: parse and interpolation . . . 28

6.10 The Sun and Moon series these transforms position . . . 29

7 Terrestrial Reference Frames, Epochs, and Datum Transformation 29
7.1 Frame versus epoch . . . 29
7.2 Transforming a velocity . . . 30
7.3 The rigid-plate approximation . . . 31
7.4 Rotation-sign conventions . . . 31
7.5 Published parameter sets and their composition . . . 31
7.6 NATRF2022 . . . 32
7.7 SIF occupations and the datasheet convention . . . 33
7.8 Monument versus ARP . . . 33
7.9 Worked example: CORV . . . 34
7.10 Deformation models and validation . . . 35

8 Orbit and Clock Product Evaluation 37
8.1 Precise orbit: derotated Neville interpolation . . . 37
8.2 Precise orbit: sidereal-trigonometric LSQ fit . . . 39
8.3 Precise clock interpolation . . . 39
8.4 Broadcast Keplerian evaluation . . . 41
8.5 Broadcast GLONASS: PZ-90 RK4 integration . . . 42
8.6 Broadcast satellite clock . . . 42
8.7 Ephemeris selection and IOD bookkeeping . . . 43

9 Atmospheric Delay Modelling: Troposphere and Ionosphere 44
9.1 The slant tropospheric delay . . . 44
9.1.1 Saastamoinen zenith hydrostatic delay . . . 44
9.1.2 The Marini mapping form . . . 45
9.1.3 GPT3 empirical pressure, temperature and VMF3 a-coefficients . . . 46
9.1.4 The gradient mapping and a degenerate fallback . . . 46
9.2 The slant ionospheric delay . . . 47
9.2.1 The $1/f^2$ scaling and its sign . . . 47
9.2.2 IONEX / GIM thin-shell slant delay . . . 47
9.2.3 The GIM iono-weighting pseudo-observation . . . 48
9.2.4 The co-estimated spherical-harmonic VTEC field . . . 48
9.2.5 Klobuchar broadcast seed . . . 51
9.3 Higher-order ionospheric corrections . . . 52
9.3.1 The geomagnetic dipole field . . . 52

10 Site Displacement and Geophysical Corrections 53
10.1 Sign convention and projection into the range . . . 53
10.2 Solid Earth tide (IERS 2010, Steps 1 + 2) . . . 54
10.2.1 Step 1: in-phase degree 2 and 3 . . . 54
10.2.2 Step 1: out-of-phase anelasticity and the L^1 term . . . 55
10.2.3 Step 2: frequency dependence (Doodson tables) . . . 55
10.3 Ocean tidal loading (OTL) . . . 56
10.3.1 Astronomical argument and the Doodson–Warburg phase bias . . . 56
10.3.2 Coefficient sources: SIF, BLQ, and the EOT20 grid . . . 56
10.4 Pole tide (rotational deformation, IERS 7.1.4) . . . 57

10.5 Ocean pole tide (Desai 2002, IERS 7.1.5) . . . 58
10.6 Atmospheric pressure loading (S1/S2 thermal tide) . . . 58
10.7 Integration into the measurement model . . . 59

11 Observation Model Physics: Antenna, Attitude, Relativity, and Range Corrections 59
11.1 Satellite attitude: the body frame . . . 60
11.2 Antenna phase-centre corrections . . . 61
11.3 Carrier-phase wind-up . . . 63
11.4 Periodic relativistic clock correction . . . 64
11.5 Shapiro (gravitational-delay) correction . . . 64
11.6 Sagnac (Earth-rotation) correction . . . 65
11.7 Group delays and inter-frequency biases . . . 65
11.8 Provenance and assembly . . . 66

III Estimation engine 66

12 Estimation engine: the Kalman filter and RTS smoother 67
12.1 Time update . . . 67
12.2 Measurement update . . . 69
12.3 Iterated relinearisation (IEKF) . . . 70
12.4 Innovation and residual gating . . . 71
12.5 Backward RTS smoother . . . 72

13 Stochastic Model and Quality Control 74
13.1 The stochastic-model dispatch . . . 74
13.2 A-posteriori variance factor . . . 76
13.3 Redundancy numbers, the Baarda w-test, and the MDB . . . 77
13.4 Data-snooping gate override . . . 79
13.5 Correlation-aware full-$\boldsymbol{R}$ kernel . . . 79
13.6 Producer-side RAIM: within-epoch adaptation . . . 80
13.7 Pooled variance factors and the per-constellation carve-out . . . 80
13.8 Station-relative deweighting of the ionosphere constraint . . . 81
13.9 State-domain emission gates . . . 82

14 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management 83
14.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed . . . 83
14.2 First-epoch state seeding by residual differencing . . . 85
14.3 BSSD pivot management . . . 87
14.3.1 Lexicographic candidate ranking . . . 87
14.3.2 Ephemeris-freshness guardrail . . . 88
14.3.3 Pivot disturbance detection and handover . . . 89

IV Cycle-slip detection and repair 89

15 Cycle-slip detection and repair **89**
15.1 Detector registry and composition . . . 90
15.2 Test-statistic combinations . . . 91
15.2.1 Melbourne–Wübbena (wide-lane) . . . 91
15.2.2 Extra-wide-lane (EWL) . . . 92
15.2.3 GRAPHIC (single-frequency half-sum) . . . 92
15.2.4 Geometry-free phase (staged) . . . 93
15.2.5 Ionosphere-free phase (staged on the user path; armed on the network path) 93
15.3 Network slip QC: raw evidence and refloat over repair . . . 95
15.4 TurboEdit dual-frequency resolution . . . 96
15.4.1 The 2×2 system . . . 96
15.4.2 Integer rounding and validation . . . 97
15.4.3 MW-only / code-event null line . . . 98
15.4.4 Gap-spanning GF bridge . . . 99
15.4.5 Atomic reconciliation . . . 99

V Integer ambiguity resolution **100**

16 Integer ambiguity resolution **100**
16.1 The integer least-squares problem . . . 100
16.2 Decorrelation: LDL^{T} factorisation and the integer $\boldsymbol{Z}$-transform . . . 101
16.3 Wide-lane / narrow-lane decomposition (PPP path) . . . 102
16.4 Partial / subset ambiguity resolution . . . 104
16.5 Validation: ratio test and fixed-failure-rate gate . . . 105
16.6 Feedback: constrained pseudo-observation update . . . 106
16.7 Network-producer AR (both mechanisms default off) . . . 107
16.8 Configuration summary . . . 109

VI Inertial aiding **109**

17 Inertial aiding **110**
17.1 Strapdown mechanization . . . 110
17.2 INS error-state dynamics . . . 111
17.3 Covariance splice and attitude feedback . . . 112
17.4 Coupling modes and the GNSS update . . . 112

VII Real-time SSR corrections **113**

18 Real-time SSR corrections **113**
18.1 Orbit correction: refines ρ_r^i . . . 114
18.2 Clock correction: refines $-c\,\mathrm{d}t^i$ . . . 114
18.3 IOD gate at correction-set boundaries . . . 115
18.4 Satellite biases: refine $-B_s^i$ and $-b_s^i$ . . . 116
18.5 Satellite-bias state gate and SSR code biases . . . 117
18.6 GLONASS . . . 118
18.7 Ionosphere: $\mu_s I_{r,1}^i$ off-wire, via the slant channel . . . 118

18.8 Provenance and code pointers . . . 119

VIII Real-time engine **120**

19 Real-time engine **120**
19.1 Streaming control flow . . . 120
19.2 NTRIP transport . . . 121
19.3 RTCM 3 framing and CRC . . . 121
19.4 MSM observation decode . . . 122
19.5 Broadcast nav, SSR and station coordinates . . . 123
19.6 Time-system handling . . . 124
19.7 Observation/SSR synchronisation . . . 124
19.8 Ephemeris management . . . 125
19.9 A-priori seeding from RTCM 1005/1006 . . . 126
19.10 The per-epoch real-time update . . . 126
19.11 Assessing a real-time run . . . 128

IX Network mode **129**

20 Network mode **130**
20.1 Producer architecture: the decomposed epoch loop . . . 130
20.2 Joint multi-station observation model . . . 134
20.3 The explicit satellite phase-bias state . . . 135
20.4 Rank deficiency and the minimum-constraint S-basis . . . 135
20.5 Per-station Schur reduction (arrowhead normals) . . . 137
20.6 Constellation scope . . . 138
20.7 The closed loop: producer → held-out user . . . 138

21 Worked example: estimability in a two-receiver network **139**
21.1 Code only: clocks and one gauge freedom . . . 139
21.2 Adding carrier phase: the satellite phase bias needs the network . . . 141

22 Between-epoch carry: arrowhead admissibility and the DD cycle basis **142**
22.1 Three doors into an arrowhead system . . . 143
22.2 The observation graph and its cycle space . . . 144
22.3 Folding the carried double differences . . . 146
22.4 Worked example: the smallest network with a cycle . . . 148
22.5 Where each object lives . . . 151

X Reference **151**

23 Pipeline and architecture diagrams **152**

A Generated reference tables **152**
A.1 Processing modes . . . 152
A.2 Execution mode and data source . . . 158
A.3 Differencing regime . . . 158
A.4 Config-to-differencing mapping . . . 158
A.5 Feature toggles . . . 158
A.6 Model-correction switches . . . 166
A.7 Logging and trace channels . . . 169
A.8 Cycle-slip detector registry . . . 171
A.9 State-vector key schema . . . 171

1 Introduction

This document is the mathematical companion to `GnssProcessor`, a C++17 post-processing and real-time GNSS engine implementing PPP, PPP-RTK (single-receiver and network), and double-differenced baseline modes. Its scope is the estimation mathematics and design rationale: the material that ages slowly because it is settled theory or algebra, not churning code.

The reference is organised as a set of *component documents*, each of which also compiles to a standalone PDF. This combined volume gathers them in roughly the order the engine applies them:

- **Foundations** (Sections 2 to 5): notation, the variable-dimension state vector, the *uncombined* pseudorange / carrier-phase observation model, and the between-satellite single-difference (BSSD) and double-difference (DD) transformations. Every other component builds on this.
- **Reference systems and measurement models** (Sections 6 to 11): the deterministic substrate and physics behind the observation model the Foundations document writes abstractly: the coordinate frames and time systems every other chapter assumes, together with the terrestrial reference frames, datum transformations, and station-coordinate epoch propagation that relate them to national frames and published coordinates; the baseline satellite orbit/clock product evaluation that SSR refines; tropospheric and ionospheric delay modelling; the geophysical site displacements (solid Earth, ocean, and pole tides) that correct the receiver coordinate; and the antenna, attitude, relativity, and range corrections that map the raw measurement to the modelled observable.
- **Estimation engine** (Sections 12 to 14): the variable-dimension Kalman filter (Joseph-form update, prior-anchored iterated relinearisation, robust gating) and the backward RTS smoother; the measurement stochastic model and the a-posteriori quality-control layer (variance factor, Baarda data-snooping, minimal detectable bias); and the supporting infrastructure that bootstraps the filter: the weighted least-squares seed, the first-epoch residual-difference state initialisation, and the pivot selection / disturbance logic.
- **Cycle-slip detection and repair** (Section 15): the linear-combination test statistics (Melbourne–Wübbena, geometry-free, GRAPHIC, extra-wide-lane) and the TurboEdit 2×2 integer repair.
- **Integer ambiguity resolution** (Section 16): the MLAMBDA integer least-squares search, wide-lane / narrow-lane decomposition, validation, constrained feedback, and the (default-off) network-producer AR machinery.

- **Inertial aiding** (Section 17): the strapdown mechanisation and 15-state INS error model of the tightly-coupled IMU path.
- **Real-time SSR corrections** (Section 18): how State Space Representation orbit, clock, and bias corrections refine the undifferenced observation model.
- **Real-time engine** (Section 19): the streaming architecture: NTRIP transport, RTCM/MSM decoding, observation/SSR time synchronisation, and the per-epoch real-time update path.
- **Network mode** (Section 20): the multi-station PPP-RTK network estimator that *generates* the SSR-style corrections consumed by the single-receiver modes: the joint observation model, the S-basis / minimum-constraint datum, the per-station Schur reduction, and the single-concern seam architecture of the per-epoch producer loop.
- **Between-epoch carry** (Section 22): how information reaches that arrowhead system from outside the current epoch: the three folds an arrowhead admits and where each must be applied, the double-difference cycle basis (the spanning tree's *chords*) that is the only gauge-free quantity worth time-linking, the Woodbury algebra that folds it, and a fully numerical minimum-parameter example.
- **Pipeline and architecture diagrams** (Section 23): the per-mode data-flow figures that place every chapter above in the epoch loop.
- **Generated reference tables** (Section A, appendix): processing modes, feature toggles, the cycle-slip detector registry, and the state-vector key schema, produced directly from the source tree by `scripts/gen_report_tables.py` rather than maintained by hand.

Implementation details (function signatures, class responsibilities, config field reference) live in the headers under `include/` and are rendered via Doxygen. When this document references a specific algorithmic component, it cites the responsible *symbol* (a function, class, or JSON config key) together with its source file (e.g. `KalmanFilter::update` in `src/core/filter/KalmanFilter.cpp`, or `network.cold_clock_seed`). Line numbers are deliberately never cited: they rot within weeks, symbol names do not.

The notation follows the IGS conventions adopted by Subirana, Zornoza, and Hernández-Pajares [23] and Teunissen and Montenbruck [27]; see Section 2 for the symbol table this document depends on.

Part I
Foundations

2 Notation and conventions

Indices and sets. We use lower-case Latin letters for indices and calligraphic upper-case for sets:

- $r \in \mathcal{R}$ is the receiver. Single-receiver modes (PPP, PPP-RTK) have $|\mathcal{R}| = 1$; baseline modes have a fixed reference and one or more rovers; the network producer (`PPP_RTK_Network`) carries a peer set of $|\mathcal{R}| \geq 2$ stations (50–305 in practice) with no reference/rover asymmetry.
- $g \in \mathcal{G} = \{\mathrm{GPS}, \mathrm{GAL}, \mathrm{BDS}, \mathrm{GLO}, \mathrm{QZS}, \dots\}$ is the constellation.

- $i \in \mathcal{S}_g$ is a satellite of constellation g. p_g denotes the pivot satellite of constellation g under the BSSD parameterisation (Section 5).
- $s \in \mathcal{B}$ is the signal/observation type, identified by its RINEX-3 three-character code (e.g. `C1C`, `L2W`, `L5X`).
- $k \in \mathbb{N}$ is the discrete epoch index.

Vectors and matrices. Bold lower-case for vectors ($\boldsymbol{x}$, $\boldsymbol{r}$); bold upper-case for matrices ($\boldsymbol{H}$, $\boldsymbol{P}$). $\boldsymbol{a}^{\mathrm{T}}$ is the transpose. A hat denotes a state estimate; a tilde a one-step prediction: $\hat{\boldsymbol{x}}_{k|k}$ vs. $\tilde{\boldsymbol{x}}_{k|k-1}$.

Observables. For each receiver/satellite/signal triple, the engine consumes two real-valued observables (in metres) per epoch:

- $P^i_{r,s}$ is the pseudorange (code), in metres.
- $\Phi^i_{r,s}$ is the carrier phase, scaled to metres via λ_s. Beyond the range, clock, and atmosphere terms it shares with the code, its ambiguous (non-range) part is $\lambda_s N^i_{r,s}$ plus the fractional phase biases $b_{r,s} - b^i_s$ (Equation (5)).

Where the distinction matters, s may further denote the band ($L_1, L_2, L_5, \ldots$) and the tracking attribute (e.g. `C`, `W`, `X`). The leading observable-type letter (`C`/`L`) selects code vs. phase; the frequency-only quantities f_s, λ_s, and $N^i_{r,s}$ depend only on the (band, attribute) part, so a code and its sibling phase share them. This matches the state keys, which carry the full phase code for ambiguities (`"G01_L1C"`) but only the band digit for receiver code biases (`"GPS_IFB2"`).

Frequencies and wavelengths. f_s is the carrier frequency (Hz) and $\lambda_s = c/f_s$ the wavelength (m). The frequency-squared scaling factor used to project the L1-equivalent slant ionospheric delay onto signal s is

$$\mu_s \;:=\; \frac{f_1^2}{f_s^2}. \tag{1}$$

Here f_1 is the GPS L1 frequency (1575.42 MHz), used as the common reference for *every* constellation, not the band-1 frequency of the satellite's own system. The estimated state $I^i_{r,1}$ is therefore GPS-L1-equivalent even for BDS/GLO/GAL satellites (`MeasurementModel::buildJacobianRow`: `f_ref = Constants::GPS_L1_FREQ`).

Differencing operators. Three differencing operations recur throughout the document:

$$\Delta q^{i,p_g}_{r,s} := q^i_{r,s} - q^{p_g}_{r,s}, \quad \nabla q^i_{r_1 r_2,s} := q^i_{r_1,s} - q^i_{r_2,s}, \quad \nabla\Delta q^{i,p_g}_{r_1 r_2,s} := \Delta\big(\nabla q^i_{r_1 r_2,s}\big).$$

Read Δq as "between-satellite single difference of q relative to pivot p_g," ∇q as "between-receiver single difference," and $\nabla\Delta q$ as "double difference."

Sign conventions. We adopt the convention that the slant ionospheric delay $I^i_{r,s}$ enters *additively* on the code observable and *subtractively* on the carrier phase observable; this is the source of the well-known "code–phase divergence." All other range-additive terms (geometry, clocks, troposphere, biases) carry the same sign on both observables. See Equations (4) and (5) for the explicit equations.

Symbol table.

Symbol	Units	Meaning
$\boldsymbol{r}_r, \boldsymbol{r}^i$	m	ECEF position of receiver/satellite
ρ_r^i	m	geometric range $\|\boldsymbol{r}^i - \boldsymbol{r}_r\|$
$\boldsymbol{e}_r^i$	none	unit LOS from r to i in ECEF
$c\,\mathrm{d}t_r$	m	receiver clock offset (scaled by c)
$c\,\mathrm{d}t^i$	m	satellite clock offset (CLK product in user modes; estimated state `"<sat>|SC"` in `PPP_RTK_Network`)
$c\,\delta t_r^{(g)}$	m	receiver inter-system time offset for constellation $g \neq g_{\mathrm{ref}}$
T_r^i	m	total tropospheric path delay
$\mathrm{ZHD}_r^i, \mathrm{ZWD}_r^i$	m	zenith hydrostatic / wet components
$I_{r,1}^i$	m	L1-equivalent slant ionospheric delay
$N_{r,s}^i$	cyc	integer carrier-phase ambiguity
$b_{r,s}, b_s^i$	m	receiver / satellite phase biases
$\bar{b}_s^i$	m	estimable satellite phase bias (network mode; Section 3)
$B_{r,s}, B_s^i$	m	receiver / satellite code biases (OSB)
$B_{r,k}^{(g,b)}$	m	per-FCN-slot GLONASS receiver IFB, band b; here $k \in [-7, +6]$ is the signed FDMA channel number (a context-local reuse, not the epoch index)
$\varepsilon^P, \varepsilon^\Phi$	m	residual (multipath + thermal)
$\boldsymbol{v}_r$	m/s	receiver velocity (optional / IMU state)
$\lambda_{\mathrm{WL}},\ \lambda_{\mathrm{NL}},\ \lambda_{\mathrm{EWL}}$	m	wide- / narrow- / extra-wide-lane wavelength $c/(f_a \mp f_b)$ for the lane's band pair
λ_{IF}	m	effective ionosphere-free wavelength $\|c_1\lambda_1 - c_2\lambda_2\|$ with the IF coefficients $c_1 - c_2 = 1$; not an integer wavelength (Section 15)
$\boldsymbol{C}_b^e$	none	body-to-ECEF attitude (direction-cosine) matrix

3 State vector layout

The Kalman filter operates on a state vector $\boldsymbol{x}_k \in \mathbb{R}^{n_k}$ whose dimension n_k varies across epochs. A per-satellite or per-(satellite, signal) state is appended when its satellite or signal is first required, and removed when it is absent from the epoch's measurement set (for example, after the satellite sets below the elevation mask). A cycle slip does not remove the ambiguity state; it re-seeds it (see the ambiguity paragraph below). `StateManager` holds all bookkeeping: the key→index maps, the appending of rows, and the compaction of $\boldsymbol{x}$ and $\boldsymbol{P}$ on removal, which shifts every later index down by one (there is no free list). `KalmanFilter` reads the state through that interface and never indexes it by hard-coded offsets.

3.1 Schematic layout

In schematic form, suppressing the epoch index k:

$$\begin{aligned}\boldsymbol{x} \;=\; \big[\; & \underbrace{\boldsymbol{r}_r^{\mathrm{T}}}_{3},\ \underbrace{\boldsymbol{v}_r^{\mathrm{T}}}_{0\text{ or }3},\ \underbrace{c\,\mathrm{d}t_r}_{1},\ \underbrace{(c\,\delta t_r^{(g)})^{\mathrm{T}}_{g\neq g_{\mathrm{ref}}}}_{|\mathcal{G}|-1},\\ & \underbrace{\mathrm{ZWD}_r,\ (g_N, g_E)}_{1+0\text{ or }1+2},\ \underbrace{(c\,\mathrm{d}t_r^{\phi})}_{0\text{ or }1},\\ & \underbrace{(I_{r,1}^{i})^{\mathrm{T}}_{i\in\mathcal{V}_k}}_{|\mathcal{V}_k|},\ \underbrace{(N_{r,s}^{i})^{\mathrm{T}}_{(i,s)\in\mathcal{A}_k}}_{|\mathcal{A}_k|},\\ & \underbrace{(B_{r,s})^{\mathrm{T}}_{s\in\mathcal{B}}}_{|\mathcal{B}|},\ \underbrace{(B_s^{i})^{\mathrm{T}}_{(i,s)\in\mathcal{D}_k}}_{|\mathcal{D}_k|}\;\big]^{\mathrm{T}}. \end{aligned} \tag{2}$$

where

- $\mathcal{V}_k$ is the set of satellites visible at epoch k (above the elevation mask, with valid orbit and clock products and at least one tracked signal);
- $\mathcal{A}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the set of phase-tracked (satellite, signal) pairs;
- $\mathcal{D}_k \subseteq \mathcal{V}_k \times \mathcal{B}$ is the subset of (satellite, signal) pairs that carry a satellite code-bias state: the pairs for which the BIA product lacks a satellite OSB or, in the network producer's satellite-OSB mode, every raw code pair (see `estimation.estimate_sat_biases` and `network.estimate_sat_code_bias`).

The schematic omits several conditional refinements, detailed below: the rate states that accompany velocity (a receiver clock rate and one inter-system clock-rate offset per non-reference constellation, both absent under BSSD); the optional per-satellite slant-ionosphere rate under `enable_iono_rate`; the receiver inter-signal code biases of the network producer; the network-producer satellite-clock, satellite-phase-bias and receiver-phase-bias states; the nine-state inertial augmentation of Section 17; and the removal of the whole receiver-side clock and inter-system-offset block under the BSSD parameterisation (Section 5.1).

The schematic groups the states by physical role; it is not the storage order. `StateManager` places a fixed core first, in the order position, receiver clock (absent under BSSD), ZWD, velocity, receiver clock rate, phase clock, the two troposphere gradients, the nine inertial states, the inter-system time offsets and the inter-system clock-rate offsets, each present only when enabled. Every dynamic state (ionosphere, ionosphere rate, ambiguity, receiver code bias, satellite code bias, satellite phase bias, receiver phase bias, satellite clock) is appended after the core in order of creation, so the categories interleave; each category is addressed only through its key map, as described under dynamic indexing below.

3.2 Component-by-component description

Position $\boldsymbol{r}_r \in \mathbb{R}^3$. ECEF Cartesian position of the antenna reference point. Always present. `Application::initializePosition` seeds it from the RINEX header approximate position, shifted from the marker to the antenna reference point by the header's antenna H/E/N offset (or from the Station Information File coordinate when that a-priori source is enabled). The seed must lie within the linearisation radius of the geometric range model (Section 4.3). The per-epoch Weighted Least-Squares (WLS) solution seeds the receiver clock (on the first epoch, after a clock jump and

after a data gap); it re-seeds the position only in the kinematic gap-recovery branch of `Application::handleGap`.

Velocity $\boldsymbol{v}_r \in \mathbb{R}^3$. Optional. Enabled by `estimation.estimate_velocity = true`; tightly-coupled IMU integration forces it on. With velocity present, position and velocity form a coupled continuous-white-noise-acceleration (CWNA, integrated-random-walk) block driven by a single acceleration spectral density, `process_noise.acceleration_psd_q` (Equation (150)); the scalar `position_q` of the velocity-free case is then unused. When BSSD is off, velocity estimation also adds a receiver clock-rate state $c\,\dot{\mathrm{d}}t_r$ and one inter-system clock-rate offset (ISBR) per non-reference constellation, each CWNA-coupled to its bias counterpart (`clock_rate_q`, `system_time_rate_offset_q`). Under BSSD both rate states are omitted, because same-system single differencing cancels them.

Receiver clock $c\,\mathrm{d}t_r$. Single state, in metres. Its process noise is large (`clock_bias_q` $\sim 10^4\,\mathrm{m}^2$ per second), so the state is in effect re-estimated each epoch. This follows oscillator drift without imposing a stochastic model that may not match the receiver's oscillator. Under the BSSD (between-satellite single-difference) parameterisation (Section 5.1) the receiver clock is unobservable and the state is not allocated; neither are the inter-system offsets below, so PPP-RTK SD mode has no receiver-side clock block.

Inter-system time offsets $c\,\delta t_r^{(g)}$. One per active constellation other than GPS, which is the fixed reference in `StateManager`. These absorb the per-constellation clock biases that arise from mixing analysis-centre clock products across systems ("GAL − GPS time offset," etc.).

Tropospheric ZWD and gradients. ZWD_r is the zenith wet delay. The zenith hydrostatic delay ZHD_r is deterministic, from the Saastamoinen model [22]. The total slant tropospheric delay is

$$T_r^i \;=\; m_h(\theta_r^i)\,\mathrm{ZHD}_r \;+\; m_w(\theta_r^i)\,\mathrm{ZWD}_r \;+\; m_w(\theta_r^i)\cot\theta_r^i\left[g_N\cos\alpha_r^i + g_E\sin\alpha_r^i\right], \tag{3}$$

with elevation θ_r^i, azimuth α_r^i, and mapping functions m_h, m_w: the Niell tables by default (labelled `Gmf` in the configuration), or VMF3 with GPT3 coefficients when configured. The gradients (g_N, g_E) are optional.

Per-satellite slant ionosphere $I_{r,1}^i$. PPP-RTK only (uncombined processing). One state per visible satellite, expressed at the L1 frequency; signal-specific delays follow Equation (1). `StateInitializer::computeIonosphereSeed` seeds it from the geometry-free combination of the satellite's BIA-covered code residuals. With fewer than two covered codes on different bands, the seed is zero and the initial standard deviation is inflated tenfold. The initial variance, like the process noise, is scaled by the obliquity factor. The process noise is a random walk whose zenith spectral density `ionosphere_q` is projected onto the slant by the squared obliquity factor $1/\sin^2\theta_r^i$ (Equation (149)), so low-elevation states vary faster. These explicit per-satellite states let PPP-RTK keep integer ambiguity recovery without forming the ionosphere-free combination, which amplifies code noise by a factor of ~ 3. Under `estimation.enable_iono_rate` (default off), each visible satellite also carries a slant-ionosphere rate, so that the pair forms a constant-velocity block $[\,I_{r,1}^i,\ \dot{I}_{r,1}^i\,]$. The rate $\dot{I}_{r,1}^i$ has no direct observation ($H = 0$); only the F-coupling into the level state drives it. It has its own spectral density `process_noise.iono_rate_psd` and its own slip response (`reinflateIonosphereRateVariance`), and lets the filter track a linear slant ramp rather than a pure random walk. The rate state is a satellite-specific dynamic row, not a core state: it is

appended with its level state, pruned with it and treated with the other per-satellite states in gap recovery, but after prunes and re-adds it need not sit next to the level state, so it is addressed only through its own map (`iono_rate_indices_`).

Carrier-phase ambiguities $N^i_{r,s}$. One state per phase-tracked (i, s) pair, in metres (the state value is λN; the `nl_wl` basis below is the exception, carrying integers in cycles). The ambiguities are float through the prediction and update cycle; when AR is enabled, the LAMBDA-family algorithms in `filter/AmbiguityResolver` [26] resolve them to integers afterwards. A cycle slip calls `StateManager::resetAmbiguity`, which decorrelates the state rather than repairing it: the ambiguity's covariance row and column are zeroed, its diagonal is reset to σ_m^2, and the key is marked pending re-seed. The state value is left unchanged by the reset; `ensureRequiredStates` (in `MeasurementModel`) then overwrites it with a geometry-based seed. This is the project's reset-over-repair convention: a slipped ambiguity has no trustworthy correlation with the rest of the state, so its cross-covariances are cleared. A reset for a key that is not in the state (for example, a signal excluded by the band filter) is refused and creates no state.

Receiver code biases $B_{r,s}$. These absorb the receiver-side inter-frequency biases that the OSB convention does not reference to zero. They are not one per signal. `StateInitializer::resolveReceiverBias` keys them per (system, frequency band): both bands of the system's clock-reference signal pair map to one per-system differential code bias (`"<system>_DCB"`), each band with its own coefficient, and every other band gets its own inter-frequency bias (`"<system>_IFB<band>"`). Like the receiver clock they are receiver-common, so under BSSD they receive no process noise and the SD transform zeroes their columns. On the network path, under `network.second_code_channel` or `processing.channel_mode:all`, a code that is not the station's latched canonical code on its band gets a separate receiver inter-signal code bias (`"<system>_ISC_<signal>"`, unit coefficient, GLONASS excluded), stored in the same map (card SCC). On the network path these columns have no Kalman time update at all, because the producer runs no `KalmanFilter` (Section 12); they are re-estimated each epoch unless `network.carry_rx_code_bias` is armed. That knob (loader default off; armed in the current golden configurations) adds a diagonal random-walk carry ($\mathrm{var} + q\,\Delta t$) held in the network run context rather than in `StateManager`. Two guards protect it: at load, the loader forces the carry off unless the code datum is time-linked (`network.code_datum_time_linked`); at inject time, an innovation gate (`rxCbInjectGate` in `filter/NetworkRunTypes`) withholds the fold when the carried level and the station's own estimate disagree at receiver-reset scale.

Per-slot GLONASS inter-frequency bias $B^{(g,b)}_{r,k}$. Optional, off by default. GLONASS FDMA assigns each slot $k \in [-7, +6]$ a distinct carrier frequency $f = f_{\mathrm{base}} + k\,\Delta f$, so the receiver group delay differs per slot and a single per-band IFB state cannot represent it. With `estimation.glonass_ifb.enable = true` and its prerequisite `estimation.estimate_ifb` (without which the loader warns and the flag is inert), `resolveReceiverBias` instead creates one state per (FDMA band $b \in \{\mathrm{G1}, \mathrm{G2}\}$, slot k) with unit measurement coefficient, keyed `"GLONASS_IFB<band>_k<±slot>"`. The central channel $k = 0$ is the held datum and has no state; the GLONASS inter-system time offset absorbs its constant. The CDMA bands (G3/G4/G6) have a slot-independent frequency; with the flag on, the GLONASS branch of `resolveReceiverBias` returns no receiver bias key for them (it handles every GLONASS signal and does not fall through to the per-band path). With the flag off, GLONASS carries no per-band IFB state on a non-reference band. Unlike the receiver-common biases, these columns survive between-satellite single differencing, because two GLONASS satellites on different slots do not cancel. `BssdTransform` and `StateManager` therefore exempt them from

the receiver-block zeroing and from the process-noise suppression (via `isGlonassPerSlotIfbKey`); only the inter-slot differences are estimable in SD.

Satellite code biases B_s^i. Per (satellite, signal) pair, but only for a pair whose OSB the BIA file does not supply, typically a third-frequency signal in a triple-frequency setup. This fallback rule is the user-path and network-V1 behaviour. In the network producer's satellite-OSB mode (`network.estimate_sat_code_bias`, which `network.estimate_iono_sh` forces on unless `network.external_sat_code_bias` is armed), the state covers every raw code (i, s) rather than only the uncovered pairs, and the external `.BIA` is not used, so the network's own estimate is the single code-bias source. The datum that makes it estimable is derived in Section 20. These states are weakly stochastic (`sat_bias_q` $\sim 10^{-10}$), because the underlying physical biases are nearly stationary over a day.

Satellite phase biases $\bar{b}_s^i$ (network mode). Present only in the network producer (`ProcessingMode::PPP_RTK_Network`) and gated by `estimation.estimate_sat_phase_biases`. The JSON default is off and the flag is inert in every other mode. In the one mode where the states exist, the network-producer loader forces the flag on unconditionally (the branch in `GnssConfigLoadNetwork` that logs *forcing estimate_sat_clocks + estimate_sat_phase_biases*), and `GnssConfig::applyNetworkConsumerCouplings` turns it off again for the closed-loop consumer, which consumes these states rather than estimating them. The user side imports the satellite phase bias as a known SSR correction (Section 18) or absorbs it into the float ambiguity. The network instead carries an explicit per-(satellite, signal) phase-bias state on the undifferenced phase rows: the estimable bias of Odijk [17], which lumps the network pivot-receiver ambiguity. On a single station's row it is collinear with the per-receiver float ambiguity; only the joint network estimator separates it, across stations (Section 20). It has its own state map, distinct from the satellite code biases above (`StateManager::sat_phase_bias_indices_`), keyed `"<sat>|<signal>|PB"`. It is near-stationary between epochs, with rate `sat_phase_bias_q`; on the network path that rate inflates a shared-border carry prior rather than entering a Kalman $\boldsymbol{Q}(\Delta t)$ (see the satellite-clock paragraph below).

Receiver phase biases $\delta_{r,s}$ (network mode). The receiver-side counterpart of the satellite phase bias, present only in the network producer and gated by `network.phase_datum_eq34` (the Eq-34 / single-receiver-chain phase datum; see Section 20). With its own estimable state $\delta_{r,s}$, the receiver fractional phase bias no longer lumps into the emitted satellite phase bias, so the emitted between-satellite reference stays integer-consistent across all satellites (each chain hop contributes only an integer). It is per (receiver, system, signal), keyed `"<system>_RPB_<signal>"` in its own map (`StateManager::receiver_phase_bias_indices_`). Unlike every other bias block described here, these are local states, each observed by a single station. A knob sets whether they carry across epochs. At `network.phase_datum_eq34_rpb_q_m2s` $= 0$ each $\delta_{r,s}$ is re-estimated every epoch (the Card-3 behaviour). At the non-zero default ($3 \times 10^{-6}\,\mathrm{m^2\,s^{-1}}$) the producer folds a local time-linking prior toward the previous estimate with variance var $+ q\,\Delta t$. The prior suppresses the per-epoch bounce that a weakly observable local state otherwise injects, anti-correlated, into every individual float ambiguity. That carry lives outside `StateManager`, in the network run context (the `rpb_carry` map of `app/NetworkRunContext`): it is read during the parallel block assembly, harvested after the solve, and erased for any (i, s) whose chain frame stepped at that epoch's election (`filter/NetworkChainDatum`). Under `network.carry_invariant` the carry is off, and the loader warns that the q is inert. In either case the receiver-side wander goes into $\delta_{r,s}$, and the time-linked integer information stays in the ambiguity carry.

Satellite clocks $c\,\Delta\delta t^i$ (network mode). Also network-producer-only, gated by `estimation.estimate_sat_clocks`. Like the phase bias above, it is off by JSON default and inert elsewhere, and the network-producer loader forces it on unconditionally at the same site. The network estimates an explicit per-satellite clock correction $\Delta\delta t^i$ (in metres, relative to the a-priori precise or real-time product clock already removed from `obs.modeled`), so the emitted clocks stay consistent with the chosen S-system datum. There is one state per satellite, with no signal field. The satellite orbit stays fixed from the product; only the clock is estimated. The state has its own map (`StateManager::sat_clock_indices_`), keyed `"<sat>|SC"`. Its cross-epoch rate `sat_clock_q` is near-white, but on the network path it is not a Kalman $\boldsymbol{Q}(\Delta t)$ entry: the producer runs no `KalmanFilter` (Section 12), and the rate instead inflates the shared-border carry prior as $\mathrm{var} + q\,\Delta t$. The same holds for `sat_phase_bias_q` and `sat_code_bias_q`. All three can be set through their legacy `filter.process_noise.*` keys or through the network-scoped `network.process_noise` class table, whose per-class fallbacks equal the legacy defaults. Between full solves the split-cadence fast tier (`network.fast_clock`, default off and currently gate-failed) may refresh the emitted clock. That tier estimates a clock delta in its own least-squares tick and creates no state-vector row, so it does not appear here; see Section 20.

Inertial augmentation. With IMU integration enabled, the core state gains nine entries: a three-axis attitude error and the three-axis accelerometer and gyroscope biases. The strapdown mechanisation propagates them, rather than the random-walk process model. Section 17 treats them in full.

Baseline differential troposphere. In baseline mode the core is the position alone; double differencing cancels the receiver clocks and inter-system offsets, and on a short baseline the ionosphere and troposphere, so none of those states exists. An optional single differential zenith-wet-delay state $\Delta\mathrm{ZWD}_{r_{\mathrm{rov}}r_{\mathrm{ref}}}$ follows the position (`baseline.estimate_differential_tropo`, Section 5.3). This is the only mode in which `StateManager` reads its `core_state_size` constructor argument; the PPP and PPP-RTK paths ignore it and derive the core layout from the configuration.

3.3 Dynamic indexing

`StateManager` addresses the variable-dimension blocks through key→index maps. `include/filter/StateKeySchema.hpp` defines every key format once, each with a `make` builder and, for the categories that are read back, a `parse` helper (the ionosphere and inter-system time-offset keys have no parser, since the key is the bare satellite or system name). Some call sites still build keys by string concatenation rather than through the builders (for example the receiver IFB and DCB keys in `resolveReceiverBias` and the double-difference keys in `MeasurementModel`, `DdPivotManager` and `KalmanFilter`); the strings they produce match the schema formats. The formats are:

- iono key: `"<sys><prn>"`, e.g. `"G05"` (delimiter-free);
- ambiguity key: `"<sys><prn>_<signal>"`, e.g. `"G05_L1C"`, split on the *first* underscore, so an iono-free combination `"G05_L1C_L5Q"` retains its compound signal;
- satellite signal-bias key (code): `"<sys><prn>|<signal>"`, e.g. `"G05|C1W"`, with a pipe delimiter and its own map, disjoint from the underscore ambiguity namespace;
- satellite phase-bias key (network): `"<sys><prn>|<signal>|PB"`, e.g. `"G05|L1C|PB"`, where the trailing `|PB` keeps it disjoint from the code-bias namespace, in its own map (`sat_phase_bias_indices_`);

- receiver phase-bias key (network): "<system>_RPB_<signal>", e.g. "Galileo_RPB_L5Q", where the _RPB_ infix keeps it disjoint from the receiver code-bias namespace (which matches on IFB/DCB), in its own map (receiver_phase_bias_indices_);
- satellite clock key (network): "<sat>|SC", e.g. "G05|SC", per satellite rather than per signal; the trailing |SC discriminator gives the clock its own map (sat_clock_indices_);
- receiver code-bias key: "<system>_IFB<band>" or "<system>_DCB"; the per-slot GLONASS variant appends an FCN marker, "GLONASS_IFB<band>_k<±slot>" (e.g. "GLONASS_IFB1_k-4");
- receiver inter-signal code-bias key (network, card SCC): "<system>_ISC_<signal>", e.g. "GPS_ISC_C1W", stored in the receiver code-bias map; the _ISC_ infix matches neither the IFB nor the DCB parser;
- inter-system time-offset key: the bare system name ("Galileo", ...; GPS is the reference and never appears);
- double-difference ambiguity key (baseline): "DD|<cons>|<sat>|<code>|<pivot>".

Addressing states by key rather than by position removes the index bookkeeping errors of hand-managed dynamic state vectors, and lets the BSSD and DD S-transforms (Section 5) operate as block operations on the relevant key sets.

Two further categories in the schema header are absent from the list above, because they are not rows of $\boldsymbol{x}$. The first is the BSSD pivot absolute-seed lookup key ("<cons>|<band>"). Its builder has no production caller and it has no parser: MeasurementModel builds the pivot_abs_seeds keys by concatenation, with the full phase signal code in place of the band ("GPS|L1C"), plus narrow-lane and wide-lane variants under the nl_wl basis. The second is the shared spherical-harmonic VTEC coefficient key ("SH|<n>|<m>|<C|S>", with the second shell on the distinct SH2 prefix), which names columns of the network estimator's shared border rather than StateManager states and has a make/parse pair. Both appear in Table 11, which is derived from the schema header by a script and is therefore the completeness reference for the list above.

Ambiguity basis. By default the ambiguity block carries one per-band float ambiguity per tracked (satellite, signal), in metres. Setting estimation.state_ambiguity_basis = "nl_wl" switches the whole run to a narrow-lane / wide-lane parameterisation. Each satellite has a reference band, the lowest-numbered band it is observed on that epoch, and carries one narrow-lane state tied to that band (key "<sat>_NL_<band>") and one wide-lane state for each other observed band, paired with the reference band ("<sat>_WL_<a>_<b>", with the band digits in ascending order, semantically $N_a - N_b$). A change of reference band produces new keys, and the old states are pruned. Both kinds are in cycles rather than metres, so the wide-lane and LAMBDA arithmetic stays native to integers; the Jacobian carries the wavelength scaling. The two bases never coexist in a single state vector.

4 Observation equations

This section gives the full *undifferenced* observation model that the filter linearises around at each epoch. The PPP-RTK parameterisation is uncombined: per-signal pseudorange and carrier phase are consumed directly, without forming ionosphere-free or wide-lane combinations as a preprocessing step. Combinations (IF, MW, GF) are still computed for diagnostic and slip-detection purposes (Section 15; see slip/MwSlipDetector, slip/GraphicSlipDetector) but they do not enter the measurement update path. This holds whenever the uncombined parameterisation is active; the loader forces uncombined_enabled=true for the whole PPP-RTK family (user, network producer,

network consumer) in `GnssConfig::loadProcessingSection_`; the same site forces it for the baseline double-difference modes, which process each signal independently (Section 5.3). Plain PPP mode leaves the flag at its default `false` and instead builds ionosphere-free combined observables (keys such as `C1W_C5X`) that *do* feed `MeasurementModel::buildJacobianRow` and the update; the sequel is written for the uncombined case.

4.1 Code (pseudorange) observation

For receiver r tracking satellite i on signal s, the observed pseudorange is

$$\boxed{P^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;+\; \mu_s\, I^i_{r,1} \;+\; B_{r,s} - B^i_s \;+\; \varepsilon^P_{r,i,s}.} \tag{4}$$

4.2 Carrier-phase observation

The corresponding carrier-phase observation, scaled to metres by λ_s, is

$$\boxed{\Phi^i_{r,s} \;=\; \rho^i_r \;+\; c(\mathrm{d}t_r - \mathrm{d}t^i) \;+\; c\,\delta t_r^{(g(i))} \;+\; T^i_r \;-\; \mu_s\, I^i_{r,1} \;+\; \lambda_s\, N^i_{r,s} \;+\; b_{r,s} - b^i_s \;+\; \varepsilon^\Phi_{r,i,s}.} \tag{5}$$

The defining structural difference between Equations (4) and (5) is the sign of the ionospheric term: $+\mu_s I$ on code, $-\mu_s I$ on phase. Three additional remarks:

1. The phase ambiguity $N^i_{r,s}$ is integer in principle but is contaminated by the receiver-plus-satellite phase biases $b_{r,s} - b^i_s$. In a *float* solution these biases are absorbed into $N^i_{r,s}$, leaving a non-integer state with no structural advantage over the code observation. Recovering integer-valued ambiguities therefore requires either external Fractional Cycle Bias (FCB) products or differencing (Section 5), which cancels common-mode bias terms.
2. The clock product $c\,\mathrm{d}t^i$ is read from the analysis-centre CLK file (post-processing) or refined from the broadcast clock by a streamed SSR correction (real-time, Section 18) and treated as deterministic in the filter; any residual product error appears on the observation and is absorbed by the receiver clock state.
3. The system time offset $c\,\delta t_r^{(g(i))}$ is zero by construction when $g(i)$ is the reference constellation (default GPS). Together with the receiver clock $c\,\mathrm{d}t_r$ this means a multi-constellation run carries $|\mathcal{G}|$ clock-like states in total.

4.3 Geometric range and linearisation

The geometric range $\rho^i_r = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|_2$ enters Equations (4) and (5) non-linearly through the receiver position $\boldsymbol{r}_r$. The filter linearises about the current state estimate $\hat{\boldsymbol{r}}_r$:

$$\rho^i_r \;=\; \hat{\rho}^i_r \;-\; (\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}(\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r) \;+\; \mathcal{O}\big(\|\boldsymbol{r}_r - \hat{\boldsymbol{r}}_r\|^2\big)\,, \tag{6}$$

with the unit line-of-sight

$$\boldsymbol{e}^i_{\hat{r}} \;:=\; \frac{\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r}{\|\boldsymbol{r}^i - \hat{\boldsymbol{r}}_r\|_2}. \tag{7}$$

The Jacobian row $\boldsymbol{H}_{(i,s)}$ contributing to the design matrix at the position columns is therefore $-(\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}$. For sub-metre residual targets this single linearisation per epoch is sufficient; the engine optionally relinearises within the epoch (`processing.relinearize_geometry`, which defaults to off

in code and is enabled in the PPP, PPP-RTK, baseline, and real-time configs but left off in the six network-producer configs) to handle long initial position offsets.

The satellite position $\boldsymbol{r}^i$ is itself a function of the transmission time. Rather than iterate the emission time against the geometric range, which depends on $\boldsymbol{r}_r$, `Geometry::computeEmissionState` solves it in a single step from the *pseudorange*: the time of flight is $\tau = P/c$, subtracted (with the satellite clock) from the receiver epoch. Because the receiver clock bias $\mathrm{d}t_r$ enters the receiver epoch as $+\mathrm{d}t_r$ and the pseudorange as $+c\,\mathrm{d}t_r$, it cancels exactly, leaving a residual $(I+T+\varepsilon)/c \approx 30$–$60$ ns of satellite displacement, sub-millimetre and below any PPP threshold. This deliberately avoids a range-based fixed point: iterating against ρ_r^i would re-introduce an uncancelled $\mathrm{d}t_r$ and drive a positive-feedback loop through the filter's receiver-clock state. The only second pass (the `model_corrections.refine_emission_time` block in `Geometry::computeSatelliteGeometry`) re-runs `computeEmissionState` on a cleaned pseudorange, having first subtracted the slant hydrostatic delay and the IF-combined ATX projection from the seed, a pseudorange-cleanliness refinement rather than a range-versus-position iteration.

4.4 Stochastic model

Observation noise covariance is structured per epoch as a block diagonal matrix with entries

$$\sigma^2_{r,i,s} \;=\; \sigma^2_{0,\mathrm{type}(s)} \;\cdot\; w(\theta_r^i)^{-1} \;\cdot\; \xi(\mathrm{C/N_0 \text{ at } } (r,i,s)), \tag{8}$$

combining a per-type baseline ($\sigma_{0,\mathrm{code}}$, $\sigma_{0,\mathrm{phase}}$ from `filter.measurement_noise`), an elevation-dependent down-weighting factor $w(\theta_r^i)^{-1} = (\sin\theta_r^i)^{-2p_{\mathrm{type}}}$ whose exponent is *continuous* and *type-dependent*: code and Doppler rows take `filter.code_elevation_power`, carrier-phase rows `filter.phase_elevation_power`, with the weighting strategy itself selected by `filter.stochastic_model` (derived, together with the low-elevation guard that leaves σ_0^2 unamplified, in Equation (166)), and an optional carrier-to-noise ratio scaling ξ parameterised by `snr_model_params`. Cross-correlations between observations are assumed zero in the undifferenced parameterisation; differencing introduces structured correlations handled in Section 5.

5 Differencing transformations

PPP-RTK and baseline modes do not consume the undifferenced observations of Equations (4) and (5) directly. Each forms a linear combination across satellites (BSSD), receivers (BRSD), or both (DD), absorbing common-mode terms into the cancellation and recovering integer-friendly residual structure. This section gives both the algebraic cancellations and the matrix-form S-transforms that map undifferenced parameters to differenced ones.

5.1 Between-satellite single difference (BSSD)

For each constellation g, choose a pivot satellite p_g (elected by `PivotManager::selectPivotCandidates` under the "sticky pivot" retention policy; the selection ordering is detailed in the *Pivot selection* note below). For any non-pivot satellite $i \in \mathcal{S}_g \setminus \{p_g\}$ and signal s, define

$$\begin{aligned} \Delta P^{i,p_g}_{r,s} &:= P^i_{r,s} - P^{p_g}_{r,s}, \\ \Delta \Phi^{i,p_g}_{r,s} &:= \Phi^i_{r,s} - \Phi^{p_g}_{r,s}. \end{aligned} \tag{9}$$

Substituting Equations (4) and (5) and cancelling the receiver-side common terms ($c\,\mathrm{d}t_r$, $c\,\delta t_r^{(g)}$, $B_{r,s}$, $b_{r,s}$) yields

$$\Delta P_{r,s}^{i,p_g} = \Delta\rho_r^{i,p_g} - c\,\Delta\mathrm{d}t^{i,p_g} + \Delta T_r^{i,p_g} + \mu_s\,\Delta I_{r,1}^{i,p_g} - \Delta B_s^{i,p_g} + \Delta\varepsilon^P, \tag{10}$$

$$\Delta\Phi_{r,s}^{i,p_g} = \Delta\rho_r^{i,p_g} - c\,\Delta\mathrm{d}t^{i,p_g} + \Delta T_r^{i,p_g} - \mu_s\,\Delta I_{r,1}^{i,p_g} + \lambda_s\,\Delta N_{r,s}^{i,p_g} - \Delta b_s^{i,p_g} + \Delta\varepsilon^\Phi. \tag{11}$$

The cancellations are exact only when both observations enter the filter with the same modelling: same tropospheric mapping (which is why ΔT is small but not zero), same antenna PCV correction, and so on.

The receiver-bias cancellation of $B_{r,s}$ in Equation (10) holds for CDMA constellations, whose group delay is a single per-signal receiver term. GLONASS is the exception: its FDMA inter-frequency biases are per-FCN-slot states, so two satellites on different slots carry different receiver delays and the term does *not* cancel between them: a residual $B_{r,s}^{(k(i))} - B_{r,s}^{(k(p_g))}$ survives the single difference. Accordingly `BssdTransform` zeroes the receiver-DCB columns after differencing *except* the per-slot GLONASS IFB states (guarded by `StateKeys::isGlonassPerSlotIfbKey`), and `PivotManager::buildEpochMaps` drops GLONASS from the BSSD candidate maps altogether, not merely from pivot election, so with the flag off the constellation contributes no single-differenced row at all. GLONASS BSSD is default-OFF (`estimation.between_satellite_sd.enable_glonass`); when enabled, Equation (10) acquires the surviving inter-slot IFB difference as an additional estimated column.

Pivot selection. Election is not by elevation alone. The primary key is a *hard* lexicographic tier on phase-signal coverage: a full-coverage candidate outranks any partial-coverage candidate regardless of its score. Only within a coverage tier does the additive score $-\min\bigl(1, \sqrt{\sigma^2_{\hat{N},\max}}\bigr) + \mathrm{el}/90 - \frac{1}{2}\mathbb{1}[\mathrm{el} < 20^\circ]$ apply, in which float-ambiguity convergence $\sqrt{\sigma^2_{\hat{N},\max}}$ and elevation enter as co-equal terms (with a hard low-elevation guardrail below 20°). Layered on top is an ephemeris-freshness bar: a candidate whose broadcast $|\mathrm{obs} - \mathrm{toe}|$ exceeds its constellation's freshest qualified peer by more than `kPivotToeSpreadMaxS` (1200 s) is barred from *pivot* election (though still usable as a non-pivot), which prevents a stale-eph pivot from stepping every SD row it anchors. Candidates below `estimation.between_satellite_sd.pivot_min_elevation` are excluded outright. The baseline DD path scores differently: `DdPivotManager` ranks on $\mathrm{avg_snr} + 0.01 \cdot \mathrm{num_sigs}$ and does not use elevation.

Pivot transitions. A handover fires on two kinds of trigger. The visibility-driven one is simple: when the incumbent pivot leaves the visible set, p_g is re-elected from the remaining satellites. But the dominant one in practice is *event-driven*: the post-fit residual analyser calls `PivotManager::requestForcedHandover` when the pivot's *observation* slipped while its filter *state* is still intact (so the resting `isPivotDisturbed` check, which tests only that the pivot ambiguity sits at exactly $(0, 0)$, would not catch it). The forced request carries an exclusion set so the handover cannot bounce onto another offset satellite, a `retired_pivots_this_epoch_` backstop that terminates cascades, a same-epoch re-entry gate (`hasPendingForcedHandover`), and a re-inflation set for states that absorbed a coherent pivot-side code bias (the information-conserving transform below cannot evict that offset on its own). The baseline DD path exposes the symmetric `DdPivotManager::requestForcedRerank`. This event response is load-bearing precisely because the pivot's states are frozen at zero variance (below): a stiff, time-linked zero-q quantity requires a wired jump response.

On election the float ambiguities $\{\Delta N_{r,s}^{i,p_g^{\text{old}}}\}_{i,s}$ must be re-expressed relative to p_g^{new}. Where the new pivot already carries an established ambiguity in the group, the engine applies the linear S-transformation $\boldsymbol{x}^{\text{new}} = \boldsymbol{S}\,\boldsymbol{x}^{\text{old}}$ with $\boldsymbol{S}$ chosen so that $\Delta N_{r,s}^{p_g^{\text{new}},p_g^{\text{old}}}$ becomes the new reference (zero), preserving the integer information accumulated since the last fix; the covariance transforms as $\boldsymbol{P}^{\text{new}} = \boldsymbol{S}\,\boldsymbol{P}^{\text{old}}\,\boldsymbol{S}^{\mathrm{T}}$. When it cannot, because the new pivot has no existing ambiguity in the (cons, code) group (`DdPivotManager`, logged as "reset-and-reseed"), or a disturbed pivot has no healthy replacement in a single-satellite constellation (`PivotManager::handleDisturbedPivot` re-zeroes $\boldsymbol{x}$ and the corresponding rows/columns of $\boldsymbol{P}$ in place), the transform is skipped and the stale rows are discarded and re-seeded. In the uncombined PPP-RTK parameterisation the same $\boldsymbol{S}\,\boldsymbol{P}\,\boldsymbol{S}^{\mathrm{T}}$ re-reference is applied not only to the ambiguity block but to *three* state groups (`reReferenceAmbiguities`, `reReferenceIonosphere`, and `reReferenceSatBiases`), which is why `BssdTransform` zeroes the pivot's ionosphere and sat-bias $\boldsymbol{H}$-columns after differencing, and why $\Delta I_{r,1}^{i,p_g}$ in Equations (10) and (11) is itself a pivot-referenced SD quantity, not an absolute slant delay. The S-basis constraint is realised operationally by freezing the pivot's ambiguity, ionosphere, and sat-bias covariances to exactly zero (and suppressing their process noise), the very condition `isPivotDisturbed` tests against.

`PivotManager::getCleanHandovers` classifies each transition as lossless-invertible (*clean*) or discontinuous (disturbed/forced/re-seeded), so the RTS smoother can inverse-reference across a clean handover rather than force a segment boundary; the losslessness claim above holds only for the clean subset. These are two distinct code paths sharing the algebra above: in PPP-RTK the pivot re-election and the "sticky-pivot" retention policy live in `filter/PivotManager`, whereas the baseline DD path performs the analogous re-reference through the ambiguity-rename S-transform in `filter/DdPivotManager`, governed by the `baseline.enable_pivot_handover` flag.

5.2 Between-receiver single difference (BRSD)

In baseline modes there is at least one rover r_{rov} and exactly one fixed-coordinate reference r_{ref}. The BRSD operator forms

$$\nabla P^{i}_{r_{\text{rov}}r_{\text{ref}},s} \;:=\; P^{i}_{r_{\text{rov}},s} - P^{i}_{r_{\text{ref}},s}, \tag{12}$$

and analogously for $\nabla\Phi$. The cancellations produced by $\nabla\cdot$ are dual to those produced by $\Delta\cdot$: BRSD cancels *satellite*-side common terms ($c\,\mathrm{d}t^{i}$, B_s^i, b_s^i) but preserves receiver clock and biases of *both* stations. No state-space estimation of satellite biases is required when ∇ is applied, since they disappear before reaching the filter.

5.3 Double difference (DD)

The composition $\nabla\Delta\cdot = \Delta\big(\nabla\,\cdot\big)$ is order-independent up to noise correlation: $\Delta\big(\nabla\,\cdot\big) = \nabla\big(\Delta\,\cdot\big)$. Applying it to the carrier-phase model and collecting common-mode cancellations:

$$\nabla\Delta\Phi^{i,p_g}_{r_{\text{rov}}r_{\text{ref}},s} \;=\; \nabla\Delta\rho^{i,p_g}_{r_{\text{rov}}r_{\text{ref}}} + \nabla\Delta T^{i,p_g}_{r_{\text{rov}}r_{\text{ref}}} - \mu_s\,\nabla\Delta I^{i,p_g}_{r_{\text{rov}}r_{\text{ref}},1} + \lambda_s\,\nabla\Delta N^{i,p_g}_{r_{\text{rov}}r_{\text{ref}},s} + \nabla\Delta\varepsilon^{\Phi}. \tag{13}$$

What survives in Equation (13) is precisely the geometry term (linear in the rover position perturbation), the differential atmospheric delays, and the integer DD ambiguity: the structural form that makes carrier-phase RTK possible.

Short vs. long baselines. On short baselines ($\lesssim 10\,\mathrm{km}$) the differential atmospheric terms in Equation (13) are negligible at the millimetre level and need not be modelled. As baseline length grows, the differential ionosphere $\nabla\Delta I$ scales linearly with separation and saturates at typically 1 m

for separations of a few hundred kilometres, swamping the carrier wavelength. Two mitigations are available in `GnssProcessor`:

- `baseline.use_iono_free` forms the dual-frequency ionosphere-free combination at the DD level, eliminating the first-order ionospheric term at the cost of a non-integer ambiguity (AR is automatically excluded for IF rows) and $\sim 3\times$ amplified code noise;
- `baseline.estimate_differential_tropo` adds a single $\Delta\mathrm{ZWD}_{r_{\mathrm{rov}} r_{\mathrm{ref}}}$ state with slow random-walk dynamics, capturing the residual differential wet delay that survives the BRSD on baselines with significant altitude difference.

5.4 S-transform structure

Both BSSD and DD admit a matrix-form representation as $\boldsymbol{x}_{\mathrm{diff}} = \boldsymbol{S}\,\boldsymbol{x}_{\mathrm{undiff}}$ with $\boldsymbol{S}$ acting as a block design transform. For BSSD restricted to the ambiguity block of one constellation g on one signal s, with $|\mathcal{S}_g|$ tracked sats, $\boldsymbol{S}$ is a $(|\mathcal{S}_g|-1)\times|\mathcal{S}_g|$ matrix of the form

$$\boldsymbol{S}_{\mathrm{BSSD}} = \begin{bmatrix} 1 & 0 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & -1 & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots & & & \\ 0 & 0 & \cdots & 0 & -1 & 0 & \cdots & 1 \end{bmatrix}, \tag{14}$$

where the column of -1's sits at the index of the pivot p_g. The DD operator is the *composition* of the two single-difference operators applied in sequence, $\boldsymbol{S}_{\mathrm{DD}} = \boldsymbol{S}_{\mathrm{BSSD}}\,\boldsymbol{S}_{\mathrm{BRSD}}$, realised in code as `BrsdTransform::apply` followed by `BssdTransform::differenceBrsd` (`include/measurement/BssdTransform.hpp`). Only on a fully populated satellite $\times$ receiver observation grid does this composition coincide with the Kronecker product $\mathbf{D}_{\mathrm{sat}} \otimes \mathbf{D}_{\mathrm{rx}}$ of the between-satellite and between-receiver differencing matrices acting on the vectorised observations, a useful mental model but not what the non-rectangular real-data case computes. The transformed covariance is $\boldsymbol{P}_{\mathrm{diff}} = \boldsymbol{S}\,\boldsymbol{P}_{\mathrm{undiff}}\,\boldsymbol{S}^{\mathrm{T}}$, a generally dense matrix even when $\boldsymbol{P}_{\mathrm{undiff}}$ is diagonal (because each differenced row borrows from the same pivot column). This dense correlation structure is the algebraic reason DD-based filters cannot be naively implemented as independent per-pair scalar updates; the correctly correlated joint update is what unlocks integer ambiguity recovery via the LAMBDA decorrelation. The general theory and a thorough catalogue of admissible S-transforms is given in Teunissen [25] and in the rank-defect treatment of Jonge [10].

Implementation notes. Two facts about how `GnssProcessor` realises the transforms above are worth stating explicitly. First, the chosen S-system is never materialised as an explicit minimum-constraint $\boldsymbol{S}$-basis matrix; it is realised *implicitly* through the pivot-differencing operator (`measurement/BssdTransform`), a faithful, S-basis-invariant realisation of the uncombined parameterisation of Odijk [17]. The three regimes (undifferenced, BSSD, and DD) are dispatched polymorphically by a `DifferencingMode` strategy in `measurement/MeasurementBuilder`, not by inline config branches. Second, integer recovery on a DD row requires that all four contributing observations share the same RINEX signal code. The row is marked `integer_capable` only when all three conjuncts hold: `!brsd_aliased` on *both* the i_i and pivot BRSD entries (rover and reference agree on each, i.e. no cross-attribute alias) *and* the rover uses the same code on i_i as on the pivot ($\texttt{rov_code}_i = \texttt{rov_code}_{\mathrm{piv}}$). The third conjunct is independent of the `brsd_aliased` flags: a row can be un-aliased on both entries yet still carry a non-integer DCB/IFB residual if the rover mixes codes across i_i and the pivot. Rows failing any conjunct are carried as float and excluded from ambiguity resolution.

Part II
Reference systems and measurement models

6 Coordinate Frames and Time Systems

Every model in this reference (the tropospheric and ionospheric mappings of Section 9, the solid-Earth, ocean, and pole tides of Section 10, the antenna, attitude, and phase-wind-up terms of Section 11, and the orbit evaluation of Section 8) assumes a fixed set of coordinate frames and time scales. This document specifies them in three parts. The first is the WGS84 ellipsoid and the three coordinate representations the engine uses: Cartesian ECEF, geodetic, and local ENU. The second is the GNSS time scales and the leap-second handling that ties them to UTC. The third is the Earth-orientation chain that rotates the celestial frame, in which the Sun, Moon, and satellite ephemerides are expressed, into the terrestrial frame of the receiver. None of these conversions is an estimator state. The one estimated frame quantity is the inter-system time offset (ISB), which absorbs the unmodelled bias between constellation time scales and belongs to Section 3. This document covers only the *deterministic* conversions that the ISB is defined relative to. Code references are to `src/core/math/Coordinates.cpp`, `src/core/math/TimeUtils.cpp`, `src/core/model/Planetary.cpp`, `src/core/products/LiveEphemerisStore.cpp`, `src/core/products/ProductParser.cpp`, and `src/core/rinex/RinexParser.cpp`.

6.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames

The terrestrial reference frame is the WGS84 Earth-Centred, Earth-Fixed (ECEF) Cartesian system: origin at the geocentre, z along the conventional pole, x toward the IERS reference meridian, y completing a right-handed triad. The defining ellipsoid has semi-major axis a and flattening f, from which the first eccentricity squared follows (`Coordinates::ecefToLlh` and `Coordinates::llhToEcef` each derive it from f):

$$e^2 \;=\; f\,(2 - f). \tag{15}$$

The radius of curvature in the prime vertical at geodetic latitude φ is

$$N(\varphi) \;=\; \frac{a}{\sqrt{1 - e^2\,\sin^2\varphi}}, \tag{16}$$

which scales the entire forward map from geodetic coordinates (φ, λ, h) to ECEF (`Coordinates::llhToEcef`):

$$\boxed{\begin{aligned} x &= \big(N + h\big)\cos\varphi\,\cos\lambda, \\ y &= \big(N + h\big)\cos\varphi\,\sin\lambda, \\ z &= \big((1 - e^2)\,N + h\big)\sin\varphi. \end{aligned}} \tag{17}$$

The factor $(1 - e^2)$ on the z component is what makes the surface an ellipsoid. On a sphere every axis would carry N unchanged. The factor is also why the geodetic normal, which is perpendicular to the ellipsoid, differs from the geocentric radius vector.

Inverse map (Bowring fixed point). The inverse problem, recovering (φ, λ, h) from (x, y, z), has no closed form for the latitude because N in (17) itself depends on φ. The longitude is immediate,

$\lambda = \operatorname{atan2}(y, x)$. For the latitude the engine runs a Bowring fixed-point iteration on the equatorial projection $p = \sqrt{x^2 + y^2}$ (`Coordinates::ecefToLlh`). The seed neglects height,

$$\varphi^{(0)} = \operatorname{atan2}\big(z,\ p\,(1 - e^2)\big), \tag{18}$$

and each pass recomputes N and the height and re-evaluates the latitude with the height feedback folded back into the eccentricity term:

$$h^{(k)} = \frac{p}{\cos\varphi^{(k)}} - N^{(k)}, \qquad \varphi^{(k+1)} = \operatorname{atan2}\left(z,\ p\left[1 - e^2\,\frac{N^{(k)}}{N^{(k)} + h^{(k)}}\right]\right). \tag{19}$$

The iteration is capped at ten passes and exits early once $|\varphi^{(k+1)} - \varphi^{(k)}| < 10^{-13}$; for terrestrial heights it converges to machine precision in two or three steps.

Pole branch. When $p < 10^{-12}\,\mathrm{m}$ the point lies on (or within a picometre of) the z axis and both the longitude and the $p/\cos\varphi$ height formula degenerate. The code takes an explicit branch (`Coordinates::ecefToLlh`, the $p < 10^{-12}$ guard): the longitude retains the $\operatorname{atan2}(y, x)$ value computed before the guard, formally undefined on the axis and immaterial there, while the branch fixes $\varphi = \pm\pi/2$ by the sign of z and reads the height directly off the polar semi-minor axis,

$$h = |z| - N\,(1 - e^2) = |z| - a\sqrt{1 - e^2} \quad (\text{at the pole},\ \sin^2\varphi = 1), \tag{20}$$

since $N(1 - e^2)$ evaluated at the pole is exactly the polar radius $a\sqrt{1 - e^2}$. This guard keeps the inverse map total over all of $\mathbb{R}^3$, which matters because the planetary and attitude code occasionally evaluates geometry for satellite positions that transit near the polar axis.

Local East–North–Up frame. Many of the downstream models (the tropospheric mapping function, the elevation mask, the antenna phase-centre projection) are most natural in the topocentric ENU frame anchored at the receiver's geodetic (φ, λ). The columns of the ENU→ECEF rotation are the three local unit vectors expressed in ECEF (`Coordinates::enuToEcefMatrix`):

$$\boxed{\boldsymbol{R}_{\mathrm{ENU}}(\varphi, \lambda) = \begin{bmatrix} \hat{\boldsymbol{e}}_{\mathrm{E}} & \hat{\boldsymbol{e}}_{\mathrm{N}} & \hat{\boldsymbol{e}}_{\mathrm{U}} \end{bmatrix} = \begin{bmatrix} -\sin\lambda & -\sin\varphi\cos\lambda & \cos\varphi\cos\lambda \\ \cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi\sin\lambda \\ 0 & \cos\varphi & \sin\varphi \end{bmatrix}.} \tag{21}$$

The Up column is the outward geodetic normal, identical to the direction along which h is measured in (17); East is its horizontal cross-product with the pole, and North completes the right-handed triad. Because the columns are orthonormal, $\boldsymbol{R}_{\mathrm{ENU}}$ is a proper rotation and its inverse is its transpose: an ECEF displacement is resolved into local components by $\boldsymbol{v}_{\mathrm{ENU}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\,\boldsymbol{v}_{\mathrm{ECEF}}$ (`Coordinates::ecefToEnu`, with `Coordinates::enuToEcef` the forward direction), avoiding any explicit matrix inversion.

Elevation and azimuth. With the receiver-to-satellite line-of-sight resolved into the local frame, $\boldsymbol{e}_{\mathrm{ENU}} = (e_{\mathrm{E}}, e_{\mathrm{N}}, e_{\mathrm{U}})^{\mathrm{T}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\boldsymbol{e}$, the elevation and azimuth that gate the measurement (the elevation mask of Section 4.3 and the mapping-function argument of Section 9) follow directly:

$$\mathrm{el} = \arcsin\big(e_{\mathrm{U}}\big), \qquad \mathrm{az} = \operatorname{atan2}\big(e_{\mathrm{E}},\ e_{\mathrm{N}}\big), \tag{22}$$

with azimuth measured clockwise from North. Equations (21)–(22) connect the ECEF state vector to every elevation-dependent weighting and antenna model in this reference.

6.2 Station reference points: marker, ARP, and phase centre

A station coordinate can refer to any of three points. The *marker* (monument) is the surveyed benchmark, and is what an IGS SINEX/SSC solution, an NGS Station Information File, and the RINEX `APPROX POSITION XYZ` header record all refer to. The *antenna reference point* (ARP) sits where the antenna meets its mount and is the origin against which the ANTEX phase-centre offsets of Section 11 are tabulated. The *antenna phase centre* is where the measurement is physically realised. The estimator's position state is the ARP, because the receiver PCO/PCV it applies are referenced to the ARP. The antenna model covers the step from ARP to phase centre, so only the step from marker to ARP belongs to this document. Treating a marker coordinate as an ARP coordinate, or the reverse, puts a constant bias equal to the eccentricity into the solution, mostly in height.

The leg itself is the RINEX `ANTENNA:DELTA H/E/N` eccentricity $\boldsymbol{e}_{\mathrm{HEN}} = (\delta H,\, \delta E,\, \delta N)^{\mathrm{T}}$, defined as the offset from the marker to the ARP in the topocentric frame *at the marker*. RINEX writes the triple as $(H, E, N) = (\mathrm{Up}, \mathrm{East}, \mathrm{North})$. The rotation of (21) expects (E, N, U), so the triple must be reordered before it is rotated (`Coordinates::markerToArp`, the single owner of this convention):

$$\boxed{\boldsymbol{r}_{\mathrm{ARP}} \;=\; \boldsymbol{r}_{\mathrm{mkr}} \;+\; \boldsymbol{R}_{\mathrm{ENU}}\big(\varphi_{\mathrm{mkr}},\, \lambda_{\mathrm{mkr}}\big) \begin{bmatrix} \delta E \\ \delta N \\ \delta H \end{bmatrix}}, \tag{23}$$

with $(\varphi_{\mathrm{mkr}}, \lambda_{\mathrm{mkr}})$ recovered from $\boldsymbol{r}_{\mathrm{mkr}}$ by the inverse map of (19). Write $\boldsymbol{e}'_{\mathrm{HEN}}$ for the permuted $(\delta E, \delta N, \delta H)^{\mathrm{T}}$ that the rotation acts on.

The inverse is not a sign flip. Reducing an ARP estimate to the monument is not the negative of (23). The local frame is defined at the marker, and the marker is the unknown. Evaluating $\boldsymbol{R}_{\mathrm{ENU}}$ at the ARP instead leaves an error of order $\|\boldsymbol{e}_{\mathrm{HEN}}\|^2 / R_{\oplus}$. The engine therefore takes one fixed-point step (`Coordinates::arpToMarker`):

$$\boldsymbol{r}^{(0)} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}_{\mathrm{ARP}})\big)\, \boldsymbol{e}'_{\mathrm{HEN}}, \qquad \boldsymbol{r}_{\mathrm{mkr}} = \boldsymbol{r}_{\mathrm{ARP}} - \boldsymbol{R}_{\mathrm{ENU}}\big(\mathrm{llh}(\boldsymbol{r}^{(0)})\big)\, \boldsymbol{e}'_{\mathrm{HEN}}, \tag{24}$$

which closes the round trip with (23) to well below the millimetre level this reference grades at.

Where the reduction is applied, and where it is not. Every position derived from a RINEX *header* is reduced: the post-processing filter seed (`Application::initializePosition` and the `PppModel` state initialiser), the real-time seed (`RealTimeEngine::initialize`), the baseline rover a priori, and the closed-loop consumer truth all take the marker through (23), some calling `Coordinates::markerToArp` directly, others through the accessor `RinexParser::getArpPositionXyz` that pairs the two header fields. A coordinate supplied *directly in the configuration* is not reduced: it is taken at face value, on the stated policy that the user owns its reference point. So the baseline *reference* coordinate is used as given while the rover a priori read from its header is reduced, an asymmetry inside one mode, and, more consequentially, the network producer's station coordinates (`StationConfig::position_ecef`, optionally SIF-resolved under `network.use_sif_coordinates`) are used as given while being held *fixed*, their three position columns dropped from the normal equations. A monument-referenced coordinate there is therefore not absorbed by a position state; it survives into the residuals and hence into the estimated clocks, biases, and ionosphere. A zero or absent eccentricity makes (23) an exact, bitwise no-op, which is what keeps header-less RTCM/SBF streams and `0/0/0` stations untouched; the ordering, sign, no-op, and round-trip properties are pinned by the `AntennaEccentricityTest` suite.

6.3 GNSS time scales and the GPS reference epoch

Time inside the engine is carried as a `GnssTime` {week, tow} pair, an integer GPS week and a floating-point time-of-week in seconds, and is expanded to a single continuous count $T = \text{week} \times 604800 + \text{tow}$ wherever a monotone scalar is needed (e.g. ERP interpolation, Section 6.9). A week boundary is 604 800 s and a day 86 400 s (`src/core/math/TimeUtils.cpp`, header constants `SECONDS_IN_WEEK`, `SECONDS_IN_DAY`). The common origin is the GPS epoch, 1980-01-06 00:00:00 UTC, whose Julian Date is fixed in the code:

$$\mathrm{JD}_{\mathrm{GPS}} = 2444244.5 \qquad (\texttt{src/core/math/TimeUtils.cpp}, \texttt{GPS_EPOCH_JD}). \tag{25}$$

GPS time is a continuous atomic scale, since it never inserts a leap second, so it sits a fixed integer offset below International Atomic Time and a fixed fractional offset below Terrestrial Time (`src/core/math/TimeUtils.cpp`, header constants `TAI_GPS_OFFSET`, `TT_GPS_OFFSET`):

$$\boxed{\mathrm{TAI} - \mathrm{GPS} = 19\ \mathrm{s}, \qquad \mathrm{TT} - \mathrm{GPS} = 51.184\ \mathrm{s} = (32.184 + 19)\ \mathrm{s}.} \tag{26}$$

The 32.184 s in the TT relation is the constant TT−TAI offset; together these let any GPS time be promoted to the TT scale on which precession and nutation are evaluated. Terrestrial Time relative to the J2000.0 epoch is measured in Julian centuries,

$$t = \frac{\mathrm{JD}_{\mathrm{TT}} - 2451545.0}{36525.0}, \tag{27}$$

the dimensionless argument of the astronomical series of Sections 6.7 and 6.10 (`TimeUtils::jdToJ2000Centuries`, constants `J2000_JD`, `DAYS_PER_JULIAN_CENTURY`).

Gregorian↔Julian Date. The calendar conversions follow the Meeus algorithm [14]. The forward map, with the month shifted so January and February count as months 13 and 14 of the prior year and a Gregorian-reform correction $b = 2 - A + \lfloor A/4 \rfloor$ for $A = \lfloor y/100 \rfloor$, is (`TimeUtils::gregorianToJulianDay`):

$$\mathrm{JD} = \lfloor 365.25\,(y + 4716) \rfloor + \lfloor 30.6001\,(m + 1) \rfloor + d + b - 1524.5 + \frac{(\text{time of day})}{86400}, \tag{28}$$

and the inverse (`TimeUtils::julianDayToGregorian`) reverses it through the integer auxiliaries $z = \lfloor \mathrm{JD}{+}0.5 \rfloor$ and the Gregorian-reform branch at $z \geq 2299161$. The code keeps two kinds of calendar date apart. A date already on the GPS scale needs no leap correction (`gregorianGpsToGpsTime`). A date on UTC must have the leap seconds at that epoch added to reach GPS time (`TimeUtils::gregorianToGpsTime`). Confusing the two produces whole-second timing errors.

6.4 Leap seconds and per-system dispatch

UTC is tied to GPS by the historical table of leap-second insertions. The engine stores the table as $\big(\mathrm{JD}_k,\ \Delta_{\mathrm{GU}}(k)\big)$ pairs, the GPS−UTC offset that takes effect at each insertion boundary, the first row being the GPS epoch itself with offset zero (`src/core/math/TimeUtils.cpp`, `LEAP_TABLE`). A query resolves the offset by a reverse scan that returns the offset of the latest boundary not after the epoch:

$$\Delta_{\mathrm{GU}}(\mathrm{JD}) = \Delta_{\mathrm{GU}}(k^{\star}), \qquad k^{\star} = \max\{\, k : \mathrm{JD}_k \leq \mathrm{JD} \,\}, \tag{29}$$

falling back to zero before the epoch (`TimeUtils::leapSecondsGpsUtc`). TAI−UTC is then simply $\Delta_{\mathrm{GU}} + 19$ s from (26) (`TimeUtils::leapSecondsTaiUtc`). The lookup must depend on the epoch. A leap second was inserted between 2005 and 2010, so the same UTC calendar instant maps to different GPS times in those two years.

The wall-clock leap constant and its runtime override. The boundary-aware table of (29) is the authority for every *epoch-referenced* conversion (a dated calendar timestamp, GLONASS day arithmetic, UTC formatting) because each of those carries the epoch at which the correct offset is known. The one conversion that does not is the promotion of the *system wall clock* to GPS time (`TimeUtils::nowContinuousGpsSeconds`, `nowGpsTime`), used only on the real-time decode and latency-stamping paths: POSIX `system_clock` is UTC and carries no epoch of its own, so the offset cannot be looked up and must instead be supplied as a fixed count. That count is the compiled constant `TimeUtils::LEAP_SECONDS` = 18 s (valid from 2017-01-01), read through an atomic accessor (`currentLeapSeconds`) so that a future insertion can be applied to a live binary without a rebuild: `setLeapSecondsOverride`, driven by the config key `realtime.leap_seconds` (`src/core/config/GnssConfigLoad.cpp`), replaces it at real-time startup. The override deliberately touches *only* the wall-clock path; the epoch-dependent (29) lookups are left alone, since they are correct at every historical epoch outside the insertion instants themselves[1] and an override there would corrupt rather than update them.

Per-constellation Gregorian dispatch. The four supported time systems differ only in how a RINEX/broadcast calendar timestamp is referred to the GPS scale, dispatched by a system tag (`TimeUtils::GnssTimeFromDate`):

- **GPS / Galileo**: already on the GPS scale; the timestamp is converted with no leap correction (GST is steered to GPS time and shares the same week/second framing to within the few nanoseconds the ISB state of Section 3 absorbs).
- **BeiDou**: the timestamp is on BDT, which runs 14 s behind GPS (Section 6.6); the code converts on the GPS-scale assumption and then *adds* 14 s, carrying any week roll-over (`TimeUtils::GnssTimeFromDate`, BeiDou branch).
- **GLONASS**: the RINEX timestamp is UTC; the full leap correction of (29) is applied to bring it onto GPS time (`TimeUtils::GnssTimeFromDate`, GLONASS branch).

GPS and Galileo therefore carry no leap correction, BeiDou a fixed 14 s, and GLONASS the full epoch-dependent leap count. Each matches the time reference that defines the constellation. There is a fourth branch: an unrecognised time-system tag falls through to the GPS-scale conversion with only a trace-level note, and a calendar date that fails validation returns the sentinel $\{0, 0\}$ rather than a fabricated epoch (`TimeUtils::GnssTimeFromDate`). The fallback logs nothing at default verbosity, so an unexpected tag produces a timing error of the full leap count with no warning.

6.5 GLONASS time-of-day to full GPS week

GLONASS broadcast and SSR records frequently carry only a time-of-day, seconds since the start of the GLONASS day, with no week and no absolute date. GLONASS time runs on UTC(SU) advanced by three hours (Moscow), so its day boundary is three hours ahead of the UTC midnight. Reconstructing a full GPS week from a bare time-of-day therefore needs an *anchor*: an approximate GPS-seconds estimate of the true epoch (typically the current obs epoch), against which the day and the ± 12 h wrap are resolved (`TimeUtils::glonassTodToGpsTime`, after RTKLIB's `adjday_glot`). The procedure, with day length $D = 86\,400$ s, half-day $D/2$, and Moscow offset $M = 3 \times 3600$ s:

[1] The comparison in `TimeUtils::leapSecondsGpsUtc` is made on the GPS scale against boundary Julian Dates tabulated at the *UTC* insertion instant, so the offset advances Δ_{GU} seconds early, a known, deferred discrepancy confined to a Δ_{GU}-second window at each boundary (18 s at the most recent insertion, 2017-01-01).

1. Convert the anchor GPS seconds to UTC by subtracting the leap count at that epoch, $t_{\mathrm{UTC}} = t_{\mathrm{anchor}} - \Delta_{\mathrm{GU}}$, then shift into the Moscow-day frame $t_{\mathrm{M}} = t_{\mathrm{UTC}} + M$ and locate that day's midnight $t_0 = \lfloor t_{\mathrm{M}}/D \rfloor\, D$, giving the anchor's time-of-day $\tau_{\mathrm{a}} = t_{\mathrm{M}} - t_0$.

2. Place the reported time-of-day τ in the anchor's day, resolving the midnight wrap toward the nearest match:

$$\boxed{\tau \leftarrow \begin{cases} \tau + D, & \tau < \tau_{\mathrm{a}} - D/2, \\ \tau - D, & \tau > \tau_{\mathrm{a}} + D/2, \\ \tau, & \text{otherwise}, \end{cases} \qquad t_{\mathrm{GPS}} = \underbrace{(t_0 + \tau)}_{\text{Moscow}} - M + \Delta_{\mathrm{GU}}.} \tag{30}$$

The last expression removes the Moscow offset to recover UTC and then adds the same leap count Δ_{GU} to return to the GPS scale. The leap term enters once with each sign and cancels. It appears only because the Moscow day must be located on the UTC scale. If no usable anchor is supplied (a non-positive value) the routine returns the sentinel $\{0, 0\}$ rather than fabricate a week, so the caller can detect the failure and fall back; the continuous-seconds-to-week split that closes the routine carries the standard negative-TOW correction (`gpsSecondsToGnssTime`).

6.6 BeiDou time: the 14 s offset and independent week base

BeiDou time (BDT) differs from GPS time by a fixed number of seconds and by an independent week numbering. Different modules handle the two parts. The *seconds* relation,

$$\mathrm{BDT} = \mathrm{GPS} - 14\ \mathrm{s}, \tag{31}$$

reflects that BDT was synchronised to UTC at its 2006 zero epoch, by which time GPS already led UTC by 14 s; it is applied in `TimeUtils` when a BeiDou calendar timestamp is converted to GPS time (`TimeUtils::GnssTimeFromDate`, BeiDou branch; Section 6.4). The *week* relation is separate. BDT counts weeks from its own zero epoch (2006-01-01), which is GPS week 1356, and this base is applied at query time by the ephemeris store, not by `TimeUtils` (the file-local constant `kBdtToGpsOffsetSec`, consumed by `LiveEphemerisStore::computeSatellitePV`). When the store lifts a BeiDou ephemeris reference time onto the common GPS scale for an honest cross-system age check it adds the combined offset

$$\boxed{\Delta_{\mathrm{BDT}\to\mathrm{GPS}} = 1356 \times 604800\ \mathrm{s} + 14\ \mathrm{s},} \tag{32}$$

the week base plus the 14 s of (31) (`kBdtToGpsOffsetSec`, applied by the file-local `toeGpsContinuous`). Without the week base every BeiDou ephemeris would appear about 1356 weeks ($\sim$26 years) old, and the staleness guard in `LiveEphemerisStore::computeSatellitePV` would reject it without a message. The offset does not cancel, because the query time is on the GPS scale while the stored t_{oe} uses the BDT week.

6.7 Legacy ECI→ECEF: the equinox-based chain

The Sun, the Moon, and for some product streams the satellite positions are given in a quasi-inertial celestial frame. The tides, attitude, and phase wind-up that use them are computed in the terrestrial ECEF frame. The Earth-orientation matrix $\boldsymbol{U}$ connects the two: $\boldsymbol{r}_{\mathrm{ECEF}} = \boldsymbol{U}\, \boldsymbol{r}_{\mathrm{ECI}}$. The default (always-available) path is the classical equinox-based chain following the IAU-1976 precession and IAU-1980 nutation conventions [18], composed as (`Planetary::eci2ecef`):

$$\boxed{\boldsymbol{U} = \boldsymbol{W}_{\mathrm{pm}}\, \boldsymbol{R}_3(\mathrm{GAST})\, \boldsymbol{N}_{\mathrm{nut}}\, \boldsymbol{P}_{\mathrm{prec}}.} \tag{33}$$

Here $\boldsymbol{R}_1, \boldsymbol{R}_2, \boldsymbol{R}_3$ are the elementary right-handed axis rotations of `Planetary::Rxyz` and each factor reverses one piece of the celestial-to-terrestrial orientation: precession and nutation orient the true equator and equinox of date, the sidereal rotation $\boldsymbol{R}_3(\mathrm{GAST})$ spins the frame about the pole by the apparent sidereal time, and polar motion $\boldsymbol{W}_{\mathrm{pm}}$ ties the celestial pole to the conventional terrestrial pole.

Precession (IAU 1976). With t the TT centuries of (27) and arcsecond coefficients converted to radians, the three equatorial precession angles ζ_A, θ_A, z_A and the mean obliquity ε_0 are (the precession block of `Planetary::eci2ecef`):

$$\begin{aligned} \zeta_A &= 2306.2181\,t + 0.30188\,t^2 + 0.017998\,t^3, \\ \theta_A &= 2004.3109\,t - 0.42665\,t^2 - 0.041833\,t^3, \\ z_A &= 2306.2181\,t + 1.09468\,t^2 + 0.018203\,t^3, \\ \varepsilon_0 &= 84381.448 - 46.8150\,t - 0.00059\,t^2 + 0.001813\,t^3 \quad (\text{arcsec}), \end{aligned} \tag{34}$$

assembled as $\boldsymbol{P}_{\mathrm{prec}} = \boldsymbol{R}_3(-z_A)\,\boldsymbol{R}_2(\theta_A)\,\boldsymbol{R}_3(-\zeta_A)$.

Nutation (IAU 1980, 106 terms). The nutation in longitude $\Delta\psi$ and in obliquity $\Delta\varepsilon$ are summed over the 106-term IAU-1980 series [18] in the five Delaunay fundamental arguments $f = (l, l', F, D, \Omega)$ (`Planetary::nutIau1980`; arguments from `Planetary::astArgs`):

$$\Delta\psi = 10^{-4}\sum_{i=1}^{106}\left(A_i^{\psi} + \dot{A}_i^{\psi}\,t\right)\sin\left(\boldsymbol{n}_i\cdot f\right), \quad \Delta\varepsilon = 10^{-4}\sum_{i=1}^{106}\left(A_i^{\varepsilon} + \dot{A}_i^{\varepsilon}\,t\right)\cos\left(\boldsymbol{n}_i\cdot f\right), \tag{35}$$

in arcseconds (the 10^{-4} scales the tabulated 0.1 mas units), then converted to radians. The nutation matrix rotates by the true obliquity $\varepsilon_0 + \Delta\varepsilon$: $\boldsymbol{N}_{\mathrm{nut}} = \boldsymbol{R}_1(-\varepsilon_0 - \Delta\varepsilon)\,\boldsymbol{R}_3(-\Delta\psi)\,\boldsymbol{R}_1(\varepsilon_0)$ (the nutation block of `Planetary::eci2ecef`).

Apparent sidereal time and the 0h-UT1 subtlety. Greenwich Apparent Sidereal Time adds the equation of the equinoxes, the nutation in longitude projected on the equator, plus the two leading complementary terms in the lunar node Ω, to the mean sidereal time (the GAST block of `Planetary::eci2ecef`):

$$\mathrm{GAST} = \mathrm{GMST} + \Delta\psi\cos\varepsilon_0 + \left(0.00264\sin\Omega + 0.000063\sin 2\Omega\right)\ \text{arcsec}. \tag{36}$$

GMST itself is built from the IAU-1982 model evaluated at the *start* of the current UT1 day plus the Earth's rotation through the elapsed day fraction (`Planetary::utc2gmst`). The reference instant must be 0h UT1 of the calendar day, and because a Julian Date is referenced to *noon*, that midnight is recovered as

$$\mathrm{JD}_{0\mathrm{h}} = \left\lfloor \mathrm{JD}_{\mathrm{UT1}} - 0.5 \right\rfloor + 0.5, \tag{37}$$

not $\lfloor \mathrm{JD} \rfloor$ (same routine). The bare floor gives the *previous* noon for any epoch after midnight. That half-day error is roughly 0.5° in GMST and, through (33), of order 10^6 km in the rotated Sun position. A comment in `Planetary::utc2gmst` records this failure.

Polar motion. The terrestrial-pole tie is $\boldsymbol{W}_{\mathrm{pm}} = \boldsymbol{R}_2(-x_p)\,\boldsymbol{R}_1(-y_p)$ for the interpolated pole coordinates (x_p, y_p) in radians (the polar-motion block of `Planetary::eci2ecef`), the small-angle two-axis rotation that aligns the celestial intermediate pole of date with the IERS conventional pole.

6.8 CIO-based ITRS↔GCRS (IAU 2000A/2006)

An opt-in path (config key `estimation.enable_cio_frame`, getter `GnssConfig::enableCioFrame`; **default off**, so the equinox chain of (33) is the shipped path) replaces the equinox chain with the modern Celestial Intermediate Origin (CIO) based transform of the IAU 2000A/2006 conventions [18], evaluated through SOFA [8] and mirroring the M-PAGES `NgsFormC2T` construction (`Planetary::celestialToTerrestrial`). The celestial-to-terrestrial matrix is composed as

$$\boxed{\boldsymbol{U} = \mathtt{POM}(x_p, y_p, s')\ \boldsymbol{R}_3(\theta_{\mathrm{ERA}})\ \mathtt{C2I}(X, Y, s),} \tag{38}$$

i.e. the SOFA composition `iauC2tcio`. The Celestial Intermediate Pole coordinates (X, Y) and the CIO locator s come from the full IAU-2000A precession-nutation model via `iauXys00a`, after which the *observed* celestial-pole offsets are injected additively,

$$X \leftarrow X + \mathrm{d}X, \qquad Y \leftarrow Y + \mathrm{d}Y, \tag{39}$$

and the celestial-to-intermediate matrix is formed by `iauC2ixys`. The diurnal rotation is the Earth Rotation Angle θ_{ERA} (a strictly linear function of UT1, `iauEra00`) rather than the sidereal time of (36), and the terrestrial-pole tie `POM` uses `iauPom00` with the TIO locator s' from `iauSp00`. Two-part Julian Dates are passed with the $(2400000.5, \mathrm{MJD})$ split for numerical conditioning (same routine). The two paths should agree. With $\mathrm{d}X = \mathrm{d}Y = 0$ the CIO result is the bare IAU-2000A model. It differs from the legacy equinox chain only by the frame bias and the model version, which is below a milliarcsecond. The engine checks this at startup by logging the angular separation of the two Sun directions (the startup diagnostic in `Planetary::computeSunMoonPosition`).

6.9 Earth-orientation parameters: parse and interpolation

Both transforms above need the time-varying Earth-orientation parameters $(x_p, y_p, \mathrm{UT1} - \mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y)$. These arrive in a standard IGS `.ERP` file, parsed by `ProductParser::parseErp` (`src/core/products/ProductParser.cpp`). The parser reads the MJD time tag and the pole/UT1 columns and applies the IGS scale factors: the pole coordinates are tabulated in 10^{-6} arcseconds and UT1−UTC in 10^{-7} seconds (`ProductParser::parseErp`):

$$x_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad y_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad (\mathrm{UT1} - \mathrm{UTC}) = (\mathrm{col}) \times 10^{-7}\,\mathrm{s}. \tag{40}$$

The MJD tag is converted to a GPS week/TOW relative to the GPS-epoch MJD (same routine). The CIP offsets $\mathrm{d}X, \mathrm{d}Y$ are *not* carried by the standard `.ERP` format and default to zero; because the IAU-2000A model of (38) already includes the modelled precession-nutation, $\mathrm{d}X = \mathrm{d}Y = 0$ is the correct nominal value, not a missing input (same routine). The parser does no interpolation. `Planetary::interpolateErp` binary-searches the chronologically sorted table on continuous GPS seconds and linearly interpolates every component between the bracketing entries:

$$p(t) = p_0 + \frac{t - t_0}{t_1 - t_0}\,(p_1 - p_0), \qquad p \in \{x_p, y_p, \mathrm{UT1}{-}\mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y\}, \tag{41}$$

clamping to the nearest endpoint outside the table span (same routine). The interpolated (x_p, y_p) feed the polar-motion factor of (33) / (38), while UT1−UTC corrects UTC to UT1 before the sidereal/ERA evaluation. The product-evaluation document (Section 8) cross-references this section rather than re-deriving the polar-motion application, since the same $\boldsymbol{W}_{\mathrm{pm}}$ rotates both the luni-solar vectors and any celestial-frame satellite product.

6.10 The Sun and Moon series these transforms position

The orientation chain exists to place the Sun and Moon in ECEF for the geophysical and observation-model corrections. The celestial-frame positions are computed first, from an analytic series in the J2000 centuries t and the Delaunay arguments f (`Planetary::sunMoonPosEci`), an enhanced RTKLIB/Meeus model [14, 15]. The Sun's ecliptic longitude is the mean longitude plus the equation of centre (three harmonics of the solar mean anomaly $M_\odot$) and small Venus, Jupiter, and lunar perturbation terms (same routine); the obliquity rotation

$$\varepsilon = 23.439291 - 0.0130042\,t \quad \text{(deg)} \tag{42}$$

carries the resulting ecliptic $(\lambda_\odot, \beta_\odot)$ into equatorial ECI components:

$$\begin{aligned} r_x &= r\,\cos\beta_\odot \cos\lambda_\odot, \\ r_y &= r\,(\cos\varepsilon\,\cos\beta_\odot \sin\lambda_\odot - \sin\varepsilon\,\sin\beta_\odot), \\ r_z &= r\,(\sin\varepsilon\,\cos\beta_\odot \sin\lambda_\odot + \cos\varepsilon\,\sin\beta_\odot), \end{aligned} \tag{43}$$

with the solar distance r from the three-harmonic radius series. The Moon's longitude, latitude, and horizontal-parallax-derived distance follow the analogous Brown-series truncation in the same five fundamental arguments (same routine). The resulting ECI vectors are then rotated to ECEF by exactly the $\boldsymbol{U}$ of (33) or (38) according to the `estimation.enable_cio_frame` toggle (default off, Section 6.8), in `Planetary::computeSunMoonPosition`; the ECEF Sun and Moon are precisely the inputs consumed by the solid-Earth and ocean tides of Section 10 and by the satellite-attitude and phase-wind-up models of Section 11. The two luni-solar models are only ∼100 km accurate in position. That is ample for the tidal and attitude geometry, which depend on the Sun/Moon *direction* to well under the milliarcsecond level that matters, and a non-finite guard substitutes a nominal 1 AU / mean-lunar-distance fallback rather than propagate a NaN into the correction stack (the output-validation tail of `Planetary::computeSunMoonPosition`).

7 Terrestrial Reference Frames, Epochs, and Datum Transformation

A terrestrial coordinate is three numbers, a frame, and an epoch. The engine works in ITRF at the epoch of observation, so the frame and the epoch are usually implicit. They become explicit when a result is compared with a national datum, a published datasheet, or a passive mark. Changing the frame and changing the epoch are different operations, and mixing them up produces decimetre-level offsets. The rule is:

Frames take a Helmert. Epochs take a velocity. (44)

Sections 7.1 to 7.3 develop the rule and its one exception. Sections 7.4 and 7.5 give the sign conventions and the published parameter sets. Section 7.6 covers NATRF2022. Sections 7.7 and 7.8 cover the NGS Site Information File and the monument-to-ARP offset. Section 7.9 is a numerical example at CORV, and Section 7.10 compares the NGS deformation model with station velocities.

7.1 Frame versus epoch

A Helmert similarity (seven parameters, or fourteen with their linear rates) maps frame A to frame B *at the same epoch*. It describes how two realisations differ in origin, orientation, and scale. It says nothing about how the ground moves.

The fourteen-parameter form is

$$p(t) \;=\; p_0 \;+\; \dot{p}\,(t - t_0), \qquad p \in \{\boldsymbol{T},\, s,\, \boldsymbol{\varepsilon}\}, \tag{45}$$

and the rates $\dot{p}$ are rates *of the parameters*: the drift of one frame relative to the other. They are not station velocities. Evaluating (45) at epoch t gives the transformation from A to B at that epoch. It does not move the coordinate through time.

Moving a coordinate between epochs requires information about the site:

$$\boldsymbol{x}(t) \;=\; \boldsymbol{x}(t_0) \;+\; \boldsymbol{v}\,(t - t_0) \;+\; \boldsymbol{d}_{\mathrm{per}}(t) \;+\; \boldsymbol{d}_{\mathrm{PSD}}(t), \tag{46}$$

with $\boldsymbol{v}$ a station velocity, $\boldsymbol{d}_{\mathrm{per}}$ the periodic (annual + semiannual) terms, and $\boldsymbol{d}_{\mathrm{PSD}}$ the post-seismic deformation. `resolveStationCoordinate` and `tools/sif_propagate` implement this form. The three site terms are observed or modelled. A transformation parameter set does not contain them, so a transformation that appears to handle an epoch change is using the rigid-plate approximation of Section 7.3.

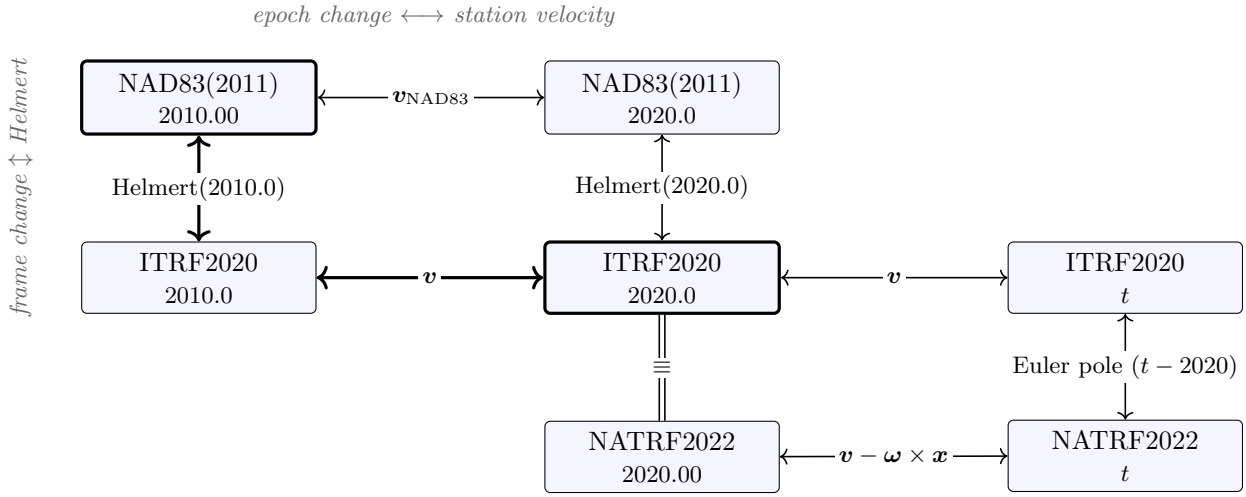


Figure 1: Frames (rows) and epochs (columns). Vertical moves change the frame at a fixed epoch; horizontal moves change the epoch within a frame and need a station velocity. Heavy lines mark the worked example of Section 7.9, from ITRF2020 at 2020.0 to NAD83(2011) at 2010.00. Because NAD83 rotates with the plate, $\boldsymbol{v}_{\mathrm{NAD83}}$ is small on the stable interior and larger at the plate margins (Section 7.3).

7.2 Transforming a velocity

Differentiating the linearised transformation with respect to time gives

$$\boldsymbol{v}_B \;=\; \boldsymbol{v}_A \;+\; \dot{\boldsymbol{T}} \;+\; \dot{s}\,\boldsymbol{x} \;+\; \dot{\boldsymbol{E}}\,\boldsymbol{x}. \tag{47}$$

Three properties of (47) matter in practice.

The rate field is rigid. The added terms $\dot{\boldsymbol{T}} + \dot{s}\boldsymbol{x} + \dot{\boldsymbol{E}}\boldsymbol{x}$ depend on the station's *position* only, not on its motion. This was checked by transforming fictitious stations at one position with different velocities: the added velocity was identical to the micron. The rates of a fourteen-parameter set are therefore a rigid velocity field, just as the seven parameters are a rigid displacement field.

Propagation and transformation commute, with one condition. Propagating in frame A and then transforming gives the same result as transforming and then propagating in frame B, to 0.000 mm, provided each propagation uses the velocity expressed in its own frame. That velocity comes from (47).

Using the wrong frame's velocity is a large error. Propagating a NAD83(2011) coordinate with an ITRF velocity costs about 20 mm/yr at CORV, or 203 mm per decade. No single-epoch check reveals it. The error exceeds every geophysical correction of Section 10 combined.

7.3 The rigid-plate approximation

The parameter rates can stand in for a station velocity in one case. A plate-fixed frame is built so that stations on the rigid plate do not move in it. Setting $\boldsymbol{v}_B = \mathbf{0}$ in (47) gives

$$\boldsymbol{v}_A \;=\; -\big(\dot{\boldsymbol{T}} + \dot{s}\,\boldsymbol{x} + \dot{\boldsymbol{E}}\,\boldsymbol{x}\big). \tag{48}$$

The right-hand side is then a *modelled* rigid-plate velocity, and (45) is being used as a one-term deformation model.

The approximation is as good as the assumption. On stable North America the horizontal velocity relative to the plate is 1–2 mm/yr, so (48) costs 1–3 cm per decade. At a plate margin it fails. CORV sits on the deforming Cascadia margin, and the fourteen-parameter-only reconstruction of Section 7.9 misses by 104 mm per decade.

7.4 Rotation-sign conventions

Two rotation conventions are in use, and the sources cited here use both. The NGS / EPSG:1056 *coordinate frame* form, linearised for small rotations, is

$$\boldsymbol{x}' \;=\; \boldsymbol{x} \;+\; \boldsymbol{T} \;+\; s\,\boldsymbol{x} \;+\; \boldsymbol{E}\,\boldsymbol{x}, \qquad \boldsymbol{E} \;=\; \begin{bmatrix} 0 & \varepsilon_z & -\varepsilon_y \\ -\varepsilon_z & 0 & \varepsilon_x \\ \varepsilon_y & -\varepsilon_x & 0 \end{bmatrix}. \tag{49}$$

The IERS / IGN *position vector* form is identical except that $\boldsymbol{E}$ carries the opposite sign throughout; equivalently, $\boldsymbol{\varepsilon} \to -\boldsymbol{\varepsilon}$.

Using one convention's parameters in the other convention's matrix applies the rotation backwards. For a rotation angle θ and a position of magnitude L the error is $2|\theta|L$, twice the size of omitting the rotation. The same holds for the rate field of Section 7.2. A rate field derived here with the wrong convention came out exactly negated, a $\sim$35 mm/yr error at CORV where the true field is $\sim$ 20 mm/yr. Check the sign of a rate field against a station of known velocity; a plausible magnitude is not evidence of a correct sign.

7.5 Published parameter sets and their composition

Table 1 lists the three sets the tooling uses. The first is in position-vector convention and the other two are in coordinate-frame convention. The IGN rotations are zero, so the composition below can add the sets without converting between the conventions of Section 7.4.

PROJ applies the pre-composed EPSG:10334 set for `EPSG:9988`→`EPSG:6317`, with `t_epoch`= 2010 and the `coordinate_frame` convention. It can be rebuilt from the other two sets in two steps:

1. **Run the IGN set to the NGS epoch.** Evaluate (45) at 2010.0 from its own $t_0 = 2015.0$: $\boldsymbol{T}(2010) = \boldsymbol{T}(2015) + \dot{\boldsymbol{T}} \cdot (-5) = (-1.4,\ -0.4,\ +0.4)$ mm.

Table 1: Published Helmert parameter sets. ITRF2020→ITRF2014 is IGN's, in position-vector convention; the two NAD83 sets are NGS/EPSG, in coordinate-frame convention. Rotations $\boldsymbol{\varepsilon}$ in milliarcseconds.

	ITRF2020→ITRF2014 IGN, position-vector $t_0 = 2015.0$	**ITRF2014→NAD83(2011)** NGS MYCS2, EPSG:8970 $t_0 = 2010.0$, coord-frame	**ITRF2020→NAD83(2011)** EPSG:10334 (pre-composed) $t_0 = 2010.0$, coord-frame
T_x	−1.4 mm	1.00530 m	1.00390 m
T_y	−0.9 mm	−1.90921 m	−1.90961 m
T_z	+1.4 mm	−0.54157 m	−0.54117 m
s	−0.42 ppb	0.36891 ppb	−0.05109 ppb
ε_x	0	26.78138 mas	26.78138 mas
ε_y	0	−0.42027 mas	−0.42027 mas
ε_z	0	10.93206 mas	10.93206 mas
$\dot{T}_x$	0.0 mm/yr	0.00079 m/yr	0.00079 m/yr
$\dot{T}_y$	−0.1 mm/yr	−0.00060 m/yr	−0.00070 m/yr
$\dot{T}_z$	+0.2 mm/yr	−0.00144 m/yr	−0.00124 m/yr
$\dot{s}$	0	−0.07201 ppb/yr	−0.07201 ppb/yr
$\dot{\varepsilon}_x$	0	0.06667 mas/yr	0.06667 mas/yr
$\dot{\varepsilon}_y$	0	−0.75744 mas/yr	−0.75744 mas/yr
$\dot{\varepsilon}_z$	0	−0.05133 mas/yr	−0.05133 mas/yr

2. **Add term by term** to the NGS set at its native 2010.0.

The result matches PROJ's set: the largest difference over the fourteen parameters is 4×10^{-15} in the respective units. Adding the sets is valid because the cross terms of the exact composition are below a micron at Earth radius. The rotation-times-translation term $\boldsymbol{\Omega T}$ is $\sim$0.3 µm and the rotation-squared term $\Omega_1\Omega_2 R$ is $\sim$0.1 µm. The two-step chain and the pre-composed set agree to 0.0000 mm at 2010.0, 2015.0, and 2020.0.

Two conditions apply. First, both sets must be in the same rotation convention before they are added. That step is unnecessary here only because the IGN rotations are zero. Second, step 1 is required: adding the IGN set at its own 2015.0 reference epoch introduces a 0.5 mm error in Y and 1.0 mm in Z.

The EPSG codes are ITRF2020 `9988` / `9990` / `9989` (geocentric XYZ, 2D, 3D), ITRF2014 `7789` / `9000` / `7912`, and NAD83(2011) `6317` / `6318` / `6319`.

7.6 NATRF2022

NATRF2022, the North American frame of NSRS2022, is ITRF2020 rotated about the North America Euler pole. It has no translation and no scale. The two frames are identical at epoch 2020.00 and separate as the plate moves. With $\boldsymbol{\omega}$ the Euler vector, positive counter-clockwise under the right-hand rule,

$$\boldsymbol{x}_{\mathrm{NATRF}}(t) = \boldsymbol{x}_{\mathrm{ITRF2020}}(t) \;-\; (\boldsymbol{\omega} \times \boldsymbol{x})\,(t - 2020.0), \tag{50}$$

$$\boldsymbol{v}_{\mathrm{NATRF}} = \boldsymbol{v}_{\mathrm{ITRF2020}} - \boldsymbol{\omega} \times \boldsymbol{x}. \tag{51}$$

The NGS BETA EPP2022 Euler poles are, in mas/yr, $\boldsymbol{\omega}_{\mathrm{NA}} = (0.046, -0.704, -0.047)$, with the companion plates $\boldsymbol{\omega}_{\mathrm{PA}} = (-0.409, 1.063, -2.188)$, $\boldsymbol{\omega}_{\mathrm{CA}} = (-0.056, -0.957, 0.589)$, and $\boldsymbol{\omega}_{\mathrm{MA}} = (-8.089, 5.937, 2.159)$ for PATRF2022, CATRF2022, and MATRF2022 respectively. These are BETA

values from https://beta.ngs.noaa.gov/NATRF2022/. Fetch the current values before production use.

The sign of (51) was checked on stable-interior stations, whose plate-fixed velocity must be near zero. The NATRF2022 velocities, ENU in mm/yr, are AMC4 (−0.49, 1.01, −1.17), GODN (−0.12, 0.47, −1.49), TXAU (−0.34, 1.73, −0.52), WES2 (0.07, −0.03, −0.12), and MIA3 (0.28, 0.90, 0.56). A sign error would double the plate rate, giving 20–30 mm/yr here.

PROJ 9.5.1 has no NATRF2022 CRS, so the caller must apply (50) directly.

7.7 SIF occupations and the datasheet convention

The NGS Site Information File (SIF) gives ITRF2020 coordinates at epoch 2020.0 from the MYCS3 solution. A station can have several **occupation** windows, each with its own reference coordinate and velocity. An occupation boundary is an equipment change or a known offset, so the coordinate series is discontinuous there. Two consequences follow.

The datasheet convention. NGS produces a datasheet's NAD83(2011) epoch-2010.00 coordinate as follows. Take the *latest* occupation. Propagate its reference coordinate back ten years with its linear velocity and **no** periodic term. Apply the fourteen-parameter transformation of Section 7.5 at 2010.0. This recipe reproduces the datasheet to 0.4 mm at CORV (PID AH2488) and 0.6 mm at CHZZ (PID AJ6961). In `tools/sif_propagate` it is the `--datasheet` option.

The engine convention. `resolveStationCoordinate`, and `tools/sif_propagate` by default, select the occupation that *covers the requested epoch*. A coordinate for an observation session must describe the equipment installed at that time. At CORV a request for 2010.0 uses the 1998–2019 occupation and differs from the datasheet by 32.8 mm. Of that, $\delta U = -32.4$ mm is the jump at the 2019-07-24 antenna change (ASH700936E_C → TRM59800.99 SCIS). Both coordinates are correct for their purpose. One rule follows:

Remark 1. The difference of two resolved coordinates across an occupation break is *not* a velocity. It includes the equipment offset, which is often the larger part.

Periodic terms are a constant offset at whole-year epochs. The SIF annual and semiannual terms are referenced to the station's reference epoch. A whole number of years from that epoch, cos = 1 and sin = 0, so the terms add a fixed vector: 0.65 mm at CORV, 1.73 mm at NEWP, 3.18 mm at CHZZ. The offset cancels in a difference between two such epochs but not in an absolute comparison. Published coordinates exclude it, so a propagated coordinate that includes it differs from the datasheet by these amounts.

7.8 Monument versus ARP

A datasheet comparison must also use the same reference point. SIF and datasheet coordinates refer to the **monument** (ground reference point, GRP). The engine's position state and the NGS CORS coordinate files refer to the **ARP**. The offset between them is the RINEX `ANTENNA:DELTA H/E/N` eccentricity, which the SIF publishes as `ARP_MINUS_GRP_ENU`. The reduction is developed in Section 6.2.

At CORV, $\delta H = 1.521$ m, and both comparisons close:

- SIF monument + 1.521 m Up versus the published ITRF2020 ARP (−2498424.007, −3802820.797, 4454737.751): 0.3 mm.

- Datasheet + 1.521 m Up versus the published NAD83(2011) ARP (−2498423.096, −3802822.020, 4454737.741): 0.5 mm.

Omitting this step at CORV gives an obvious 1.5 m height error. The harder case is a site with an eccentricity of a few centimetres, where the error is small enough to go unnoticed.

7.9 Worked example: CORV

This example works every step above with numbers, at the monument, using the latest occupation (2019-08-04 to open). `tools/sif_propagate/corv_worked_example.py` reproduces it.

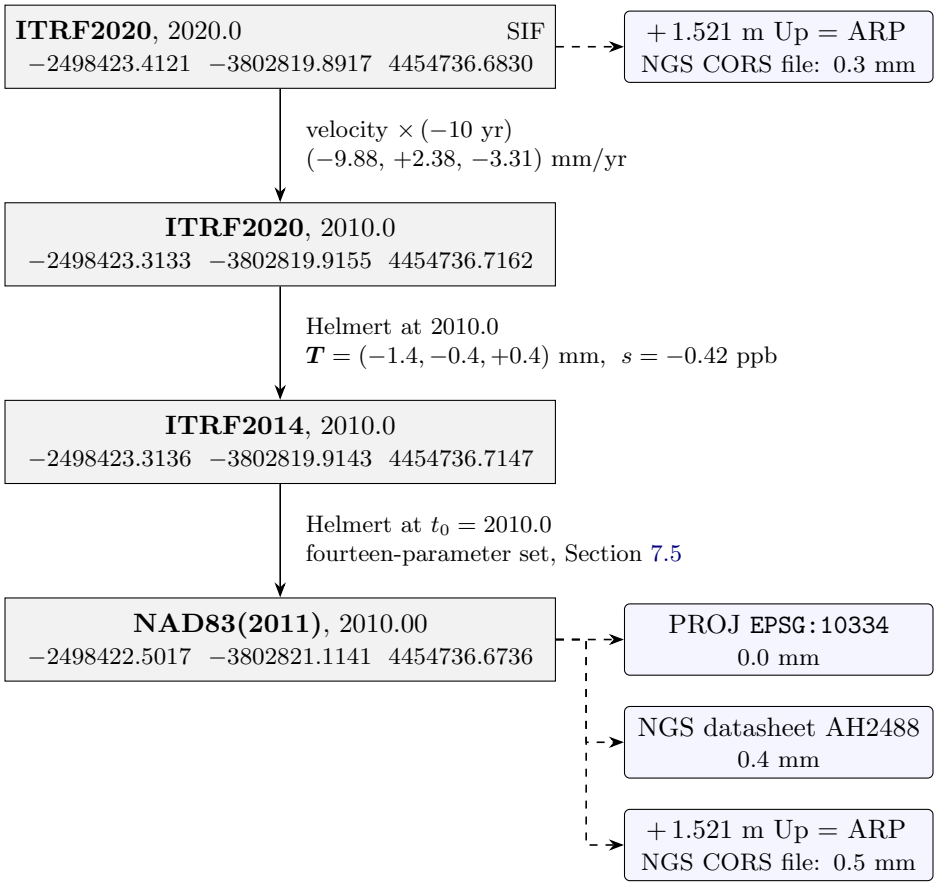


Figure 2: The CORV worked example, modern → legacy, at the monument (metres). Solid arrows are the computation; dashed arrows are independent checks, with the 3-D closure. The antenna reference point lies 1.521 m above the monument (RINEX `DELTA H`).

Starting point (SIF, ITRF2020 @ 2020.0). $\boldsymbol{x}$ = (−2498423.4121, −3802819.8917, 4454736.6830) m, h = 105.5881 m; velocity in ECEF (−9.8836, 2.3812, −3.3134) mm/yr, i.e. ENU (−9.57, −4.77, +0.12) mm/yr.

Step 2a: propagate to 2010.0 in ITRF2020. $\boldsymbol{x} - 10\boldsymbol{v}$ = (−2498423.3133, −3802819.9155, 4454736.7162) m; the station moved 106.9 mm.

Step 2b: ITRF2020 → ITRF2014 at 2010.0. With $\boldsymbol{T} = (-1.4, -0.4, +0.4)$ mm (the IGN set run to 2010.0) and $s = -0.42$ ppb: (−2498423.3136, −3802819.9143, 4454736.7147) m.

Step 2c: ITRF2014 → NAD83(2011) at 2010.0. Translation (1.00530, −1.90921, −0.54157) m; rotation term (−0.19247, 0.71082, 0.49885) m; scale term (−0.00092, −0.00140, 0.00164) m. Result (−2498422.5017, −3802821.1141, 4454736.6736) m. PROJ gives the same result (0.0 mm).

Closure against the datasheet. NGS datasheet AH2488 gives $\varphi = 44°\,35'\,07.91300''$ N, $\lambda = 123°\,18'\,16.51681''$ W, $h = 105.953$ m, i.e. (−2498422.5015, −3802821.1143, 4454736.6734) m. Hand computation minus datasheet is $\delta E = -0.3$, $\delta N = +0.2$, $\delta U = +0.1$ mm, which is 0.4 mm in three dimensions.

The reverse direction. Going from the datasheet to ITRF2020 at 2020.00, which is also NATRF2022 at that epoch (Section 7.6), requires ten years of station velocity. The miss against the SIF coordinate depends on the source of that velocity:

Table 2: CORV reverse reconstruction: miss against the SIF coordinate by velocity source. The last row is the rigid-plate approximation of Section 7.3, which does not hold at CORV.

Velocity source	3D miss	Dominant component
SIF velocity	0.4 mm	$\delta E = 0.3$
IFDM2022 grid (−8.68, −4.78, +0.04) mm/yr ENU	9.3 mm	$\delta E = 9.2$
BETA NCAT	9.6 mm	δE 6.9, δN − 5.4, δU 4.0
14-parameter only ($\boldsymbol{v}_{\mathrm{NAD83}} := \mathbf{0}$)	104.2 mm	δE − 54.6, δN − 88.2, δU 10.4

The same point in four frames. CORV velocity, ENU mm/yr: ITRF2020 (−9.57, −4.77, 0.12); plate motion $\boldsymbol{\omega} \times \boldsymbol{x}$ (−13.20, −13.12, −0.04); NATRF2022 (3.63, 8.35, 0.17); NAD83(2011) (5.49, 8.80, −1.05). The NATRF2022 velocity is what remains after removing rigid North America. At a deforming margin it is not small: a NATRF2022 coordinate at CORV moves (21.8, 50.1, 1.0) mm by 2026.0 and (36.3, 83.5, 1.7) mm by 2030.0 from its 2020.00 value. A plate-fixed coordinate is not a static one.

7.10 Deformation models and validation

A passive mark has no observed velocity, so a model must supply one. NGS supplies IFDM2022 with BETA NCAT. It is a global 0.1° ENU velocity grid with per-node standard deviations, in ITRF2020 (`epsg:9990`), plus 33 earthquake displacement grids. It was compared with SIF velocities at 21 stations in CONUS, Alaska, and British Columbia, using one occupation per station:

- $\|\boldsymbol{v}_{\mathrm{IFDM}} - \boldsymbol{v}_{\mathrm{SIF}}\|$: median 1.34 mm/yr, maximum 3.11 mm/yr.
- The grid's formal σ is optimistic by roughly a factor of two (median ratio 2.1, up to 5.9 in the stable interior). The comparison is *in-sample*, because the grid was built from these CORS. A passive mark should expect a larger error.
- In the stable interior the IFDM horizontal velocity equals the EPP2022 plate motion to 0.01 mm/yr. The model assigns zero intraplate velocity there.

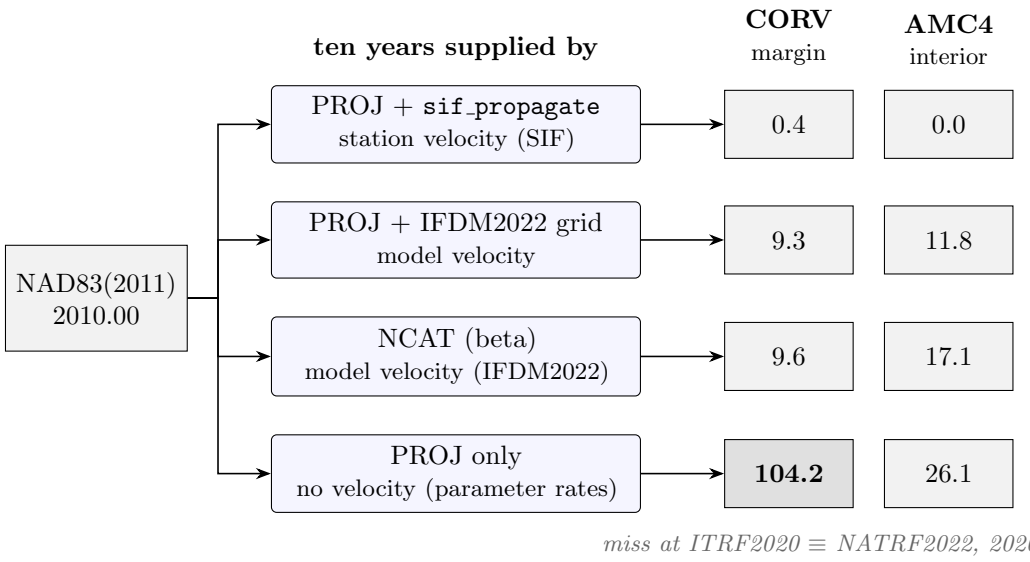


Figure 3: Legacy → modern, NAD83(2011) epoch 2010.00 to ITRF2020 ≡ NATRF2022 epoch 2020.00, by four routes. PROJ is the inverse of `EPSG:10334`, evaluated at 2010.0 (at 2020.0 in the last row, where it stands alone). The number is the 3-D miss against the SIF coordinate at 2020.0. CORV starts from NGS datasheet AH2488; AMC4 (Colorado Springs, single occupation) starts from its SIF-derived 2010.00 coordinate, so its first row is zero by construction.

- Velocity spread across a single grid cell is 0.03–0.05 mm/yr in the interior and up to 3.7 mm/yr in the Aleutians. Interpolation error matters only where the field is steep.

NCAT's transformation from NAD83(2011) at 2010.00 to NATRF2022 at 2020.00 was compared with the SIF coordinate at 2020.0. The mean difference is (+4.9, −6.1, −1.3) mm and the median 3D difference is 18 mm, within NCAT's stated uncertainty of about 2 cm for most stations. The small mean confirms that NATRF2022 and ITRF2020 coincide at 2020.00 to a few millimetres, as Section 7.6 requires. Five stations are outliers: FAIR 115, SNI1 82, AB02 78, KYW1 61, and HOLP 43 mm. Four of them lie inside IFDM earthquake displacement grids. The SIF represents those events as occupation *breaks* rather than PSD terms, so the two sources model them differently. This attribution rests on grid coverage only. The displacement magnitudes were not evaluated, and KYW1 is unexplained.

Limitations of beta NCAT. It offers only fixed epochs: NAD83(2011) at 2010.00 and NATRF2022 at 2020.00. Its offline CLI performs coordinate *conversions* only, not datum transformations. Its XYZ input is rounded to 1 mm. The post-seismic terms of both tools are untested here, because no station in the comparison has a PSD term.

Mixing velocity sources. Within NSRS2022 the deformation model is the convention, so using IFDM (or HTDP) for a passive mark is correct by definition. The risk is mixing sources. A network that propagates its CORS with SIF velocities and its marks with grid velocities introduces a 1–2 mm/yr difference between the two groups, which distorts the network over a decade. The grid also gives only a diagonal covariance, and an optimistic one by the ratio above. SIF propagation gives the full

$$\boldsymbol{C}_{xx}(t) \;=\; \boldsymbol{C}_{xx} \;+\; \Delta t\,\big(\boldsymbol{C}_{xv} + \boldsymbol{C}_{xv}^{\mathrm{T}}\big) \;+\; \Delta t^{2}\,\boldsymbol{C}_{vv}, \tag{52}$$

including the position–velocity cross-covariance, which dominates (52) at short Δt and which the grid does not provide.

8 Orbit and Clock Product Evaluation

Every processing mode in the engine (post-processed PPP, PPP-RTK, the baseline modes, and the real-time engine) needs a position $\boldsymbol{r}^i$, velocity $\dot{\boldsymbol{r}}^i$, and clock bias δt^i for each satellite at each query epoch. These are evaluated *before* any state-space refinement. In post-processing they are final. In the real-time engine they are the broadcast values that the SSR overlay (Section 18) corrects with a RAC orbit delta and a scalar clock delta. This document derives the two precise interpolation paths, the two broadcast evaluation paths, and the ephemeris selection that feeds them. It stops where the SSR delta is added: the RAC frame, the IOD gate, and the freshness checks are in Section 18. The Sagnac (Earth-rotation) range correction, which makes $\boldsymbol{r}^i$ at transmit time consistent with the line of sight, is in Section 11. The ERP and polar-motion transforms between the terrestrial and celestial frames are in Section 6.

There is a *precise* path and a *broadcast* path. The precise path *interpolates* tabulated SP3 orbit samples and CLK clock samples (IGS Final / Rapid products). It is the post-processing default and is accurate to centimetres or better. The broadcast path evaluates a Keplerian ephemeris from the navigation message, or for GLONASS a numerically integrated PZ-90 ephemeris. It is accurate to metres and is what the live SSR stream corrects. By default the two paths do not mix for one satellite at one epoch: a loaded precise product that spans the query is used, and otherwise the engine falls back to the broadcast evaluation (`src/core/model/ModelGeometry.cpp`, `src/core/corrections/GnssCorrections.cpp`). The choice is made per quantity, though. The real-time RTB-4 gate `realtime.use_precise_orbits` selects the orbit and clock sources independently (`GnssConfig::realtimeUsesLiveOrbitSource` vs `realtimeUsesLiveOrbitClockSource`, `ProductParser::loadProducts(bool, bool)`). With that gate armed, an "ultra-rapid orbits + network clocks" configuration uses a precise interpolated orbit *and* a broadcast+SSR clock for the same satellite and epoch. The flag defaults to false. The orbit gate then returns the same value as the clock gate, the result is byte-identical, and the no-mixing statement holds for every non-real-time mode and for the default real-time path.

8.1 Precise orbit: derotated Neville interpolation

SP3 files tabulate the satellite position in the ECEF frame on a coarse grid (typically 900 s for IGS Final, 300 s for Rapid). The ECEF trajectory is *not* a smooth low-degree polynomial. The rotating frame adds an oscillation at the sidereal frequency, and a degree-10 fit cannot follow it to better than a few millimetres near the SP3 epoch boundaries. The engine therefore *derotates* each node into a common near-inertial frame anchored at the query epoch before interpolating (`src/core/model/ModelGeometry.cpp`, following the RTKLIB / cssrlib convention [24]). For a node sampled at t_i and a query at t_q, with the Earth-rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}$ rad/s (`Constants::EARTH_ANGULAR_VELOCITY`), the in-plane coordinates are rotated by the angle the frame swept between the two epochs:

$$\boldsymbol{r}^i_{\mathrm{I}}(t_i) = \boldsymbol{R}_z(\omega_\oplus\,(t_i - t_q))\;\boldsymbol{r}^i(t_i), \qquad \boldsymbol{R}_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{53}$$

Concretely, with $\alpha_i = \omega_\oplus\,(t_i - t_q)$ the derotated in-plane components written to the interpolation arrays are $\tilde{x}_i = \cos\alpha_i\, x_i - \sin\alpha_i\, y_i$ and $\tilde{y}_i = \sin\alpha_i\, x_i + \cos\alpha_i\, y_i$, with z unchanged (`ModelGeometry.`

cpp). The rotation angle is zero at $t_i = t_q$, so the interpolated value at the query is already the ECEF position at t_q and needs no rotation back.

Bracket-centred window. The choice of node window determines whether the interpolated position is *continuous*. The engine selects an IGS_POINTS = 11-node window centred on the *bracket* $[t_a, t_{a+1}]$ that contains t_q, not the 11 nodes nearest to t_q (`ModelGeometry.cpp`). A nearest-N rule changes a node at the *midpoint* between SP3 epochs. There the two candidate polynomials disagree by a few millimetres, which produces the known 15-minute sawtooth in PPP position, synchronous across constellations. A bracket-centred rule changes the window when t_q crosses a node t_{a+1}. That node belongs to both windows, so the Neville value is the same on either side and the orbit is continuous. A gap guard re-picks a narrower MIN_POINTS = 9-node window and rejects the satellite if the residual span still exceeds the fit interval (thresholds in `ModelGeometry.cpp`; tuning values cited, not reproduced). The "fit interval" is not read from the product header. It is the *median* of the leading consecutive-node spacings (`ModelGeometry.cpp`). The reason is that a multi-file SP3 merge can be non-uniformly sampled. For example, a 5-minute ultra-rapid file concatenated with a 15-minute one keeps the finer file's off-grid nodes, because the 1 s dedup window correctly treats them as distinct (`ProductParser` warns on the mixed-rate concat). The median spacing gives a stable MAX_SPAN guard on such a series, where a nominal grid is not defined.

Neville value and derivative. Within the window the engine runs Neville's algorithm simultaneously for the value and its analytic derivative (`src/core/math/Interpolate.cpp`, `Interpolate::nevilleInterpDeriv`). The node abscissae are shifted to $s_i = t_i - t_q$ to avoid cancellation: raw continuous GPS seconds are $\sim 1.36 \times 10^8$, so an unshifted difference loses ~ 8 digits. The shift changes neither the polynomial value nor its derivative. With the tableau column P_i for the value and D_i for the derivative, the coupled recurrence over degree j (evaluating at the shifted query $s_q = 0$) is

$$P_{i,j} = \frac{(s_q - s_{i+j})\, P_{i,j-1} - (s_q - s_i)\, P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{54}$$

$$D_{i,j} = \frac{(s_q - s_{i+j})\, D_{i,j-1} + P_{i,j-1} - (s_q - s_i)\, D_{i+1,j-1} - P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{55}$$

seeded by $P_{i,0} = y_i$ and $D_{i,0} = 0$; the converged scalars are $P_{0,n-1}$ and $D_{0,n-1}$ (`Interpolate.cpp`). The derivative recurrence (55) is $\partial/\partial s_q$ of the value recurrence (54), so one pass gives both and no finite difference is needed.

Inertial-to-ECEF velocity. The Neville derivative is taken on the *derotated* channels, so it is the velocity in the inertial frame fixed at t_q. The Doppler line-of-sight rate, the orbit-normal attitude, and the relativistic $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ term need the ECEF velocity, which follows by removing the frame rotation $\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \dot{\boldsymbol{r}}^i_{\mathrm{I}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$. For $\boldsymbol{\omega}_\oplus = (0, 0, \omega_\oplus)$ this is the boxed transform applied in `ModelGeometry.cpp`:

$$\boxed{\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \begin{bmatrix} v_{\mathrm{I},x} + \omega_\oplus\, \boldsymbol{r}^i_y \\ v_{\mathrm{I},y} - \omega_\oplus\, \boldsymbol{r}^i_x \\ v_{\mathrm{I},z} \end{bmatrix}}. \tag{56}$$

The relativistic clock term is unaffected, since $\boldsymbol{r}^i \cdot (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i) = 0$. Only the Doppler and attitude computations see the difference (`ModelGeometry.cpp`).

8.2 Precise orbit: sidereal-trigonometric LSQ fit

A config-gated alternative (`config.enableTrigSp3Interp()`) replaces the Neville polynomial with an over-determined least-squares fit on a *sidereal-trigonometric basis*, run on the identical derotated channels and time base (`ModelGeometry.cpp`; `Interpolate::trigSvdInterp`). The basis is a constant plus four sine/cosine harmonic pairs of the fundamental sidereal angular rate, nine terms in all:

$$\boldsymbol{r}^i_{\mathrm{c}}(\tau) = c_0 + \sum_{f=1}^{4}\big[c_{2f-1}\sin(f\,\Omega_0\,\tau) + c_{2f}\cos(f\,\Omega_0\,\tau)\big], \qquad \tau = t - t_0, \tag{57}$$

where t_0 is the first node. The fitted curve does not depend on this choice, because the constant term absorbs the offset and the sine and cosine pairs absorb the phase. The fundamental rate is the angular frequency of the *mean sidereal day*. With the sidereal-day ratio $\kappa = 0.99726956634$ and a nominal solar day of 86400 s,

$$\boxed{\Omega_0 = \frac{2\pi}{\kappa \cdot 86400\ \mathrm{s}} \quad [\mathrm{rad/s}], \qquad \kappa = 0.99726956634,} \tag{58}$$

expressed *per second*, so τ stays in the caller's seconds and the analytic derivative is per second (`Interpolate.cpp`). M-PAGES uses rad/day and divides the velocity by 86400, which is numerically identical. The constant $\kappa = 0.99726956634$ is the IERS ratio of the mean sidereal day to the solar day. The model depends on it directly, so it is quoted here.

The fit is solved by Eigen's `JacobiSVD` with a relative singular-value threshold of 10^{-15}, which yields the minimum-norm least-squares solution, the Numerical-Recipes `svbksb` convention M-PAGES' `svdfit` uses [19]. Per-node weights derive from the SP3 `++` accuracy codes; because those are a single value per satellite, the weight set is uniform and the fit is effectively unweighted, matching M-PAGES. The analytic value and derivative at the query $\tau_q = t_q - t_0$ are

$$\boldsymbol{r}^i_{\mathrm{c}}(\tau_q) = c_0 + \sum_{f=1}^{4}\big[c_{2f-1}\sin(f\Omega_0\tau_q) + c_{2f}\cos(f\Omega_0\tau_q)\big], \tag{59}$$

$$\dot{\boldsymbol{r}}^i{}_{\mathrm{c}}(\tau_q) = \sum_{f=1}^{4}(f\Omega_0)\big[c_{2f-1}\cos(f\Omega_0\tau_q) - c_{2f}\sin(f\Omega_0\tau_q)\big], \tag{60}$$

with the per-harmonic angular weight $w_f = f\,\Omega_0$ (`Interpolate.cpp`). The fit is an *over-determined smoother*. The window has up to 11 nodes against 9 basis terms, so for $N > 9$ the fitted curve does *not* pass through the nodes. This differs from the M-PAGES 9-point exact fit, and is kept so that the trig branch can reuse the derotated window unchanged. The same inertial-to-ECEF velocity conversion (56) is then applied to $\dot{\boldsymbol{r}}^i{}_{\mathrm{c}}$.

8.3 Precise clock interpolation

The CLK product tabulates the scalar satellite clock bias δt^i on its own grid (commonly 30 s). Clock noise behaves like a random walk in bias with occasional rate discontinuities, so the engine offers three interpolation modes, selected by `config.getSatelliteClockInterpolationMode()` (`src/core/corrections/GnssCorrections.cpp`). All three are bracket-centred for the same reason as the orbit window: the window must change at a shared node, never at a midpoint between epochs.

(a) Linear (MMSE for Brownian motion). The default is a two-node linear interpolation across the bracket $[t_0, t_1]$ containing t_q:

$$\delta t^i(t_q) = \delta t_0^i + \left(\delta t_1^i - \delta t_0^i\right) \frac{t_q - t_0}{t_1 - t_0}, \tag{61}$$

which is the minimum-mean-square-error estimator for a Wiener process sampled at the two bracketing epochs. If the gap exceeds a guard threshold the interpolation is rejected (`GnssCorrections.cpp`; gap value cited, not reproduced).

(b) Rate-anchored weighted LSQ. The `LsqSatClock` mode fits a local line $\delta t^i(t) \approx a + b\,(t - t_q)$ over an $\sim$11-node window by weighted least squares, with the centring shift $s_k = t_k - t_q$ making the fitted value at the query equal to the intercept a. The CLK product may also carry a clock *rate* m_j at the coarse 5-minute epochs. Each rate is added as a direct observation of the slope, $m_j = b + \mu_j$. A value equation $y_k = a + b\,s_k + \nu_k$ has $\partial/\partial a = 1$ and $\partial/\partial b = s_k$. A rate equation has $\partial/\partial a = 0$ and $\partial/\partial b = 1$, so the rate observations enter the slope row only. With value weights $w_k^{(y)} = 1/\sigma_{y,k}^2$ and rate weights $w_j^{(m)} = 1/\sigma_{m,j}^2$, the 2×2 weighted normal equations are

$$\boxed{\begin{bmatrix} \sum_k w_k^{(y)} & \sum_k w_k^{(y)} s_k \\ \sum_k w_k^{(y)} s_k & \sum_k w_k^{(y)} s_k^2 \;+\; \sum_j w_j^{(m)} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_k w_k^{(y)} y_k \\ \sum_k w_k^{(y)} s_k y_k \;+\; \sum_j w_j^{(m)} m_j \end{bmatrix}}, \tag{62}$$

solved in closed form for the intercept

$$a = \frac{\left(S_{wxx} + S_m\right) S_{wy} - S_{wx}\left(S_{wxy} + S_{my}\right)}{S_w\left(S_{wxx} + S_m\right) - S_{wx}^2}, \tag{63}$$

with the running sums named exactly as in `GnssCorrections.cpp` ($S_w = \sum w_k^{(y)}$, $S_{wx} = \sum w_k^{(y)} s_k$, $S_{wxx} = \sum w_k^{(y)} s_k^2$, $S_{wy} = \sum w_k^{(y)} y_k$, $S_{wxy} = \sum w_k^{(y)} s_k y_k$, $S_m = \sum w_j^{(m)}$, $S_{my} = \sum w_j^{(m)} m_j$). If the determinant in the denominator vanishes (a window of coincident times) the code falls back to linear interpolation. The estimate $\delta t^i(t_q) = a$ is returned. The fit averages $\sim N$ samples where linear interpolation uses two, so the observation-noise part of the estimate at the bracket midpoint falls by $\sim\sqrt{N/2}$ relative to (61). This matters over seconds to minutes, where CLK observation noise exceeds the per-sample process noise. The cost is that the fit no longer passes through the nodes, so millimetre-level discontinuities appear at the clock epochs when the window moves.

(c) Barycentric Lagrange. For high-rate (1 Hz) products a barycentric-form Lagrange interpolation over a LAGRANGE_POINTS = 9-node window is available (`GnssCorrections.cpp`; `Interpolate::barycentricWeights` / `barycentricEval`). The barycentric form

$$\delta t^i(t_q) = \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}\,\delta t_i^i \;\Bigg/\; \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}, \qquad w_i = \prod_{k \neq i} \frac{1}{t_i - t_k}, \tag{64}$$

is the numerically stable evaluation of the degree-$(n-1)$ Lagrange polynomial [2]. An extrapolation guard falls back to two-node linear interpolation when t_q lies outside the node span. This mode is *not* recommended for standard 30 s IGS Final clocks, because the polynomial oscillates between sparse samples. It is kept for high-rate input only.

8.4 Broadcast Keplerian evaluation

When no precise product spans the query, GPS, Galileo, BeiDou and QZSS are evaluated from their Keplerian broadcast ephemeris (`src/core/products/LiveEphemerisStore.cpp`, `keplerPosition`). The live SSR stream corrects this evaluation. Each system carries its own gravitational parameter μ and Earth-rotation rate $\omega_\oplus$ (`LiveEphemerisStore.cpp`): GPS / QZSS use $\mu = 3.986005 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$; Galileo / BeiDou use $\mu = 3.986004418 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, with BeiDou taking the slightly different $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$. BeiDou is evaluated on its native BeiDou Time (BDT) scale, where BDT week 0 is GPS week 1356 and BDT = GPS − 14 s. Both the query epoch and the stored t_{oe} stay on BDT, so the node-rotation term below uses BDT seconds-of-week (`LiveEphemerisStore.cpp`).

With the semi-major axis $a = (\sqrt{a})^2$ and the time-from-ephemeris $t_k = t - t_{oe}$ reduced into $[-302400, +302400]\,\mathrm{s}$ (week rollover), the corrected mean motion adds the broadcast δn to the Keplerian value, and the mean anomaly propagates linearly:

$$n = \sqrt{\frac{\mu}{a^3}} + \delta n, \qquad M = M_0 + n\,t_k. \tag{65}$$

Kepler's equation $M = E - e\sin E$ is solved by Newton iteration (up to 15 steps, convergence at $|\Delta E| < 10^{-13}$):

$$E_{k+1} = E_k + \frac{M - \big(E_k - e\sin E_k\big)}{1 - e\cos E_k}. \tag{66}$$

The true anomaly and the argument of latitude follow, then the broadcast second-harmonic corrections refine the argument of latitude u, the orbital radius r, and the inclination i:

$$\nu = \operatorname{atan2}\big(\sqrt{1-e^2}\,\sin E,\ \cos E - e\big), \qquad \phi = \nu + \omega, \tag{67}$$

$$\begin{aligned} u &= \phi + c_{us}\sin 2\phi + c_{uc}\cos 2\phi,\\ r &= a\big(1 - e\cos E\big) + c_{rs}\sin 2\phi + c_{rc}\cos 2\phi,\\ i &= i_0 + \dot{i}\,t_k + c_{is}\sin 2\phi + c_{ic}\cos 2\phi. \end{aligned} \tag{68}$$

The corrected longitude of the ascending node includes the node rate *relative to* the rotating Earth and the fixed node offset at the ephemeris epoch:

$$\boxed{\Omega = \Omega_0 + \big(\dot{\Omega} - \omega_\oplus\big)\,t_k - \omega_\oplus\,t_{oe},} \tag{69}$$

where the final $-\omega_\oplus\,t_{oe}$ uses the seconds-of-week of t_{oe} on the native scale (`LiveEphemerisStore.cpp`). With the in-plane position $(x_o, y_o) = (r\cos u,\ r\sin u)$ the ECEF position is the standard rotation through Ω and i:

$$\boldsymbol{r}^i = \begin{bmatrix} x_o\cos\Omega - y_o\cos i\,\sin\Omega \\ x_o\sin\Omega + y_o\cos i\,\cos\Omega \\ y_o\sin i \end{bmatrix}. \tag{70}$$

The additional fixed rotation for BeiDou GEO satellites is not applied, because GEO satellites are rarely used for PPP-RTK. Velocity is a central finite difference of (70), $\dot{\boldsymbol{r}}^i \approx [\boldsymbol{r}^i(t+\tfrac{1}{2}) - \boldsymbol{r}^i(t-\tfrac{1}{2})]/(2\,\Delta t)$ with $\Delta t = 0.5\,\mathrm{s}$, using the same position evaluator (`LiveEphemerisStore.cpp`).

8.5 Broadcast GLONASS: PZ-90 RK4 integration

GLONASS does not broadcast Keplerian elements. It broadcasts an ECEF state vector (position, velocity, and a luni-solar acceleration) valid at t_b, and the ICD prescribes numerical integration of the PZ-90 equations of motion to the query epoch [6]. The engine uses a fixed-step fourth-order Runge–Kutta integrator (`src/core/products/LiveEphemerisStore.cpp`, `glonassState`). The per-axis acceleration combines the central gravitational term, the J_2 oblateness perturbation, the centrifugal and Coriolis terms of the rotating frame, and the broadcast luni-solar acceleration held constant over the integration. Writing $r = \|\boldsymbol{r}^i\|$ and the oblateness coefficient

$$\kappa_{J_2} = \tfrac{3}{2}\, J_2\, \frac{\mu\, a_e^2}{r^5}, \qquad J_2 = 1.0826257 \times 10^{-3}, \tag{71}$$

(with $\mu = 3.9860044 \times 10^{14}\,\mathrm{m^3/s^2}$, $a_e = 6378136\,\mathrm{m}$, $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$, all PZ-90 constants at `LiveEphemerisStore.cpp`), the accelerations are

$$\boxed{\begin{aligned} \ddot{x} &= -\frac{\mu}{r^3}x + \kappa_{J_2}\, x\big(1 - 5\,z^2/r^2\big) + \omega_\oplus^2 x + 2\omega_\oplus \dot{y} + a_x, \\ \ddot{y} &= -\frac{\mu}{r^3}y + \kappa_{J_2}\, y\big(1 - 5\,z^2/r^2\big) + \omega_\oplus^2 y - 2\omega_\oplus \dot{x} + a_y, \\ \ddot{z} &= -\frac{\mu}{r^3}z + \kappa_{J_2}\, z\big(3 - 5\,z^2/r^2\big) + a_z, \end{aligned}} \tag{72}$$

where (a_x, a_y, a_z) is the broadcast luni-solar acceleration. The oblateness term on the z axis has the factor $3 - 5z^2/r^2$, against $1 - 5z^2/r^2$ on the in-plane axes. The centrifugal $\omega_\oplus^2$ and Coriolis $2\omega_\oplus$ terms appear only on the in-plane axes, as the PZ-90 rotating frame requires. The integrator uses a maximum step of $h = 30\,\mathrm{s}$, signed toward the query, and shortens the last step so that it ends on t_q. Each step is the classical RK4 update $y \leftarrow y + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ (`LiveEphemerisStore.cpp`).

8.6 Broadcast satellite clock

The broadcast satellite clock for the Keplerian systems is the standard second-order polynomial in the time-from-clock-epoch $t - t_{oc}$ (on the native scale, with BeiDou on BDT as for the orbit node term):

$$\boxed{\delta t^i = a_{f0} + a_{f1}\,(t - t_{oc}) + a_{f2}\,(t - t_{oc})^2.} \tag{73}$$

GLONASS instead uses its two-term broadcast model with the 1020-message τ_n (DF123) and γ_n (DF120) about $t_b = t_{oe}$:

$$\delta t^i = -\tau_n + \gamma_n\,(t - t_b) \tag{74}$$

(`LiveEphemerisStore.cpp`). The shipped real-time replay configurations exclude GLONASS from `active_systems`, so they do not use this path. The default `active_systems` and the post-processing configurations enable GLONASS and do use it.

The precise-clock-baseline convention. The broadcast clock (73) *omits* both the periodic relativistic eccentricity term and the group delay (TGD/BGD). This is intentional. The IGS-RTS clock correction is generated against the bare polynomial, so the engine must *not* apply the periodic relativity or TGD before adding the SSR delta. Doing so would count them twice. The periodic relativistic term $-2\,\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i / c^2$ is added later, in the observation-physics path; see Section 11 for its derivation. With this convention the orbit, the bare clock, and the SSR overlay all refer to the same ephemeris.

8.7 Ephemeris selection and IOD bookkeeping

The broadcast paths assume that the correct ephemeris record has been selected. The live store holds a `current` and an optional `previous` record per satellite. The rules below decide which one is evaluated (`src/core/products/LiveEphemerisStore.cpp`).

Freshest-toe within a source. Within one source, the record with the newer reference epoch is kept. An incoming record with an older t_{oe} (a stale re-broadcast) is rejected. A record with the same epoch, or one differing by less than a second, overwrites the held record (`LiveEphemerisStore.cpp`). At query time the store chooses between `current` and `previous` by *closest* t_{oe} on a common GPS-continuous scale. BeiDou's BDT t_{oe} is first converted to GPS seconds (`toeGpsContinuous`). Without that, every BeiDou record would appear $\sim$1356 weeks old and be skipped.

IGS-source authority over receiver nav. When the engine ingests both an IGS broadcast stream and the receiver's own decoded navigation, the IGS record takes priority. The IGS-RTS SSR delta is computed against the IGS broadcast ephemeris, so the rover must pair the delta with the *same* ephemeris. Even at an identical IOD, the receiver's locally assembled orbit has slightly different fit parameters: a live Galileo I/NAV assembly was observed to differ by more than 100 m. Applying IGS-SSR to a receiver ephemeris therefore corrupts the single-difference observations. A receiver record never overwrites a current IGS record with an equal or older t_{oe}. It is accepted only when it is newer than the IGS record by more than one constellation refresh (1800 s, the GLONASS nominal step), which indicates a *stalled* IGS feed. A dead IGS stream then falls back to receiver navigation and the satellite stays usable (`LiveEphemerisStore.cpp`).

IODE-change demote and SSR re-referencing. When the IOD changes, the `current` record becomes `previous` and is not discarded, so the superseded orbit remains available across the cutover (`LiveEphemerisStore.cpp`). This is needed because the receiver can hold an ephemeris one version ahead of the SSR stream. When an SSR delta names an IOD, `pickForSsr` returns whichever held record (`current` or `previous`) matches it. The orbit and clock are then evaluated on the ephemeris the correction was computed for, and the satellite is not dropped to broadcast accuracy because `current` and the SSR disagree. Keeping both records also supports a second use of the broadcast clock polynomial (73): `LiveEphemerisStore::ephCutoverShiftMetres` evaluates $c\,(\delta t^{i}_{\text{new}} - \delta t^{i}_{\text{old}})$ across an ephemeris version change (`computeSatelliteClock` run once on the freshest pick and once pinned to the old IODE), and returns nothing when the old version is no longer held. The network producer's `network.apriori_source = brdc_rts` eph-cutover re-level uses this value. That re-level is derived in Section 20.

The GNSS-IOD matching convention. The SSR "GNSS IOD" field (IGS-SSR / RTCM `4076`, IDF012) is an 8-bit quantity whose definition differs by constellation and differs from the raw broadcast IODE stored with the ephemeris. `ssrMatchIode` converts the held record's IODE to the SSR convention, in one place (`LiveEphemerisStore.cpp`):

$$\mathrm{IOD}_{\mathrm{SSR}}(i) = \begin{cases} \text{IODE} & \text{GPS, QZSS (8-bit LNAV, passthrough),} \\ \text{IODnav \& } \texttt{0xFF} & \text{Galileo (low 8 bits of the 10-bit I/NAV IODnav),} \\ \left\lfloor t_{oe}^{\mathrm{sow}}/720 \right\rfloor \bmod 240 & \text{BeiDou (toe-derived, per IGS-SSR v1.00),} \end{cases} \tag{75}$$

with t_{oe}^{sow} the BDT seconds-of-week. The BeiDou broadcast AODE that the decoder stores in the IODE field is a *different* quantity and never matches the SSR field, so the toe-derived value is used

instead. The raw IODE is kept for the record bookkeeping above; only the value passed to the SSR gate is converted. Section 18.3 describes how that value gates the SSR orbit and clock delta: the delta is accepted only on an exact IOD match, and otherwise the satellite falls back to broadcast accuracy.

9 Atmospheric Delay Modelling: Troposphere and Ionosphere

The undifferenced code and phase equations, Equation (4) and Equation (5) of the Foundations document, each contain two propagation terms that are computed from *models* rather than estimated directly: the slant tropospheric delay T_r^i and the first-order slant ionospheric delay $\mu_s I_r^i$. The troposphere has an a-priori *zenith hydrostatic* part, which is computed and removed before the filter sees the measurement. The small *zenith wet* residual and two horizontal gradients are estimated (Section 3, Equation (3)). In the uncombined parameterisation the ionosphere is one slant state per satellite (Section 3.3). The models in this chapter supply its a-priori seed, an optional pseudo-observation that ties the relative slants to a Global Ionosphere Map, and the small second- and third-order corrections that the $1/f^2$ scaling does not cover. The network producer also co-estimates its own spherical-harmonic VTEC field on one thin shell, or optionally two (Section 9.2.4), so it needs no external GIM. The column layout of that field in the network normal equations is in Section 20. This document derives the mapping functions, the pressure reductions, the thin-shell geometry, the geomagnetic field, and the sign with which each term enters the modelled range.

9.1 The slant tropospheric delay

The neutral atmosphere is non-dispersive at GNSS frequencies. Its delay is the same on code and phase, and no ionosphere-free or geometry-free combination removes it. The standard decomposition splits the slant delay into a hydrostatic ("dry") and a wet zenith delay, each carried to the line of sight by its own elevation mapping function, plus an azimuthal gradient [23]:

$$\boxed{T_r^i = m_{\mathrm{h}}(\varepsilon)\,\mathrm{ZHD} + m_{\mathrm{w}}(\varepsilon)\,\mathrm{ZWD} + m_{\mathrm{g}}(\varepsilon)\big(G_{\mathrm{N}}\cos\alpha + G_{\mathrm{E}}\sin\alpha\big).} \tag{76}$$

ZHD is supplied a priori and removed from the prefit residual. ZWD, G_{N} and G_{E} are estimated states whose Jacobian columns are the mapping coefficients $m_{\mathrm{w}}(\varepsilon)$ and $m_{\mathrm{g}}(\varepsilon)\{\cos\alpha, \sin\alpha\}$ (`src/core/measurement/MeasurementModel.cpp`; Equation (3)). The rest of this subsection derives each factor.

9.1.1 Saastamoinen zenith hydrostatic delay

The total surface pressure determines the zenith hydrostatic delay to better than a millimetre, through the Saastamoinen reduction [22] as coded in `src/core/atmosphere/Troposphere.cpp`:

$$\boxed{\mathrm{ZHD} = \frac{0.0022768\ P}{1 - 0.00266\cos 2\phi - 0.00028\,h_{\mathrm{km}}},} \tag{77}$$

with P the total pressure in millibar at the antenna, ϕ the geodetic latitude, and $h_{\mathrm{km}} = h/1000$ the ellipsoidal height in kilometres (`Troposphere::computeSaastamoinenZHD`, denominator `f_lat`). The coefficient $0.0022768\ \mathrm{m\,mbar^{-1}}$ combines the refractivity constant $k_1 = 77.604\ \mathrm{K\,mbar^{-1}}$ with the dry-gas constant and standard gravity. The denominator $f(\phi, h)$ accounts for the variation of gravity with latitude and height.

When neither a surface met sensor nor the GPT3 grid (below) is available, P is taken from a standard-atmosphere barometric reduction of a configured sea-level pressure P_0 and temperature T_0 (`Troposphere.cpp`, `pressureFromHeightStdAtm`):

$$P(h) = P_0 \left(1 - \frac{L\,h}{T_0}\right)^{g/(RL)}, \qquad L = 0.0065\ \mathrm{K\,m^{-1}}, \quad \frac{g}{RL} \approx 5.25588, \tag{78}$$

with $g = 9.80665\ \mathrm{m\,s^{-2}}$ and $R = 287.05\ \mathrm{J\,kg^{-1}\,K^{-1}}$ the specific gas constant of dry air. The base $(1 - Lh/T_0)$ is clamped to $[0.1, 2.0]$ and the result to $[100, 1100]$ mbar as a numerical guard.

9.1.2 The Marini mapping form

Both the hydrostatic and wet mappings use the normalised three-parameter continued fraction of Marini, normalised to unity at zenith (`Troposphere::mariniMapping`):

$$\boxed{m(\varepsilon;\, a, b, c) = \cfrac{1 + \cfrac{a}{1 + \cfrac{b}{1 + c}}}{\sin\varepsilon + \cfrac{a}{\sin\varepsilon + \cfrac{b}{\sin\varepsilon + c}}}.} \tag{79}$$

The numerator is the denominator evaluated at $\varepsilon = 90^\circ$ ($\sin\varepsilon = 1$), so $m(90^\circ) = 1$. The hydrostatic and wet mappings, and the different models, differ only in the triple (a, b, c). Two families are implemented.

Niell / GMF coefficients. The default mapping interpolates the Niell tables [16] of (a, b, c) given at five reference latitudes $15^\circ, 30^\circ, 45^\circ, 60^\circ, 75^\circ$, each as a mean plus an annually varying amplitude (the coefficient tables inside `Troposphere::getTroposphereMapping`). The seasonal value is

$$v(\phi, t) = \bar{v}(\phi) - \delta v(\phi)\,\cos\frac{2\pi\,(\mathrm{doy} - \mathrm{doy}_0)}{365.25}, \qquad \mathrm{doy}_0 = \begin{cases} 28 & \phi \geq 0 \\ 28 + 182.625 & \phi < 0 \end{cases} \tag{80}$$

(`Troposphere::getTroposphereMapping`, over the day of year returned by `Troposphere::getDayOfYear`). Two details matter. The amplitude term is *subtracted*. The Southern Hemisphere is put in anti-phase by adding half a year to the phase reference, not by changing the sign. The latitude argument is the absolute latitude, linearly interpolated and clamped to $[15^\circ, 75^\circ]$. The wet coefficients have mean values only, with no annual term.

A height correction lifts the hydrostatic mapping from sea level to the station, identical in the Niell and VMF3 paths (`Troposphere::getTroposphereMapping`):

$$m_\mathrm{h}(\varepsilon, h) = m_\mathrm{h}(\varepsilon, 0) + \left[\frac{1}{\sin\varepsilon} - m(\varepsilon;\, a_\mathrm{ht}, b_\mathrm{ht}, c_\mathrm{ht})\right] h_\mathrm{km}, \qquad \begin{aligned} a_\mathrm{ht} &= 2.53 \times 10^{-5}, \\ b_\mathrm{ht} &= 5.49 \times 10^{-3}, \\ c_\mathrm{ht} &= 1.14 \times 10^{-3}, \end{aligned} \tag{81}$$

with the same Marini form (79) evaluated at the height triple. Only the hydrostatic mapping receives this correction.

VMF3 coefficients. When configured for VMF3 [12], the wet/hydrostatic a-coefficients come from the GPT3 grid (next paragraph) and the b, c coefficients are synthesised on the fly by a degree- and order-12 spherical-harmonic expansion of their seasonal columns (`src/core/atmosphere/Vmf3.cpp`). A Cartesian-form associated Legendre recursion builds the real and imaginary solid harmonics V_{nm}, W_{nm} at the station co-latitude and longitude, the $91 = \sum_{n=0}^{12}(n+1)$ harmonic terms are accumulated against the stored `Vmf3Coeffs` tables, and each of $b_\mathrm{h}, c_\mathrm{h}, b_\mathrm{w}, c_\mathrm{w}$ is then evaluated as a five-coefficient mean-plus-annual-plus-semiannual harmonic, with all three steps inside `Vmf3::computeMapping`:

$$v(\mathrm{doy}) = A_0 + A_1\cos\omega + B_1\sin\omega + A_2\cos 2\omega + B_2\sin 2\omega, \qquad \omega = \frac{2\pi\,\mathrm{doy}}{365.25}. \tag{82}$$

The synthesised (a, b, c) enter the same Marini fraction (79), followed by the height correction (81). If the grid query fails the code falls back to the Niell path without a message, so one bad sample does not stop the run.

9.1.3 GPT3 empirical pressure, temperature and VMF3 a-coefficients

The GPT3 5° grid [12] supplies, per cell, the five harmonic coefficients of pressure, temperature, temperature lapse rate, geoid undulation, and the VMF3 hydrostatic and wet a-coefficients (`Gpt3::load`, in `atmosphere/Gpt3`). Each quantity is the same annual-plus-semiannual harmonic as (82) evaluated per cell (`Gpt3::harmonic`):

$$V(\mathrm{doy}) = a_0 + A_1\cos\omega + B_1\sin\omega + A_2\cos 2\omega + B_2\sin 2\omega. \tag{83}$$

The grid value sits at the cell's orthometric reference height H_s; it is carried to the station orthometric height $h_\mathrm{ortho} = h - N$ (undulation N) by a barometric pressure reduction and a linear temperature lapse (`Gpt3::query`):

$$P_\mathrm{stn} = P_\mathrm{grid}\big(1 - K_\mathrm{P}\,\Delta h\big)^{n_\mathrm{P}}, \quad T_\mathrm{stn} = T_\mathrm{grid} + \frac{\mathrm{d}T}{\mathrm{d}z}\,\Delta h, \qquad \begin{aligned} K_\mathrm{P} &= 2.26\times10^{-5}\,\mathrm{m}^{-1}, \\ n_\mathrm{P} &= 5.225, \end{aligned} \tag{84}$$

with $\Delta h = h_\mathrm{ortho} - H_s$ and $\mathrm{d}T/\mathrm{d}z$ the grid lapse in $\mathrm{K\,km^{-1}}$ applied to Δh in kilometres. The exponent 5.225 is the same $g/(RL)$ ratio as (78), rounded for the grid. Four corners are read and combined by bilinear interpolation in latitude and (wrapped) longitude (also `Gpt3::query`); the resulting P_stn drives the Saastamoinen ZHD of (77), and the interpolated $a_\mathrm{h}, a_\mathrm{w}$ drive the VMF3 mapping.

9.1.4 The gradient mapping and a degenerate fallback

The azimuthal gradient in (76) uses the Chen–Herring mapping [3], coded identically in the Jacobian and the prefit residual (`MeasurementModel::buildJacobianRow`, `StateInitializer::computeRawResidual`):

$$\boxed{m_\mathrm{g}(\varepsilon) = \frac{1}{\sin\varepsilon\tan\varepsilon + 0.0032}, \qquad T^\mathrm{grad} = m_\mathrm{g}(\varepsilon)\big(G_\mathrm{N}\cos\alpha + G_\mathrm{E}\sin\alpha\big).} \tag{85}$$

The partial derivatives $\partial\mathrm{T}/\partial G_\mathrm{N} = m_\mathrm{g}\cos\alpha$ and $\partial\mathrm{T}/\partial G_\mathrm{E} = m_\mathrm{g}\sin\alpha$ are the Jacobian columns of the north and east gradient states (Section 3).

A cold-start guard applies to the *elevation* (hydrostatic and wet) mapping, not to the gradient mapping. When the receiver position is not yet usable (norm below 10^3 m, or non-finite), the

geodetic latitude and height needed by the Niell and VMF3 coefficients are unavailable. The hydrostatic and wet mappings then use the geometry-only Black–Eisner form $1/(\sin\varepsilon + a/(\tan\varepsilon + b))$ (the fallback branch of `Troposphere::getTroposphereMapping`), which keeps the mapping defined during cold start.

9.2 The slant ionospheric delay

The ionosphere is dispersive. To first order it advances the phase and delays the code by the same amount, $40.3\,\mathrm{STEC}/f^2$, where STEC is the slant total electron content in $\mathrm{el\,m^{-2}}$. The uncombined model carries one slant state I_r^i per satellite, referenced to GPS L1, and scales it to each signal by the dispersion ratio.

9.2.1 The $1/f^2$ scaling and its sign

For a signal of frequency f on a state referenced to $f_{\mathrm{ref}} = f_{\mathrm{L1}}^{\mathrm{GPS}}$, the dispersion coefficient is

$$\boxed{\mu_s = \frac{f_{\mathrm{ref}}^2}{f_s^2}, \qquad \frac{\partial P_s}{\partial I} = +\mu_s, \qquad \frac{\partial \Phi_s}{\partial I} = -\mu_s,} \tag{86}$$

so the slant-iono Jacobian column is $+\mu_s$ on code and $-\mu_s$ on phase (`MeasurementModel::buildJacobianRow`). This defines the μ_s of Equation (1). Every constellation's slant state is referenced to GPS L1, which lets the between-satellite and between-receiver differencing operators (Section 5) cancel the receiver iono datum. Because $f_{\mathrm{ref}} = f_{\mathrm{L1}}$, the L1 signal has $\mu = 1$ and the state is the L1 slant delay in metres.

9.2.2 IONEX / GIM thin-shell slant delay

A Global Ionosphere Map gives vertical TEC on a latitude/longitude grid at a single thin shell of height H above a base radius R_E. The slant delay along a ray at elevation ε follows from single-layer geometry (`computeIppGeometry` in `src/core/atmosphere/SphericalHarmonicIonosphere.cpp`, the shared helper that `GlobalIonosphereMap::slantIonoDelay` and the co-estimated spherical-harmonic VTEC model both call so the two never drift). The zenith angle z' at the ionospheric pierce point (IPP) obeys

$$\sin z' = \frac{R_E}{R_E + H}\cos\varepsilon, \qquad \text{mapping } M(\varepsilon) = \frac{1}{\cos z'}, \tag{87}$$

so that $\mathrm{STEC} = M(\varepsilon)\,\mathrm{VTEC}$ (`GlobalIonosphereMap::slantIonoDelay`). The earth-centred angle to the IPP is $\psi = \frac{\pi}{2} - \varepsilon - z'$, and the IPP latitude and longitude come from the spherical-triangle solution along the satellite azimuth α (`computeIppGeometry`). The helper treats the receiver's geodetic latitude ϕ_r as geocentric when crossing the thin shell. The approximation is worth well under a millimetre of L1 delay. The higher-order model (105) later uses the IPP in the same frame:

$$\phi_{\mathrm{IPP}} = \arcsin\big(\sin\phi_r\cos\psi + \cos\phi_r\sin\psi\cos\alpha\big), \tag{88}$$

$$\lambda_{\mathrm{IPP}} = \lambda_r + \arcsin\frac{\sin\psi\,\sin\alpha}{\cos\phi_{\mathrm{IPP}}}. \tag{89}$$

The VTEC at the IPP is interpolated bilinearly in space and, in time, *sun-fixed*: each bracketing map is sampled at the IPP longitude rotated by the Earth's rotation between the map epoch and the query epoch, $\Delta\lambda = (t - t_{\mathrm{map}}) \cdot 360^\circ/86400$ s, before the two are linearly blended

(`GlobalIonosphereMap::interpolateVtec`, over `GlobalIonosphereMap::sampleMap`). The slant delay in metres at frequency f then uses the dispersion constant

$$\boxed{I^{\mathrm{GIM}} = \mathrm{STEC} \cdot \frac{40.3 \times 10^{16}}{f^2} \quad [\mathrm{m}], \qquad \mathrm{STEC} = M(\varepsilon)\,\mathrm{VTEC}\ [\mathrm{TECU}],} \tag{90}$$

the factor 10^{16} converting VTEC from TECU to $\mathrm{el\,m^{-2}}$ and $40.3 = 40.3\ \mathrm{m\,Hz^2\,(el\,m^{-2})^{-1}}$ the first-order refractivity constant (both the constant and the metre conversion live in `GlobalIonosphereMap::slantIonoDelay`).

9.2.3 The GIM iono-weighting pseudo-observation

The GIM is too coarse to constrain the *absolute* slant of one satellite, but it constrains the *relative* slants across a constellation well. The constraint is applied as a Kalman pseudo-observation (`Application::applyGimIonoUpdate`), one row per satellite that owns a slant state above the elevation cutoff. The innovation is the GIM slant minus the current state,

$$z_i = I_i^{\mathrm{GIM}} - I_r^i, \qquad \boldsymbol{H} = \boldsymbol{I}\ \text{(selection of the iono columns)}, \tag{91}$$

with the GIM evaluated at GPS L1 to match the state reference (`Application::applyGimIonoUpdate`). The measurement-noise matrix carries the structure. Per constellation g, with $\mathbf{1}$ the all-ones vector over that constellation's rows,

$$\boxed{\boldsymbol{R} = \bigoplus_{g \in \mathcal{G}} \Big(\mathrm{diag}(\sigma_i^2) + \sigma_c^2\, \mathbf{1}\mathbf{1}^{\mathrm{T}} \Big),} \tag{92}$$

(also `Application::applyGimIonoUpdate`). The rank-one $\sigma_c^2\,\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block gives a very large variance to the per-constellation *common mode* of the innovations. That common mode is the receiver iono datum, an arbitrary offset shared by all satellites of a constellation, which no model can supply. The pseudo-observation therefore leaves it free and constrains only the between-satellite differences, which the GIM does describe. The update uses the same Joseph-form pseudo-update as every other measurement (Section 12). The iono process noise is documented there (Equation (149)).

Why frozen pivots are excluded. In the BSSD parameterisation each constellation's pivot satellite holds its slant-iono state frozen at value 0 with variance 0 as the rank-fixing datum. Such a row is skipped entirely (the frozen-pivot guard in `Application::applyGimIonoUpdate`). A zero-variance state receives zero Kalman gain, so the GIM cannot improve it. It does share the common-mode $\mathbf{1}\mathbf{1}^{\mathrm{T}}$ block of (92) with the other satellites of its constellation. If it were included, its innovation $I_{p_g}^{\mathrm{GIM}} - 0 \sim 5$–$30$ m would enter the gain of every other satellite through the off-diagonal of $\boldsymbol{S}^{-1}$ and shift the relative slants by the pivot's full absolute STEC. Excluding the pivot row is required for a correct result. The common-mode block absorbs the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

9.2.4 The co-estimated spherical-harmonic VTEC field

The network producer can estimate the vertical TEC field itself instead of importing a map. The field is a truncated real spherical-harmonic expansion on the same kind of thin shell, solved jointly with the satellite code biases. The coefficients are *shared* columns of the network normal equations, registered through `NetworkEstimator::registerVtecCoeff` and reduced onto the shared border

like any other network-wide family (Section 20). This section gives the model, its pseudo-observation, and its datum.

The switch is `network.estimate_iono_sh`. It defaults to false, but the standing network configuration arms it and most shipped network configurations set it, so it is the ionosphere model the producer normally runs. Arming it forces `network.estimate_sat_code_bias` on, because the field is what makes the satellite code biases observable. It also suppresses external GIM loading, so the two ionosphere sources are never applied to the same run (`config/GnssConfigLoadNetwork.cpp`; the SH-versus-GIM branch guards the `networkAppendGimIonoRows_` call site in `app/ApplicationNetworkAssembly`).

Basis and normalisation. With β the geographic latitude of the pierce point and s its sun-fixed longitude, the field is

$$\boxed{\mathrm{VTEC}(\beta, s) = \sum_{n=0}^{N} \sum_{m=0}^{n} \bar{P}_{nm}(\sin\beta)\,\big[a_{nm}\cos(ms) + b_{nm}\sin(ms)\big] \quad [\mathrm{TECU}],} \tag{93}$$

truncated triangularly at degree N (`estimation.global_vtec_sh.max_degree`), which gives $(N{+}1)^2$ real coefficients: one cosine term per $(n, m \geq 0)$ and one sine term per $(n, m \geq 1)$, enumerated once in a fixed order that the shared-column layout mirrors (`SphericalHarmonicIonosphere::coeffs`). The normalisation is

$$\bar{P}_{nm} = N_{nm}\,P_{nm}, \qquad N_{nm} = \sqrt{(2-\delta_{0m})\,(2n+1)\,\frac{(n-m)!}{(n+m)!}}, \tag{94}$$

chosen so the real surface harmonics $\{\bar{P}_{nm}\cos(ms),\ \bar{P}_{nm}\sin(ms)\}$ are orthonormal under $(1/4\pi)\int(\cdot)\,\mathrm{d}\Omega$ [23]. Because (93) is linear in the coefficients, the basis values are the partial derivatives. `SphericalHarmonicIonosphere::basis` returns $\partial\mathrm{VTEC}/\partial a_{nm}$ and $\partial\mathrm{VTEC}/\partial b_{nm}$ directly, and the Jacobian needs no numerical differentiation. One consequence is used repeatedly below: $\bar{P}_{00} \equiv 1$, so a_{00} is the global mean VTEC.

Frame and geometry. The expansion uses a sun-fixed frame, $s = \lambda_{\mathrm{IPP}} - \lambda_{\odot}$, with $\lambda_{\odot} = \alpha_{\odot} - \mathrm{GMST}$ the sub-solar geographic longitude (`meanSunLongitude`). In this frame the field changes slowly over a session; in a geographic frame it would move westward at the Earth's rotation rate. `estimation.global_vtec_sh.sun_fixed` selects geographic longitude instead. The pierce point and the thin-shell mapping $M(\varepsilon)$ that carries VTEC to STEC come from *the same* `computeIppGeometry` helper that `GlobalIonosphereMap::slantIonoDelay` calls, evaluated at `estimation.global_vtec_sh.layer_height_km` over base radius `earth_radius_km`. Equations (87), (88) and (89) are therefore also the geometry of the co-estimated model. The external map and the co-estimated field share one implementation so that they stay consistent.

The pin row. No measurement observes the field directly. One pseudo-observation per line of sight ties it to the estimated slant states (`Application::networkAppendShIonoRows_`). For every GPS or Galileo LOS above `ionosphere_weighting.elevation_cutoff_deg` whose slant state lives

in that station's local block, the row asserts that the slant state equals the field's prediction:

$$\boxed{\begin{aligned} & g_r^i \equiv I_r^i \;-\; \kappa\, M(\varepsilon) \sum_k \bar{Y}_k(\mathrm{IPP})\, c_k \;=\; 0, \qquad \kappa = \frac{40.3 \times 10^{16}}{f_{\mathrm{L1}}^2}, \\ & \frac{\partial g_r^i}{\partial I_r^i} = +1, \qquad \frac{\partial g_r^i}{\partial c_k} = -\,\kappa\, M(\varepsilon)\, \bar{Y}_k(\mathrm{IPP}). \end{aligned}} \tag{95}$$

Here c_k runs over the field coefficients, $\bar{Y}_k$ over the corresponding basis functions of (93), and κ is the TECU-to-L1-metre conversion already introduced in (90). The $+1$ goes in the station's private block and the $-\kappa M \bar{Y}_k$ terms go in the *shared* border. The second write is what turns an ionosphere prior into a joint estimate. The right-hand side is $-I_r^i$ at the current state, and the shared coefficients enter the absolute solve at zero offset.

The measurement noise on these rows is *the same* common-mode-free per-constellation block as the GIM pseudo-observation, (92), built from the same `ionosphere_weighting.common_mode_sigma_m` over a per-LOS representativeness sigma

$$\sigma_i = \max\bigl(\texttt{floor_m},\ \sigma_{\mathrm{rep}}\, M(\varepsilon)\, \kappa\bigr), \qquad \sigma_{\mathrm{rep}} = \texttt{global_vtec_sh.representation_sigma_tecu}, \tag{96}$$

after which the block is whitened by the Cholesky factor of $\boldsymbol{R}$ into unit-information rows before it joins the normal equations. (92) is reused here for a modelling reason. The rank-one $\sigma_c^2\, \mathbf{1}\mathbf{1}^{\mathrm{T}}$ block leaves the iono datum of each (receiver, constellation) pair free. The receiver DCB is part of that common mode, so the field constrains only the *between-satellite* slant structure, which is all a global VTEC surface can describe. As a result a_{00} is only weakly observable, through the spread of $M(\varepsilon)$ across the network, and the datum below has to handle that. GLONASS is excluded from the field (`networkAppendShIonoRows_` skips it). Its FDMA per-slot inter-frequency bias does not cancel in the common-mode block and would leak into the global coefficients.

The Kaula–Tikhonov prior as the estimability datum. The coefficients are shared columns. A coefficient whose pierce-point support has moved over an ocean or polar gap is nearly unobservable in the reduced normal equations and would fail the all-or-nothing rank gate (`NetworkSbasis::assertNullSpaceRank`). A degree-dependent prior is therefore applied to *every* coefficient before the gate runs (`networkInjectShVtecPrior`):

$$\boxed{\begin{aligned} & \sigma_n = \frac{\texttt{kaula_scale_tecu}}{n} \quad (n \geq 1), \qquad \sigma_0 = \texttt{mean_vtec_prior_sigma_tecu}, \\ & \bar{c}_k = \begin{cases} \texttt{mean_vtec_prior_tecu}, & c_k = a_{00}, \\ 0, & \text{otherwise}, \end{cases} \end{aligned}} \tag{97}$$

combined in information space with an optional random-walk anchor toward the previous epoch's estimate, inflated by a per-degree spectral density $q_n = \texttt{q_per_degree}/n^2$ for $n \geq 1$. The prior has two separate roles. The Kaula term provides *estimability*. It keeps every coefficient positive definite, so the declared null-space dimension equals the number of constellations whatever the coverage, and it pulls unobserved coefficients toward climatology each epoch so that they do not random-walk. The a_{00} term is a deliberately *loose* climatology anchor. a_{00} trades against the absolute satellite-DCB level, whose only rigorous datum is the per-(constellation, signal) sum-to-zero constraint of Section 20. Fixing a_{00} to an external mean VTEC would bring back the GIM dependency that co-estimation removes, and would bias every emitted satellite code bias by the difference.

Open extensions: default OFF, not yet field-validated. Two extensions of this model are built, off by default, and still under validation.

(i) A second thin shell (`estimation.global_vtec_sh.layer2_max_degree`, default 0 = single layer) registers a second coefficient set at its own height, and the pin row (95) then sums the shells, each evaluated at *its own* pierce geometry, $\sum_l \kappa\, M_l(\varepsilon)\, \bar{Y}_k(\mathrm{IPP}_l)\, c_{k,l}$ (`ShLayer`, assembled by `shLayers`, which every consumer of the field loops so a shell can never be evaluated at another shell's geometry). The second shell's a_{00} anchor is deliberately *tight*, for the opposite reason to the first shell's. Ground-only geometry barely observes how the mean VTEC divides between the layers: only the difference of the two mapping profiles is sensitive to it. Prior information must therefore hold the split, while the first shell's loose anchor leaves the *total* mean to the data.

(ii) A per-(station, satellite) representativeness deviation (`estimation.global_vtec_sh.rep_error_tau_s`, default 0 = off) replaces the pin-to-zero of (95) with a pin to a tracked first-order Gauss–Markov deviation d, carried in vertical TECU so its level and steady state are elevation-stationary. Writing d^-, P^- for the aged prior, $\hat{u}, \hat{V}$ for the solved posterior mean and variance of the pin functional, and f^2 for the pin-row floor variance, the exact scalar posterior is

$$\hat{d} = d^- + \frac{P^-}{a}\big(\hat{u} - d^-\big), \qquad P^+ = P^-\,\frac{U + f^2}{U + a}, \qquad a = P^- + f^2, \quad U = \frac{\hat{V}\,a}{a - \hat{V}}, \tag{98}$$

with U the data-only information recovered from $\hat{V}$ (`shDevExactUpdate`). Two treatments against ratcheting are built on it, also default OFF.

`rep_error_demean` (`shDevDemean`) removes the information-weighted global and per-station means of the carried priors once per epoch. A deviation component that is common in vertical TECU has row signature $\kappa\, M(\varepsilon)$, the same as the a_{00} column. The pair is therefore exactly quasi-null, and the cycle of harvesting, ageing, and re-asserting the prior ratchets it. The de-mean enforces the state's own contract, which is that it carries per-LOS structure only.

`rep_error_d5_keep` (`shDevD5Retap`) makes the slip response depend on which detector fired. A slip confirmed by a raw detector still *erases* the deviation, following the rule that a stiff time-linked carry must have a wired slip response (Section 15). A re-tap triggered only by the innovation gate keeps the level and resets the variance to the Gauss–Markov steady state.

The warm-LOS counts that arm the de-mean are constants in the seam, not configuration. The loader warns that both treatments are inert unless `rep_error_tau_s` is positive, and that arming the erase-policy split without the de-mean allows the ratchet again.

9.2.5 Klobuchar broadcast seed

The broadcast Klobuchar half-cosine model [11] is used only as a single-frequency a-priori: the cold-start seed of the slant state. It is never used as a dual-frequency correction (`Ionosphere::computeKlobucharDelay`). With elevation and azimuth in semicircles, the earth-centred angle, geomagnetic IPP latitude, local time and obliquity are

$$\psi = \frac{0.0137}{\varepsilon/180 + 0.11} - 0.022, \qquad \phi_i = \mathrm{clip}_{\pm 0.416}\big(\phi/180 + \psi\cos\alpha\big), \tag{99}$$

$$\phi_m = \phi_i + 0.064\cos\big((\lambda_i - 1.617)\pi\big), \qquad F = 1 + 16\,(0.53 - \varepsilon/180)^3, \tag{100}$$

with $\lambda_i = \lambda/180 + \psi\sin\alpha/\cos(\phi_i\pi)$ and local time $t_i = 43200\,\lambda_i + t \pmod{86400}$. Cubic polynomials in the geomagnetic latitude ϕ_m give the amplitude and period,

$$\mathrm{AMP} = \sum_{k=0}^{3} a_k\,\phi_m^k\ (\geq 0), \qquad \mathrm{PER} = \sum_{k=0}^{3} b_k\,\phi_m^k\ (\geq 72000\ \mathrm{s}), \tag{101}$$

and the zenith delay is the truncated half-cosine, mapped to slant by F and scaled to the requested frequency by the f_{L1}^2/f^2 dispersion ratio of (86) (the tail of `Ionosphere::computeKlobucharDelay`):

$$I^{\mathrm{Klob}} = c\,F\Big(5\times10^{-9} + \mathrm{AMP}\left[1 - \tfrac{X^2}{2} + \tfrac{X^4}{24}\right]\Big)\frac{f_{\mathrm{L1}}^2}{f^2}, \quad X = \frac{2\pi(t_i - 50400)}{\mathrm{PER}},\ |X| < 1.57, \tag{102}$$

and the constant 5×10^{-9} s night term alone when $|X| \geq 1.57$. The leading c converts the modelled signal delay from seconds to metres.

9.3 Higher-order ionospheric corrections

The $1/f^2$ scaling covers the first-order term. The second- and third-order delays scale as $1/f^3$ and $1/f^4$ and are applied as deterministic corrections to the modelled range (`HigherOrderIonosphere::slantHigherOrderComponents`). The second order is the magnetic-field–weighted delay

$$\boxed{I_2^P = 7527\,c\,\mathrm{STEC}\,\frac{\boldsymbol{B}\cdot\hat{\boldsymbol{k}}}{f^3}, \qquad I_2^\Phi = -\tfrac{1}{2}\,I_2^P,} \tag{103}$$

where STEC is in $\mathrm{el\,m^{-2}}$, $\boldsymbol{B}$ is the geomagnetic flux density in tesla at the IPP, $\hat{\boldsymbol{k}}$ the receiver→satellite unit vector, and c the speed of light. The constant $7527\,c = e\,A/(2\pi m_e) \approx 2.2564 \times 10^{12}$ combines the electron charge, the first-order refractivity constant A, and the electron mass. The constants and the $-\frac{1}{2}$ phase factor are both in `HigherOrderIonosphere::slantHigherOrderComponents`. The third order depends on the path integral of N_e^2, which a shape factor η relates to the F2-peak density and STEC (also `HigherOrderIonosphere::slantHigherOrderComponents`):

$$\boxed{I_3^P = 2437\,\eta\,N_{\max}\,\frac{\mathrm{STEC}}{f^4}, \qquad N_{\max} = \kappa\,\mathrm{STEC}, \qquad I_3^\Phi = -\tfrac{1}{3}\,I_3^P,} \tag{104}$$

with $\eta = 0.66$ the Hoque–Jakowski shape factor [7], $2437 = 3A^2/8$, and $N_{\max} = \kappa\,\mathrm{STEC}$ the linear Hoque–Jakowski fit ($\kappa = (20-6)\times10^{12}/((4.55-1.38)\times10^{18})$ $\mathrm{el\,m^{-3}/(el\,m^{-2})}$) of the F2-peak electron density to slant STEC. The third-order term is therefore *linear* in $N_{\max}\,\mathrm{STEC}$, not in STEC^2. Both corrections are below a millimetre at L1. They differ by satellite, so between-satellite differencing does not remove them, and each is applied with its own sign on code and on phase.

9.3.1 The geomagnetic dipole field

The field at the IPP is a centred, tilted magnetic dipole evaluated in ECEF (`MagneticField::fieldAtEcef`):

$$\boxed{\boldsymbol{B}(\boldsymbol{r}) = B_0\left(\frac{R_E}{r}\right)^3\big(\hat{\boldsymbol{m}} - 3(\hat{\boldsymbol{m}}\cdot\hat{\boldsymbol{r}})\,\hat{\boldsymbol{r}}\big),} \tag{105}$$

with $r = \|\boldsymbol{r}\|$, $\hat{\boldsymbol{r}} = \boldsymbol{r}/r$, $B_0 = 30\times10^{-6}$ T the equatorial surface field, $R_E = 6371.2$ km the IGRF reference radius, and $\hat{\boldsymbol{m}}$ the unit axis pointing to the geographic location of the geomagnetic north pole, IGRF-13 epoch 2020.0 at $(80.65°, -72.68°)$ (`MagneticField::dipoleAxisEcef`). The Arctic geomagnetic pole is the dipole's *south* pole. With $\hat{\boldsymbol{m}}$ pointing at it, (105) makes $\boldsymbol{B}$ point horizontally toward magnetic north at the magnetic equator, as a compass does. This convention sets the sign of $\boldsymbol{B}\cdot\hat{\boldsymbol{k}}$ and therefore of I_2 in (103). IONEX is a thin-shell model, so the IPP ECEF position

is rebuilt from the geocentric IPP latitude and longitude at the IONEX base-plus-shell radius (`HigherOrderIonosphere::slantHigherOrderComponents`).

The a-priori ZHD, the estimated ZWD and gradient states, the uncombined slant-iono state, and these corrections account for every atmospheric term of the undifferenced observation equation. Section 5 describes the operators that then remove the receiver-common parts.

10 Site Displacement and Geophysical Corrections

The receiver coordinate that enters Equation (7) and Equation (6) of the Foundations document is not the instantaneous position of the crust. It is the conventional tide-free station position, to which the IGS/ITRF products are referenced, plus a sum of *a priori* geophysical corrections. Each correction is a modelled 3D displacement $\boldsymbol{d}(t)$ of the antenna reference point. The ARP is the position state, which the engine obtains from the RINEX monument and the header eccentricity (Section 6.2). None of the corrections is estimated. All are computed from the a priori position (good to centimetres), the Sun and Moon ephemerides, and the Earth-orientation parameters. They are then projected onto the line of sight and added to the modelled range. This document derives the models as the IERS 2010 conventions [18] intend them, so it can be used to settle questions of sign and phase in the implementation.

Write the corrected receiver position as

$$\boldsymbol{r} = \boldsymbol{r}_0 + \boldsymbol{d}_{\mathrm{SET}} + \boldsymbol{d}_{\mathrm{OTL}} + \boldsymbol{d}_{\mathrm{PT}} + \boldsymbol{d}_{\mathrm{OPT}} + \boldsymbol{d}_{\mathrm{APL}}, \tag{106}$$

with $\boldsymbol{r}_0$ the conventional a priori position and each $\boldsymbol{d}$ an ECEF vector of millimetres to decimetres. The five terms are:

- the solid Earth tide $\boldsymbol{d}_{\mathrm{SET}}$, the largest, up to $\sim$30 cm radially;
- ocean tidal loading $\boldsymbol{d}_{\mathrm{OTL}}$, centimetres near coasts;
- the solid (rotational-deformation) pole tide $\boldsymbol{d}_{\mathrm{PT}}$, below a centimetre to a centimetre, secular plus Chandler;
- the ocean pole tide $\boldsymbol{d}_{\mathrm{OPT}}$, below a millimetre to $\sim$2 mm;
- atmospheric pressure loading $\boldsymbol{d}_{\mathrm{APL}}$, 2–20 mm.

Only the sun-synchronous S_1/S_2 thermal-tide part of the atmospheric loading is modelled. The non-tidal (synoptic) part is a known gap (Section 10.6). All five are computed in `src/core/corrections/GnssCorrections.cpp` and `src/core/corrections/OceanTideModel.cpp`. The LOS projection and the summation into the modelled range are in `src/core/model/ModelGeometry.cpp` and `src/core/model/PppModel.cpp`. Each term has a config flag (`enable_solid_earth_tide`, `enable_ocean_tide_loading`, `enable_pole_tide`, `enable_ocean_pole_tide`, `enable_atmospheric_loading`). With a flag off, that term is a zero vector.

10.1 Sign convention and projection into the range

Every displacement is projected onto the unit line of sight before it enters the modelled range. Let $\boldsymbol{e} = (\boldsymbol{r}^i - \boldsymbol{r})/\|\boldsymbol{r}^i - \boldsymbol{r}\|$ be the receiver-to-satellite unit vector (the convention of Equation (7); see `ModelGeometry.cpp` where `los = (sat_pos -rx_pos)/range`). A site displacement $\boldsymbol{d}$ changes the

geometric range by its component along the line of sight. The range shortens when the antenna moves *toward* the satellite. The scalar correction added to the modelled range is

$$\boxed{\delta\rho \;=\; \boldsymbol{d}\cdot(-\boldsymbol{e}) \;=\; \boldsymbol{d}\cdot\frac{\boldsymbol{r}-\boldsymbol{r}^{i}}{\|\boldsymbol{r}-\boldsymbol{r}^{i}\|},} \tag{107}$$

so a displacement toward the satellite (a positive component along $\boldsymbol{e}$) gives $\delta\rho < 0$. The ocean-loading, atmospheric-loading, and pole-tide projections use this form (`Geometry::computeSatelliteGeometry`: `otl_corr = otl_ecef.dot(-los)`, `apl_corr = apl_ecef.dot(-los)`; `PppModel::computePoleTideProjection`: `pole_tide_ecef.dot(-los)`), and the modelled range adds them in (`PppModel::computeBaseModeledValue` return sums `+ otl_corr + apl_corr + solid_tide` and the pair processors add `+ pole_tide_corr`). The solid-Earth-tide helper does its own projection with the same vector $(\boldsymbol{r}-\boldsymbol{r}^{i})/\|\boldsymbol{r}-\boldsymbol{r}^{i}\|$ (`GnssCorrections.cpp`, `computeSolidEarthTide`: `los = (receiver_pos -sat_pos)/rho_norm`, then `disp.dot(los)`), so all the tide terms use one sign convention. The rest of this document gives each model as a 3D ECEF (or local) displacement $\boldsymbol{d}$ and leaves Equation (107) implicit.

The Sun and Moon positions $\boldsymbol{r}^{\odot},\boldsymbol{r}^{\mathrm{m}}$ on which the body tides depend, and the ECI↔ECEF rotation and Earth-orientation parameters they require, come from `src/core/model/Planetary.cpp` and are documented in Section 6. Here $\boldsymbol{r}^{\odot},\boldsymbol{r}^{\mathrm{m}}$ are given geocentric ECEF vectors.

10.2 Solid Earth tide (IERS 2010, Steps 1 + 2)

The solid Earth tide is the elastic deformation of the whole Earth under the lunisolar tide-generating potential. The implementation is a port of the IERS Conventions `dehanttideinel` routine [18] and has two steps. Step 1 is evaluated in the time domain from the instantaneous Sun and Moon geometry. Step 2 is a frequency-domain correction for the few constituents whose Love and Shida numbers differ from the nominal degree-2 values. **The whole Step 1+2 block is validated against reference values**: it reproduces the official IERS `dehanttideinel` test vector and the in-tree POINT reference to below a millimetre, cross-checked by `scripts/solid_tide_reference.py` (`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement` and the `SolidEarthTideTest` suite). The permanent (zero-frequency) tide is *retained*, so the corrected coordinate is conventional tide-free, as in IGS/ITRF (see the routine's header comment).

10.2.1 Step 1: in-phase degree 2 and 3

For each tide-raising body $b\in\{\odot,\mathrm{m}\}$ at geocentric position $\boldsymbol{r}^{b}$ ($r_b=\|\boldsymbol{r}^{b}\|$) and the station at $\boldsymbol{r}_0$ ($r=\|\boldsymbol{r}_0\|$, geocentric latitude φ via direction cosines), the degree-2 in-phase displacement is built from the cosine of the body zenith angle $\cos Z_b=\hat{\boldsymbol{r}}_0\cdot\hat{\boldsymbol{r}}^{b}$ and a body scale factor

$$\mathrm{fac}_b \;=\; \frac{M_b}{M_\oplus}\,R_e\left(\frac{R_e}{r_b}\right)^{3}, \tag{108}$$

with $R_e=6378136.55$ m and mass ratios $M_\odot/M_\oplus=332945.943062$, $M_{\mathrm{m}}/M_\oplus=0.012300034$ (`GnssCorrections.cpp` tide-helper constants `kTideRe`/`kTideMassRatioSun`/`kTideMassRatioMoon`). The radial and tangential weights are

$$p_2^{b}=3\Big(\tfrac{h_2}{2}-l_2\Big)\cos^2 Z_b-\tfrac{h_2}{2}, \tag{109}$$

$$x_2^{b}=3\,l_2\cos Z_b, \tag{110}$$

(`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement`), where h_2, l_2 are the *latitude-dependent* degree-2 Love and Shida numbers of IERS Eq. 7.2,

$$h_2 = 0.6078 - 0.0006\,(1 - \tfrac{3}{2}\cos^2\varphi), \qquad l_2 = 0.0847 + 0.0002\,(1 - \tfrac{3}{2}\cos^2\varphi) \tag{111}$$

(`GnssCorrections.cpp`, `computeSolidEarthTideDisplacement`). An analogous degree-3 term uses fixed $h_3 = 0.292$, $l_3 = 0.015$ (`kTideH3`/`kTideL3`) and $\mathrm{fac}_b^{(3)} = \mathrm{fac}_b\,(R_e/r_b)$ (same routine). The Step-1 in-phase displacement sums the radial weight along the station unit vector and the tangential weight along each body's unit vector:

$$\boxed{\boldsymbol{d}^{(1)}_{\mathrm{SET}} = \sum_{b\in\{\odot,\mathrm{m}\}} \mathrm{fac}_b\Big(x_2^b\,\hat{\boldsymbol{r}}^b + p_2^b\,\hat{\boldsymbol{r}}_0\Big) \;+\; (\text{degree-3 analogue}),} \tag{112}$$

matching the per-component accumulation in `computeSolidEarthTideDisplacement`.

10.2.2 Step 1: out-of-phase anelasticity and the L^1 term

Mantle anelasticity makes the diurnal and semidiurnal degree-2 Love numbers complex; the small imaginary parts produce out-of-phase displacements `ST1IDIU` (diurnal, $\delta h = -0.0025$, $\delta l = -0.0007$) and `ST1ISEM` (semidiurnal, $\delta h = -0.0022$, $\delta l = -0.0007$) (`GnssCorrections.cpp`, helpers `st1Diurnal`, `st1SemiD`). The latitude (L^1) dependence of the Love numbers contributes a horizontal-only correction `ST1L1` with $l^{(1)}_{\mathrm{diu}} = 0.0012$, $l^{(1)}_{\mathrm{sem}} = 0.0024$ (helper `st1L1`). These three helpers return local radial/north/east displacements rotated into ECEF and are added to Equation (112) inside `computeSolidEarthTideDisplacement`. Their trigonometric forms follow the IERS reference and are not reproduced here.

10.2.3 Step 2: frequency dependence (Doodson tables)

Step 2 corrects the constituents whose actual Love numbers differ from the nominal degree-2 value, summing harmonic terms keyed by the Doodson fundamental arguments. The astronomical arguments τ, s, h, p, N', p_s (lunar time, Moon/Sun mean longitudes, lunar perigee, negative node, solar perigee) are polynomials in Julian centuries T from J2000, with the 15°/hr Earth-rotation term entering τ through the fractional UT hour H:

$$s = 218.31664563^\circ + 481267.88194\,T - 0.0014663889\,T^2 + \ldots, \tag{113}$$

$$\tau = 15^\circ H + 280.4606184^\circ + 36000.7700536\,T + \ldots - s, \tag{114}$$

with the remaining h, p, N', p_s polynomials given to fourth order in `GnssCorrections.cpp` (`computeAngles`). For each table line the phase is $\theta = \tau + n_s s + n_h h + n_p p + n_{N'} N' + n_{p_s} p_s (+\,\lambda$ for diurnal) and the in-/out-of-phase radial and tangential amplitudes (millimetres) are applied with a latitude weight.

Diurnal band (Table 7.3a). Thirty-one lines, dominated by K_1 and P_1 (`GnssCorrections.cpp`, `st2Diurnal`). The displacement for one line is

$$\begin{aligned} \delta R &= (A_R^{\mathrm{in}}\sin\theta + A_R^{\mathrm{out}}\cos\theta)\,2\sin\varphi\cos\varphi, \\ \delta N &= (A_T^{\mathrm{in}}\sin\theta + A_T^{\mathrm{out}}\cos\theta)\,\cos 2\varphi, \\ \delta E &= (A_T^{\mathrm{in}}\cos\theta + A_T^{\mathrm{out}}\sin\theta)\,\sin\varphi, \end{aligned} \tag{115}$$

(`st2Diurnal`), summed over the table and rotated radial/north/east → ECEF. The reference amplitudes (mm) include K_1 : (12.00, −0.80, −0.67, −0.03) and P_1 : (−1.23, −0.07, 0.06, 0.01) in the order $(A_R^{\mathrm{in}}, A_R^{\mathrm{out}}, A_T^{\mathrm{in}}, A_T^{\mathrm{out}})$ (the `st2Diurnal` table `T`).

Long-period band (Table 7.3b). Five zonal lines, with no longitude or τ dependence and no east component (`GnssCorrections.cpp`, `st2Lon`). The radial weight is the zonal Legendre form $(3\sin^2\varphi - 1)/2$ and the north weight is $\sin 2\varphi$:

$$\delta R = (A_R^{\text{in}} \cos\theta + A_R^{\text{out}} \sin\theta)\, \frac{3\sin^2\varphi - 1}{2}, \qquad \delta N = (A_T^{\text{in}} \cos\theta + A_T^{\text{out}} \sin\theta)\, \sin 2\varphi, \tag{116}$$

(`st2Lon`). The full solid-tide displacement is $\boldsymbol{d}_{\text{SET}} = \boldsymbol{d}^{(1)}_{\text{SET}}$ + ST1IDIU + ST1ISEM + ST1L1 + STEP2DIU + STEP2LON (`computeSolidEarthTideDisplacement`).

10.3 Ocean tidal loading (OTL)

Ocean tidal loading is the elastic response of the crust to the moving mass of the ocean tides. It reaches centimetres near coasts and is computed as a harmonic sum over the eleven principal constituents M2, S2, N2, K2, K1, O1, P1, Q1, Mf, Mm, Ssa (`OceanTideModel.hpp`, `OTL_CONSTITUENT_NAMES`). Each constituent k contributes, on each local component $c \in \{\text{up}, \text{west}, \text{south}\}$,

$$\boxed{dr_c(t) \;=\; \sum_{k=1}^{11} A_{c,k}\, \cos\!\big(\chi_k(t) - \phi_{c,k}\big),} \tag{117}$$

where $A_{c,k}$ is the station amplitude, $\phi_{c,k}$ the Greenwich phase lag, and $\chi_k(t)$ the astronomical argument (the eleven-constituent accumulation of `OceanTideModel::computeDisplacement`). The internal accumulation is in the $(\text{up}, \text{west}, \text{south})$ frame; the conversion to ENU is $(E, N, U) = (-\text{west}, -\text{south}, \text{up})$, then ENU→ECEF (same routine).

10.3.1 Astronomical argument and the Doodson–Warburg phase bias

The argument χ_k is assembled from Doodson multipliers $[\,\tau, s, h, p, N', p_s\,]$ acting on the variables $[\,\text{GMST} + \pi,\ s,\ h,\ p,\ N',\ p_s\,]$, themselves derived from the Delaunay fundamental arguments $s = F + \Omega,\ h = s - D,\ p = s - l,\ N' = -\Omega$ (`OceanTideModel::computeTidalArguments`). The BLQ Greenwich phase lags and the EOT20/FES complex coefficients are both referenced to a tide argument that carries a *species-dependent Doodson–Warburg phase bias*. For the diurnal band the bias is a quarter cycle ($\pm 90°$), because the Cartwright–Tayler–Edden tide-generating potential refers to a *sine* rather than a cosine when $(\text{degree} + n_\tau)$ is odd. The bias per constituent (cycles) is

M2	S2	N2	K2	K1	O1	P1	Q1	Mf	Mm	Ssa
0	0	0	0	+0.25	−0.25	−0.25	−0.25	0	0	0

(118)

applied as $\chi_k \mathrel{+}= 2\pi\,(\text{bias}_k)$ (the `CONSTITUENT_PHASE_BIAS_CYCLES` step of `OceanTideModel::computeTidalArguments`; the table itself in `OceanTideModel.hpp`). **Without this bias every diurnal constituent (K1/O1/P1/Q1) is wrong in phase by** 90°, and the whole diurnal OTL contribution is mismodelled. The bias must be present, with the signs of Equation (118). This settles the question of the diurnal OTL phase.

10.3.2 Coefficient sources: SIF, BLQ, and the EOT20 grid

The per-station amplitudes and phases come from the first available of four sources (`Geometry::loadData`, OTL coefficient block): a Station Information File (SIF), a station BLQ block, the global EOT20 grid, and finally zero.

The SIF gives the best coefficients. When `enableStationInfo()` is set and the SIF has an entry for the RINEX marker, it supplies per-station three-component FES2014b coefficients. The SIF stores Up/East/North amplitude and phase, while the model accumulates in Up/West/South (Equation (117)). Since East = −West and South = −North, the horizontal components keep their amplitude and shift phase by +180° (`d.phase_west = c.east_phase + 180`, `d.phase_south = c.north_phase + 180`). Up is unchanged.

Without a SIF entry the coefficients come from a station BLQ block. Without that, they are synthesised by bilinear interpolation of the global EOT20 complex load grid, which gives the radial component only. The grid is sampled at the RINEX header *marker* position (`RinexParser::getApproxPositionXyz`), not at the ARP position state. The difference is immaterial at this grid resolution, but the two points are distinct in the code. With no source the displacement is zero.

The EOT20 netCDF packs the real and imaginary load coefficients as `int16` with a `scale_factor`/`add_offset`. The netCDF C library does *not* apply these automatically, so the code unpacks them as $v = v_{\text{int16}} \cdot \text{scale} + \text{offset}$ (`OceanTideModel::parseEot20Load`: its `getScaleOffset` lambda and the manual unpack that follows). The scale is $\approx 5.34 \times 10^{-5}$ m/LSB, and omitting it inflates the amplitudes by ∼20000×. The complex load $Z = \text{Re} + i\,\text{Im}$ yields the real signal $h(t) = \text{Re}\{Z\, e^{i\chi}\} = A\cos(\chi - \phi)$ with

$$A = \sqrt{\text{Re}^2 + \text{Im}^2}, \qquad \phi = \text{atan2}(-\text{Im},\, \text{Re}) \tag{119}$$

(ϕ in BLQ format being the Greenwich phase lag in degrees; same routine). Because EOT20 orders its 17 constituents differently from the BLQ convention, the synthesis remaps the BLQ slot k to the EOT20 index via $\{4, 13, 8, 3, 2, 9, 10, 11, 6, 7, 15\}$ (the `EOT20_INDICES_FOR_BLQ` table in `OceanTideModel::parseEot20Load`).

10.4 Pole tide (rotational deformation, IERS 7.1.4)

The solid pole tide is the elastic deformation caused by the centrifugal potential of polar motion. As the rotation pole moves relative to its secular mean, the bulge it raises moves with it. The wobble is the polar-motion offset from the IERS 2018 secular mean pole. In milliarcseconds, with t_y the decimal year,

$$\bar{x}_p = 55.0 + 1.677\,(t_y - 2000), \qquad \bar{y}_p = 320.5 + 3.460\,(t_y - 2000) \tag{120}$$

(`GnssCorrections.cpp`, `computePoleTideCorrection`). The post-2018 IGS products used by the engine are referenced to this linear model. The wobble components (arcseconds) are

$$m_1 = x_p - \bar{x}_p, \qquad m_2 = -\big(y_p - \bar{y}_p\big), \tag{121}$$

with x_p, y_p the polar-motion values linearly interpolated from the ERP table at the query epoch (`computePoleTideCorrection`; the $\bar{x}_p, \bar{y}_p$ millarcsecond means are scaled to arcseconds in the difference). **The wobble must be in arcseconds, not radians.** The displacement coefficients below are in metres per arcsecond, so a wobble in radians scales the whole pole tide by $\sim 4.85 \times 10^{-6}$. The wobble comment in `computePoleTideCorrection` records this as a bug that was once present.

IERS 2010 Eq. 7.24 gives the rotational-deformation displacement in colatitude $\theta = 90° - \varphi$; substituting geodetic latitude ($\sin 2\theta = \sin 2\varphi$, $\cos 2\theta = -\cos 2\varphi$, $\cos\theta = \sin\varphi$, north $= -S_\theta$) yields the radial, north, and east components (metres):

$$\boxed{\begin{aligned} \boldsymbol{d}^R_{\text{PT}} &= -0.033\, \sin 2\varphi \,\big(m_1 \cos\lambda + m_2 \sin\lambda\big), \\ \boldsymbol{d}^N_{\text{PT}} &= -0.009\, \cos 2\varphi \,\big(m_1 \cos\lambda + m_2 \sin\lambda\big), \\ \boldsymbol{d}^E_{\text{PT}} &= +0.009\, \sin\varphi \,\big(m_1 \sin\lambda - m_2 \cos\lambda\big), \end{aligned}} \tag{122}$$

(`computePoleTideCorrection`). **The east factor is $\sin\varphi$ ($=\cos\theta$), not $\cos\varphi$.** A factor of $\cos\varphi$ is inconsistent with the radial and north terms, which use geodetic φ. It gives a scale error of $\sim$1.48$\times$ at mid-latitude and the wrong latitude dependence, vanishing at the equator instead of the pole. This settles the question of the pole-tide east term. The local (R, N, E) vector is rotated into ECEF at the end of `computePoleTideCorrection`.

10.5 Ocean pole tide (Desai 2002, IERS 7.1.5)

Polar motion also redistributes ocean mass. The load of this ocean pole tide deforms the crust by a further amount, from below a millimetre to $\sim$2 mm. It is added to Equation (122) when the station's six Desai grid load coefficients are available (the `has_opt_coeffs_` branch of `computePoleTideCorrection`). Following Desai (2002) [4], the same wobble m_1, m_2 drives a complex pole-tide coefficient

$$c^R = m_1\,\gamma_2^R + m_2\,\gamma_2^I, \tag{123}$$

$$c^I = m_2\,\gamma_2^R - m_1\,\gamma_2^I, \tag{124}$$

with the IERS/Desai load Love combination $\gamma_2^R = 0.6870$, $\gamma_2^I = 0.0036$ (`kOptGammaReal`/`kOptGammaImag`). Each of the six per-station coefficients u_c^R, u_c^I (north, east, radial real and imaginary parts), bilinearly interpolated from the Desai grid, gives the local displacement

$$\boxed{\boldsymbol{d}^c_{\mathrm{OPT}} = \kappa\left(c^R\,u_c^R + c^I\,u_c^I\right), \qquad c \in \{E, N, U\},} \tag{125}$$

(`GnssCorrections.cpp`, `computeOceanPoleTideDisplacement`). The scale κ combines the IERS load-deformation constants with the arcsecond-to-radian conversion, so the wobble stays in arcseconds as in Equation (121):

$$\kappa = \frac{4\pi\,G\,a\,\rho_w\,H_p}{3\,g_E}\,\frac{\pi}{648000}, \qquad H_p = \sqrt{\tfrac{8\pi}{15}}\,\frac{\Omega^2 a^4}{GM_\oplus}, \tag{126}$$

with $\rho_w = 1025\ \mathrm{kg\,m^{-3}}$, $a = 6378136.6$ m, $g_E = 9.7803278\ \mathrm{m\,s^{-2}}$, $\Omega = 7.292115\times10^{-5}\ \mathrm{rad\,s^{-1}}$, $G = 6.67428\times10^{-11}$, $GM_\oplus = 3.986004418\times10^{14}$ (the `kOpt*` constants; κ assembled in `oceanPoleTideKappa`). The whole ocean-pole-tide path reproduces the NGS M-PAGES implementation and the independent Python oracle `scripts/ocean_pole_tide_reference.py` to $\sim10^{-12}$ (see the constants' header comment).

10.6 Atmospheric pressure loading (S1/S2 thermal tide)

Atmospheric pressure loading (APL) is the response of the crust to changes in surface pressure. It is $\sim$2–20 mm and mostly vertical. It has two parts. The S_1/S_2 tidal part is sun-synchronous and thermally driven (diurnal and semidiurnal), and is predictable from local solar time. The non-tidal (synoptic) part is driven by weather systems. The engine models the *tidal* part only; the non-tidal part is a gap (see the note below).

The thermal-tide displacement uses per-station diurnal (S_1) and semidiurnal (S_2) harmonic coefficients evaluated against the local-solar-time argument $\Theta_n = n\,(2\pi\,\mathrm{UT_{frac}} + \lambda)$, $n \in \{1, 2\}$, with $\mathrm{UT_{frac}}$ the UT fraction of day and λ the station longitude (`GnssCorrections.cpp`, `computeAtmosphericTideDisplacement`):

$$\boxed{\boldsymbol{d}^c_{\mathrm{APL}} = a^{\cos}_{c,S_1}\cos\Theta_1 + a^{\sin}_{c,S_1}\sin\Theta_1 + a^{\cos}_{c,S_2}\cos\Theta_2 + a^{\sin}_{c,S_2}\sin\Theta_2, \qquad c \in \{E, N, U\},} \tag{127}$$

formed in local ENU and rotated to ECEF, with a 1 cm sanity guard. The twelve coefficients per station are ordered $[\,a_E^{\cos}, a_E^{\sin}, a_N^{\cos}, a_N^{\sin}, a_U^{\cos}, a_U^{\sin}\,]$ for S_1 (indices 0–5) then the same six for S_2 (indices 6–11). They are bilinearly interpolated from a Ray–Ponte-style S_1/S_2 load grid (`loadAtmosphericTideFromGrid`) at the same RINEX header marker position as the OTL grid (Section 10.3.2). The per-epoch displacement is evaluated at the ARP position state. The term is gated by `model_corrections.enable_atmospheric_loading` (default `false`). It reads its grid from `files.atm_tide_grid_path`, which defaults to `<data>/static/apl/s1_s2_load_cm.dat`. That filename is PROVISIONAL and will be set to the supplied product. The coefficients are loaded in `Geometry::loadData` and projected to `apl_corr` alongside `otl_corr` in `computeSatelliteGeometry`. With the flag off, or with no usable grid, $\boldsymbol{d}_{\mathrm{APL}} = \mathbf{0}$ in Equation (106) and runs are byte-identical.

What is not modelled. Only the sun-synchronous S_1/S_2 tidal APL is modelled. The non-tidal (synoptic) pressure loading is larger and is driven by weather. Modelling it needs a surface-pressure field convolved with a Green's function, or a gridded non-tidal loading product. It is *not* modelled, and it is the leading candidate for the next site-displacement term.

10.7 Integration into the measurement model

Each epoch, the geometry builder evaluates every enabled displacement and projects it with Equation (107). The modelled-range assembly then adds the scalar corrections. In `Geometry::computeSatelliteGeometry` the OTL ECEF displacement is formed and projected to `otl_corr`, the S_1/S_2 atmospheric tide to `apl_corr`, and the solid tide is evaluated against the cached Sun/Moon positions and already returns its LOS-projected scalar. In `PppModel` the pole tide (and, when enabled, the ocean pole tide additively inside it) is projected by `computePoleTideProjection` and the base modelled value (`computeBaseModeledValue`) sums `+ otl_corr + apl_corr + solid_tide` while the pair processors add `+ pole_tide_corr`. The result is the $\rho(\boldsymbol{r})$ of Equation (6) with $\boldsymbol{r}$ replaced by Equation (106).

The differenced observables of Section 5.1 and Section 5.3 formulations contain whatever part of these corrections survives differencing. The solid Earth tide and the pole tide are highly correlated in space and largely cancel over short baselines. Ocean loading and atmospheric loading, the latter largely unmodelled in its non-tidal part, do not cancel. They are the dominant differential site-displacement error on long baselines.

11 Observation Model Physics: Antenna, Attitude, Relativity, and Range Corrections

This document covers the *deterministic* corrections of the observation model, other than geometry, the atmosphere, and the streamed SSR overlay. Section 4 (Equation (4) and Equation (5)) introduce the satellite and receiver hardware biases B_s^i, b_s^i, $B_{r,s}$, $b_{r,s}$ only as symbols. Section 4.3 gives the geometric range ρ_r^i and its linearisation. This document supplies every other deterministic term between the raw measurement and the modelled range:

- the antenna phase-centre geometry, and the satellite attitude frame it depends on;
- the carrier-phase wind-up;
- the relativistic clock and propagation corrections;
- the Sagnac (Earth-rotation) frame correction;

- the bias terms, realised through broadcast group delays and the precise `.BIA` product.

The streamed SSR overlay corrects the same terms in real time and is described in Section 18. The two mechanisms differ in frames, sign conventions, and gating, and the differences are noted where they matter.

`PppModel::computeBaseModeledValue` in `model/PppModel.cpp` assembles every term below into one modelled observable. Both the iono-free-pair and the uncombined observation paths call it. It adds the terms to the geometric range from `model/ModelGeometry.cpp`. For the code observable the sum is

$$\hat{P}^i_{r,s} = \rho^i_r \;-\; c\,\mathrm{d}t^i \;-\; c\,\delta t_{\mathrm{rel}} \;+\; T^i_r \;+\; A^i_{r,s} \;+\; B_{\mathrm{grp}} \;+\; S^i_r \;+\; d_{\mathrm{atm}} \;+\; d_{\mathrm{tide}}, \tag{128}$$

where $A^i_{r,s}$ is the total antenna correction (Section 11.2), δt_{rel} the periodic relativistic clock term (Section 11.4), S^i_r the Shapiro delay (Section 11.5), B_{grp} the deterministic code/phase bias (Section 11.7), and d_{atm}, d_{tide} the loading/site-displacement terms documented in Section 10. The Sagnac correction (Section 11.6) is part of ρ^i_r and is not added separately; the reason is given below. The signs in (128) follow the return statement of `PppModel::computeBaseModeledValue` (`model/PppModel.cpp`): the satellite clock and relativistic terms are *subtracted*, and the antenna, Shapiro, loading and tide terms are *added*.

11.1 Satellite attitude: the body frame

The satellite phase-centre offset, the phase-centre variation, and the carrier wind-up are all defined in the *spacecraft body frame*, so that frame is constructed first. The engine uses the IGS/ANTEX convention: a right-handed orthonormal triad $(\hat{\boldsymbol{x}}_i, \hat{\boldsymbol{y}}_i, \hat{\boldsymbol{k}}_i)$ with $+Z$ toward the Earth's centre (nadir / antenna boresight), $+X$ in the Sun–satellite–Earth plane pointing toward the Sun, and $+Y$ completing the triad along the solar-panel (cross-track) axis (`model/SatAttitude.cpp`, `model/Antenna.cpp`).

Nominal yaw-steering. With $\boldsymbol{r}^i$ the satellite ECEF position and $\hat{\boldsymbol{e}}_\odot = (\boldsymbol{r}_\odot - \boldsymbol{r}^i)/\|\boldsymbol{r}_\odot - \boldsymbol{r}^i\|$ the unit direction to the Sun, the nadir axis and the cross-track axis are

$$\hat{\boldsymbol{k}}_i = -\frac{\boldsymbol{r}^i}{\|\boldsymbol{r}^i\|}, \qquad \hat{\boldsymbol{y}}_i = \frac{\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot}{\|\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot\|}, \qquad \hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i, \tag{129}$$

in the operand order of `SatAttitude::compute` (`model/SatAttitude.cpp`). The antenna path calls that routine (`Antenna::getAntennaCorrection`) and does not rebuild the frame. The wind-up path does contain an inline copy of the construction, noted in Section 11.3. Forming $\hat{\boldsymbol{y}}_i$ first and then $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$ is the Gram–Schmidt orthogonalisation of the Sun direction against nadir. By the $\boldsymbol{a} \times (\boldsymbol{b} \times \boldsymbol{c})$ identity,

$$\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i = (\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot) \times \hat{\boldsymbol{k}}_i = \hat{\boldsymbol{e}}_\odot - \hat{\boldsymbol{k}}_i\,(\hat{\boldsymbol{k}}_i \cdot \hat{\boldsymbol{e}}_\odot), \tag{130}$$

so $\hat{\boldsymbol{x}}_i$ is the normalised component of $\hat{\boldsymbol{e}}_\odot$ orthogonal to nadir. The $+X$ axis therefore points toward the Sun as far as the nadir constraint allows, as the ANTEX convention requires. The body-frame matrix with these three columns,

$$\boldsymbol{F} = \left[\, \hat{\boldsymbol{x}}_i \;\; \hat{\boldsymbol{y}}_i \;\; \hat{\boldsymbol{k}}_i \,\right], \tag{131}$$

is orthonormal, so its inverse is its transpose. This is used below to rotate a body-frame offset into ECEF.

Sun elevation and the mode switch. Non-nominal attitude (eclipse seasons, the noon and midnight yaw turns) depends on the Sun's elevation above the orbital plane, β. With the orbit angular-momentum (normal) unit vector $\hat{\boldsymbol{n}} = (\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i)/\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|$,

$$\boxed{\beta = \arcsin\big(\hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{e}}_\odot\big), \qquad \hat{\boldsymbol{n}} = \frac{\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i}{\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|},} \tag{132}$$

signed by the projection of the Sun direction onto the $+(\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i)$ normal (`SatAttitude::computeBeta`, `model/SatAttitude.cpp`; the sign is noted in-code as unvalidated against an external attitude product). The mode selector (`SatAttitude::determineMode`) keeps GPS, GLONASS and Galileo in yaw-steering unconditionally, and switches BeiDou and QZSS to orbit-normal attitude below a constellation-specific β threshold (4.0° BeiDou, 20.0° QZSS, `model/SatAttitude.cpp`). In orbit-normal mode the $+Y$ axis is pinned to the orbit normal, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{n}}$, and the triad is re-orthogonalised $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{x}}_i$. The legacy default path also forces a hard-coded list of BeiDou-2 GEO PRNs (C01–C05, C59, C60) into orbit-normal mode whatever the value of β. The β-driven table is active only when `enableOrbitNormalAttitude()` is set. With the flag off the result is bit-identical to the earlier inline frame (`model/Antenna.cpp`, `model/SatAttitude.cpp`).

Eclipse gating. The body frame above assumes a known yaw angle. That assumption fails during Earth-shadow passes. The actual yaw during eclipse and the recovery after it depend on the satellite block and are not modelled here, so the antenna PCO/PCV and the wind-up are unreliable for an eclipsing satellite. When the user opts in (`processing.exclude_eclipsing_satellites`), the satellite is dropped at the geometry stage by a *cylindrical umbra* test (`Planetary::isSatelliteEclipsing`, `model/Planetary.cpp`, called from `model/ModelGeometry.cpp`). With $\hat{\boldsymbol{u}}_\odot = \boldsymbol{r}_\odot/\|\boldsymbol{r}_\odot\|$ the Earth-to-Sun direction, the satellite's along-axis and perpendicular distances are $s_\parallel = \boldsymbol{r}^i \cdot \hat{\boldsymbol{u}}_\odot$ and $\boldsymbol{s}_\perp = \boldsymbol{r}^i - s_\parallel\, \hat{\boldsymbol{u}}_\odot$, and the predicate is

$$\text{eclipsed} \iff \big(s_\parallel < 0\big) \ \wedge \ \big(\|\boldsymbol{s}_\perp\| < \kappa\, R_\oplus\big), \tag{133}$$

i.e. the satellite is on the anti-Sun side *and* inside the shadow cylinder of radius $\kappa R_\oplus$, with $R_\oplus = 6\,371\,230$ m (`common/Constants.hpp`) and κ the user shadow scale (`processing.eclipse_shadow_scale`, default unity; cited rather than reproduced as a tuning knob). This is a conservative geometric umbra test. It does not model the penumbra.

11.2 Antenna phase-centre corrections

The range is measured between the satellite and receiver *antenna phase centres*. The estimated states refer to the satellite centre of mass and the receiver antenna reference point (ARP). The position state must be the ARP because the ANTEX receiver PCO/PCV are referenced to it. The antenna correction $A^i_{r,s}$ connects the two, as the sum of four geometric terms and the wind-up:

$$\boxed{A^i_{r,s} \;=\; -\,\delta^i_{\mathrm{PCO}} \;+\; \delta^i_{\mathrm{PCV}} \;+\; \delta^r_{\mathrm{PCO}} \;+\; \delta^r_{\mathrm{PCV}} \;+\; \varphi_{\mathrm{wu}},} \tag{134}$$

as accumulated in `Antenna::getAntennaCorrection` (`model/Antenna.cpp`). The line of sight used in this section is the *satellite-to-receiver* unit vector, $\boldsymbol{e} = (\boldsymbol{r}_r - \boldsymbol{r}^i)/\|\boldsymbol{r}_r - \boldsymbol{r}^i\|$ (`Antenna.cpp`). It is the negative of the receiver-to-satellite $\boldsymbol{e}^i_r$ of Section 4.3 (Equation (7)). The sign of each term below follows from this choice.

Which coordinate is the ARP. Every position the engine derives from a RINEX *header* is an ARP position. The marker coordinate is reduced to the ARP through the header eccentricity before it seeds the state (`Coordinates::markerToArp`, packaged as `RinexParser::getArpPositionXyz`; Section 6.2). A coordinate supplied directly in the configuration is *not* reduced. It is used as given, on the policy that the user owns its reference point. This matters in the network producer. Its station coordinates come from the configuration (`StationConfig::position_ecef`) and are held fixed, with their position columns dropped from the normal equations. A monument coordinate there is not absorbed by a position state. The offset goes into the residuals, and from there into the estimated clocks, biases, and ionosphere.

Satellite PCO (subtracted). The satellite phase-centre offset $\boldsymbol{p}^i_{\text{body}}$ is published in the body frame (millimetres in ANTEX). It is rotated to ECEF by the body-frame matrix; because $\boldsymbol{F}$ collects the body axes as *columns*, $\boldsymbol{F}$ itself maps body $\rightarrow$ ECEF (and the orthonormal $\boldsymbol{F}^{\mathrm{T}} = \boldsymbol{F}^{-1}$ maps ECEF $\rightarrow$ body). It is then projected onto the LOS:

$$\delta^i_{\text{PCO}} = \left(\boldsymbol{F}\,\boldsymbol{p}^i_{\text{body}}\right) \cdot \boldsymbol{e}. \tag{135}$$

The antenna phase centre is *nearer the receiver* than the centre of mass, so the true range is shorter than the CoM range and the correction is *subtracted* in (134) (`correction -= sat_pco_corr`, `model/Antenna.cpp`). There is one real-time exception. When an applied SSR orbit correction is referenced to the antenna phase centre, the corrected orbit already points at the phase centre. The `skip_satellite_pco` path of `getAntennaCorrection` then drops the satellite-PCO term so that it is not counted twice. The path is gated by `realtime.honor_ssr_reference_point` and is off by default. The SSR reference-point convention is documented in Section 18.

Satellite PCV (added). The satellite phase-centre variation is a function of the *nadir angle*, the angle between the boresight $\hat{\boldsymbol{k}}_i$ and the line of sight, and of the azimuth in the body frame. With $\cos\theta_{\text{nad}} = \hat{\boldsymbol{k}}_i \cdot \boldsymbol{e}$,

$$\theta_{\text{nad}} = \arccos\!\left(\hat{\boldsymbol{k}}_i \cdot \boldsymbol{e}\right), \qquad \alpha_{\text{body}} = \operatorname{atan2}\!\left((\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{e})_y,\, (\boldsymbol{F}^{\mathrm{T}}\,\boldsymbol{e})_x\right), \tag{136}$$

and the PCV value is read by bilinear interpolation of the ANTEX grid over (zenith/nadir ring, azimuth sector), the look-up argument being $\theta_{\text{nad}} - \texttt{zen1}$ in grid units (`Antenna::interpolatePcv`, `model/Antenna.cpp`). The satellite PCV is *added* to the range correction.

Receiver PCO (added). The receiver PCO is published as a North/East/Up (NEU) triplet relative to the ARP. It is reordered to ENU, rotated to ECEF by the local ENU$\rightarrow$ECEF rotation $\boldsymbol{R}_{\text{enu}}(\phi,\lambda)$ at the receiver geodetic latitude/longitude (`Coordinates::enuToEcefMatrix`), and projected on the LOS:

$$\delta^r_{\text{PCO}} = \Big(\boldsymbol{R}_{\text{enu}}(\phi,\lambda)\,\begin{bmatrix} p_E & p_N & p_U \end{bmatrix}^{\mathrm{T}}\Big) \cdot \boldsymbol{e}, \qquad \begin{bmatrix} p_E\, p_N\, p_U \end{bmatrix} = \begin{bmatrix} p_{\text{NEU},2} & p_{\text{NEU},1} & p_{\text{NEU},3} \end{bmatrix}, \tag{137}$$

where the index swap $(N,E,U)\rightarrow(E,N,U)$ is explicit in `model/Antenna.cpp`. It is *added* to the correction.

Receiver PCV (added). The receiver PCV is indexed by *zenith angle* $z = 90^\circ - \text{el}$ (and, optionally, azimuth corrected for the antenna mounting orientation when `enableRxAntennaOrientation()` is set), with the same bilinear interpolation over $z - \texttt{zen1}$ (`model/Antenna.cpp`). It is *added* to the correction.

ATX frequency fallback. ANTEX entries often omit the newest carriers: L5/E5a/B2a on older spacecraft, and many receiver antennas published before triple-frequency tracking. Setting the antenna correction to zero would leave a vertical bias of several centimetres. The engine instead substitutes the *nearest carrier* when the requested ANTEX frequency key is absent (`pickAtxFrequencyKey`, `model/Antenna.cpp`). The chains are ordered by carrier-frequency proximity, e.g. for GPS `G05` $\rightarrow$ {`G02`, `G01`} and for Galileo `E05` $\rightarrow$ {`E08`, `E07`, `E06`, `E01`}. The PCO of one physical antenna typically changes by less than a centimetre between frequencies. The PCO and PCV chains are searched *independently*, because an antenna may have a PCO but no PCV at a given key.

For the *satellite* antenna only the chain within the same system is allowed; another constellation's spacecraft entry would be meaningless. For the *receiver* antenna a *cross-system* chain to the nearest GPS band is allowed after the same-system chain is exhausted, and only when `enable_receiver_cross_system_atx_fallback` is set (`pickReceiverAtxFrequencyKey`). This is justified because the receiver phase pattern depends on frequency, not on constellation. The important mappings are exact carrier matches (`E01` $\rightarrow$ `G01`, `E05` $\rightarrow$ `G05` at 1575.42 and 1176.45 MHz). The rest are approximations over $\sim$10–70 MHz (`model/Antenna.cpp`). Each substitution sets a coverage flag and writes a log line. The same fallback policy is used for biases: substitute and report, and do not exclude the measurement.

11.3 Carrier-phase wind-up

A right-hand circularly polarised GNSS signal gains a phase shift equal to the relative rotation of the transmitting and receiving antenna dipoles about the line of sight [28]. This wind-up is the same in cycles on every signal, so as a range it is proportional to wavelength. It is computed in `GnssCorrections::computePhaseWindUp` (`corrections/GnssCorrections.cpp`). With $\boldsymbol{k}$ the unit line-of-sight, $(\hat{\boldsymbol{x}}_i, \hat{\boldsymbol{y}}_i)$ the satellite body $\pm X, +Y$ axes and $(\boldsymbol{x}_r, \boldsymbol{y}_r)$ the receiver local North/East axes, the *effective dipoles* are

$$\boldsymbol{D}_i = \hat{\boldsymbol{x}}_i - \boldsymbol{k}\,(\boldsymbol{k}\cdot\hat{\boldsymbol{x}}_i) - \boldsymbol{k}\times\hat{\boldsymbol{y}}_i, \qquad \boldsymbol{D}_r = \boldsymbol{x}_r - \boldsymbol{k}\,(\boldsymbol{k}\cdot\boldsymbol{x}_r) - \boldsymbol{k}\times\boldsymbol{y}_r, \tag{138}$$

which is the standard Wu construction. Each dipole is the antenna X axis projected into the plane orthogonal to $\boldsymbol{k}$, *minus* the $\boldsymbol{k}\times Y$ quadrature term (`corrections/GnssCorrections.cpp`).

The sign of the quadrature term is fixed by the antenna axes. It is $-\operatorname{sgn}\big((\boldsymbol{x}\times\boldsymbol{y})\cdot\boldsymbol{k}\big)$, which is negative when the antenna's third axis points along $+\boldsymbol{k}$. That is the case for both triads here: the satellite's $\boldsymbol{e}_z$ is the geocentric nadir, and the receiver's implied third axis is North $\times$ East = Down. The wrong sign gives $|\boldsymbol{D}| = 1-\cos\varepsilon$, a dipole that *vanishes on boresight*. RTKLIB's $(-,+)$ pairing is stated with $\boldsymbol{y}_r = -$East and is the same construction; with the +East convention used here it becomes $(-,-)$. *Earlier revisions of this document printed* $(+,+)$*, copied from a defect in the code that was corrected on 2026-08-21; see* ***`docs/SPEC_phase_windup_sign.md`****.*

The receiver North and East axes are built from the geodetic latitude ϕ and longitude λ, $\boldsymbol{x}_r = (-\sin\phi\cos\lambda,\ -\sin\phi\sin\lambda,\ \cos\phi)$ and $\boldsymbol{y}_r = (-\sin\lambda,\ \cos\lambda,\ 0)$ (`GnssCorrections.cpp`). The signed wind-up angle is the atan2 of the cross and dot products of the unit dipoles. This covers the full range $(-\pi, \pi]$ and avoids the loss of precision of an arccos near 0 and π:

$$\boxed{\Delta\varphi = \operatorname{atan2}\Big(\boldsymbol{k}\cdot\big(\hat{\boldsymbol{D}}_i\times\hat{\boldsymbol{D}}_r\big),\ \hat{\boldsymbol{D}}_i\cdot\hat{\boldsymbol{D}}_r\Big), \qquad \varphi_{\mathrm{wu}} = \Delta\varphi\,\frac{\lambda_s}{2\pi},} \tag{139}$$

with $\hat{\boldsymbol{D}}_i, \hat{\boldsymbol{D}}_r$ the normalised dipoles and λ_s the signal wavelength (`corrections/GnssCorrections.cpp`); the $\boldsymbol{k}\cdot(\hat{\boldsymbol{D}}_i\times\hat{\boldsymbol{D}}_r)$ argument fixes the sign of the rotation about the line of sight. The result enters (134) as φ_{wu} on phase observables only (the code branch passes zero wavelength).

Fractional vs. accumulated modes. The atan2 returns the wind-up wrapped to one cycle. The default *fractional* mode uses this wrapped angle directly. The float ambiguity $N^i_{r,s}$ absorbs any whole-cycle part, so the wrapping does no harm. Integer ambiguity resolution needs the ambiguity to stay a constant integer across the arc. For that the engine offers an *accumulated* (unwrapped) mode (`config.accumulatePhaseWindUp()`), which unwraps each epoch relative to the previous stored angle, $\Delta\varphi_t = \Delta\varphi_{t-1} + \mathrm{wrap}_{2\pi}(\Delta\varphi_t - \Delta\varphi_{t-1})$, provided the gap between epochs is below the 300 s continuity threshold. A larger gap, or a backward time step, restarts the accumulator from the raw angle (`corrections/GnssCorrections.cpp`). A gap that long invalidates the unwrap reference, so the engine starts a new arc, as it does for an ambiguity.

Known frame duplication. The satellite body frame feeding the wind-up dipole $\boldsymbol{D}_i$ (138) is built *inline* inside `computePhaseWindUp` (`corrections/GnssCorrections.cpp`) rather than calling the shared `model/SatAttitude.cpp` helper used by the antenna path (129). The two constructions are kept bit-for-bit identical: the same operand order, the same BeiDou-2 GEO orbit-normal special case, and the same velocity fallback. Both source files note the duplication. It is recorded here as a *maintenance* obligation. A change to one frame must be copied to the other. Otherwise the antenna PCV and the wind-up would use different frames for the same satellite, with no error reported.

11.4 Periodic relativistic clock correction

A clock on an eccentric orbit runs at a varying rate relative to a clock on the geoid. The broadcast or precise clock polynomial absorbs the constant and secular parts. The *periodic* part, caused by orbital eccentricity, must be applied separately. The standard expression [23], in `GnssCorrections::computeRelativisticCorrection` (`corrections/GnssCorrections.cpp`), is

$$\boxed{\delta t_{\mathrm{rel}} \;=\; -\,\frac{2\,(\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i)}{c^2} \quad [\mathrm{s}], \qquad \text{range term} \;= -\,c\,\delta t_{\mathrm{rel}},} \tag{140}$$

applied to the range by the $-c\,\delta t_{\mathrm{rel}}$ term of the assembly (128) (`model/PppModel.cpp`). Two points need stating.

First, the dot product $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ is the same whether the position and velocity are expressed in the inertial frame or in the rotating ECEF frame. The ECEF velocity is $\dot{\boldsymbol{r}}^i_{\mathrm{ecef}} = \dot{\boldsymbol{r}}^i_{\mathrm{inertial}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$, and $\boldsymbol{r}^i \cdot (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i) = 0$ because the cross product is orthogonal to $\boldsymbol{r}^i$. The ECEF velocity from the broadcast ephemeris can therefore be used directly.

Second, the periodic term is computed here because the IGS precise CLK product, by convention, contains only the constant and drift parts of the clock. Applying (140) supplies the periodic part consistently for both precise and broadcast clock sources.

11.5 Shapiro (gravitational-delay) correction

The Earth's gravitational potential slows and bends the signal. The resulting Shapiro delay is a few centimetres [23]. With $r_i = \|\boldsymbol{r}^i\|$, $r_r = \|\boldsymbol{r}_r\|$ the geocentric radii and $r_{ir} = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ the satellite–receiver distance, the delay (in metres) is

$$\boxed{S^i_r \;=\; \frac{2\,GM}{c^2}\,\ln\frac{r_i + r_r + r_{ir}}{r_i + r_r - r_{ir}},} \tag{141}$$

with $GM = 3.986005 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$ (`Constants::GRAVITATIONAL_CONSTANT`, `common/Constants.hpp`) and the formula as coded in `GnssCorrections.cpp`. Since $r_{ir} < r_i + r_r$ the logarithm is

positive, so $S_r^i > 0$ and the term is *added* to the modelled range in (128) (`model/PppModel.cpp`). It is ∼1–2 cm for a typical MEO geometry.

11.6 Sagnac (Earth-rotation) correction

The pseudorange relates the satellite position at *signal emission* to the receiver position at *signal reception*, and both must be expressed in one ECEF frame. The ECEF frame rotates with the Earth during the ∼70 ms transit, so the emission-time satellite coordinates are rotated into the reception-time frame. The engine applies the full rotation, not a scalar approximation. With the geometric time-of-flight $\tau = \|\boldsymbol{r}_{\mathrm{em}}^i - \boldsymbol{r}_r\|/c$ and the Earth rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$ (`common/Constants.hpp`), the emission-time position $\boldsymbol{r}_{\mathrm{em}}^i$ is rotated by $\theta = \omega_\oplus \tau$ about the Z axis,

$$\boxed{\boldsymbol{r}^i = \boldsymbol{R}_z(\omega_\oplus \tau)\, \boldsymbol{r}_{\mathrm{em}}^i, \qquad \boldsymbol{R}_z(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix},} \tag{142}$$

after which $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line of sight are computed from the rotated $\boldsymbol{r}^i$ (`model/ModelGeometry.cpp`, the `rotateAndComputeTopo` lambda). The Sagnac correction is therefore *part of the geometric range*, which is why (128) has no separate Sagnac term. An equivalent scalar form, $(\omega_\oplus/c)\,(x_i y_r - y_i x_r)$, exists in the code as a *diagnostic* only (`GnssCorrections::computeSagnacCorrection`, `corrections/GnssCorrections.cpp`). It is the first-order expansion of the rotation and is not used for modelling. The Section 8 refers to this section for the Sagnac rotation. The same rotation $\boldsymbol{R}_z(\omega(t_{\mathrm{node}} - t_{\mathrm{query}}))$ is used at SP3 interpolation time to rotate ephemeris nodes into the query-epoch frame (`model/ModelGeometry.cpp`), for numerical conditioning, as described with the products.

11.7 Group delays and inter-frequency biases

Two products supply deterministic values for the bias terms of the Foundations observation equations: the satellite code and phase biases B_s^i, b_s^i and the receiver biases $B_{r,s}$, $b_{r,s}$. This section relates the products to those symbols.

Broadcast group delay. The broadcast navigation message carries a per-system inter-frequency group delay (GPS TGD, Galileo BGD, BeiDou TGD). It refers each signal's code hardware delay to the ionosphere-free combination used to generate the broadcast clock. In the uncombined parameterisation these delays are part of the deterministic code bias applied through the bias term of `model/PppModel.cpp`. They realise B_s^i at broadcast accuracy and are used when no precise bias product is available.

Precise OSB / DCB biases. When a precise bias product is loaded, the deterministic correction is the observable-specific bias (OSB) / differential code bias (DCB) entry for the exact (i,s) pair, time-windowed against its validity interval and returned by `GnssCorrections::getBiasCorrection` (`corrections/GnssCorrections.cpp`). This single scalar realises B_s^i (code) or b_s^i (phase) for that signal:

$$B_{\mathrm{grp}} = \mathrm{bias}(i, s, t), \qquad t_{\mathrm{start}} \le t \le t_{\mathrm{end}}, \tag{143}$$

with the application gated by `enable_code_phase_bias` (`model/PppModel.cpp`). The bias index (`GnssCorrections::buildBiasIndex`) excludes two kinds of entry.

Zero-valued code entries, such as the MGEX placeholder `C2I`/`C6I` for BeiDou, are treated as *absent*. Otherwise the model would apply a null correction and also suppress the estimated bias state for that (i,s), leaving a real delay of tens of metres unmodelled.

GLONASS code entries are also treated as missing. The FDMA per-channel receiver delay cannot be represented by one value per (i,s), so those signals use the estimated-state path (`corrections/GnssCorrections.cpp`).

Estimated vs. deterministic biases. The estimator handles whatever the deterministic bias does not correct: mainly the receiver hardware delay $B_{r,s}$, $b_{r,s}$, and any satellite delay that no product covers. The receiver biases are common to all satellites on a signal. They are absorbed by the receiver clock and inter-system bias states, or cancel under between-satellite differencing (Section 5). The per-(i,s) satellite code bias may be *estimated* as a state when no product covers it (`estimation.estimate_sat_biases`, `measurement/MeasurementModel.cpp`). The state-allocation gate that prevents estimating a bias the products already supply, hasBIA(i, s) from the static product, OR'd with hasSsrCB(i, s) from the live stream, is documented with the SSR overlay (Section 18, Equation (272) there); the present document supplies only the *deterministic* half of that OR.

Relation to the SSR overlay. The streamed SSR code/phase OSB (Section 18) refines the *same* B_s^i, b_s^i terms in real time, summed onto the predicted observable as `ssr_bias_term` in `measurement/MeasurementModel.cpp` alongside the deterministic B_{grp} here. The SSR bias is a per-(i,s) scalar like (143). The SSR *orbit* delta is different: it is expressed in a radial/along/cross (RAC) frame (Section 18's Equation (264)), which is *not* the satellite body frame (131) of this document. The RAC frame is built from the broadcast position and velocity to express an orbit correction. The body frame is built from the Sun direction to orient the antenna. The two frames differ in construction and in purpose.

11.8 Provenance and assembly

All the deterministic corrections are combined in `model/PppModel.cpp`. `computeBaseModeledValue`, shared by the iono-free-pair and uncombined paths, returns the sum (128). It calls `model/ModelGeometry.cpp` for the Sagnac-rotated geometric range (142), `model/Antenna.cpp` for the total antenna correction (134) (which in turn calls `model/SatAttitude.cpp` for the body frame and `corrections/GnssCorrections.cpp` for the wind-up), and `corrections/GnssCorrections.cpp` for the relativistic (140), Shapiro (141) and bias (143) terms. The Sun and Moon ephemeris feeding the attitude frame (129), the Sun-elevation (132), the eclipse predicate (133) and the wind-up (138) is produced by `model/Planetary.cpp` (the rigorous IAU precession/nutation Sun position, cached per epoch) and is documented in Section 6. Every term is invariant physics; the only user-facing knobs are enable toggles and the eclipse-shadow scale, all cited rather than reproduced because they tune *whether* a correction is applied, never its functional form.

Part III
Estimation engine

12 Estimation engine: the Kalman filter and RTS smoother

This section documents the engine's recursive estimator. It is a variable-dimension extended Kalman filter (EKF) with an optional prior-anchored iterated (IEKF) relinearisation pass, robust gating of post-fit residuals, and a backward Rauch–Tung–Striebel (RTS) smoother for post-processing. The state vector $\boldsymbol{x}$ is the uncombined PPP / PPP-RTK parameterisation of Section 3: receiver position (and optionally velocity), receiver clock and per-system time offsets, zenith wet troposphere and gradients, per-satellite slant ionosphere, receiver and satellite code and phase biases, and one float ambiguity per tracked carrier. The dimension changes from epoch to epoch as satellites rise and set and as ambiguities are reset. `filter/StateManager` owns the layout and the dynamic key→index maps. The filter operates on that state and assumes no fixed indices (`src/core/filter/KalmanFilter.cpp`).

Throughout we write $\boldsymbol{F}$ for the state-transition matrix, $\boldsymbol{Q}$ for the discrete process-noise covariance, $\boldsymbol{R}$ for the measurement-noise covariance, $\boldsymbol{S}$ for the innovation covariance, $\boldsymbol{K}$ for the Kalman gain, and $\boldsymbol{\nu}$ for the innovation (measurement minus predicted measurement). Predicted (a-priori) and updated (a-posteriori) quantities use the tilde/hat and $k|k{-}1$ / $k|k$ conditioning convention fixed in Section 2.

12.1 Time update

State transition. The transition matrix $\boldsymbol{F}(\Delta t)$ for epoch interval $\Delta t = t_k - t_{k-1}$ is identity plus a small number of coupling blocks, assembled by `StateManager::getTransitionMatrix` (`src/core/filter/StateManager.cpp`). The kinematic-integrator off-diagonal couplings are

$$\boldsymbol{F}[\,\boldsymbol{r},\boldsymbol{v}\,] = \Delta t\,\boldsymbol{I}_3, \qquad \boldsymbol{F}[\,c\mathrm{d}t,\ c\dot{\mathrm{d}}t\,] = \Delta t, \qquad \boldsymbol{F}[\,c\delta t^{(g)},\ c\dot{\delta} t^{(g)}\,] = \Delta t, \tag{144}$$

present only when the corresponding rate state exists (`hasVelocity()`, clock-rate, per-system rate). A fourth coupling appears when the constant-velocity slant-iono model is armed (`estimation.enable_iono_rate`, default off): a per-satellite level/rate integrator

$$\boldsymbol{F}[\,I^i,\ \dot{I}^i\,] = \Delta t \qquad \text{for every non-frozen-pivot satellite,} \tag{145}$$

which gives $I^i(k{+}1) = I^i(k) + \Delta t\,\dot{I}^i(k)$. Every other state has $\boldsymbol{F} = \boldsymbol{I}$ and is a random walk with constant mean: ambiguities, zenith wet delay, gradients, code and satellite biases, any inter-system offset without a rate, and (without `enable_iono_rate`) the per-satellite ionosphere.

Under the BSSD (single-difference) parameterisation some states are unobservable, because their $\boldsymbol{H}$ columns are zero: the receiver clock, the phase clock (`getPhaseClockIndex`), the inter-system offsets, and the frozen-pivot per-satellite ionosphere (`getFrozenPivotSatellites`). Their rows in Equation (144) are suppressed. After propagation their cross-covariances are set to zero and their diagonal variances are kept (the `freeze_state` block in `src/core/filter/KalmanFilter.cpp`).

The iono level/rate coupling of Equation (145) is *not* disabled under BSSD. The slant-iono level stays observable under single-differencing, since only the pivot column is zero, so every satellite other than the frozen pivot keeps its integrator. The frozen pivot is skipped. Its rate is held at $(0, 0)$ and its process noise is suppressed, so the coupling cannot move the pivot level away from the SD datum.

The propagation itself is the standard EKF time update

$$\boxed{\begin{aligned}\tilde{\boldsymbol{x}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\hat{\boldsymbol{x}}_{k-1|k-1}, \qquad (146)\\ \tilde{\boldsymbol{P}}_{k|k-1} &= \boldsymbol{F}(\Delta t)\,\boldsymbol{P}_{k-1|k-1}\,\boldsymbol{F}(\Delta t)^{\mathrm{T}} + \boldsymbol{Q}(\Delta t), \qquad (147)\end{aligned}}$$

implemented in `KalmanFilter::predict`. A non-positive Δt (a duplicate or out-of-order epoch) does nothing: the routine returns before Equation (147).

Process-noise model. The discrete process noise $\boldsymbol{Q}(\Delta t)$ is built from continuous spectral densities by `StateManager::getProcessNoiseMatrix`. Two block structures appear. Pure random-walk states have a scalar discrete variance increment

$$\boldsymbol{Q}_{ii} = q_i\,\Delta t, \tag{148}$$

where q_i is the continuous variance rate (m^2/s) read from `filter.process_noise.*`. Representative entries are zenith wet troposphere (`troposphere_q`), the receiver code bias (`code_bias_q`), the satellite *code* bias (`sat_bias_q`, applied to `StateManager::sat_bias_indices_`), the satellite *phase* bias (`sat_phase_bias_q`, a separate density on the separate `sat_phase_bias_indices_` map), phase clock (`phase_clock_q`), and the float ambiguity (`ambiguity_q`). Both satellite-bias blocks, and the satellite-clock block beside them, set $\boldsymbol{Q}$ to zero for a frozen-pivot satellite. The frozen state is held at (value, variance) $= (0, 0)$ as the datum, and any process noise would move it off that datum. The ambiguity density defaults to a numerical floor (10^{-12}), not a physical value. Per-key overrides are supported for reseeding individual arcs. The per-satellite slant ionosphere uses a random walk whose zenith density is mapped to the slant by the squared obliquity factor,

$$\boldsymbol{Q}_{ii} = \frac{q_{\mathrm{iono}}}{\sin^2\theta^i}\,\Delta t, \qquad \sin^2\theta^i \text{ floored at } 10^{-2}\ (\theta \approx 5.7^\circ), \tag{149}$$

with q_{iono} = `ionosphere_q` and optional per-satellite overrides driven by the adaptive-iono scheme. Under the constant-velocity model of Equation (145) this random walk is used only for satellites without a rate state. Otherwise the level/rate pair takes the CWNA block of Equation (150) below, with the acceleration PSD scaled by obliquity, $q = q_{\dot{\mathrm{iono}}}/\sin^2\theta^i$, where $q_{\dot{\mathrm{iono}}}$ = `process_noise.iono_rate_psd` (an acceleration PSD, $\mathrm{m}^2/\mathrm{s}^3$, default 10^{-10}). The adaptive-monitor override enters this branch as the *dimensionless* ratio $q_{\mathrm{ov}}/q_{\mathrm{iono}}$, which multiplies the rate PSD. The Sage–Husa storm response therefore acts on $\dot{I}^i$, not on the level.

Coupled position/velocity, clock/clock-rate, inter-system offset/rate, and, under `enable_iono_rate`, slant-iono level/rate pairs use the continuous white-noise-acceleration (CWNA, integrated-random-walk) discretisation. For a coupled pair $(\xi, \dot{\xi})$ driven by acceleration PSD q,

$$\boldsymbol{Q}_{[\xi,\dot{\xi}]} = \begin{bmatrix} \frac{1}{3}\,q\,\Delta t^3 & \frac{1}{2}\,q\,\Delta t^2 \\ \frac{1}{2}\,q\,\Delta t^2 & q\,\Delta t \end{bmatrix}, \tag{150}$$

with q given by `acceleration_psd_q` (position/velocity, the 3×3 block-diagonal extension of Equation (150) over the ECEF axes), `clock_rate_q`, `system_time_rate_offset_q`, and the obliquity-scaled `iono_rate_psd`$/\sin^2\theta^i$ respectively. When the velocity state is present the rate-free position density `position_q` is unused. Baseline (double-difference) mode reduces $\boldsymbol{Q}$ to the position block, a per-ambiguity random-walk term, and an optional differential-ZWD entry $\boldsymbol{Q}_{ii} = \sigma^2\Delta t$ governed by `baseline.differential_tropo_process_noise_m_per_sqrt_s`.

Scope: the recursive estimator only. These densities apply to the *recursive* estimator. The network producer does not run a `KalmanFilter`. `getTransitionMatrix` and `getProcessNoiseMatrix` are called only from `KalmanFilter::predict` / `predictImu` / `predictAided` and from the smoother's `Application::capturePriorSnapshot`. The producer solves per-epoch normal equations for corrections to each station's linearisation point. It links epochs with explicit information-form carry priors of effective variance $\mathrm{var}_{\mathrm{eff}} = \mathrm{var} + q\,\Delta t$, not through $\boldsymbol{Q}(\Delta t)$ (Section 20). There the satellite-side densities `sat_clock_q`, `sat_phase_bias_q` and `sat_code_bias_q` are read as those carry rates. They can also be set through the network-scoped `network.process_noise` class table, which is always active. Its per-class fallbacks equal the legacy `filter.process_noise.*` defaults, so an unmodified config gives the same values either way.

Aided time update. When inertial aiding is active, `KalmanFilter::predictImu` mechanises the nominal position, velocity, and attitude (strapdown). It inserts a 15×15 INS error-state transition $\boldsymbol{\Phi}$ and noise $\boldsymbol{Q}_{\mathrm{ins}}$, with error order $[\delta\boldsymbol{p},\ \delta\boldsymbol{v},\ \delta\boldsymbol{\psi},\ \delta\boldsymbol{b}_a,\ \delta\boldsymbol{b}_g]$, into the GNSS-dimensioned $\boldsymbol{F}$ and $\boldsymbol{Q}$ over the integrated interval. Equation (147) then runs on the full state, followed by the symmetrisation $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$. The general `predictAided` path does the same for any aiding source. With no aiding payload it calls the GNSS-only Equation (147) and gives a byte-identical result.

12.2 Measurement update

The measurement update (`KalmanFilter::update`) is a *batch* update: all observation rows of the epoch are processed together, not one at a time. A sparse-$\boldsymbol{H}$ optimisation speeds up the four products that involve the design matrix ($\boldsymbol{HP}$, $\boldsymbol{HPH}^{\mathrm{T}}$, $\boldsymbol{KH}$, and the $\boldsymbol{H}$ parts of the gain). It agrees with the dense computation to a 10^{-12} structural threshold (`src/core/filter/KalmanFilter.cpp`).

Given the design matrix $\boldsymbol{H}$ from the linearisation of the Equation (4) and Equation (5) of the Foundations document (position rows $-(\boldsymbol{e}^i_{\hat{r}})^{\mathrm{T}}$, the Equation (6)), the innovation $\boldsymbol{\nu}$ (`measurement.innovations`), and the measurement-noise covariance $\boldsymbol{R}$ (a full matrix when supplied, otherwise the diagonal $\boldsymbol{R} = \mathrm{diag}(\sigma^2_{r,i,s})$ of the Equation (8)), the innovation covariance and gain are

$$\boldsymbol{S} = \boldsymbol{H}\,\tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}, \tag{151}$$

$$\boldsymbol{K} = \tilde{\boldsymbol{P}}_{k|k-1}\,\boldsymbol{H}^{\mathrm{T}}\,\boldsymbol{S}^{-1}. \tag{152}$$

The gain is not formed by inverting $\boldsymbol{S}$. $\boldsymbol{S}$ is symmetric, so the engine factorises it once with an $\mathrm{LDL}^{\mathrm{T}}$ decomposition and obtains $\boldsymbol{K} = \left(\boldsymbol{S}^{-1}\boldsymbol{H}\tilde{\boldsymbol{P}}\right)^{\mathrm{T}}$ from one solve against $\boldsymbol{H}\tilde{\boldsymbol{P}}$ (`ldlt.solve(HP).transpose()`). If the factorisation fails it falls back to a complete-orthogonal (pseudo-inverse) solve. The state update is

$$\delta\boldsymbol{x} = \boldsymbol{K}\,\boldsymbol{\nu}, \qquad \hat{\boldsymbol{x}}_{k|k} = \tilde{\boldsymbol{x}}_{k|k-1} + \delta\boldsymbol{x}. \tag{153}$$

Joseph-form covariance update. The covariance update uses the *Joseph* form, not the short form $(\boldsymbol{I} - \boldsymbol{KH})\tilde{\boldsymbol{P}}$:

$$\boxed{\boldsymbol{P}_{k|k} = (\boldsymbol{I} - \boldsymbol{KH})\,\tilde{\boldsymbol{P}}_{k|k-1}\,(\boldsymbol{I} - \boldsymbol{KH})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}.} \tag{154}$$

The Joseph form keeps the covariance symmetric and positive semi-definite for any gain. That property is needed because the robust paths of Section 12.4 use a gain that is not the optimal one. The result is symmetrised again, $\boldsymbol{P} \leftarrow \frac{1}{2}(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}})$, to remove round-off. The factor $(\boldsymbol{I} - \boldsymbol{KH})$ is built from the sparse $\boldsymbol{KH}$ product by adding one on the diagonal. The same kernel, with the explicit gain $\boldsymbol{K} = \boldsymbol{PH}^{\mathrm{T}}\boldsymbol{S}^{-1}$, is the static routine `josephPseudoUpdate`. It is reused for scalar pseudo-observation constraints (for example the GIM ionosphere prior via `applyPseudoObservation`) and for the IEKF pass of Section 12.3.

Post-fit residuals. After the update the post-fit residuals and their (diagonal) covariance are

$$\boldsymbol{r} = \boldsymbol{\nu} - \boldsymbol{H}\,\delta\boldsymbol{x}, \qquad S_{r,ii} = R_{ii} - \left(\boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}\right)_{ii}, \tag{155}$$

By default only the diagonal of S_r is formed. The full symmetrised $S_r = \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}_{k|k}\,\boldsymbol{H}^{\mathrm{T}}$ is formed and stored when two conditions hold: the off-diagonal-R kernel is enabled (`enable_full_r_kernel`, default off), *and* a dense $\boldsymbol{R}$ was supplied (BSSD and baseline-DD epochs, whose $\boldsymbol{R}$ carries SD/DD cross-covariance). The redundancy and w-test QC of Section 12.4 then account for correlated observations through S_r^{-1}. The diagonal, and so Equation (156), is the same either way. The standardised residual used for gating (Section 12.4) is

$$s_i = \frac{r_i}{\sqrt{\max\{\, S_{r,ii},\, R_{ii},\, 10^{-12} \,\}}}. \tag{156}$$

The denominator is floored at the *working* R_{ii}, the inflated noise actually used in $\boldsymbol{R}$, not the formal noise. This is a deliberate compromise. When an ambiguity absorbs the measurement (double-differenced phase with an ambiguity gain near one), the rigorous denominator $S_{r,ii}$ shrinks toward $R\,\boldsymbol{S}^{-1}R$, and residuals near zero would register as outliers of many σ. The strictly correct standardisation uses $S_{r,ii}$ alone. The floor gives up that rigour for stability, and the source code says so.

12.3 Iterated relinearisation (IEKF)

When the initial position is far from the truth, one linearisation of the Equation (6) per epoch may not be enough. When `processing.relinearize_geometry` is enabled, and the run is not tightly-coupled IMU, the engine performs a prior-anchored iterated update (a Gauss–Newton, or iterated EKF, step) after the primary update.

Anchor. The iteration is anchored to the *stored pre-update prior* $(\boldsymbol{x}^-, \boldsymbol{P}^-)$ that the primary update Equations (147) and (154) used. It is captured only when relinearisation is armed (`getLastUpdatePriorX` / `getLastUpdatePriorP`). It is the prior *after* any in-epoch ambiguity augmentation and outlier reset, so its dimension matches the posterior. It is not a snapshot from the top of the epoch loop.

Gauss–Newton step. Let $\boldsymbol{x}_i$ be the current linearisation iterate (the committed state at which `measurement` was linearised, with innovations $\boldsymbol{z} - h(\boldsymbol{x}_i)$ and $\boldsymbol{H} = \partial h/\partial \boldsymbol{x}\big|_{\boldsymbol{x}_i}$). The prior-anchored iterate (`KalmanFilter::updateFromPrior`, via `josephPseudoUpdate`) is

$$\boxed{\begin{aligned} \boldsymbol{\nu} &= \left(\boldsymbol{z} - h(\boldsymbol{x}_i)\right) - \boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i), && (157)\\ \boldsymbol{K} &= \boldsymbol{P}^-\boldsymbol{H}^{\mathrm{T}}\left(\boldsymbol{H}\boldsymbol{P}^-\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{R}\right)^{-1}, \qquad \boldsymbol{x}^+ = \boldsymbol{x}^- + \boldsymbol{K}\,\boldsymbol{\nu}, && (158)\\ \boldsymbol{P}^+ &= (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\,\boldsymbol{P}^-\,(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})^{\mathrm{T}} + \boldsymbol{K}\,\boldsymbol{R}\,\boldsymbol{K}^{\mathrm{T}}. && (159) \end{aligned}}$$

The offset term $-\boldsymbol{H}(\boldsymbol{x}^- - \boldsymbol{x}_i)$ in Equation (157) expresses the innovation about the fixed prior, which makes this a MAP (Gauss–Newton) iteration. $\boldsymbol{K}$ and $(\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}^-$ are always computed from the same fixed prior $(\boldsymbol{x}^-, \boldsymbol{P}^-)$, so the covariance does *not* shrink a second time and no information is counted twice.

Firing condition. The driver `Application::runGeometryRelinearization` (`src/core/app/Application.cpp`) fires the iteration when all of the following hold:

1. `processing.relinearize_geometry` is on and a prior was stashed at the primary update;
2. the position change since the primary update exceeds a data-dependent threshold, `relinearize_dpos_threshold_phase_m` (default 1 m) when phase is present, or `relinearize_dpos_threshold_code_only_m` (default 100 m) for code-only epochs;
3. the per-run cap `relinearize_max_count` (-1 = unlimited) is not exhausted.

The inner loop recomputes the geometry and re-applies Equations (157) to (159) against the *fixed* exclusion set from the primary update. It runs at most five iterations and stops when the position step falls below 10^{-4} m. GNSS geometry is nearly linear at the metre level, so one pass usually converges, and the threshold of item 2 means the iteration rarely fires. The tightly-coupled IMU configuration takes a single legacy *naive* second pass, re-applying the measurement against the posterior, because error-state attitude and bias feedback conflicts with re-deriving the update from the prior.

12.4 Innovation and residual gating

The engine performs *no* χ^2 test on the innovation covariance. Outlier rejection (`KalmanFilter::validatePostFitResiduals`) tests the standardised *post-fit* residuals s_i of Equation (156) against a fixed scalar threshold. Robust median-based detectors for pivot slips supplement it. The two top-level thresholds are hardcoded `GnssConfig` defaults (`residual_threshold_` $= 6.0$ and `bad_meas_percent_` $= 0.5$ in `src/core/config/GnssConfig.cpp`). They are not exposed to JSON, so there are no config keys of those names. The per-row rejection threshold is $s_{\max} = 6.0$ (read at runtime via `getResidualThreshold`). The fraction that triggers a complete reset is 0.5 (`getBadMeasPercent`). The Baarda data-snooping gate below, which is exposed to JSON, can replace both the dispatched row and the criterion.

Per-row rejection. If $\max_i |s_i| > 6.0$, the worst row is handled according to its type. A phase outlier triggers an ambiguity reset, with BSSD pivot handling and a sub-cycle λ-fraction floor. A code outlier is excluded from the epoch. Correlated phase rows of the same satellite on other bands are reset with it.

Baarda data-snooping gate (optional override). When `estimation.enable_data_snooping` is set, the fixed-σ gate triple $(\text{gate stat}, \text{gate row}, \text{gate crit}) = (\max_i |s_i|, \arg\max, 6.0)$ is *replaced* in `validatePostFitResiduals` by the Baarda w-test. The test runs over *checkable* rows, those whose redundancy $r_i = S_{r,ii}/R_{ii}$ is at least 0.1. Rows with lower redundancy are skipped, because a blunder there goes into the estimate and not the residual. The row with the largest $|w_i|$ is selected, and the gate fires when $|w_i| >$ `estimation.data_snooping_critical` (default 3.29). Both the criterion *and* the dispatched row can therefore differ from the σ gate. The handling of the selected row by type is unchanged. The w-test statistic and redundancy are produced by the a-posteriori QC diagnostics layer (`computeResidualQc_`, gated by `estimation.enable_variance_factor`): the variance factor $\hat{\sigma}_0^2 = \left(\sum_i v_i^2/R_{ii}\right)/\text{dof}$, the per-row redundancy r_i, the w-test, and the minimal detectable bias (MDB). This layer accounts for a correlated R through S_r^{-1} when the full-R kernel is active (see Equation (155)).

Complete-reset backstop. If the number of rejected rows exceeds $0.5\times$ the original observation count (before any rows were removed), the filter covariance is re-inflated (`resetFilterCovariance`). This is a hard reset. In the smoother it marks a segment boundary (Section 12.5).

Robust pivot-slip detectors (BSSD). Before the per-row gate, single-difference phase rows are grouped by (pivot, signal) and screened for a pivot slip common to the group, using median and median-absolute-deviation (MAD) statistics. A forced pivot handover fires when four conditions hold: the group is large enough, a majority of its rows exceed the threshold, the group median exceeds a metre-scale floor, and the MAD-to-median ratio is small. A small ratio means a tight cluster, which indicates a common shift and not scattered noise. Cross-band pooled and code-side variants apply similar tests with their own floors. Slips on non-pivot satellites across several bands are confirmed by integer-cycle consistency: $|r|/\lambda_{\text{band}}$ must round to the same nonzero integer on each band.

Down-weighting (not gating). Two paths inflate $\boldsymbol{R}$ and do not reject. A persistent EWMA-bias term adds the squared mean bias to R_{ii} once an arc has warmed up. A Huber post-fit scheme inflates R for sub-cycle rows whose $|s_i|$ exceeds the Huber knee, either once on the innovation or iteratively on the post-fit residual (IRLS), toggled by `enable_huber_r_inflation` and `enable_huber_innovation`. Both change the working $\boldsymbol{R}$ used in Equations (151) and (156). The standardised residual is therefore scaled by the working noise, not the formal noise.

12.5 Backward RTS smoother

In post-processing modes, enabling `processing.run_smoother` runs a fixed-interval Rauch–Tung–Striebel recursion [20] over the stored forward trajectory (`include/filter/Smoother.hpp`, `src/core/filter/Smoother.cpp`). The flag is forced off under `AppMode::RealTime`, because smoothing uses future data.

Forward-pass capture. The backward pass does not reconstruct the a-priori from $\boldsymbol{F}$ and $\boldsymbol{Q}$. The forward pass stores, per epoch, the one-step prediction $\tilde{\boldsymbol{x}}_{k|k-1}, \tilde{\boldsymbol{P}}_{k|k-1}$, the posterior $\hat{\boldsymbol{x}}_{k|k}, \boldsymbol{P}_{k|k}$, the matrix $\boldsymbol{F}_k$, and all core and dynamic key→index maps (`capturePriorSnapshot` / `capturePosteriorSnapshot` in `src/core/app/Application.cpp`). The order of capture makes the stored prior equal to $\boldsymbol{F}_k\hat{\boldsymbol{x}}_{k-1|k-1}$. The a-priori is recorded immediately after `predict()`, before any cycle-slip reseeding or measurement update. The a-posteriori is recorded *before* integer ambiguity resolution, so the smoother uses *float* ambiguities only.

Recursion. Initialised at the terminal posterior $\hat{\boldsymbol{x}}_{N|N} = \hat{\boldsymbol{x}}_{N|N}$, $\boldsymbol{P}_{N|N} = \boldsymbol{P}_{N|N}$, the backward recursion is

$$\boxed{\begin{aligned} \boldsymbol{G}_k &= \boldsymbol{P}_{k|k}\,\boldsymbol{F}_{k+1}^{\mathrm{T}}\,\tilde{\boldsymbol{P}}_{k+1|k}^{-1}, && (160)\\ \hat{\boldsymbol{x}}_{k|N} &= \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{G}_k\big(\hat{\boldsymbol{x}}_{k+1|N} - \tilde{\boldsymbol{x}}_{k+1|k}\big), && (161)\\ \boldsymbol{P}_{k|N} &= \boldsymbol{P}_{k|k} + \boldsymbol{G}_k\big(\boldsymbol{P}_{k+1|N} - \tilde{\boldsymbol{P}}_{k+1|k}\big)\boldsymbol{G}_k^{\mathrm{T}}. && (162) \end{aligned}}$$

The smoother gain $\boldsymbol{G}_k$ is computed by an $\mathrm{LDL}^{\mathrm{T}}$ solve, not an explicit inverse. The predicted covariance is symmetrised, then $\boldsymbol{G}_k = \big(\tilde{\boldsymbol{P}}_{k+1|k}^{-1}(\boldsymbol{P}_{k|k}\boldsymbol{F}^{\mathrm{T}})^{\mathrm{T}}\big)^{\mathrm{T}}$ via `ldlt.solve(...).transpose()`. The smoothed covariance Equation (162) is symmetrised after the update. If the factorisation fails, the step is skipped and epoch k keeps its forward posterior.

Variable-dimension reconciliation. Because $\dim \boldsymbol{x}_k \neq \dim \boldsymbol{x}_{k+1}$ in general, Equations (160) to (162) are restricted at each step to the *common-key subspace*, the states present at both k and $k{+}1$, found by intersecting the dynamic key→index maps (`collectCommonIndices`, less any keys in the handover exclusion set of the segment-boundary paragraph below). Let $\boldsymbol{\Pi}_k$ and $\boldsymbol{\Pi}_{k+1}$ select the common rows in each epoch's state space; the implemented state update is then

$$\hat{\boldsymbol{x}}_{k|N} = \hat{\boldsymbol{x}}_{k|k} + \boldsymbol{\Pi}_k^{\mathrm{T}} \boldsymbol{G}_{k,C}\big(\boldsymbol{\Pi}_{k+1}\hat{\boldsymbol{x}}_{k+1|N} - \boldsymbol{\Pi}_k\tilde{\boldsymbol{x}}_{k+1|k}\big), \tag{163}$$

with the analogous projection on Equation (162). States outside the subspace keep their forward a-posteriori value. The indexing needs care. $\tilde{\boldsymbol{x}}_{k+1|k}$, $\tilde{\boldsymbol{P}}_{k+1|k}$ and $\boldsymbol{F}_{k+1}$ are sized for epoch k's layout, because the predict step does not grow the state. They are addressed with epoch k's common indices. $\hat{\boldsymbol{x}}_{k+1|N}$ and $\boldsymbol{P}_{k+1|N}$ use epoch $k{+}1$'s layout.

Segment boundaries at resets and frame changes. Four mechanisms stop or restrict the backward flow of information at ambiguity resets, teardowns, and pivot handovers:

- A *complete* filter reset at $k{+}1$ (the covariance re-inflation of Section 12.4, recorded as `is_discontinuity`) is a hard segment boundary. It breaks the dynamic link $\tilde{\boldsymbol{x}}_{k+1|k} = \boldsymbol{F}\hat{\boldsymbol{x}}_{k|k}$, so $\boldsymbol{G}_k$ has no meaning. Backward flow stops and epoch k keeps its forward posterior.
- A *single-satellite* ambiguity reset is handled softly. That satellite's ambiguities are excluded from the common subspace for the $k \to k{+}1$ step. The re-seeded prior at $k{+}1$ has a large variance and carries no information backward, so smoothing it would only add noise. The exclusion matches by satellite prefix and is slightly broader than necessary. The extra states simply keep their forward values.
- A *dimension change* $\dim \tilde{\boldsymbol{x}}_{k+1|k} \neq \dim \hat{\boldsymbol{x}}_{k|k}$ is a hard boundary. Predict never grows the state, so a size mismatch means a long-gap teardown before predict (`Application::handleGap` prunes ambiguity, iono and sat-bias states before `predict()`). Across it, epoch k's indices would address the wrong states in the $k{+}1$ a-priori. Backward flow stops and epoch k keeps its forward posterior.
- A *BSSD pivot handover* between k and $k{+}1$ re-references one or more constellations in place. Each constellation is either reconciled or segmented, as described next.

BSSD pivot-handover reconciliation. When the pivot map changes between epochs ($\texttt{pivot_satellites}_k \neq \texttt{pivot_satellites}_{k+1}$) the smoothed posterior at $k{+}1$ is in the *new* pivot frame. The cached a-priori $\tilde{\boldsymbol{x}}_{k+1|k}$ was captured before the handover, in epoch k's layout, and is still in the *old* frame. The two cannot be compared directly in Equation (161). `Smoother::prepareHandoverFrame` makes them comparable, per constellation, on *copies* of the $k{+}1$ smoothed estimate. The reported $k{+}1$ result stays in its own frame. A handover that `PivotManager` recorded as *clean* (old and new pivots matching `clean_handovers`) is inverted back to the old pivot p_0, one signal group at a time, by the public static `Smoother::inverseReReferenceGroup`. This is the exact inverse of the forward re-reference. For a group with old-pivot datum row d,

$$x'(i) = x(i) - x(d), \quad x'(d) = 0, \qquad P'(i,\cdot) = P(i,\cdot) - P(d,\cdot), \quad P'(\cdot,j) = P(\cdot,j) - P(\cdot,d), \tag{164}$$

applied over *full* rows and columns, with the datum row and column set to zero last. Full rows are needed because the forward re-reference coupled the group to every other state. A signal group can be inverted only if the old pivot still has a live state on it *and* the forward step re-referenced that signal to the new pivot. Everything else is *segmented*: orphaned groups (the new pivot lacked the signal), disturbed or forced handovers, and the ionosphere and satellite-bias states of every handover constellation. Their keys are added to the `exclude_keys` set, which `collectCommonIndices` removes from the common subspace, and those states keep their forward posterior.

Static-position degeneracy. In static PPP the receiver position is nearly constant. The smoother gain for the position block then applies almost the same correction to every epoch, and the smoothed position is constant across epochs *by construction*. This is expected behaviour. The smoother improves the other states (troposphere, ionosphere, float ambiguities), and the position in kinematic runs, where it really changes from epoch to epoch.

13 Stochastic Model and Quality Control

This document covers the *measurement* noise of the estimator. It describes the strategies that fill the observation-noise covariance $\boldsymbol{R}$, and the a-posteriori quality-control (QC) layer that tests an epoch's residuals against that $\boldsymbol{R}$. The $\boldsymbol{R}$ here has the block-diagonal structure of Equation (8), and this document gives the variance each strategy puts in it. The QC layer uses the post-fit residual statistics that the filter already computes for outlier rejection, and presents them in the classical geodetic detection theory of Baarda and Teunissen [27].

Two topics are outside this document and are only cross-referenced. The first is the reaction to a detected blunder: the standardised-residual outlier path ($|s| > 6$), the Huber re-weighting, and the EWMA inflation of $\boldsymbol{R}$. These are in Section 12.4. This document explains how the QC statistics drive an *alternative* gate but does not repeat the dispatch. The second is the process-noise covariance $\boldsymbol{Q}$ (clock random walk, ZWD, ambiguity, IMU bias), which belongs to the dynamics and is described with the smoother (Equation (149)). Everything below is measurement noise.

13.1 The stochastic-model dispatch

Every undifferenced observation (r, i, s) is assigned a scalar variance by a strategy object selected from the `StochasticModelType` enum (`include/config/GnssConfigTypes.hpp`, the four-way {`Constant`, `Elevation`, `SNR`, `SatelliteBlock`} set). The dispatch is in `src/core/model/StochasticModel.cpp`. Each strategy returns a variance in m^2, which becomes one diagonal entry of $\boldsymbol{R}$. The three strategies that use geometry share a low-elevation guard. When $\sin\theta \leq 10^{-6}$ the elevation amplification is skipped and the unamplified variance is returned, so a satellite at the horizon does not receive a numerically infinite variance (`computeElevation`). The `Constant` strategy has no elevation term. It returns its a-priori variance directly (`computeConstant`) and never reaches the guard.

(a) Constant. This strategy returns the a-priori variance of unit weight unchanged,

$$\sigma^2_{r,i,s} = \sigma^2_{0,\mathrm{type}(s)}, \tag{165}$$

with σ_0 drawn from `filter.measurement_noise` (`computeConstant`). The other strategies scale this value.

(b) Elevation. The classical elevation-dependent model scales the zenith-referenced variance by an inverse power of $\sin\theta$,

$$\boxed{\sigma^2_{r,i,s} = \sigma^2_{\mathrm{zen}} \left(\sin\theta^i_r\right)^{-2p_{\mathrm{type}}}, \qquad p_P = p_\Phi = 1.0.} \tag{166}$$

The shipped defaults of `filter.code_elevation_power` and `filter.phase_elevation_power` are both one, so the shipped model is *symmetric*: code and phase both follow the legacy $\csc^2\theta$ variance law (`computeElevation`). The exponent is $-2p_{\mathrm{type}}$ because the strategy works in variance, not σ:

$\left(\sin\theta\right)^{-2p} = \left(\sin\theta^{-p}\right)^2$. Rows whose observable code does not begin with `L` take the code power. That is every row except carrier phase, including Doppler and the fallback paths with an empty `obs_type`.

Both exponents are configurable per observable class. The asymmetric setting $p_\Phi = 0.5$ is the one alternative with a physical argument, so the reasons it is an *option* and not the default are recorded here. It gives a $\csc\theta$ variance ($\csc^{1/2}\theta$ in σ), which amplifies much less at low elevation.

The argument for it has two parts. Phase noise near the horizon is dominated by carrier-loop SNR and by antenna phase-centre miscalibration, and neither follows the $\csc^2\theta$ law that governs code multipath and pseudorange path-length error. The milder amplification also keeps low-elevation phase within the useful range of the standardised-residual cycle-slip test, where a steeper law would deweight it until the test has no effect.

The argument against it is comparability: the code branch and every archived benchmark run at one. The loader therefore ships one (`GnssConfig::getPhaseElevationPower`, whose documentation gives the amplification ratios at a 10° mask).

(c) SNR (SIGMA-ε). When carrier-to-noise density is available the strategy uses the Hadás–Bosy variance-additive form [23]. Writing the measured (or synthesised) density as $\mathrm{C/N_0}$ in dB Hz,

$$\boxed{\sigma^2_{\mathrm{SNR}} = a^2 + b^2\,10^{-\mathrm{C/N_0}/10},} \tag{167}$$

where a is the high-SNR floor ($\sigma \to a$ as $\mathrm{C/N_0} \to \infty$) and b scales the SNR-dependent contribution; both have separate code/phase values (`snr_params_.code_a/b`, `phase_a/b`). Adding *variances*, as in Equation (167), is the correct way to combine two independent noise sources. The legacy alternative adds standard deviations, $\sigma = a + b\,10^{-\mathrm{C/N_0}/10}$. `computeSnr` rejects it, because squaring it produces a cross-term $2ab\,10^{-\mathrm{C/N_0}/10}$ that corresponds to no noise process: it treats the two independent contributions as correlated.

When no `S...` observable is present the model keeps the same functional form. It *synthesises* a density from elevation, so the noise curve stays continuous (`synthesizeCn0FromElevation`):

$$\mathrm{C/N_0}(\theta) = 50 - 15\left(1 - \max(0, \sin\theta)\right) \ \ [\mathrm{dB\,Hz}], \tag{168}$$

which is a maximum of 50 dB Hz at zenith, falling by 15 dB at the horizon. The density is then clamped to the configured working range [`snr_floor_dbhz`, `snr_ceil_dbhz`]. Two refinements follow.

On code rows, a per-band scale s^2_{band} from the SatelliteBlock code table multiplies σ^2_{SNR}. Wider-bandwidth signals such as L5/E5a have $\sim\sqrt{10}$ less correlator noise in σ, a factor of $\sim 1/10$ in variance. Phase has no band scale, because PLL noise depends on the carrier-loop bandwidth, not the chip rate.

The Wieser hybrid step then passes the SNR variance through Equation (166). Pure SNR weighting under-weights multipath that is correlated with elevation, because the carrier loop can track cleanly while the range is corrupted. The elevation factor accounts for it.

(d) SatelliteBlock. The most detailed strategy keys the zenith σ on the transmitting satellite's ANTEX block type, then applies observable-specific frequency-band and tracking-attribute scales (`computeSatBlock`):

$$\boxed{\sigma^2_{r,i,s} = \left(\sigma_{\mathrm{block}(i),\,\mathrm{type}}\; s_{\mathrm{band}}\; s_{\mathrm{attr}}\right)^2 \left(\sin\theta\right)^{-2p_{\mathrm{type}}}.} \tag{169}$$

Here $\sigma_{\mathrm{block}(i),\mathrm{type}}$ is looked up by substring-matching the satellite's ATX type string against `block_code_std_m` / `block_phase_std_m` (falling back to `default_code_std_m` / `default_phase_std_m`

when no block matches); s_{band} is the per-band factor keyed on the RINEX band character (`obs_type[1]`); and s_{attr} is the tracking-attribute factor keyed on `obs_type[2]` via a *per-system* table, because a RINEX-3 attribute means different things in different constellations: a `C` attribute is the noisy GPS C/A code but the high-precision Galileo E1C pilot. The attribute lookup order is `[system][attr]` → `[default][attr]` → 1.0. Code and phase have separate band and attribute tables (`SatBlockNoiseParams::ObservableScale`). Chip-rate calibration governs code, while carrier-tracking calibration governs phase and varies much less between bands. As with the other geometry strategies, the block variance is finally amplified by Equation (166). Phase reaches this path only when `apply_to_phase` is set. Otherwise it takes the flat `default_phase_std_m` route in the top-level dispatch.

All four strategies fill diagonal entries only, so the undifferenced $\boldsymbol{R}$ is block-diagonal, as in Equation (8). Off-diagonal correlation appears only after differencing (Section 13.5).

13.2 A-posteriori variance factor

The Kalman-side QC layer of this section and the next three is a diagnostic that is *off by default*. `KalmanFilter::update` calls `computeResidualQc_` only when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set. The correlation-aware kernel of Section 13.5 also requires `estimation.enable_full_r_kernel`. All three default to false. With them off, the only outlier gate is the fixed $|s| > 6$ path of Section 12.4, the variance factor stays at its initial value of one, and the redundancy numbers, w-tests and minimal detectable biases below are not computed.

After an epoch's update, the post-fit residuals $\hat{\boldsymbol{v}}$ show whether the assumed $\boldsymbol{R}$ was correctly scaled. The a-posteriori *variance factor* (estimated variance of unit weight) is

$$\boxed{\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \Omega = \hat{\boldsymbol{v}}^{\mathrm{T}} \boldsymbol{R}^{-1} \hat{\boldsymbol{v}} = \sum_i \frac{\hat{v}_i^{\,2}}{R_{ii}} \,\Bigg|_{\boldsymbol{R}\ \text{diag}}, \qquad \text{dof} = \operatorname{tr} \boldsymbol{\Theta},} \tag{170}$$

where Ω is the weighted sum of squared residuals and the degrees of freedom dof is the trace of the redundancy matrix $\boldsymbol{\Theta}$ (Equation (175) below). This is the *effective* redundancy of a recursively filtered epoch, not the $m - n$ of a single batch (`computeResidualQc_`, `src/core/filter/KalmanFilter.cpp`). $\boldsymbol{R}$ is in absolute units, so the a-priori variance of unit weight is $\sigma_0^2 = 1$. A well-scaled model gives $\hat{\sigma}_0^2 \approx 1$. $\hat{\sigma}_0^2 \gg 1$ means the assumed noise is too small (optimistic), and $\hat{\sigma}_0^2 \ll 1$ means it is too large (pessimistic).

The variance factor is used for *reporting only*. It never feeds back into the propagated covariance $\boldsymbol{P}$. When `enableVarianceFactor()` is set, the filter *logs* a copy of the position σ scaled by $\sqrt{\hat{\sigma}_0^2}$ and leaves the filter state unchanged (`validatePostFitResiduals`, `src/core/filter/KalmanFilter.cpp`).

The same quantity has a batch form in the normal-equation accumulator of the PPP NEQ-export path (constructed only under `enableNeqExport()` in `Application::run`). Given normal matrix $\boldsymbol{N}$, right-hand side $\boldsymbol{u}$, and the pre-reduced $\boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l}$ term, the accumulator forms (`NeqAccumulator::solve`)

$$\Omega = \boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l} - \boldsymbol{u}^{\mathrm{T}}\hat{\boldsymbol{x}}, \qquad \text{dof} = n_{\text{obs}} - (n_{\text{params}} + n_{\text{local}}), \tag{171}$$

$$\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \sigma_i = \sqrt{\hat{\sigma}_0^2 \left(\boldsymbol{N}^{-1}\right)_{ii}}. \tag{172}$$

Here $\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{u}$ is the least-squares solution. The redundancy subtracts the Schur-reduced local parameters n_{local} as well as the retained parameters n_{params}. The formal σ of each retained parameter

is the square root of the corresponding diagonal of the inverse normal matrix, scaled by $\hat{\sigma}_0^2$. This is the batch counterpart of the recursive dof in Equation (170).

The network producer uses neither of these as its *operational* variance factor. The undifferenced network estimator does not form Ω/dof. It estimates a robust unit-weight σ as a scaled median absolute deviation over the weighted phase-leveling residual rows $\tilde{v}_i = |\hat{v}_i|\sqrt{w_i}$,

$$\hat{\sigma}_{0,\mathrm{net}} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \mathrm{median}_i\,\tilde{v}_i\big), \tag{173}$$

where the factor 1.4826 converts the MAD to a Gaussian σ and `raim_sigma0_floor` (default 1.0) is a lower bound. Unlike the Kalman variance factor of Equation (170), $\hat{\sigma}_{0,\mathrm{net}}$ is *not* used for reporting only. It controls three things:

- a within-epoch RAIM adaptation loop, whenever it exceeds `network.raim_sigma0_trigger`;
- a whole-epoch and a per-constellation product withhold, above `network.raim_withhold_sigma0`;
- the scale of the emitted URA.

Three separate seams implement them. `networkSolveGlue` (`filter/NetworkSolveGlue`) owns the within-epoch loop and both withhold gates, the per-constellation one through `NetworkEstimator::decidePerConstWithhold`. `networkPostFitRaim` (`filter/NetworkRaim`) owns the post-fit pass and the exponentially weighted fold of $\hat{\sigma}_{0,\mathrm{net}}$ into the emitted URA scale. `networkSolPrep` (`filter/NetworkSolPrep`) owns the emission-side prune that removes a withheld constellation's satellites. Sections 13.6 and 13.7 develop the adaptation loop and the pooled statistics. The emitted product is not treated here.

The network path has a third variance-factor instrument, which is not operational. `NetworkCarryVce` estimates Floor and Rate variance components of the carry-row stochastic model, by residual-form least-squares variance-component estimation at fixed weight. Its only purpose is to check the hand-set carry floors against a measured $\hat{\sigma}$. It is off by default (`network.carry_vce`) and never feeds back into the solve. The feedback setting `network.carry_vce_apply` is a hard error at configuration load, and the class has no apply method.

13.3 Redundancy numbers, the Baarda w-test, and the MDB

The detection theory rests on the post-fit residual covariance $\boldsymbol{Q}$ (the covariance of $\hat{\boldsymbol{v}}$),

$$\boldsymbol{Q} \;=\; \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}^{+}\,\boldsymbol{H}^{\mathrm{T}}, \tag{174}$$

where $\boldsymbol{P}^{+}$ is the *updated* (post-fit) state covariance. `KalmanFilter::update` builds the diagonal $\boldsymbol{Q}_{ii}$ per row as $R_{ii} - (\boldsymbol{H}\boldsymbol{P}^{+}\boldsymbol{H}^{\mathrm{T}})_{ii}$. The full matrix is formed only under the off-diagonal kernel of Section 13.5.

Redundancy number. The (local) redundancy of observation i is

$$\boxed{r_i \;=\; \frac{\boldsymbol{Q}_{ii}}{R_{ii}} \;=\; (\boldsymbol{\Theta})_{ii}, \qquad \boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i \in [0,1],} \tag{175}$$

clamped to $[0,1]$ for numerical safety in both branches of `computeResidualQc_`. When $r_i \to 1$ the geometry checks the observation fully, and a blunder appears almost entirely in the residual. When $r_i \to 0$ the observation is *uncheckable*: the estimate absorbs a blunder and the residual

shows nothing. The sum dof $= \sum_i r_i = \operatorname{tr} \boldsymbol{\Theta}$ is the effective number of degrees of freedom used in Equation (170).

The network producer computes Equation (175) in the batch (arrowhead) algebra of the Schur-conditioned shared covariance, $r_i = 1 - w_i \, \boldsymbol{a}_i^{\mathrm{T}} \operatorname{Cov}(\boldsymbol{x}) \, \boldsymbol{a}_i$ with $\boldsymbol{a}_i = [\, \boldsymbol{a}_{s,i} \mid \boldsymbol{a}_{r,i} \,]$ and the same $[0, 1]$ clamp. This is algebraically the batch form of $r_i = \boldsymbol{Q}_{ii}/R_{ii}$ (`NetworkNormalEquations::rowRedundancy`, gated by `network.raim_redundancy_normalize`, default off). Rows below the `kRedFloor` redundancy floor are uncheckable and are dropped from the $\hat{\sigma}_{0,\mathrm{net}}$ pool of Equation (173). This corresponds to the `kUncheckableRedundancy` cutoff of the KF snooping gate below. The network constant is much smaller, because it protects only the $\sqrt{r_i}$ standardisation and not a reset decision.

One case of $r_i = 0$ is exact, and it determines the design of the network producer's phase-side QC. Suppose each (station, satellite, signal) arc has its own free ambiguity, which is the case whenever `network.phase_leveling` is off. Then every phase row is fitted *exactly* by its own parameter. The post-fit phase residual is identically zero, r_i is exactly zero, and the residuals cannot reveal a phase fault of any size. An undifferenced network with per-arc float ambiguities has *no* phase-side redundancy.

The phase rows gain redundancy, and so the power to detect faults, only when a carried prior constrains the ambiguities. That is why the producer's phase RAIM (Section 13.6) requires phase-leveling: without it the test has no power at all. The measure-only instrument `networkPostfitSigma0` (`filter/NetworkFrameDiag`) reports the effect. Its pooled phase variance factor is identically zero in that configuration, and the zero is the correct result. Its pools exclude two classes of row for related reasons. Rows that touch a shared spherical-harmonic column are pseudo-observations, not data (Section 13.8). The carry-coupling rows describe the carry, not the epoch's measurements.

w-test. Baarda's data-snooping statistic standardises each residual by its own post-fit standard deviation. The diagonal (uncorrelated) form is

$$w_i \;=\; \frac{\hat{v}_i}{\sqrt{\boldsymbol{Q}_{ii}}} \tag{176}$$

(the diagonal branch of `computeResidualQc_`). The divisor is $\sqrt{\boldsymbol{Q}_{ii}}$, the *true* residual standard deviation, not $\sqrt{R_{ii}}$. This is the Studentised residual, and under the null hypothesis $w_i \sim \mathcal{N}(0, 1)$. The s used by the fixed gate floors the denominator at R and is therefore conservative. When the residual covariance is not diagonal, the correlated Teunissen generalisation is used (the `use_full` branch of the same routine):

$$\boxed{w_i \;=\; \frac{\left(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\right)_i}{\sqrt{\left(\boldsymbol{Q}^{-1}\right)_{ii}}}.} \tag{177}$$

Equation (177) reduces to Equation (176) when $\boldsymbol{Q}$ is diagonal, but accounts for the off-diagonal correlation among residuals introduced by differencing [25, 27].

Minimal detectable bias. The smallest blunder on observation i that the w-test will detect, at the configured significance and a conventional 80% test power, is

$$\boxed{\mathrm{MDB}_i \;=\; \frac{\sigma_i \, \delta_0}{\sqrt{r_i}}, \qquad \delta_0 = \texttt{dataSnoopingCritical()} + 0.8416,} \tag{178}$$

with $\sigma_i = \sqrt{R_{ii}}$ the a-priori observation σ (both branches of `computeResidualQc_`). The non-centrality δ_0 is the critical value plus $\Phi^{-1}(0.80) = 0.8416$, for a test power of 80% ($\beta = 0.20$). For the default critical value $k = 3.29$ this gives $\delta_0 \approx 4.13$. The factor $1/\sqrt{r_i}$ expresses the geometry. A poorly checked observation ($r_i \to 0$) has an arbitrarily large MDB, and the code returns $+\infty$ when r_i is below `kRedFloor`. A blunder on such a row cannot be detected from the residual. The MDB exists only in the Kalman-filter QC. The network producer computes no minimal detectable bias and relies on the robust $\hat{\sigma}_{0,\text{net}}$ RAIM loop of Equation (173).

13.4 Data-snooping gate override

When `enableDataSnooping()` is set, the QC statistics drive an *alternative* outlier gate, which can select a row that the fixed $|s| > 6$ path missed. Both gates use the same downstream dispatch in `KalmanFilter::validatePostFitResiduals`. Only the *statistic, index, and critical value* differ. By default the gate is seeded from the fixed-σ triple ($\max_i |s_i|$, arg max, 6), so the dispatch is byte-identical when snooping is off.

When snooping is enabled the gate is seeded from the w-test over *checkable* rows only. Rows with redundancy below the `kUncheckableRedundancy` cutoff are skipped. Their residuals are unreliable, and resetting the corresponding ambiguity would discard good information. The same cutoff filters the summary log, so the log and the gate agree. Over the remaining rows the gate fires on

$$\max_{\{i\,:\,r_i \geq \texttt{kUncheckableRedundancy}\}} |w_i| \;>\; \texttt{dataSnoopingCritical()}, \tag{179}$$

and the checkable row with the largest $|w_i|$ becomes the handled index. w_i uses the true residual σ (Equation (176)), not the R-floored s. The gate can therefore catch a blunder on a high-leverage row whose floored s stayed below 6, and in that case it overrides the s choice. The downstream Melbourne–Wübbena tiebreaker still works in the σ domain. When the w-test selects a row that the σ cut missed, no σ candidate exists, and the w-selected index is handled directly. The fixed standardised-residual path is described in Section 12.4.

13.5 Correlation-aware full-R kernel

The diagonal forms above assume that $\boldsymbol{R}$ and $\boldsymbol{Q}$ are diagonal, which is true for undifferenced PPP. For BSSD and baseline-DD epochs the differencing operator $\boldsymbol{D}$ of Section 5 turns the diagonal undifferenced covariance into a *dense* one, $\boldsymbol{R}_{\text{SD/DD}} = \boldsymbol{D}\,\boldsymbol{R}_{\text{ZD}}\,\boldsymbol{D}^{\mathrm{T}}$. It contains the cross-covariance between observations that share a pivot or reference. The off-diagonal kernel (`estimation.enable_full_r_kernel`, default off) also requires that a dense $\boldsymbol{R}$ was supplied for the epoch, which `KalmanFilter::update` checks with `dense_r_supplied`. Under it the full residual covariance Equation (174) is built densely and symmetrised, and the QC quantities follow without approximation:

$$\boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i = \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \text{dof} = \operatorname{tr}\boldsymbol{\Theta}, \tag{180}$$

$$\Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}, \qquad w_i = \frac{\big(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{(\boldsymbol{Q}^{-1})_{ii}}}, \qquad \text{MDB}_i = \frac{\sqrt{R_{ii}}\,\delta_0}{\sqrt{r_i}}. \tag{181}$$

$\boldsymbol{R}^{-1}$ and $\boldsymbol{Q}^{-1}$ come from LDL^{T} solves against the identity, since both matrices are symmetric positive definite. When the off-diagonals vanish, the diagonal of the full $\boldsymbol{\Theta}$ and the full Teunissen w equal the per-row quantities of Section 13.3. The *outlier-handling path is therefore unchanged behind the flag.* The flag only makes the statistics correct on differenced epochs, where the residuals really are correlated. Its scope is narrow. The gain and the Joseph-form update already use a dense

$\boldsymbol{R}$ whenever one is supplied, with or without the flag. The flag decides only whether the *post-fit residual* covariance is formed densely. Section 5 describes the corresponding correlation on the model side.

13.6 Producer-side RAIM: within-epoch adaptation

When $\hat{\sigma}_{0,\mathrm{net}}$ exceeds its trigger, the network producer chooses the *granularity* of its response from a robust two-way decomposition of the epoch's standardised residuals. Taking the largest standardised residual m_{ri} of each (station r, satellite i) carried-ambiguity cell as a sparse two-way table and applying a Tukey median polish,

$$m_{ri} \;\approx\; \mu \;+\; \alpha_r \;+\; \beta_i \;+\; \gamma_{ri}, \tag{182}$$

splits the contamination into a grand effect μ, a per-station row effect α_r, a per-satellite column effect β_i, and an isolated-arc interaction γ_{ri} (`medianPolish2Way`, `filter/NetworkRobustStats`). The polish sweeps medians, not means, so a minority of very large cells (the outliers) does not distort the additive effects. A bad satellite, which raises a whole column, can then be told apart from a bad station, which raises a whole row. Isolated arc slips remain in the interaction term.

Each effect is a candidate culprit. To be eligible it must satisfy effect$/\hat{\sigma}_{0,\mathrm{net}} > k_{\mathrm{RAIM}}$ for the configured gate (`network.raim_k_sigma`). The eligible candidates are ranked with the narrowest response first: on equal score an arc ranks above a satellite, and a satellite above a station. The station branch is also reduced by a bias factor (`network.raim_station_drop_bias`, active under `network.raim_surgical_station`). Excluding a station discards good arcs with the bad, so it is chosen only when it dominates by that margin. A consensus rule avoids removing arcs one at a time. When at least `network.raim_min_stations` of one satellite's arcs exceed the gate in one iteration, the arc candidate is replaced by the whole satellite. A corrupted *shared* clock or phase bias is a network-wide fault, not a set of independent local ones.

The response distinguishes a bad *measurement* from a stale *parameter*. This distinction is the central design decision. A station culprit is excluded, because its data are bad and exclusion is the right response to bad data. A satellite or arc culprit is almost always a cycle slip. A slipped ambiguity does not need less weight; it needs a new *value*. The epoch is therefore solved again on a rebuilt normal-equation system with the culprit's ambiguities *refloated*. Their measurements keep full weight, so the local block stays positive definite and no singularity is introduced. Their carried prior is withheld, so the new solve can find the new value and absorb the slip locally. The loop repeats until $\hat{\sigma}_{0,\mathrm{net}}$ falls below the trigger or a small fixed number of passes is reached. Only the rare contaminated epochs pay for a second solve, and any failure reverts to the last good solution (`networkSolveGlue`).

`network.raim` and `network.raim_within_epoch` both default to false. *Both* tiers also require the phase-leveling carry to be warm: the within-epoch loop inside `networkSolveGlue`, and the post-fit pass at the `networkPostFitRaim` call site. With `network.phase_leveling` off there are no carried ambiguities to refloat. By the redundancy argument of Section 13.3 there is also no phase-side redundancy to test. The producer's RAIM then does nothing, as a consequence of the model and not of a policy choice.

13.7 Pooled variance factors and the per-constellation carve-out

Equation (173) pools only the carried-ambiguity *phase* rows. Two extensions widen or partition that set of rows without changing the estimator. Both are off by default.

Code and iono-constraint pools. Originally no tier screened the code rows or the iono-constraint rows that tie each line of sight to the spherical-harmonic field. Under `network.raim_we_code_iono` the same scaled-MAD estimator is applied to each of them over its own disjoint set of rows,

$$\hat{\sigma}_{0,\text{code}} = 1.4826\ \text{median}_{i\in\mathcal{C}}\,\tilde{v}_i, \qquad \hat{\sigma}_{0,\text{iono}} = 1.4826\ \text{median}_{i\in\mathcal{I}}\,\tilde{v}_i, \tag{183}$$

each with its own trigger. Every row belongs to exactly one of the three sets. Neither new pool has the floor of Equation (173), and the phase pool still drives the whole-epoch ceiling. The adaptation gains two responses suited to what these rows can diagnose: a station exclusion for the epoch, and a network-wide *deweighting* of one satellite's code rows. The second is a fallback and not a drop, in line with the project's preference for degrading a measurement over discarding it.

Per-constellation carve-out. When the global ceiling trips, the default response withholds the whole epoch's product. Under `network.per_const_withhold` the same pool is instead partitioned by the ambiguity's satellite constellation and each part judged on its own,

$$\hat{\sigma}_{0,c} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \text{median}\{\tilde{v}_i : i \in \mathcal{P}_c\}\big), \tag{184}$$

over the same rows $\mathcal{P}_c$ that the global statistic pools. The statistic and the ceiling are the same, and only the set of rows is restricted. A constellation is published if and only if $\hat{\sigma}_{0,c}$ is below that ceiling *and* $|\mathcal{P}_c|$ meets a minimum row count. Too few rows cannot show that a constellation is clean inside a contaminated epoch (`NetworkEstimator::decidePerConstWithhold`).

This is an approximation, in two respects. The residuals in Equation (184) are evaluated at the *joint* solution. A contamination in one constellation perturbs the shared per-station nuisance parameters (receiver clock, tropospheric delay, slant ionosphere) and leaks into the residuals of another. Each row's redundancy is also its share of the joint system's redundancy, not of a separate per-constellation solve. $\hat{\sigma}_{0,c}$ therefore under-separates coupling between constellations. The outcome "every constellation individually clean while the global statistic tripped" is a consequence of that approximation. It is published with a warning and counted. The rank and observability gate keeps whole-epoch semantics. A rank deficiency invalidates the declared datum, so the product would be a meaningless least-norm split and no healthy subset exists.

13.8 Station-relative deweighting of the ionosphere constraint

A single-layer spherical-harmonic VTEC surface cannot represent the slant structure of individual lines of sight. Most LOS residuals against the fitted field exceed 3σ at any degree and any network size. The exceedance is a persistent background that is roughly common to all stations, so an *absolute* 3σ gate would engage every station and diagnose nothing. The difficult sites are equatorial-anomaly and scintillation-belt stations, whose true slant ionosphere the field cannot represent. They stand out only as a *relative* excess. The gate below is therefore robust and relative to the network (`NetworkIonoDeweightGate`).

Each station's per-epoch misfit statistic m_r is first smoothed by an exponentially-weighted moving average with forgetting factor $\phi = 2^{-1/W}$, i.e. a half-life of W epochs, then standardised against the network's own robust centre and scale. Writing $\mathcal{M}$ for the set of mature, non-statically-listed stations,

$$z_r \;=\; \frac{\bar{m}_r - \text{med}}{1.4826\,\text{MAD}}, \qquad \text{med} = \text{median}_{r'\in\mathcal{M}}\,\bar{m}_{r'}, \qquad \text{MAD} = \text{median}_{r'\in\mathcal{M}}\big|\bar{m}_{r'} - \text{med}\big|. \tag{185}$$

Centring on the median removes the common background. The MAD scale tolerates a contaminated minority, with a breakdown point of 50%, which an absolute threshold does not have. The robust

centre is formed only when enough mature stations are available. Statically listed stations are excluded from it, so a known bad site cannot mask the detection of others.

Detection is hysteretic: a station engages at $z_r > \sigma_{\mathrm{hi}}$ and releases at $z_r < \sigma_{\mathrm{lo}}$, with σ_{lo} forced below σ_{hi}. While engaged the target weight follows a smooth IGG-style ramp, unity at the release threshold and equal to a configured floor $w_{\min}$ at and above the engage threshold, quadratic between:

$$\boxed{w_r^* = w_{\min} + (1 - w_{\min})\,(1 - t)^2, \qquad t = \frac{\mathrm{clamp}\big(z_r,\, \sigma_{\mathrm{lo}},\, \sigma_{\mathrm{hi}}\big) - \sigma_{\mathrm{lo}}}{\sigma_{\mathrm{hi}} - \sigma_{\mathrm{lo}}}.} \tag{186}$$

The realised weight slews toward w_r^* by at most a fixed increment per clean epoch and is clamped to $[w_{\min}, 1]$. The slew limit stops the gate oscillating in response to its own effect on the field fit.

The gate acts only on $\boldsymbol{R}$. The factor w_r multiplies the *information* of that station's pseudo-observation rows between the spherical-harmonic field and the slant ionosphere, which is equivalent to $\sigma^2 \mapsto \sigma^2 / w_r$. Those rows form one joint pseudo-observation that couples the station's per-LOS slant ionosphere to the shared field in both directions, so one w_r relaxes both. As $w_r \to 0$ the station's ionosphere decouples from the field and is determined by its own geometry-free combination. It stays observable and no rank deficiency appears. The station still contributes to the shared clocks, biases, datum graph and position. This is a narrower response than dropping a difficult station. It changes weights only: no state is removed and no datum is altered.

Two limits apply. Epochs with a datum change, including a pivot handover, are excluded. The gate is neither fed nor advanced on them, because their residual spikes come from the datum change and not from station misfit, and every weight is held across them. The adaptive path is off by default (`network.iono_constraint_deweight_adaptive`). With it off and the static station list empty, every w_r is one and the producer is byte-identical to a build without the gate. The gate is advanced once per epoch from the serial post-solve harvest in `networkShHealth` (`filter/NetworkShHealth`) and is read, const and free of races, during the parallel per-station assembly.

13.9 State-domain emission gates

Everything above tests *residuals* against $\boldsymbol{R}$. The producer also has two gates that test the *solved state* and act on the emitted product, not on the estimation. They are a different kind of test, and neither lives in the seam whose name suggests it.

Spherical-harmonic field health. Under `network.sh_health_gate` (default off) the solved VTEC coefficients are read back out of the shared solution, the dual-layer degree-zero terms are summed to a total mean VTEC, and that value is tested against a physically admissible band together with a ceiling on the RMS of all coefficients. An out-of-band epoch must repeat for a configured number of consecutive epochs before the gate engages, so a single-epoch excursion does not trip it. The verdict affects *emission only*. The solve, the carry stash and every QC tier continue to run while emission is withheld, so an era lasting hours does not expire the carry, as skipping the epochs would. If a coefficient lookup fails, the gate state is left unchanged; a release is never invented. The gate has no policy for release or degradation: a bad era withholds the product for its full duration. That is an open policy item, not a design conclusion. The verdict is formed in `networkSolveGlue`, the only seam before emission that holds the solved shared vector. `networkShHealth` (`filter/NetworkShHealth`) owns the health *probe*, the deweight fold of Section 13.8 and the carry variance-component update, but not this verdict.

Clock-runaway quarantine. Under `network.clock_runaway_check` (default off) the epoch's satellite clock solutions are grouped by constellation, and a constellation is dropped from the emission when the median of $|\text{clock} - \text{median}_{\text{all}}|$ exceeds a magnitude ceiling, or the median |drift| exceeds a rate ceiling. Like the carve-out of Section 13.7, this reduces the scope of the product and does not withhold all of it. The code documents one hazard and does not defend against it. The magnitude test is referenced to the global median over *all* clock satellites. If the runaway constellation supplies half or more of them, the comparison inverts: the healthy minority exceeds the ceiling and is quarantined, and the runaway passes. The degenerate single-constellation case is warned at configuration load. The quarantine is in `networkSolPrep` (`filter/NetworkSolPrep`). `networkClockQc` (`filter/NetworkClockQc`) is a different tier. It contains the two-state oscillator model with its normalised-innovation fault test, the decoupled emit-clock-rate estimator, and the per-satellite sanity monitor for carried clocks.

14 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management

The Kalman filter of Section 12 is a *local* linear estimator. Every update assumes the state is close to the linearisation point. Every dynamic state (ambiguity, per-satellite ionosphere, receiver code bias, satellite bias, or inter-system offset; Section 3) needs a value and a covariance before it can be updated. This document covers the machinery that provides them:

- the weighted-least-squares (WLS) epoch solve, which places the receiver close enough for the linearisation to hold (Section 14.1);
- the first-epoch seeds formed from residual differences, which give each new state a physically reasonable prior (Section 14.2);
- the between-satellite pivot selection and disturbance logic, which keep the single-difference (SD) reference stable across an arc (Section 14.3).

None of this is filtering. It is what makes the filter's local-linear assumptions valid at the first epoch and after every discontinuity.

The lossless covariance re-referencing (the $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ S-transform that moves a group onto a new pivot) is *out of scope* here. It is derived in Section 5 and implemented once in `include/filter/CovarianceReReference.hpp`, which both the single-receiver and the network handover paths call. This section notes where the pivot logic calls it.

14.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed

Before the filter can run, the receiver position must be close enough to the truth that the geometry Jacobian Equation (6) is valid to below a decimetre. Broadcast pseudoranges are sufficient for this. The engine runs a GPS-only, code-only iterated Gauss–Newton solve over the core state

$$\boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{r}^{\mathrm{T}} & c\,\mathrm{d}t_r & \mathrm{ZWD} \end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^5, \tag{187}$$

with $\boldsymbol{r} \in \mathbb{R}^3$ the ECEF receiver position, $c\,\mathrm{d}t_r$ the receiver clock in metres, and ZWD the zenith wet delay (`src/core/math/WlsEstimator.cpp`). At least `kCoreSize` $= 5$ code observations are required before the solve is attempted; the iteration cap is `kMaxIter` $= 5$.

For each code measurement i the prefit residual removes every modelled term and the current state's contribution,

$$\widetilde{z}_i = P_i - \left[\, P_i^{\mathrm{mod}} + (\rho_i - \rho_i^0) + c\,\mathrm{d}t_r + m_w(i)\,\mathrm{ZWD} \,\right], \tag{188}$$

where P_i^{mod} is the modelled pseudorange, $\rho_i - \rho_i^0$ the change in geometric range relative to the range at the iteration's linearisation point, and $m_w(i)$ the wet mapping function (`WlsEstimator::predictObs`, evaluated once per row from the design-row loop of `WlsEstimator::estimate`). The geometric range is *re-evaluated every iteration* about the current position estimate, so each iteration is a Gauss–Newton relinearisation.

The design row for satellite i is the negated line-of-sight stacked with the clock and wet-mapping partials, assembled in the same loop:

$$\boldsymbol{A}_i = \begin{bmatrix} -\boldsymbol{e}_i^{\mathrm{T}} & 1 & m_w(i) \end{bmatrix} = \begin{bmatrix} -e_x & -e_y & -e_z & 1 & m_w(i) \end{bmatrix}, \tag{189}$$

with $\boldsymbol{e}_i$ the receiver-to-satellite unit vector (the same line-of-sight that drives Equation (7)). Stacking all rows into $\boldsymbol{A}$, the prefit residuals into $\widetilde{z}$, and the inverse measurement variances onto the diagonal weight $\boldsymbol{W}$ (clamped from below by `kMinVariance` $= 10^{-6}$ in that same loop), the normal-equations step is the boxed Gauss–Newton update

$$\boxed{\Delta\boldsymbol{\xi} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\widetilde{z},} \tag{190}$$

applied once per Gauss–Newton iteration of `WlsEstimator::estimate`. Iteration terminates on a position increment $\|\Delta\boldsymbol{r}\| <$ `kTol` $= 0.1\,\mathrm{m}$ or at `kMaxIter`. The normal matrix is guarded against rank deficiency: if

$$\det\!\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right) \le 10^{-6} \tag{191}$$

the solve aborts and the epoch is not seeded (the determinant guard immediately before the step). This protects against degenerate single-constellation, nearly coplanar geometry, which would otherwise produce a very large position step.

On convergence the estimator returns both the state $\boldsymbol{\xi} = [\boldsymbol{r}^{\mathrm{T}},\ c\,\mathrm{d}t_r,\ \mathrm{ZWD}]^{\mathrm{T}}$ and its formal covariance, the inverse normal matrix

$$\boldsymbol{P}_{\boldsymbol{\xi}} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1} \tag{192}$$

(the result assembly at the end of `WlsEstimator::estimate`). The filter uses only the *clock* from this result. `Application::handleWlsSeeding` writes $c\,\mathrm{d}t_r$ into the receiver-clock state when one of three triggers fires: the first epoch, a data gap, or a clock *jump* of at least `kClockJumpThreshSec` $= 0.5\,\mathrm{ms}$ between the WLS and filter clock estimates. In practice the jump is the most common trigger.

The position, the zenith wet delay, and the covariance (192) are computed but are not used as filter seeds. The position prior is the RINEX-header `approx_position` plus the antenna offset. The WLS position re-seeds the filter only on a kinematic (non-IMU) cold start after a long gap (`Application::handleGap`). The ZWD estimate goes only to a trace-CSV column, and nothing in the filter reads the formal covariance. The whole seeding path is skipped under BSSD, where the filter has no receiver-clock state (`handleWlsSeeding` returns early on `!hasClock()`).

The main use of the clock seed is *re-seeding after a gap*. The receiver clock is the most volatile state across a data gap, because an unsteered oscillator can drift freely while no measurements constrain it. After a gap the WLS clock estimate resets $c\,\mathrm{d}t_r$ before the phase filter resumes. Otherwise the filter would correct a stale clock prior through a series of large innovations.

14.2 First-epoch state seeding by residual differencing

The WLS solve of Section 14.1 seeds the receiver clock and, on a kinematic cold start, the position. It does not touch the *differential* states: inter-system bias, phase clock, uncombined receiver code bias, per-satellite ionosphere, and carrier ambiguity. Each of these is created empty and must be seeded from the measurement residuals. The common idea (`src/core/measurement/StateInitializer.cpp`) is that each state is what remains in a particular *difference* of residuals after all modelled terms are removed. The seed is that difference, taken as a median over satellites for robustness. The shared prefit residual is

$$\widetilde{z}^{\,i}_{r,s} = \Phi/P^{i}_{r,s} - \big(\text{modelled} + c\,\mathrm{d}t_r + m_w\,\mathrm{ZWD} + \mathrm{ISB} + g_{\mathrm{trop}} + \alpha_s \widehat{I}^{\,i} + \beta_s b_r + b^i\big), \tag{193}$$

the observation minus every term the model can already account for, evaluated at the current state (`StateInitializer::computeRawResidual`, which also forms the tropospheric gradient projection $g_{\mathrm{trop}} = m_g(g_N \cos\mathrm{az} + g_E \sin\mathrm{az})$). The five *differential* seeds (a)–(e) below are differences of (193); a sixth, network-only pre-seed (f) anchors the receiver clock itself.

(a) Inter-system bias. Each non-GPS constellation g carries a receiver clock offset relative to the GPS time scale (Section 3). The GPS clock is the datum, and the WLS step has already put it into $c\,\mathrm{d}t_r$. The ISB of g is therefore the median code residual of the constellation minus the median GPS residual:

$$\boxed{\mathrm{ISB}_g = \mathrm{med}_{i\in g}\ \widetilde{z}^{\,i} - \mathrm{med}_{i\in \mathrm{GPS}}\ \widetilde{z}^{\,i}} \tag{194}$$

(`StateInitializer::initializeInterSystemBiases`). Medians are used throughout, so one gross code outlier cannot spoil the seed. A guard requires at least one GPS code observation at the seeding epoch (the GPS-median guard in the same routine). If GPS is absent the routine returns `false` and *tries again at the next epoch*. It does not mark the ISB states as initialised against an arbitrary datum. Without the guard, whichever constellation is acquired first would become the implicit zero, and a GPS satellite rising later would then carry the whole offset.

(b) Phase clock. In the uncombined model the phase observations see a receiver clock that differs from the code clock by the receiver phase–code hardware delay (Section 3). Its seed is the median over satellites carrying *both* a code and a phase observation of the code-minus-phase residual:

$$\delta_\Phi = \mathrm{med}_i\ \big(\widetilde{z}^{\,i}_P - \widetilde{z}^{\,i}_\Phi\big) \tag{195}$$

(`StateInitializer::initializePhaseClockOffset`). The covariance of the seed is set from the code noise. It is tighter than the default initial standard deviation, and it does not depend on the number n of differences in the median (the covariance-seeding code at the end of that routine):

$$\sigma_{\delta_\Phi} = \max\!\Big(\tfrac{\sigma_P}{\sqrt{n}},\ \sigma_P\Big), \qquad \boldsymbol{P}(\delta_\Phi,\delta_\Phi) = \sigma^2_{\delta_\Phi}, \tag{196}$$

The corresponding row and column of $\boldsymbol{P}$ are set to zero first, so the seeded state starts uncorrelated. Since $\sigma_P/\sqrt{n} \le \sigma_P$ for every $n \ge 1$, the max always selects σ_P, and the seeded variance is σ_P^2 whatever the value of n. The seed is floored at the noise of a single code measurement and does not improve as $1/\sqrt{n}$.

(c) Uncombined receiver code bias / IFB. In the uncombined PPP-RTK model each non-reference code band carries a receiver differential code bias (DCB) or inter-frequency bias (IFB) state relative to the constellation's BIA clock-reference band (Section 3). Forming the residual difference between band j and the reference band, removing the differential ionospheric contribution (the two bands scale the same slant ionosphere by different α), and dividing by the difference of receiver-bias Jacobian coefficients gives

$$\boxed{\widehat{b}_{r,j} = \frac{\Delta\widetilde{z}_j - \Delta(\alpha_j\,\widehat{I})}{\Delta\beta_j} = \frac{(\widetilde{z}_j - \widetilde{z}_{\mathrm{ref}}) - (\alpha_j - \alpha_{\mathrm{ref}})\,\widehat{I}^i}{\beta_j - \beta_{\mathrm{ref}}}} \tag{197}$$

(`StateInitializer::initializeCodeBiases`), median-aggregated over satellites and subject to the same $|\widehat{b}| > 200\,\mathrm{m}$ reject as the ionosphere seed below. The Jacobian coefficient β_s depends on the bias parameterisation (`StateInitializer::resolveReceiverBias`): for a pure IFB state it is the unit coefficient $\beta = 1$, whereas for a geometry-free / ionosphere-free DCB state against reference bands of frequencies f_1, f_2 it is

$$\mu_1 = 1, \quad \mu_2 = \frac{f_1^2}{f_2^2}, \quad \Delta\mu = \mu_1 - \mu_2, \qquad \beta_j = \begin{cases} \mu_1/\Delta\mu & j = \text{band 1} \\ \mu_2/\Delta\mu & j = \text{band 2}, \end{cases} \tag{198}$$

guarded by $|\Delta\mu| \geq 10^{-12}$ to avoid dividing by a near-degenerate frequency separation.

For GLONASS a third branch runs before both cases. Under `enable_glonass_ifb` each FDMA band (G1, G2) has one bias state *per FCN slot* k with $\beta = 1$ (`StateKeys::makeGlonassPerSlotIfbKey`). The per-slot carrier $f = f_{\mathrm{base}} + k\,\Delta f$ makes the receiver group delay depend on the slot, which one state per band cannot represent. The central channel $k = 0$ is the datum and has no state. The CDMA bands (G3/G4/G6) use the per-band treatment above. The branch runs before the clock-reference lookup because GLONASS usually has no clock-reference pair in the `.BIA` product, and the routine would otherwise return before reaching any IFB logic. With the flag off, a non-reference GLONASS band has no IFB state (the per-slot and non-reference-band branches of `resolveReceiverBias`).

(d) Per-satellite slant ionosphere. Each tracked satellite's slant ionospheric delay (Section 3) is recovered from the geometry-free combination of two code bands. With ionospheric scale factors $\alpha_s = f_{\mathrm{ref}}^2/f_s^2$ referenced to GPS L1 (Equation (1)), the seed is

$$\boxed{\widehat{I}^i = \frac{(P_1 - P_1^{\mathrm{mod}}) - (P_2 - P_2^{\mathrm{mod}})}{\alpha_1 - \alpha_2}} \tag{199}$$

(`StateInitializer::computeIonosphereSeed`), where P_k^{mod} is the modelled pseudorange on band k. Two guards apply. The denominator must satisfy $|\alpha_1 - \alpha_2| \geq 10^{-6}$. The seed is rejected unless it is finite and $|\widehat{I}^i| < 200\,\mathrm{m}$. A slant ionospheric delay above 200 m is not physical. It indicates mis-paired bands or a gross code outlier, and must not be stored in a per-satellite state.

Two BIA-coverage gates also apply, so that the differenced code is consistent in its biases. The first is *presence*: a `.BIA` entry exists on the band, with the zero-valued BeiDou C2I/C6I entries treated as missing. The second is coverage *honesty* (`bias_coverage_requires_apply`). It skips the seed when an entry is loaded but not *applied* to $(P - \mathrm{mod})$, because the raw hardware bias is then still present. The network producer uses presence only. The honesty gate is needed because the failure it prevents, a zero-valued BeiDou entry that injects 30–60 m, is well below the 200 m reject and would pass it.

(e) Carrier ambiguity. The float ambiguity that subsequently feeds integer resolution (Section 16) is the part of the *phase* residual that remains after every other modelled and estimated term is removed (`StateManager::computeAmbiguitySeed`, the routine that every phase-seeding call site uses),

$$\widehat{N}^i_{r,s} = \Phi^i_{r,s} - \text{modelled} - \left(c\,\mathrm{d}t_r + \delta_\Phi\right) - m_w\,\mathrm{ZWD} - \mathrm{ISB} - g_{\mathrm{trop}} + \alpha_s\,\widehat{I}^i, \tag{200}$$

This is the phase observation minus its model, the receiver clock *and* the phase clock δ_Φ (seed (b)), the wet delay, the inter-system bias, and the tropospheric gradient, plus the slant ionospheric phase advance $+\alpha_s\,\widehat{I}^i$ (seed (d)). The phase is advanced by the same α_s that delays the code, so the iono term has the opposite sign to (193). The phase clock and the slant iono are removed because they are estimated states in the uncombined model. One term cannot be removed this way: a receiver clock that is still zero on a network cold start. Pre-seed (f) handles that case.

The seeded ambiguity therefore contains the integer cycle count plus the remaining receiver and satellite phase biases. AR recovers the integer part once the satellite phase bias is supplied and the receiver phase bias cancels under between-satellite differencing.

(f) Cold receiver-clock pre-seed (network path). Seeds (a)–(e) assume the receiver clock has already been solved. That holds on the PPP path, where the WLS clock seed of Section 14.1 runs first. The network producer does not run the WLS seed, so nothing solves the receiver clock before the ambiguity of (e) is seeded. `computeAmbiguitySeed` then subtracts a clock term of zero, and the whole free-running receiver-clock offset goes into every cold ambiguity seed.

The opt-in `seedColdReceiverClock` (`network.cold_clock_seed`, default off) fixes this. It writes the median GPS code residual into x(clock). This is the `gps_median` that ISB seed (a) computes and discards. The seed overwrites only a *cold* clock. Any $|x(\text{clock})| >$ `kColdClockEpsilonM` $= 10^{-6}$ m is treated as already solved and is left unchanged. When the epoch has no GPS code observation the routine returns `false`, so it tries again at the next epoch and does *not* mark the clock as seeded. This matches the GPS-reference contract of seed (a) (`StateInitializer::seedColdReceiverClock`).

14.3 BSSD pivot management

Between-satellite single differencing (BSSD, Section 5.1) re-expresses each constellation's ambiguities relative to a designated *pivot* satellite p_g. This cancels the receiver phase bias and makes the SD ambiguities integer. The choice of pivot matters. A poor pivot degrades every SD row that references it, and an unstable one forces costly re-referencing handovers. `src/core/filter/PivotManager.cpp` selects and maintains the pivot. The baseline double-difference path uses the similar `src/core/filter/DdPivotManager.cpp`.

14.3.1 Lexicographic candidate ranking

Candidates are ordered *lexicographically*: first by a signal-coverage tier, then by a continuous score within the tier. Coverage comes first because the pivot must track every signal that any non-pivot satellite tracks. A non-pivot signal that the pivot lacks has no reference and is dropped. *Any* full-coverage candidate therefore ranks above *any* partial-coverage candidate, whatever their scores (the lexicographic comparator in `PivotManager::buildEpochMaps`). Candidate a beats b if and only if its phase-signal count is higher, or the counts are equal and its score is higher.

Within a coverage tier the tie-break score combines float-convergence and geometry

(`PivotManager::scorePivotCandidate`):

$$\boxed{\text{score}(i) = -\min\!\left(1,\ \sqrt{v_{\max}(i)}\right) + \frac{\text{el}(i)}{90} - \tfrac{1}{2}\,\mathbb{1}[\text{el}(i) < 20^\circ]\,,} \tag{201}$$

where $v_{\max}(i)$ is the worst (largest) float-ambiguity variance among the satellite's tracked phase signals and $\text{el}(i)$ is its elevation in degrees.

The first term penalises a pivot that has not converged. On handover the new pivot's variance is added to every SD state in its group. This is the diagonal of the Section 5, $\boldsymbol{P}'(i,i) = \boldsymbol{P}(i,i) + \boldsymbol{P}(p_g,p_g) - 2\boldsymbol{P}(i,p_g)$, implemented in `include/filter/CovarianceReReference.hpp`. A pivot whose own state is still settling therefore passes its uncertainty to the whole constellation. The penalty saturates at -1, equivalent to one metre of 1σ ambiguity uncertainty. A candidate with *no* state yet receives a flat -1 (the negative-`max_amb_variance` sentinel branch of the score). On the first BSSD epoch every candidate in a tier has this sentinel, so the score reduces to elevation and no special first-election branch is needed.

The el/90 term prefers high-elevation pivots. The $-\frac{1}{2}$ penalty below 20° discourages a low pivot, whose tropospheric and multipath residuals exceed the formal stochastic model however well its float state has converged.

Both terms only *rank* candidates that have already passed a hard elevation gate. A satellite below `pivot_min_el` (`getBssdPivotMinElevation`) cannot be elected at all: not in the freshest-peer scan, not in the online pick, and not in the sticky-retention test, all three in `PivotManager::selectPivotCandidates`. The $-\frac{1}{2}$ penalty therefore only breaks ties among the survivors. When *no* satellite passes the mask (a code-only float constellation), the gate is bypassed. The pick reverts to the precomputed best or highest-elevation satellite, so the candidate set is never empty (the fallback from `pickExclusionAwareBest` to the precomputed `best_sat` / `highest_sat`).

Sticky retention is the code's *elevation hysteresis*, and it prevents the pivot changing back and forth between epochs. It uses the same *absolute* gate and does not compare the incumbent with a challenger. The incumbent is kept whenever it still passes `pivot_min_el`, is not excluded, and stays within the toe spread of Section 14.3.2 (the sticky branch of `selectPivotCandidates`). No score is evaluated on that branch, so a better challenger cannot displace a qualifying incumbent. The score ranking runs only after the incumbent has *failed* the gate.

14.3.2 Ephemeris-freshness guardrail

A pivot with a stale broadcast ephemeris puts a step into *every* SD row that references it. Its extrapolated orbit and clock drift away from those of the fresher satellites it is differenced against. This was observed in production: a Galileo pivot held a one-hour-old I/NAV record while the other satellites' records were minutes old, and the whole SD solution was corrupted. The guardrail is gated on the *spread* of ages. A candidate is barred from pivot election when its absolute ephemeris age $|t_{\text{obs}} - t_{\text{oe}}|$ exceeds the constellation's freshest qualified peer by more than

$$\texttt{kPivotToeSpreadMaxS} = 1200\ \text{s} = 20\ \text{min} \tag{202}$$

(the `freshestToeAgeAbs` scan in `PivotManager::selectPivotCandidates`, applied both at the online pick and at the sticky-retention test). Because the gate is on the *spread* and not on an absolute age, the ranking is unchanged when all candidates are about equally fresh. Only a candidate that is much staler than the rest is displaced. The barred satellite can still be used as a non-pivot. The freshest satellite always passes, since its difference from itself is zero, so the candidate set is never empty.

14.3.3 Pivot disturbance detection and handover

In BSSD the pivot's own SD ambiguity is zero with zero variance, because the pivot *is* the reference: $x(p_g) = 0$ and $\boldsymbol{P}(p_g, p_g) = 0$ exactly. Any departure from that state means the filter has perturbed the reference, through process noise or an update, and the SD states are no longer consistent. `isPivotDisturbed` flags this when, for any of the pivot's phase signals,

$$\boxed{\left|x(p_g)\right| > 10^{-6} \quad \text{or} \quad \boldsymbol{P}(p_g, p_g) > 10^{-12}} \tag{203}$$

(`PivotManager::isPivotDisturbed`). The two thresholds are tolerances for numerical zero, not tuned gates. 10^{-6} m on the value and $10^{-12}\,\mathrm{m}^2$ on the variance catch any real leakage onto the pivot and tolerate floating-point round-off in the re-reference arithmetic. The predict step must not create a false disturbance, so the pivot's process noise is suppressed during prediction and $\boldsymbol{P}(p_g, p_g)$ stays at zero (Section 12). Without the suppression, every epoch's $\boldsymbol{Q}$ would trip (203) and force an unnecessary handover.

A handover fires when the incumbent is disturbed or when the ranking elects a different satellite. The outgoing pivot is then classified. In a *clean* handover the old pivot's state is invertible and consistent. It can be re-referenced without loss, and the RTS smoother can later invert it. A *forced* or disturbed handover cannot be inverted cleanly and marks a discontinuity.

In both cases `CovReReference::applyBssd` in `include/filter/CovarianceReReference.hpp` re-references the covariance in place. It subtracts the new pivot's row and column from each non-pivot member of the group, then sets the new pivot's row and column to zero. This is the one shared implementation of the $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ transform Section 5.

The baseline DD path has a different structure. It negates the new-pivot row and column and subtracts from the others, which is a different S-transform, and then renames the affected ambiguity states with `StateManager::renameAmbiguity` (`DdPivotManager::handleHandover`). The two paths share only the positive-definiteness guard applied after the transform (`enforcePsd`). The single-receiver user path runs the transform without the guard. At user scale the groups are small and handovers are rare, so floating-point error in $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ does not accumulate.

Part IV
Cycle-slip detection and repair

15 Cycle-slip detection and repair

The uncombined filter (Section 4) forms no linear combination in its measurement update. The per-signal $P^i_{r,s}$ and $\Phi^i_{r,s}$ enter Equations (4) and (5) directly. Cycle slips are screened by a set of *diagnostic* combinations. In each one the geometric range ρ^i_r, the clocks $c(\mathrm{d}t_r - \mathrm{d}t^i)$ and the troposphere T^i_r cancel. What remains varies between epochs only by measurement noise and by any integer discontinuity of the carrier.

The status of the detectors is as follows.

- Melbourne–Wübbena (`src/core/slip/MwSlipDetector.cpp`) is the primary detector and the only one armed by default.
- GRAPHIC (`src/core/slip/GraphicSlipDetector.cpp`) and the extra-wide-lane detector (`src/core/slip/EwlSlipDetector.cpp`), which needs a third frequency, are compiled and registered. Both are opt-in (Section 15.1), and no bundled config enables them.

- The ionosphere-free phase detector (`src/core/slip/IfPhaseSlipDetector.cpp`) is *compiled and armed on the network path*. It is instantiated per non-pivot station whenever `network.cycle_slip.if_phase_td` is set (Section 15.2.5). On the single-receiver user and kinematic path it stays staged, because the position-uncertainty gate it needs is unsafe there.
- The geometry-free phase detector (`src/core/slip/GfSlipDetector.cpp`) is *staged*. It is in the tree but commented out of the explicit CMake source list, unlike the IF detector beside it. Its algebra is recorded here for completeness.

The user and kinematic set is composed at run time from the config block `estimation.cycle_slip`. The network producer composes its own per-station set from a parallel `network.cycle_slip` block (Section 15.1).

Throughout this section the receiver index r is suppressed (single-receiver PPP / PPP-RTK), and $f_1 > f_2$ denotes the two carriers of a band pair, sorted by frequency so that $f_1 - f_2 > 0$ and $\lambda_1 < \lambda_2$ (`src/core/slip/MwSlipDetector.cpp`, the band-frequency sort). Carrier phase is read from RINEX in cycles and scaled by $\lambda_s = c/f_s$ before combining, so Φ_s^i below is always in metres, consistent with Equation (5). The operator $\delta\cdot$ denotes a *temporal* integer slip step: for example δN_s is the change in the ambiguity on signal s across the slip epoch. It is distinct from the between-satellite single difference $\Delta\cdot$ of Section 5, which also uses Δ.

15.1 Detector registry and composition

The set of detectors is configurable. Each detector implements the interface `ISlipDetector` (`include/slip/ISlipDetector.hpp`), which has one entry point per epoch

$$\texttt{detect}(\text{SlipEpochContext}) \longrightarrow \{\,\text{SlipEvent}\,\},$$

plus lifecycle hooks (`resetSatellite`, `resetSatelliteSignal`, `enableTracing`, `setCorrections`). A `SlipEvent` is the union of the legacy per-detector result structs. Its `granularity` field distinguishes two kinds of flag. A *per-signal* flag (GRAPHIC, GF, IF) names one carrier. A *band-difference* flag (MW, EWL) reports a wide-lane integer $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$, which alone cannot assign the jump to one signal. Equation (218) resolves those two unknowns downstream. The inputs are bundled in `SlipEpochContext`, so every detector has the same signature. Only the IF detector uses the 3D position sigma and receiver ECEF it carries. On the network path, where that detector is armed, the position gate is always satisfied, because the estimator holds the station positions fixed.

`SlipDetectorRegistry::createEnabled` (`src/core/slip/SlipDetectorRegistry.cpp`) builds the set selected by `estimation.cycle_slip.{mw,graphic,ewl,gf,if}` in a fixed order. MW comes first. It is the primary detector and the only source of the wide-lane integers used by ambiguity resolution (Section 16) and by the BSSD pivot logic (`src/core/filter/PivotManager.cpp`). The auxiliary detectors follow. MW defaults on and the auxiliaries default off, which keeps the historical MW-only behaviour.

A kind that is enabled in config but has no creator in the registry is skipped with a warning, so a config written for a later version still runs. Two kinds are skipped, for different reasons. The `.cpp` of `gf` is excluded from the build (Section 15.2.4). The `.cpp` of `if` *is* compiled, but the network path instantiates it directly (Section 15.2.5) and not through this registry, so `SlipDetectorRegistry` registers no `IonoFreePhase` creator.

`CycleSlipManager` drives the whole set through one loop. It reaches the MW detector's additional API (WL-lock queries, pivot-handover hints, per-satellite quality scoring) through a typed down-cast (`asMelbourneWubbena`), which keeps the base interface small.

The network producer does not use this registry for any detector. The `buildStation` lambda of `Application::networkSetup_` (`src/core/app/ApplicationNetworkSetup.cpp`) constructs each station's set directly from a *parallel* config block, `network.cycle_slip.{mw_detect, turbo_edit, if_phase_td}`. These keys are distinct from their `estimation.cycle_slip` namesakes, and all three default off. The two paths share the detector *classes*. They do not share the composition mechanism, the config block, or the response: the network refloats where the user path repairs (Section 15.3).

Composing by hand has one important consequence. With `network.cycle_slip.mw_detect` on and `network.cycle_slip.turbo_edit` off, the station receives an `MwSlipDetector` and a null `TurboEditResolver`. The null pointer is what directs the detection branch away from integer repair and to the refloat path.

`turbo_edit` (Section 15.4) is *not* a detector kind. It is a *coordination mode*. When the MW detector is enabled, it pairs the MW wide-lane integer with the geometry-free phase jump to recover per-signal integers. It is documented separately from the detectors below.

15.2 Test-statistic combinations

15.2.1 Melbourne–Wübbena (wide-lane)

The Melbourne–Wübbena (MW) observable is the wide-lane carrier phase minus the narrow-lane code. Introducing the wide- and narrow-lane wavelengths

$$\lambda_{\mathrm{WL}} \;:=\; \frac{c}{f_1 - f_2}, \qquad \lambda_{\mathrm{NL}} \;:=\; \frac{c}{f_1 + f_2}, \tag{204}$$

the engine forms (`MwSlipDetector::computeMwMeters`)

$$\boxed{\mathrm{MW}^i \;=\; \underbrace{\frac{f_1\,\Phi_1^i - f_2\,\Phi_2^i}{f_1 - f_2}}_{\text{wide-lane phase}} \;-\; \underbrace{\frac{f_1\,P_1^i + f_2\,P_2^i}{f_1 + f_2}}_{\text{narrow-lane code}}.} \tag{205}$$

The range-additive terms common to Equations (4) and (5), namely geometry, clocks and troposphere, cancel between the two bracketed combinations. The slant ionosphere cancels too. The wide-lane phase has the ionospheric coefficient $-(f_1\mu_1 - f_2\mu_2)/(f_1 - f_2)$ and the narrow-lane code has $+(f_1\mu_1 + f_2\mu_2)/(f_1 + f_2)$. With $\mu_s = f_1^2/f_s^2$ (Equation (1)) the two are equal, so (205) removes the first-order ionosphere exactly. What remains is nearly constant, equal to the wide-lane ambiguity plus hardware bias,

$$\mathrm{MW}^i \;\approx\; \lambda_{\mathrm{WL}}\,N_{\mathrm{WL}}^i \;+\; b_{\mathrm{MW}}^i \;+\; \varepsilon_{\mathrm{MW}}, \qquad N_{\mathrm{WL}}^i = N_1^i - N_2^i, \tag{206}$$

so a wide-lane slip $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$ appears as a step of $\lambda_{\mathrm{WL}}\,\delta N_{\mathrm{WL}}$. For GPS L1/L2, $\lambda_{\mathrm{WL}} \approx 0.861\,\mathrm{m}$, and the dominant error is the narrow-lane *code* noise. With code noise $\sigma_P \sim 0.3\,\mathrm{m}$ the MW noise is $\sigma_{\mathrm{MW}} \approx \sigma_P/(\sqrt{2}\,\lambda_{\mathrm{WL}}) \sim 0.25$ wide-lane cycles, so the observable is limited by code noise. The integer-ready (BIA-corrected) variant subtracts the satellite MW bias b_{MW}^i, which has the same algebra as Equation (205) applied to the OSB phase and code biases of Equations (4) and (5).

The series $\mathrm{MW}^i(k)$ is tested against a robust adaptive gate. `estimation.cycle_slip.mw_detect_reference` selects the statistic.

The recommended choice is the first time difference $\mathrm{TD1}(k) = \mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$, gated against the median absolute deviation of its own history. Every bundled real-time and replay config sets it explicitly. The second time difference $\mathrm{TD2}(k) = \mathrm{MW}^i(k) - 2\,\mathrm{MW}^i(k-1) + \mathrm{MW}^i(k-2)$ is

available (`mw_td_order`). It is insensitive to curvature, at a noise penalty of $\sqrt{6}$ against $\sqrt{2}$ for TD1. `MwSlipDetector::activeStatScale` holds the white-noise scaling of each statistic.

TD1 is *not* the loader default. A config that leaves `estimation.cycle_slip.mw_detect_reference` unset uses the legacy Hatch mean. It remains the default for backward compatibility and as an A/B arm, alongside the windowed local median (`hatch_mean`, `local_median`).

The threshold is derived from the data alone, and no σ from the stochastic model enters it. It combines a per-arc MAD, an EWMA-smoothed MAD floor across satellites, and a $1/\sin\theta$ elevation shape. For sparse or cold-start arcs the pool widens in steps: band pair, then constellation, then all satellites except GLONASS. The empirical construction is in `MwSlipDetector::computeAdaptiveThreshold` and is not reproduced here.

15.2.2 Extra-wide-lane (EWL)

For satellites with three or more frequencies the MW algebra (Equation (205)) is applied to a pair of *non-L1* carriers (`EwlSlipDetector::computeEwlMwMeters`; band digit ≥ 2). The two carriers are close in frequency, so $f_1 - f_2$ is small and the extra-wide-lane wavelength

$$\lambda_{\mathrm{EWL}} \;=\; \frac{c}{f_1 - f_2} \tag{207}$$

is very long: $\lambda_{\mathrm{EWL}} \approx 5.86\,\mathrm{m}$ for GPS L2/L5 and $\approx 9.77\,\mathrm{m}$ for Galileo E5a/E5b. Metre-level code noise is then $\lesssim 0.05$ EWL cycles. The EWL integer $N_{\mathrm{EWL}} = N_2 - N_5$ resolves in one epoch, and the size of a slip δN_{EWL} is determined almost without error. With the L1/L2 wide-lane (Equation (206)) this gives two independent band-difference constraints,

$$\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2, \qquad \delta N_{\mathrm{EWL}} = \delta N_2 - \delta N_5, \tag{208}$$

which leave the per-signal triple $(\delta N_1, \delta N_2, \delta N_5)$ with one degree of freedom [5].

15.2.3 GRAPHIC (single-frequency half-sum)

The GRAPHIC combination averages the code and phase on a *single* band (`GraphicSlipDetector::computeGraphic`):

$$\mathrm{GR}_s^i \;=\; \tfrac{1}{2}\big(P_s^i + \Phi_s^i\big), \tag{209}$$

with Φ_s^i in metres. The slant ionosphere enters as $+\mu_s I_{r,1}^i$ on the code and $-\mu_s I_{r,1}^i$ on the phase of the *same* signal (Equations (4) and (5)), so the half-sum cancels it to first order. GRAPHIC is therefore ionosphere-free with one frequency, but it keeps half the code noise. The combination still contains the geometric range, the clocks, the troposphere, and the constant half-ambiguity $\tfrac{1}{2}\lambda_s N_s^i$. The between-epoch first difference

$$\mathrm{dGR}_s^i(k) \;=\; \mathrm{GR}_s^i(k) - \mathrm{GR}_s^i(k-1) \tag{210}$$

removes the constant ambiguity, and reduces range and clocks to the level of receiver motion and code noise. A slip of δN cycles on signal s produces a jump of $\tfrac{1}{2}\lambda_s\,\delta N$, and the integer estimate is $\delta N = \mathrm{round}\big(2\,\mathrm{dGR}/\lambda_s\big)$. With $\sigma_P \sim 0.3\,\mathrm{m}$ the noise after differencing is $\sim 0.3\,\mathrm{m}$, so the detector can resolve only a few L1 cycles. On quiet arcs the hard floor `kMinThresholdM` $= 0.5\,\mathrm{m}$ (≈ 5 L1 cycles) sets the threshold. GRAPHIC is therefore a coarse backstop and not a sub-cycle detector [1, 29].

15.2.4 Geometry-free phase (staged)

The geometry-free (GF) observable is the between-band phase difference

$$\mathrm{GF}^i \;=\; \Phi_1^i - \Phi_2^i \;\approx\; (\mu_2 - \mu_1)\, I_{r,1}^i \;+\; (\lambda_1 N_1^i - \lambda_2 N_2^i), \tag{211}$$

which removes geometry, clocks and troposphere exactly but keeps the *difference* of the two ionospheric delays. $I_{r,1}^i$ varies smoothly, so a second-order time difference

$$\mathrm{TD2}_{\mathrm{GF}}(k) \;=\; \mathrm{GF}^i(k) - 2\,\mathrm{GF}^i(k-1) + \mathrm{GF}^i(k-2) \tag{212}$$

cancels the ionospheric trend up to quadratic order over the sampling interval and leaves noise plus any slip (`GfSlipDetector::computeGfMeters` for Equation (211); the TD2 update is the test statistic for a level jump). The detector needs a high data rate ($\geq 1\,\mathrm{Hz}$). At a coarse rate the remaining ionospheric curvature is too large for (212). The detector is in the tree but excluded from the build.

The TD2 statistic (212) is still used elsewhere. `slip/IonoVariabilityMonitor` forms the same second difference per satellite on the two lowest RINEX bands (`selectPrimaryPair`, `computeGfMeters`), and it *is* compiled. It is a process-noise *monitor*, not a detector. Its per-satellite output is the clamped squared ratio of two MAD-derived values of σ for $\mathrm{TD2}_{\mathrm{GF}}$, one over a recent window and one over a baseline window,

$$m^i \;=\; \mathrm{clamp}\!\left(\left(\hat{\sigma}_{\mathrm{recent}}/\hat{\sigma}_{\mathrm{baseline}}\right)^2,\;\; 1,\;\; m_{\max}\right), \tag{213}$$

The ratio normalises itself. The slowly varying obliquity and curvature content of Equation (211) is present in *both* windows and cancels. A quiet sky gives $m^i \approx 1$ at any elevation, and the $1/\sin^2\theta$ obliquity scaling already in the ionospheric process noise is not counted twice. The monitor treats a cycle slip as an isolated single-epoch outlier, which the MAD is designed to *reject*. That is the opposite of a detection statistic, so the monitor is not a substitute for the staged detector.

Both arming keys default off. On the user path, `estimation.adaptive_iono_process_noise.enable` makes m^i scale the static ionospheric spectral density into a per-satellite zenith override, which `StateManager::setIonoProcessNoiseOverrides` consumes. On the network path, `network.adaptive_iono_q` (per station) makes it scale the deviation-channel innovation of the only time-linked slant-iono element the producer carries. The window lengths and the clamp $m_{\max}$ are empirical and stay in config.

15.2.5 Ionosphere-free phase (staged on the user path; armed on the network path)

The dual-frequency ionosphere-free (IF) phase combination uses the coefficients

$$c_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \qquad c_2 = \frac{f_2^2}{f_1^2 - f_2^2}, \qquad c_1 - c_2 = 1, \tag{214}$$

giving $\Phi_{\mathrm{IF}}^i = c_1\Phi_1^i - c_2\Phi_2^i$ and the effective wavelength $\lambda_{\mathrm{IF}} = \left|c_1\lambda_1 - c_2\lambda_2\right|$ (`IfPhaseSlipDetector::computeIfWavelengthMeters`); for GPS L1/L2 $\lambda_{\mathrm{IF}} \approx 0.107\,\mathrm{m}$. The detector does not test the raw combination. It forms the IF observed-minus-computed (OMC) value and differences it twice. The first difference is between satellites, against a high-elevation pivot p_g. It removes the receiver clock, the inter-system offset $c\,\delta t_r^{(g)}$ and the satellite phase clock. The second is between epochs. It removes the slowly varying wet troposphere and the constant ambiguity:

$$\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g}(k) = \left[\Phi_{\mathrm{IF}}^i(k) - \Phi_{\mathrm{IF}}^{p_g}(k)\right] - \left[\Phi_{\mathrm{IF}}^i(k-1) - \Phi_{\mathrm{IF}}^{p_g}(k-1)\right], \tag{215}$$

which is near zero for a clean arc and jumps by $\sim \delta N\,\lambda_{\mathrm{IF}}$ on a slip. The between-satellite step does not cancel the differential geometric error $(\boldsymbol{e}^{i} - \boldsymbol{e}^{p_g})^{\mathrm{T}}\delta\boldsymbol{r}$, so flagging must be gated by the filter's 3D position sigma. The gate requires a static site with a precise a priori position. The single-receiver kinematic path cannot guarantee that. There the detector is never instantiated, and the user toggle `estimation.enable_if_slip_detector` has no effect. The network estimator holds every station position *fixed*, so the gate is always satisfied, and the detector is armed there.

On the network path (the `buildStation` lambda of `Application::networkSetup_`, gated on `network.cycle_slip.if_phase_td`) an `IfPhaseSlipDetector` is instantiated per *non-pivot* station. The pivot's ambiguities are fixed by the gauge, so on the legacy datum path the pivot has nothing to refloat. Under the prioritised phase datum `network.phase_datum_prioritized` the detector is instantiated for every station, because the former pivot then has arcs that are not anchored and still need detection. The detector is built with `setDeferFlagging(true)`. It caches each per-epoch $\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g}$ of Equation (215) and does *not* apply a threshold locally. The driver (`networkSlipIfTdDriver` in `src/core/slip/NetworkSlipQc.cpp`) *pools* the deferred jumps across stations by signal pair and removes their median med. The median is the network common mode, a wobble of the frame or datum. It does not depend on elevation, so it is removed in raw metres. Each residual is then normalised to a zenith equivalent,

$$z \;=\; \left(\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g} - \mathrm{med}\right)\,\sin\theta, \tag{216}$$

and the pooled robust scale $\hat{s} = 1.4826\,\mathrm{median}\,|z|$ is formed. At each satellite's threshold the scale is multiplied by $1/\sin\theta$ again, so *one* scale serves the whole sky. The map is the free function `ifElevMap` ($1/\sin\theta$, clamped at 5° so that it stays bounded near the horizon, which is already elevation-masked). The effective scale $\hat{s}_{\mathrm{eff}}$ is an empirical-Bayes blend of the pooled prior with the station pair's own EWMA scale. The shrinkage weight is $w = n/(n+\kappa)$, with n the number of clean samples of the station pair. A quiet reference station and a noisy station with modernised signals therefore each get their own envelope, and the majority does not set one threshold for all. A satellite at elevation θ is flagged when

$$\left|\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g} - \mathrm{med}\right| \;>\; \max\!\left(\frac{k\,\hat{s}_{\mathrm{eff}}}{\sin\theta},\;\; c_{\mathrm{floor}}\,\lambda_{\mathrm{IF}}\right), \tag{217}$$

where k is a loose MAD multiplier. The floor is proportional to the $(1,1)$ IF jump $\lambda_{\mathrm{IF}} = c/(f_1+f_2) \approx 10.7\,\mathrm{cm}$ and is kept in raw metres, so a simultaneous one-cycle slip is caught at any elevation. Only the clean survivors are folded into the station-pair EWMA, so the scale does not absorb the event it has to catch. The five knobs (κ, k, c_{floor}, the minimum pool size and the EWMA weight) live under `network.cycle_slip.if_phase_td_*` and are read once per call inside the driver.

A flag never repairs and never resets filter state in place. It restarts the detector's own arc unconditionally (arc hygiene). Its two estimator-side consequences are gated separately inside `refloat_pair`. The flagged pair enters the Card-4 invariant-carry slip funnel only when that carry is armed. The band ambiguities are marked pending-reseed (the refloat itself) only when the phase-leveling carry is warm. On a cold or absent carry the flag is emitted as diagnostic telemetry and changes nothing, so a clean run log is no evidence that the arm acted; the run-end counters are. The loose gate is intentional. The driver is a coarse first pass, and the sharp integer decision is left to a post-estimation phase validation that is not yet built; it is not the discriminating solver of Section 15.4.

The pool cannot see a slip on the between-satellite reference itself. Such a jump is common to every satellite of that station's pool and is removed with med. The driver therefore folds in a second, independent flag source: the detector's own reference-satellite reset keys, parsed by

parseIfPairResetKey and refloated through the same refloat_pair. Both bands of the pair are refloated, because the IF combination cannot localise which one slipped. The exceptions are the band-scope and common-band-scope knobs, which, when armed, keep one band of the pair pinned. The reference-satellite election that produces those keys is gated by IfPhaseSlipDetector::RefGateParams. The gate is pushed only on the network path and armed by network.cycle_slip.if_phase_td_ref_gate (default off); the staged user/kinematic path keeps the legacy fixed gate.

15.3 Network slip QC: raw evidence and refloat over repair

The IF-phase driver of Section 15.2.5 is one arm of the network producer's slip layer. The arms are free functions in slip/NetworkSlipQc, called in a fixed serial, station-ordered sequence from the per-epoch PASS-A slip seam (Application::networkPassASlipScan_). Two further arms are described here: networkSlipDrainTORepairs, which applies the integer repairs of the network TurboEdit resolver to the phase-leveling carry, and networkSlipDrainMwRefloat, which refloats on an admitted wide-lane slip candidate or on the receiver's own LLI loss-of-lock bit. The same seam then runs the wind-up restart drain, the two gap re-entry drains (per satellite and per signal) and the harness-only injected-free drain, each behind its own default-off knob. The shared evidence is a separate component, slip/NetworkRawObservables.

Shared evidence. Once per epoch per (station, satellite), networkComputeRawObservables forms the MW combination of Equation (205) and the GF combination of Equation (211) on the two lowest distinct bands whose phase *and* matching code are both present. It also records the first band's phase-minus-code level and the per-band RINEX LLI byte. These are absolute levels, not differences. networkQualifyRawObservables then adjudicates each observable's step against the immediately adjacent processed epoch, that is $\mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$ and $\mathrm{GF}^i(k) - \mathrm{GF}^i(k-1)$. The threshold has the shape of Section 15.2.5: a multiple of an empirical scale, floored from below. The empirical term is a per-station robust EWMA of that station's own clean steps, built from the same $1.4826\,\mathrm{median}\,|\cdot|$ estimator (exposed for test as networkRobustStepScale). The floor is a physical integer bound kept in raw metres: a fraction of a wide-lane cycle λ_{WL} for MW (default 0.5 cycle) and a few centimetres for GF (default 4 cm); the default multiplier is 5 for both. The gate therefore falls back to the physical bound when the empirical term is cold or degenerate, never to a stochastic-model σ. The MW adaptive threshold of Section 15.2.1 follows the same rule. The ordering also matches the IF-TD pool arm: each satellite is thresholded with the pre-update scale, and only the clean survivors are folded into the EWMA, so the scale does not absorb the event it has to catch. A step is measurable only against an adjacent predecessor. Across a gap it cannot be measured, and the code treats that absence of evidence as neither evidence of a slip nor evidence of continuity. The multipliers and floor values live in SlipQcParams.

Response doctrine. On the network path a detection refloats: it marks the ambiguity pending-reseed so that PASS B re-estimates it from that epoch's data, and it never applies an integer. This differs from the user-path TurboEdit repair of Section 15.4. The reason is the network's missing satellite phase biases. Without them the network wide-lane integer is not trustworthy, and a wrong integer written into a carried value would bias the gauge, whereas a refloat costs only re-convergence. The one exception is networkSlipDrainTORepairs, active only when the network TurboEdit key of Section 15.4 is armed. It shifts the phase-leveling carry by $\delta N_s\,\lambda_s$ continuously, with the variance unchanged and the covariance preserved. It does so only for an integer that is both confident and of sane magnitude. Both bounds are network knobs, separate from the resolver's own constants: a

rounding residual (Equation (223)) of at most 0.15 cycle and a magnitude of at most 50 cycles by default. A low-confidence detection is left untouched: it is neither repaired, refloated nor dropped. The float wide-lane's low-confidence repairs are measured to be mostly false positives, and acting on them discards a good carry. The resolver's reset set is not acted on either; without satellite phase biases it flags tens of satellites per epoch.

Status. Every arm is default off in the loader. `network.cycle_slip.mw_detect` arms the MW/LLI drain, `network.cycle_slip.mw_lli_corroborate` the LLI qualification and `network.cycle_slip.gap_instrument` the arc-resumption instrument. The raw observables are computed only when a consumer asks for them (`SlipQcParams::anyConsumer`). The adjudication above runs only when a decision card is armed; a run with the raw trace alone computes the levels and adjudicates nothing. Every use of the verdicts in the drains is therefore guarded on whether the evidence exists this epoch. Without it the LLI arm stays decisive, because a gate that fired on missing data would do more harm than the chatter it is meant to suppress. The qualification never excludes an LLI assertion. An assertion acts when a measured MW or GF step corroborates it, or when the station's own LLI behaviour is still judged credible by a two-state hysteresis on the EWMA of its LLI-set fraction (`networkLliCredibilityStep`). Only an uncorroborated assertion at a station whose LLI is judged not credible is downgraded. It is then emitted as suppressed and counted, not dropped.

The raw step evidence has a second consumer outside this chapter. With `network.carry_invariant_innov_corroborate` armed (default off), the Card-4 innovation gate on the carried double differences does not re-tap a chord whose arc the raw observables measured as continuous. The gate is fail-safe in one direction only: a measured flat arc suppresses the re-tap, and absence of evidence still re-taps. The slip response is therefore narrowed and never removed. The gate is part of the estimator and is described in Section 20.

15.4 TurboEdit dual-frequency resolution

The TurboEdit resolver is a pure MW+GF joint integer solver (`src/core/slip/TurboEditResolver.cpp` and its header `include/slip/TurboEditResolver.hpp`). It owns the per-arc geometry-free jump history and the integer arithmetic. It returns per-signal integer repairs and a reset-fallback set, and it does not modify filter state. It is wired into the post-processing `CycleSlipManager` and the real-time engine, and is enabled by `estimation.cycle_slip.turbo_edit`.

That key is default off. The bundled real-time and SBF-replay configs turn it on; the internal PPP-RTK AR config leaves it off. The network producer has a separate key, `network.cycle_slip.turbo_edit`, which builds one resolver per station and feeds the repair drain of Section 15.3. With that key off and `network.cycle_slip.mw_detect` on, the same MW detector runs as a pure detector, and the network refloats instead of repairing (Section 15.3). Of the sub-toggles below, the soft-LLI bridge (`turbo_edit_bridge_lli`) is default off and the MW-only code-domain veto (`turbo_edit_mw_only_veto`) is default on.

15.4.1 The 2×2 system

For a dual-frequency phase pair (s_1, s_2) on satellite i, ordered higher frequency first ($f_{s_1} > f_{s_2}$, hence $\lambda_{s_1} < \lambda_{s_2}$), the resolver pairs two scalar slip observables. The first is the MW wide-lane integer of Equation (206). It is taken as input from the MW detector and never recomputed, so the empirically tuned MW path of Section 15.2.1 is the single source of δN_{WL}:

$$\delta N_{\mathrm{WL}} \;=\; \delta N_{s_1} - \delta N_{s_2} \qquad \text{(one equation, two unknowns).} \tag{218}$$

The second is the GF phase jump in metres. Writing the GF level as $\mathrm{GF}(t) = \lambda_{s_1} L_{s_1}(t) - \lambda_{s_2} L_{s_2}(t)$ (carrier in cycles), a simultaneous slip steps its ambiguity content by

$$\delta\mathrm{GF} \;=\; \lambda_{s_1}\,\delta N_{s_1} - \lambda_{s_2}\,\delta N_{s_2}, \tag{219}$$

estimated within the arc by the second-order time difference $\mathrm{TD2}(t) = \mathrm{GF}(t) - 2\,\mathrm{GF}(t-1) + \mathrm{GF}(t-2)$ of Equation (212). TD2 cancels the ionospheric delay up to quadratic order, so at the slip epoch it equals the integer jump plus phase noise. The TD2 chain requires three consecutive epochs; a non-consecutive history re-seeds the chain (Section 15.4.4).

Stacking Equations (218) and (219) gives the linear map from the integer slip pair to the two observed jumps:

$$\boxed{\begin{pmatrix} \delta N_{\mathrm{WL}} \\ \delta\mathrm{GF} \end{pmatrix} \;=\; \underbrace{\begin{pmatrix} 1 & -1 \\ \lambda_{s_1} & -\lambda_{s_2} \end{pmatrix}}_{\boldsymbol{A}} \begin{pmatrix} \delta N_{s_1} \\ \delta N_{s_2} \end{pmatrix}.} \tag{220}$$

The determinant $\det \boldsymbol{A} = \lambda_{s_1} - \lambda_{s_2}$ is non-zero for any cross-band pair. It is the centimetre-scale quantity `denom` of Equation (222), $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, and is distinct from the $\sim 0.86\,\mathrm{m}$ wide-lane wavelength. The system is therefore always invertible:

$$\boldsymbol{A}^{-1} = \frac{1}{\lambda_{s_1} - \lambda_{s_2}} \begin{pmatrix} -\lambda_{s_2} & 1 \\ -\lambda_{s_1} & 1 \end{pmatrix}. \tag{221}$$

The implementation does not invert the full matrix. It solves the second row for δN_{s_2} and back-substitutes into the first (`TurboEditResolver.cpp`):

$$\delta N_{s_2} \;=\; \frac{\delta\mathrm{GF} - \lambda_{s_1}\,\delta N_{\mathrm{WL}}}{\lambda_{s_1} - \lambda_{s_2}}, \qquad \delta N_{s_1} \;=\; \delta N_{s_2} + \delta N_{\mathrm{WL}}, \tag{222}$$

where $\delta\mathrm{GF}$ is the in-arc TD2 or, across a data gap, the bridged jump of Section 15.4.4. The denominator $\lambda_{s_1} - \lambda_{s_2}$ (the variable `denom` in the source) is $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2. The amplification factor on the wide-lane integer is therefore $\lambda_{s_1}/(\lambda_{s_1} - \lambda_{s_2}) \approx -3.53$. Despite that lever, the GF equation must localise δN_{s_2} to well within ± 0.5 cycle; the split-capability gate below enforces this.

15.4.2 Integer rounding and validation

The float δN_{s_2} from Equation (222) is rounded and its residual recorded:

$$d_2 = \mathrm{round}\big(\delta N_{s_2}\big), \quad f = \big|\delta N_{s_2} - d_2\big|, \quad d_1 = d_2 + \delta N_{\mathrm{WL}}. \tag{223}$$

A candidate is accepted as a repair only if three conditions hold. The rounding is clean ($f \le 0.30$, the constant `kMaxFractional`, which leaves a $1 - 0.30 = 0.70$ cycle margin to the nearest wrong integer). The magnitudes are sane ($|d_1|, |d_2| \le$ `kMaxSlipCycles`, which is 1000 cycles). The pair is non-trivial ($d_1 \neq 0 \vee d_2 \neq 0$). An accepted candidate yields `action="repair"`. The pair $(d_1, d_2) = (0, 0)$ resolves as no slip, and the arc continues. Any other outcome resets the satellite.

The joint solve is trusted only when the GF evidence is sharp enough to discriminate the integer lattice. The float δN_{s_2} carries $\sigma_{\mathrm{TD2}}/|\lambda_{s_1} - \lambda_{s_2}|$ cycles of noise, so the solve is permitted only when

$$\sigma_{\mathrm{TD2}} \;\le\; \texttt{turbo_edit_split_max_sigma} \cdot \big|\lambda_{s_1} - \lambda_{s_2}\big| \qquad (\texttt{splitCapable}), \tag{224}$$

Here σ_{TD2} is the arc's own MAD-derived σ when the arc is mature, otherwise the cross-satellite tier value, otherwise NaN. NaN fails the gate, because the absence of an estimate is no evidence of sharpness. The default of `estimation.cycle_slip.turbo_edit_split_max_sigma` is 0.2 cycles. This gate is the only mechanism that makes the resolver rate-adaptive. Millimetre-level noise at 1 s passes easily; the ionospheric residual at a coarse rate (30 s) passes only while the arc is quiet. An MW-flagged pair that fails the gate falls back to a satellite reset. A GF-only exceedance on an arc that fails the gate is ignored, not reset, and the Kalman residual gate remains the backstop. When $\delta N_{\mathrm{WL}} = 0$ (an equal slip $\delta N_{s_1} = \delta N_{s_2}$, to which MW is blind), a GF-only flag fires when |TD2| exceeds the adaptive threshold $\tau = \max(k_\sigma \hat{\sigma}_{\mathrm{TD2}}, 0.04\,\mathrm{m})$, with k_σ the `turbo_edit_sigma_multiplier` (default 6). The floor sits just below $|\lambda_{s_1} - \lambda_{s_2}| \approx 0.054\,\mathrm{m}$, so a ± 1-cycle equal slip remains detectable. The joint solve takes a zero wide-lane jump for a GF-only flag only when the MW detector affirmatively reports the pair clean this epoch. MW silence alone is not taken as a zero wide-lane jump: a locked or immature MW arc can miss a real unequal slip, and forcing zero would alias it into a confident wrong repair. Without that evidence the wide-lane integer is unknown and the satellite is reset. An unstable ionosphere on the constellation (the cross-satellite IVI above 0.06 m) also resets the satellite instead of solving.

15.4.3 MW-only / code-event null line

The asymmetric case is an MW flag with a GF jump that is affirmatively zero. For a fixed $\delta N_{\mathrm{WL}} = k$, every integer pair $(\delta N_{s_1}, \delta N_{s_1} - k)$ predicts, through Equation (219), the GF jump

$$\delta\mathrm{GF} \;=\; \lambda_{s_2} k - (\lambda_{s_2} - \lambda_{s_1})\,\delta N_{s_1}, \tag{225}$$

with its zero crossing at $\delta N_{s_1} = \lambda_{s_2} k/(\lambda_{s_2} - \lambda_{s_1}) \approx 4.53\,k$ for GPS L1/L2 ($\approx 3.95\,k$ for L1/L5). Every small pair with $|\delta N_{s_1}| \le 3$ then predicts $|\delta\mathrm{GF}| \ge 8\,\mathrm{cm}$, orders of magnitude above phase noise. A flat GF therefore leaves only *null-line* pairs of $(4,3)$ / $(9,7)$-class magnitude: multi-cycle simultaneous slips on both signals with no loss of lock, which are implausible. The simpler explanation is a narrow-lane code excursion (multipath, NLOS, DLL re-lock), since $\mathrm{MW} = \Phi_{\mathrm{WL}} - P_{\mathrm{NL}}$ and one wide-lane cycle corresponds to $\approx 0.86\,\mathrm{m}$ of narrow-lane code.

The pair is therefore classified as a code-domain artifact, and the caller skips the phase reset, when the GF jump is affirmatively zero within a gate bounded by the lattice discrimination:

$$|\mathrm{TD2}| \le \underbrace{\texttt{kCodeEventZeroSigma} \cdot \hat{\sigma}_{\mathrm{MAD}}}_{\text{zero gate}} \quad\wedge\quad \text{zero gate} \le \underbrace{\texttt{kVetoZeroGateMaxLambdaFrac} \cdot |\lambda_{s_1} - \lambda_{s_2}|}_{\text{discrimination cap}}, \tag{226}$$

with `kCodeEventZeroSigma` $= 3.5$ ($\approx 99.95\%$ two-sided) and `kVetoZeroGateMaxLambdaFrac` $= 0.5$. The cap is the rate-adaptive safeguard. A zero gate wider than half the smallest GF-visible quantum cannot distinguish stillness from a small slip; the veto then stands down and the MW flag governs. The classification has further trust conditions: the veto is enabled, MW flagged, GF did not exceed its threshold, no post-slip cooldown is active, the constellation's ionosphere is stable, neither signal carries an LLI, and the MAD maturity is ≥ 10 samples. The last condition is stricter than the 5 samples detection needs, because affirming stillness preserves state. The implied null-line pair is computed for the trace only and never applied; `did_slip` stays false so the GF chain continues. A real null-line slip is still caught. It moves that signal's Kalman phase innovation by $\ge |4\lambda_{s_1}| \approx 0.76\,\mathrm{m}$ in the same epoch, far beyond any residual gate, so the residual-driven reset corrects it one epoch late. The behaviour is toggled by `estimation.cycle_slip.turbo_edit_mw_only_veto` (default on).

15.4.4 Gap-spanning GF bridge

The TD2 chain needs three consecutive epochs. Any per-signal dropout or receiver gap re-seeds the chain, and the post-gap TD2 is then blind to a re-lock slip whose GF level jump falls between the two chains. When a chain break occurs within `estimation.cycle_slip.turbo_edit_max_gap_s`, the resolver stores the pre-gap GF level $\mathrm{GF}_{\mathrm{ref}}$, its timestamp, the arc's EWMA GF rate $\dot{\mathrm{GF}}$ and the pre-gap spacing dt_0. On return it forms the drift-corrected jump across the gap:

$$\boxed{\delta\mathrm{GF}_{\mathrm{gap}} \;=\; \mathrm{GF}_{\mathrm{now}} - \mathrm{GF}_{\mathrm{ref}} - \dot{\mathrm{GF}}\,\Delta t, \qquad \Delta t = t_{\mathrm{now}} - t_{\mathrm{ref}},} \tag{227}$$

which then takes the place of $\delta\mathrm{GF}$ in Equation (222). Here $\dot{\mathrm{GF}}$ is an EWMA ($\alpha = 0.1$) of the per-epoch GF rate, chosen so that its noise is not amplified when projected over the gap. The uncancelled ionospheric residual is a random walk over the gap. The detection threshold and the code-event zero gate are therefore widened by the factor $w = \sqrt{\Delta t/dt_0}$ (for $\Delta t > dt_0$), and the split-capability σ becomes $\sigma_{\mathrm{bridge}} = \hat{\sigma}_{\mathrm{MAD}}\, w$. The discrimination cap of the zero gate is not widened, because the integer lattice of Equation (220) does not stretch with the gap. The bridge cap defaults to 15 s in code and is raised to 60 s in the bundled real-time and replay configs.

An MW-flagged pair across the gap has three possible outcomes.

1. A *code event* (`action="code_event"`) when $\delta\mathrm{GF}_{\mathrm{gap}}$ is affirmatively zero within the widened gate. This is the null-line logic of Equation (226) applied across the gap, and it preserves phase through a re-acquisition DLL transient.

2. A *repair* or *moot* outcome when the joint solve runs with $\delta\mathrm{GF} = \delta\mathrm{GF}_{\mathrm{gap}}$. Each signal is dispatched by continuity in the observation domain. A signal observed at every epoch through the gap kept its ambiguity, so its nonzero δN is a *repair*. A signal that missed an epoch was pruned and reseeded on return, so its nonzero δN is *moot*: already absorbed, with the reset suppressed.

3. A *reset* when the across-gap σ is too coarse for the split, or when the candidate set holds no integer or more than one and no deferral applies.

The GF equation alone is degenerate along the null-line lattice of Equation (225): $+0.735$ m fits both $(0, -3)$ and $(9, 4)$ at the millimetre level, and only the integer k separates them. The bridged solve therefore requires a unique surviving candidate after three guards. The first is the fractional gate of Equation (223). The second is a GF-separation guard: the candidate's own predicted jump must clear the zero gate, $|\lambda_{s_1} d_1 - \lambda_{s_2} d_2| >$ `zero_gate`, so that a marginal noise exceedance cannot produce a multi-cycle repair. The third is a sibling-pair zero affirmation. Another GF arc of the same satellite that stayed chain-continuous and unflagged through the bridged interval affirms $\delta N = 0$ for both its signals, and any candidate that needs a nonzero integer on an affirmed signal is pruned. The wide-lane integer for the bridged solve is taken in order of trust: the published MW integer k; failing that, at the flag epoch, $\mathrm{round}(f) \pm 3$ around the instantaneous re-acquisition float (biased by the narrow-lane code); and on deferral epochs the settled across-gap MW Hatch mean. When the float wide-lane evidence is unsettled or the lattice is ambiguous (two or more acceptable integers), the solve is deferred instead of guessed, up to `estimation.cycle_slip.turbo_edit_wl_settle_max_s` (default 45 s). A deferral ends in a later repair, a reset or a timeout reset.

15.4.5 Atomic reconciliation

Per-satellite candidates are reconciled atomically. A cross-pair integer disagreement on a shared signal demotes the whole satellite to reset. A satellite that ends in the reset set has its code-event,

moot, clean and pending classifications withdrawn, so contradictory evidence always resolves to the conservative outcome. Arc bookkeeping maintains the TD2 precondition of consecutive epochs. A non-consecutive update partially resets the chain but keeps the slowly varying MAD calibration window. A gap beyond the arc's maximum clears the arc. The callers set that maximum to 1.5 times the data interval; the resolver's own default, 120 s, applies only when no caller sets it. An accepted slip re-seeds the chain with a cooldown of 3 epochs that suppresses the TD2 echo. Only clean TD2 samples (not exceeding the threshold, not bridged) feed the MAD window.

Part V
Integer ambiguity resolution

16 Integer ambiguity resolution

Integer ambiguity resolution (AR) maps the real-valued, bias-contaminated phase ambiguities of the float solution to the integer carrier-cycle counts they represent. The engine runs MLAMBDA [26] on a between-satellite-differenced ambiguity sub-problem extracted from the filter state and covariance. The operative quantity is the ambiguity differenced against a per-constellation pivot p_g (Section 5). The single-receiver and baseline modes share the integer-estimation machinery, but the condition that makes the differenced ambiguity an integer differs by mode.

The undifferenced ambiguity $N^i_{r,s}$ of the phase observation equation (Equation (5)) carries a receiver phase bias $b_{r,s}$ and a satellite phase bias b^i_s. In single-receiver PPP and PPP-RTK the pivot difference is a *between-satellite single difference* $\Delta\cdot$ (Section 5.1). It cancels the receiver bias $b_{r,s}$, which is common to every tracked satellite, and leaves the satellite bias. The residual $\Delta N^{i,p_g}_{r,s}$ is therefore integer only after b^i_s is removed by a consistent external phase-bias product (BIA or SSR, Section 18). In baseline mode the state holds true double differences $\nabla\Delta\cdot$ (Section 5.3); the additional between-receiver difference also cancels the satellite bias, so $\nabla\Delta N$ is integer without an external product. The equations below keep the resolver's internal label $\nabla\Delta$ for the differenced ambiguity, since the algebra is the same in both cases: it denotes the between-satellite single difference Δ in the single-receiver case and the full double difference only in baseline mode.

The implementation separates a driver from a numerical kernel. The driver, `src/core/filter/AmbiguityResolver.cpp`, assembles the state sub-vector, rescales between metres and cycles (metre↔cycle), looks up the per-pair λ, forms the differences, performs the WL/NL decomposition and applies the returned integer fix. The numerical core lives in the `GnssProcessor::ils` namespace of `src/core/filter/IlsCore.{hpp,cpp}`: the LDL^{T} factorisation (in the factor order of (232)), the $\boldsymbol{Z}$-reduction, the Schnorr–Euchner search, the bootstrap success rate and the full acceptance battery. These are pure free functions over ($\hat{\boldsymbol{N}}$ cycles, $\boldsymbol{Q}_{\hat{N}}$ cycles^2), with no dependence on `StateManager`, `KalmanFilter` or the configuration. The PPP-RTK network producer (Section 20) calls the same `ils` core under its own gate set, as a read-only fixability probe only, because the producer is float-only (Section 16.7). The single-receiver and baseline paths below are configured under the JSON path `estimation.ambiguity_resolution.*`.

16.1 The integer least-squares problem

The float solution gives a real-valued ambiguity estimate $\hat{\boldsymbol{N}}$ (in cycles) with covariance $\boldsymbol{Q}_{\hat{N}}$, both extracted from the filter state $\boldsymbol{x}$ and covariance $\boldsymbol{P}$. Integer ambiguity resolution is the weighted closest-lattice-point problem

$$\boxed{\check{\boldsymbol{N}} \;=\; \arg\min_{\boldsymbol{a}\in\mathbb{Z}^n} \big(\hat{\boldsymbol{N}}-\boldsymbol{a}\big)^{\mathrm{T}}\, \boldsymbol{Q}_{\hat{N}}^{-1}\, \big(\hat{\boldsymbol{N}}-\boldsymbol{a}\big),} \tag{228}$$

that is, the minimisation of the squared Mahalanobis distance from $\hat{\boldsymbol{N}}$ to a lattice point $\boldsymbol{a}$ in the metric of $\boldsymbol{Q}_{\hat{N}}^{-1}$. This integer least-squares (ILS) problem is NP-hard. LAMBDA and its MLAMBDA variant make it tractable in three stages, none of which has a tuning parameter: (i) an LDL^{T} factorisation of $\boldsymbol{Q}_{\hat{N}}$, (ii) a decorrelating integer $\boldsymbol{Z}$-transform (lattice reduction), and (iii) a depth-first shrinking-ellipsoid search over the reduced lattice. The hat/check convention of the notation block applies: $\hat{\boldsymbol{N}}$ is the float estimate and $\check{\boldsymbol{N}}$ the integer-fixed solution.

Float extraction and rescaling to cycles. The filter stores per-band ambiguity states in metres ($\lambda_s N$, one per tracked satellite and signal). The resolver therefore forms the between-satellite-differenced float and covariance by explicit differencing against the pivot p_g, then rescales by wavelength. In baseline mode the state already holds the double difference, and the differencing step reduces to a direct selection (Section 16.6). For a differenced pair k with satellite index $i(k)$ and pivot index $p(k)$ (`src/core/filter/AmbiguityResolver.cpp`):

$$y_k^{\nabla\Delta} = \boldsymbol{x}_{\,i(k)} - \boldsymbol{x}_{\,p(k)}, \qquad \text{(DD float, m)} \tag{229}$$

$$\boldsymbol{Q}_{kl}^{\nabla\Delta} = \boldsymbol{P}_{i_k i_l} - \boldsymbol{P}_{i_k p_l} - \boldsymbol{P}_{p_k i_l} + \boldsymbol{P}_{p_k p_l}, \qquad \text{(DD covariance, m}^2\text{)} \tag{230}$$

$$\hat{\boldsymbol{N}}_k = y_k^{\nabla\Delta}/\lambda_k, \qquad (\boldsymbol{Q}_{\hat{N}})_{kl} = \boldsymbol{Q}_{kl}^{\nabla\Delta}/(\lambda_k\lambda_l). \qquad \text{(cycle domain)} \tag{231}$$

The rescaling (231) is required. On metre-domain floats MLAMBDA would round toward integer-metre values, which have no carrier-cycle meaning, and the ratio test (Section 16.5) would return ≈ 1 irrespective of float quality. The per-pair wavelength λ_k uses the GLONASS FDMA per-satellite frequency where applicable. $\boldsymbol{Q}^{\nabla\Delta}$ is symmetrised, $\boldsymbol{Q}^{\nabla\Delta} \leftarrow \frac{1}{2}(\boldsymbol{Q}^{\nabla\Delta} + \boldsymbol{Q}^{\nabla\Delta\mathrm{T}})$, before factorisation.

The resolver elects its own DD reference: in each (constellation, signal) group of AR-eligible ambiguities, the highest-elevation member is the reference and every other member is differenced against it. This reference differs from the filter's BSSD pivot. Under BSSD the pivot's ambiguity is frozen at zero variance, which excludes it from the candidate set (Section 16.4); the resolver therefore differences against a second reference, and the resulting quantity is a difference of single differences. The two elections also use different rules. The BSSD pivot is chosen on a hard phase-coverage tier, a convergence/elevation score and an ephemeris-freshness bar (Section 5); the AR reference is chosen on elevation alone (`AmbiguityResolver::buildDdPairs`).

16.2 Decorrelation: LDL^{T} factorisation and the integer $\boldsymbol{Z}$-transform

Factorisation. MLAMBDA factorises the symmetric positive-definite ambiguity covariance as

$$\boldsymbol{Q}_{\hat{N}} \;=\; \boldsymbol{L}^{\mathrm{T}}\, \boldsymbol{D}\, \boldsymbol{L}, \qquad \boldsymbol{L} \text{ unit lower-triangular}, \;\; \boldsymbol{D} = \mathrm{diag}(d_1,\ldots,d_n), \;\; d_i > 0, \tag{232}$$

eliminated in reverse index order, from the last row up (the Chang–Yang MLAMBDA convention). The transposed unit lower-triangular factor stands on the left, as written in (232); the shorthand "LDL-transpose" used in this section names that factorisation, not the reversed factor order. A non-positive pivot $d_i \le 0$ signals a covariance that is not positive definite and aborts the fix. After reduction, d_i is the *conditional variance* of the i-th decorrelated ambiguity given the later ones; the success-rate gate of Section 16.5 uses it directly.

The $\boldsymbol{Z}$-transform. The central device of LAMBDA [26] is a unimodular integer transform $\boldsymbol{Z} \in \mathbb{Z}^{n\times n}$, $|\det \boldsymbol{Z}| = 1$, that re-parameterises

$$\boldsymbol{z} = \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}, \qquad \boldsymbol{Q}_{\hat{z}} = \boldsymbol{Z}^{\mathrm{T}}\boldsymbol{Q}_{\hat{N}}\boldsymbol{Z}, \tag{233}$$

so that the transformed ambiguities are nearly decorrelated (small off-diagonal $\boldsymbol{L}$ entries) and their conditional variances are ordered. The decorrelated float vector is $\bar{\boldsymbol{z}} := \boldsymbol{Z}^{\mathrm{T}}\hat{\boldsymbol{N}}$, with components $\bar{z}_k$; it is the quantity the search of Section 16.2 rounds, computed as `z_float = Z.transpose()*a` in `ils::mlambda` (`src/core/filter/IlsCore.cpp`). Because $\boldsymbol{Z}$ is unimodular, $(\boldsymbol{Z}^{\mathrm{T}})^{-1}$ is also integer and the integer minimiser maps back exactly, $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1}\check{\boldsymbol{z}}$. Decorrelation therefore leaves the solution of (228) unchanged. It makes the search ellipsoid closer to spherical, which reduces the number of enumerated points by orders of magnitude. The reduction interleaves two operations in a sweep from $j = n-2$ downward.

Integer Gauss transformations reduce the sub-diagonal $\boldsymbol{L}$ entries by integer column operations,

$$\mu = \lceil L_{ij} \rfloor, \qquad \boldsymbol{L}_{(:,j)} \mathrel{-}= \mu\,\boldsymbol{L}_{(:,i)}, \qquad \boldsymbol{Z}_{(:,j)} \mathrel{-}= \mu\,\boldsymbol{Z}_{(:,i)}, \tag{234}$$

where $\lceil\cdot\rfloor$ is round-to-nearest, keeping every $|L_{ij}| \le \frac{1}{2}$.

Permutations (conditional adjacent swaps) reorder two ambiguities when the swap reduces the later conditional variance. With $\bar{d}_j = d_j + L_{j+1,j}^2\, d_{j+1}$, the swap fires when

$$\bar{d}_j + \varepsilon \;<\; d_{j+1}, \qquad \varepsilon = 10^{-6}, \tag{235}$$

after which d_j, d_{j+1}, the affected rows/columns of $\boldsymbol{L}$ and the columns of $\boldsymbol{Z}$ are updated by the standard rank-one formulas ($\eta = d_j/\bar{d}_j$, $\lambda = d_{j+1}L_{j+1,j}/\bar{d}_j$, $d_j \leftarrow \eta\, d_{j+1}$, $d_{j+1} \leftarrow \bar{d}_j$), and the sweep restarts from $j = n-2$. The guard $\varepsilon = 10^{-6}$ prevents repeated swaps of near-equal variances. It is a numerical safeguard rather than an estimator setting and has no configuration key.

The shrinking-ellipsoid search. The reduced problem is solved by depth-first enumeration of the lattice points inside the ellipsoid $\sum_i (z_i - \hat{z}_i)^2/d_i \le \chi^2$, in Schnorr–Euchner order (nearest integer first, then zig-zagging outward). Recursing from the last level $k = n-1$ down to 0,

$$\hat{z}_k = \bar{z}_k + \sum_{i>k} (z_i - \hat{z}_i)\, L_{ik}, \qquad \text{(conditional centre)} \tag{236}$$

$$z_k = \lceil \hat{z}_k \rfloor, \qquad \mathrm{dist}_k = \mathrm{dist}_{k+1} + \frac{(\hat{z}_k - z_k)^2}{d_k}. \qquad \text{(partial distance)} \tag{237}$$

A complete integer vector that reaches level 0 with cumulative distance below the current bound χ^2 is recorded. Once the candidate list is full, the bound shrinks to the worst retained candidate, so the ellipsoid contracts as better candidates are found. The search (`ils::search` in `src/core/filter/IlsCore.cpp`) retains the $m = 2$ best candidates for the ratio test, sorted by ascending squared distance, under a hard iteration cap (`kMaxIter` $= 10\,000$). Fewer than two candidates is a failure. The outputs are the two smallest objective values $s_1 \le s_2$ of (228) and the best integer vector $\check{\boldsymbol{z}}$, back-transformed via $\check{\boldsymbol{N}} = (\boldsymbol{Z}^{\mathrm{T}})^{-1}\check{\boldsymbol{z}}$.

16.3 Wide-lane / narrow-lane decomposition (PPP path)

PPP and PPP-RTK AR use a *wide-lane-first* decomposition when Melbourne–Wübbena (MW) resolved wide-lane (WL) integers are available. It halves the search dimension and exploits the long WL wavelength [13]. The driver attempts the WL-constrained narrow-lane (NL) fix before

any unconstrained LAMBDA. When WL integers exist and the constrained fix fails, the epoch stays float: the driver does not fall through to an unconstrained search, because that search would include DDs whose WL is not locked and could return integers that contradict the MW wide lane. A wrong-WL fix is worse than no fix, so the enforced rule is "no NL AR for satellites whose WL is float" (`src/core/filter/AmbiguityResolver.cpp`).

WL from Melbourne–Wübbena (integer by contract). The DD wide-lane integer is not searched. It is the difference of the MW detector's single-difference WL integers (`src/core/slip/MwSlipDetector`):

$$\nabla\Delta N_{\mathrm{WL}} = N^{i}_{\mathrm{WL}} - N^{p_g}_{\mathrm{WL}} \in \mathbb{Z}, \qquad \lambda_{\mathrm{WL}} = \frac{c}{f_1 - f_2}. \tag{238}$$

NL search variable. For each dual-frequency DD pair the transform $\boldsymbol{T}$ forms the float narrow-lane ambiguity in NL cycles. The default row is the narrow-lane combination below. It is not ionosphere-free: its slant-ionosphere sensitivity is 0.959 NL cycles per TECU, so the default per-pair cap (244) of 0.15 cycles implicitly requires the slant ionosphere to about 0.156 TECU.

$$\hat{n}_{\mathrm{NL}} = \frac{f_1\,\nabla\Delta N_1 + f_2\,\nabla\Delta N_2}{f_1 + f_2} = \frac{1}{f_1 + f_2}\left(\frac{f_1}{\lambda_1}\,\nabla\Delta_1 + \frac{f_2}{\lambda_2}\,\nabla\Delta_2\right), \qquad \lambda_{\mathrm{NL}} = \frac{c}{f_1 + f_2}, \tag{239}$$

with reduced covariance $\boldsymbol{Q}_{\mathrm{NL}} = \boldsymbol{T}\,\boldsymbol{Q}^{\nabla\Delta}\,\boldsymbol{T}^{\mathrm{T}}$. Only NL-paired DDs enter LAMBDA; unpaired single-signal DDs are excluded so that they cannot distort the ratio test. An optional ionosphere-free search row (configuration key nl_iono_free_row, default off) targets the same L1 integer with zero slant-ionosphere sensitivity. For GPS L1/L2 it multiplies the WL sensitivity by 8.06 (a 1-cycle WL error moves the L1 integer target by 3.53 cycles instead of 0.438) and the searched variance by about 65. The equations of this subsection describe the default narrow-lane row.

Integer-recentring shift. The float $\hat{n}_{\mathrm{NL}}$ is not integer-valued in general. Substituting $N_2 = N_1 - \mathrm{WL}$ gives $\hat{n}_{\mathrm{NL}} = N_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$, which for GPS L1/L2 with WL = 1 lies at $N_1 - 0.4379$. Neither neighbouring integer is then materially closer, and the ratio test settles near 1.6 regardless of float quality. The resolver therefore adds the known-WL offset, which makes the search variable integer in expectation,

$$\tilde{n}_k = \hat{n}_{\mathrm{NL},k} + \frac{f_2}{f_1 + f_2}\,\nabla\Delta N_{\mathrm{WL},k} \;\approx\; N_{1,k} \;\in\; \mathbb{Z} + \text{noise}, \tag{240}$$

runs MLAMBDA on $\tilde{\boldsymbol{n}}$ (a constant per-row shift leaves $\boldsymbol{Q}_{\mathrm{NL}}$ unchanged), then back-shifts $\check{n}_{\mathrm{NL}} = \check{N}_1 - \frac{f_2}{f_1+f_2}\,\mathrm{WL}$.

Back-transform to per-signal DDs. The fixed integers map back to the two per-signal DDs in metres,

$$\nabla\Delta_1^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} + \tfrac{f_2}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_1, \qquad \nabla\Delta_2^{\mathrm{fix}} = \left(\check{n}_{\mathrm{NL}} - \tfrac{f_1}{f_1+f_2}\,\nabla\Delta N_{\mathrm{WL}}\right)\lambda_2, \tag{241}$$

and re-enter the filter through the constrained update of Section 16.6.

State-basis variants. When `estimation.state_ambiguity_basis = nl_wl` the state holds NL/WL ambiguities directly, and there are no per-band DD pairs to enumerate. In this basis only the MW-resolved WL integers are applied, as constraint rows; a full NL LAMBDA search with the WL pre-constrained is not implemented. The complementary WL-only mode (`estimation.ambiguity_resolution.wl_only`, in the `per_band` basis) applies the cycle-domain constraint $\nabla\Delta_1/\lambda_1 - \nabla\Delta_2/\lambda_2 = \nabla\Delta N_{\mathrm{WL}}$ as one pseudo-observation per pair, without an NL search. When MW re-locks at a new integer, the covariance of the affected single-difference states is inflated so that the state moves to the new integer within one epoch.

Both WL-constraint paths (this one and the `nl_wl`-basis constraint) take their pseudo-observation noise from a separate setting, $\boldsymbol{R} = \sigma^2_{\mathrm{wl}}\boldsymbol{I}$ with $\sigma^2_{\mathrm{wl}} =$ `wl_only_pseudo_var` (10^{-4} cycles^2, in the cycle domain), rather than the metre-domain σ^2_{hold} of the NL/DD feedback in (246). Both paths apply three acceptance gates to the MW integers. The first is the per-arc fractional-residual point test (`wl_max_frac`, default 0.40; on both arcs in the `per_band wl_only` path, on the single arc in the NL/WL basis). The second is a bootstrap success-rate gate (key wl_only_ffr, default on): the product of per-arc rounding success rates, taken over distinct arcs, must reach the threshold min_success_rate of (243), and the weakest arc is dropped until it does. The third is an innovation screen (key wl_only_max_innov_cyc, default 0.5 cycle) that rejects a constraint whose asserted WL integer differs from the predicted DD wide lane by more than that amount. The cross-frequency parity check runs only in the `nl_wl` basis (`applyMwWlConstraintsNlWl_` in `src/core/filter/AmbiguityResolver.cpp`). For a triple-frequency satellite it rejects all three WL DDs when the telescoping canonical cycle counts are inconsistent, $\nabla\Delta_{ab} - \nabla\Delta_{ac} + \nabla\Delta_{bc} \neq 0$. The `per_band wl_only` path has no cross-frequency consistency check.

16.4 Partial / subset ambiguity resolution

Partial AR operates at two levels: a static candidate gate that decides which ambiguities enter the integer problem, and an optional backward-elimination search that looks for a fixable subset when the full set fails validation.

Upstream candidate gating (always on). Subset selection begins when the DD pairs are built (`src/core/filter/AmbiguityResolver.cpp`). A satellite/signal ambiguity enters the integer set only if it passes every filter:

- **Initialised**: the state entry is non-zero (seeded by data), and **non-frozen**: its variance is non-zero, which excludes a BSSD-frozen pivot.
- **Elevation**: $\theta^i_r \geq \theta_{\mathrm{AR}}$ (`elevation_mask`). In the single-receiver path the elevation comes from the epoch's observations; an ambiguity whose satellite has no observation is given elevation 0 and is removed by any positive mask (the consequence for fix-and-hold is stated in Section 16.6).
- **Resolvable**: the ambiguity is not flagged unresolvable for lack of bias or antenna (BIA, ATX) coverage. This applies to the single-receiver path; in baseline DD the analogous filter is the `exclude_keys` set of non-integer-capable DDs from `MeasurementModel`.
- **Constellation**: $g(i) \in$ `active_systems`, when that list is non-empty.
- **Pairable**: the (constellation, signal) group holds ≥ 2 satellites, so a pivot difference exists.
- **Mature** (`min_arc_info`, default 0 = off): both constituent arcs of a candidate (the higher-frequency arcs of the satellite and of the reference) have accumulated at least $\sum_k \sin^2\theta^i_{r,k}$ of elevation-weighted code information since arc birth, in epoch-equivalents at zenith. Maturity is measured in information rather than epochs: under the elevation stochastic model an arc rising

off the mask and one at zenith accumulate information at very different rates, so two arcs of equal age are not equally determined. The counter restarts on both ways an arc can end: the key leaving the state vector, and a refloat in place. A refloat keeps the key and would otherwise let the new arc inherit the pre-slip age (`AmbiguityResolver::refreshArcAges_`, keyed on `StateManager::isAmbiguityPendingReseed`). The gate applies only to the WL-constrained NL candidate set of Section 16.3, not to the baseline DD, WL-only or unconstrained full-set paths. It guards against wrong fixes, since one immature candidate can fail the joint ratio test of (242) for the whole set. It does not speed convergence: it admits a candidate only once the candidate is well determined, which is when the float solution gains least from a fix.

The constellation filter keeps a system in the float solution while excluding it from the candidate set. It serves a constellation whose residual model bias would lower the ratio test.

Dynamic backward-elimination subset (`enable_partial_ar`, default off). With the candidate set fixed, the default is all-or-nothing: the full set is searched and either passes all gates of Section 16.5 or the epoch stays float. When `enable_partial_ar` is set, a failure of the full-set battery starts a greedy backward elimination (`ils::selectFixableSubset` in `src/core/filter/IlsCore.cpp`, called through the driver-side `selectFixableSubset_` configuration adapter) before the epoch falls back to float. A failure of MLAMBDA itself on the full set (a covariance that is not positive definite, or fewer than two candidates) does not start the elimination. Starting from the full candidate set, each iteration re-runs the complete MLAMBDA pipeline of Sections 16.1 to 16.2 on the current subset and re-tests the same acceptance battery: the ratio lower and upper bounds (242), the bootstrap success rate (243), and the per-entry fractional cap (244). On any failure it drops one entry and retries, down to a single ambiguity:

- if the *fractional cap* is the failing gate, the entry with the largest $|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k|$ (nearest a half-cycle boundary) is removed;
- otherwise (including a failure of MLAMBDA on the subset) the entry with the largest formal float variance $(\boldsymbol{Q}_{\hat{N}})_{kk}$ is removed: the least precise ambiguity, whose removal is expected to improve the bootstrap rate and the search conditioning most.

The accepted subset is always re-validated by the full battery on its own covariance sub-block. The procedure can therefore only fix fewer ambiguities than the full set, never accept a set that fails the battery: a poor elimination order costs fixes, not integrity. With the option off, the path is bit-identical to the all-or-nothing behaviour. The same routine serves the per-band SD, baseline DD and WL-constrained NL paths and operates only in the cycle domain, independent of the decorrelation. Each caller maps the surviving indices back to its own state rows and applies them through the constrained update of Section 16.6.

16.5 Validation: ratio test and fixed-failure-rate gate

A fix is committed only if it passes all three of the following gates. The code evaluates them in order (ratio lower bound, ratio upper bound, bootstrap success rate, per-pair cap) and stops at the first failure. The ratio and fixed-failure-rate (FFR) gates bound different things: the ratio bounds the discrimination between the two best candidates, the FFR gate the absolute success probability.

Ratio test (discrimination).

$$\boxed{\text{ratio} = \frac{s_2}{s_1} \;\geq\; \tau_{\text{ratio}}, \qquad s_1 \leq s_2 \text{ the two smallest ILS objectives,}} \tag{242}$$

with s_1, s_2 from the search of Section 16.2 and threshold τ_{ratio} (`ratio_threshold`). An upper-bound rejection, ratio $> \tau_{\text{ratio}}^{\max}$ (`ratio_upper_bound`), catches a degenerate covariance direction: a near-zero s_1, typically left by an earlier wrong-integer hold, produces a very large but meaningless ratio. A non-positive squared distance for either candidate rejects the fix before the ratio is formed.

Fixed-failure-rate gate (bootstrap success rate). The gate uses a lower bound on the ILS success probability that follows from the problem geometry,

$$\boxed{P_{s,\text{boot}} = \prod_{i=1}^{n} \operatorname{erf}\left(\frac{1}{2\sqrt{2\,d_i}}\right) \;\geq\; P_s^{\min},} \tag{243}$$

where the d_i are the post-reduction LDL^{T} conditional variances of (232) and $P_s^{\min}$ is `min_success_rate`. Each factor is $\Pr\big(|\mathcal{N}(0, d_i)| < \frac{1}{2}\big)$, the probability that sequential bootstrapped rounding lands on the correct integer in that dimension. The product is Teunissen's lower bound on the ILS success rate. The acceptance threshold is therefore tied to the problem geometry rather than to a fixed ratio alone.

Per-pair residual backstop. The joint ratio test can pass on a borderline set in which one pair lies near a half-cycle boundary. The fix is therefore rejected if the fractional residual of any single DD exceeds a cap,

$$\max_k \big|\hat{\boldsymbol{N}}_k - \check{\boldsymbol{N}}_k\big| \;\leq\; \kappa_{\text{frac}} \quad \text{(cycles)}, \tag{244}$$

with $\kappa_{\text{frac}} =$ `nl_max_per_pair_frac`.

Convergence guards. Before any search the resolver skips AR while the float solution is immature: during its first `min_epochs` invocations (a global counter of resolver calls, not a per-candidate age), and while the position standard deviation $\sqrt{\boldsymbol{P}_{00} + \boldsymbol{P}_{11} + \boldsymbol{P}_{22}}$ exceeds `max_position_sigma_m`. A marginal fix accepted before the covariance settles can lock onto wrong integers and corrupt every later epoch.

16.6 Feedback: constrained pseudo-observation update

Accepted integers are fed back through a finite-variance pseudo-observation rather than the classical perfect back-substitution (the $R = 0$ conditioning form)

$$\boldsymbol{x}_{\text{fix}} = \boldsymbol{x} - \boldsymbol{Q}_{\boldsymbol{aa}}\, \boldsymbol{Q}^{\nabla\Delta-1}\big(y^{\nabla\Delta} - \check{\boldsymbol{N}}\big), \qquad \boldsymbol{P}_{\text{fix}} = \boldsymbol{P} - \boldsymbol{Q}_{\boldsymbol{aa}}\, \boldsymbol{Q}^{\nabla\Delta-1}\boldsymbol{Q}_{\boldsymbol{aa}}^{\mathrm{T}}, \tag{245}$$

The update is a *Joseph-form pseudo-observation* (`src/core/filter/AmbiguityResolver.cpp`; kernel `KalmanFilter::josephPseudoUpdate` in `include/filter/KalmanFilter.hpp`). The reason is structural. (245) drives the fixed ambiguity rows of $\boldsymbol{P}$ to ≈ 0 variance, so the next epoch's LAMBDA sees a near-singular $\boldsymbol{Q}^{\nabla\Delta}$ and returns large, meaningless ratios that keep tripping the upper-bound guard of (242). A finite-variance pseudo-observation is intended to keep $\boldsymbol{P}$ well-conditioned, so that later epochs can re-confirm the integer (converging toward it) or detect a slip through its residual.

This intent is not yet validated. Reducing σ_{hold}^2 (over the range 1e-3 to 1e-7, in square metres) reduced the fix count and made the kinematic solution diverge, which is the signature of a constraint that drives the covariance singular rather than tightening it. The resolver therefore carries a read-only conditioning probe (`qbCondition` / `rebuildQbCycles` in `src/core/filter/AmbiguityResolver.`

cpp) that eigen-decomposes the cycle-domain DD covariance before every search and after every fix, and logs the conditioning on every fix and every rejection. The mechanism described below is the design intent, not a validated result.

For each accepted DD pair k the constraint row, innovation and noise are

$$\boxed{\boldsymbol{H}_{(k,\,i)} = +1, \quad \boldsymbol{H}_{(k,\,p)} = -1, \qquad v_k = \lambda_k \check{\boldsymbol{N}}_k - \big(\boldsymbol{x}_i - \boldsymbol{x}_p\big), \qquad \boldsymbol{R} = \sigma^2_{\text{hold}}\, \boldsymbol{I},} \tag{246}$$

followed by $(\boldsymbol{x}_{\text{fix}}, \boldsymbol{P}_{\text{fix}}) = \textsc{JosephUpdate}(\boldsymbol{x}, \boldsymbol{P}, \boldsymbol{H}, \boldsymbol{v}, \boldsymbol{R})$, with $\sigma^2_{\text{hold}} =$ `hold_variance`. The DD/baseline path uses a single-column selector $\boldsymbol{H}_{(k,\,\text{idx})} = +1$, because the state value already is the double difference. The same update applies a partial-AR subset, with fewer rows. The updated state and covariance replace the filter's on every epoch with an accepted fix, in every mode other than 0. The resolver does not distinguish modes 1 (continuous) and 2 (instantaneous); only mode 0 (float) and mode 3 (fix-and-hold) change its behaviour. The $R = 0$ form (245) is not implemented; it appears in the code only in comments, as the rejected alternative.

Fix-and-hold mode (`estimation.ambiguity_resolution.mode` = 3) applies a second pseudo-observation update on each epoch with an accepted fix, holding the differenced ambiguities at their fixed values with the same hold variance. In the baseline path (`AmbiguityResolver::holdAmbiguitiesDd`) the hold covers every DD-keyed ambiguity in the state, including keys excluded from the fix. In the single-receiver path (`AmbiguityResolver::holdAmbiguities`, called by both the post-processing and the real-time engines) the hold is inert under the default configuration. It rebuilds the DD pairs with an empty observation list, so every ambiguity is assigned elevation 0 and the AR elevation mask (default 20°) removes every pair. PPP fix-and-hold therefore holds nothing unless the mask is 0 or negative; mode 3 then behaves like modes 1 and 2. Under the NL/WL state basis the hold is a no-op by design.

16.7 Network-producer AR (both mechanisms default off)

The PPP-RTK network producer of Section 20 is float-only. Its per-epoch seam sequence contains two integer mechanisms, both default-off; with the defaults, neither changes the emitted satellite-bias product. They are documented here because their algebra is settled, not because they are in use.

Producer AR: a log-only fixability probe. Under `network.ambiguity_resolution.enable` (default `false`) the seam `filter/NetworkProducerAr` reconstructs each station's float ambiguities from the reduced solution (`NetworkEstimator::reconstructStation`) and forms an independent between-station double-difference basis: a star topology per (constellation, signal), restricted to GPS and Galileo because GLONASS FDMA leaves a per-slot receiver term that survives the difference. It then builds the exact DD covariance from `NetworkEstimator::jointConditionalCovariance`, rescales it to cycles by (231), and calls `ils::ilsFix`. The result is logged and discarded: nothing is re-pinned or re-solved, and the run is byte-identical whether the block executes or not. The probe is meaningful because the between-station DD ambiguity

$$\nabla\Delta N^{i,p_g}_{r_1 r_2,s} \;=\; \Delta N^{i,p_g}_{r_1,s} - \Delta N^{i,p_g}_{r_2,s} \tag{247}$$

is invariant under the producer's choice of phase-bias S-basis: the satellite bias b^i_s cancels in the between-receiver difference exactly as it does in the baseline DD of Section 5.3. The probe therefore reports the network's intrinsic integer recoverability rather than a property of the datum the producer holds. Its convergence gauge is the median DD formal standard deviation in cycles. That value must fall well below the half-cycle scale before (228) is reachable, and it is a better progress

indicator than the fix count. Two configuration notes apply. `ils_max_dim` (default 0 = uncapped) exists because the search does not return on the full-network basis at operational station counts; a positive value restricts the search to the lowest-formal-σ subset, in the manner of Section 16.4. `wl_cascade` is parsed and echoed in the load-time log but has no consumer in the code, so it is inert.

SD extra-wide-lane integers: Route B and the A2 datum. The second mechanism (`filter/NetworkSdEwlAr`, gated on `network.sd_ewl.fix_enable`, default `false`) decides an integer for the between-satellite single-differenced extra-wide-lane ambiguity. The motivation is a rank argument. The float Melbourne–Wübbena row that the enclosing SD-EWL layer folds is a linear combination of observables already in the epoch's normal equations, so it adds no rank; the only new information the row can supply is integrality. Rounding the SD statistic directly is circular, because it requires the satellite bias difference $b^{i}_{\mathrm{EWL}} - b^{p_g}_{\mathrm{EWL}}$ that the network estimates. Route B avoids the circularity by differencing the SD MW statistic between stations, which gives the same double difference as (247):

$$\boxed{\left(\mathrm{MW}^{i}_{r_1} - \mathrm{MW}^{p_g}_{r_1}\right) - \left(\mathrm{MW}^{i}_{r_2} - \mathrm{MW}^{p_g}_{r_2}\right) \;=\; \lambda_{\mathrm{EWL}}\,\nabla\Delta N^{i,p_g}_{r_1 r_2,\mathrm{EWL}} \;+\; \nabla\Delta\varepsilon^{\mathrm{MW}},} \tag{248}$$

in which the satellite bias pair cancels exactly: the satellite pair is the same on both sides, and each station's receiver term has already cancelled inside its own single difference. What remains is an integer plus averaged MW noise at the extra-wide-lane wavelength. An integer decided this way uses no a-priori satellite bias and cannot be corrupted by a wrong one, which is why Route B is preferred over rounding against a bias product. The cancellation in (248) has a precondition that belongs to the algebra, not to the implementation: b_{EWL} is a per-signal combination, so both stations' arcs must use the same two tracking channels. Otherwise a per-signal bias difference is folded in as though it were an integer. The seam therefore elects a network-common channel pair per satellite by station consensus and admits only stations that hold that pair.

Equation (248) yields a double difference, while the constrained row needs a single-differenced integer per (station, satellite). The A2 datum supplies one. Within each connected component a reference station r_0 is elected, $\Delta n^{i,p_g}_{r_0} := 0$ is defined, and the rounded DD fixes are propagated outward along a spanning tree; the link satellite absorbs the per-station free integer. The asserted quantity is then

$$\Delta n^{i,p_g}_{r} \;=\; \Delta N^{i,p_g}_{r,\mathrm{EWL}} - \Delta N^{i,p_g}_{r_0,\mathrm{EWL}}, \tag{249}$$

the true SD integer minus a constant per (satellite pair, component), which is the classical wide-lane fractional-cycle-bias convention modulo λ_{EWL} [5, 13]. The constant is absorbed into the estimated bias the network reports. Two consequences of (249) matter for the design. First, the per-station free integer, and with it the spanning tree, cancels at the point of use, so a change of spanning tree between epochs is not a defect. Second, r_0's own integers are absorbed into the constant, so the choice of r_0 is a datum decision rather than a measurement.

Acceptance is the scalar case of the battery in Section 16.5. A rounded DD is admitted on its fractional residual, as in (244), and on the $n = 1$ instance of the bootstrap bound (243), each with its own threshold (network.sd_ewl.fix_max_frac_cyc, default 0.15 cycle, and network.sd_ewl.fix_min_ffr, default 0.999):

$$P_{s,\mathrm{round}} = \mathrm{erf}\left(\frac{1}{2\sqrt{2}\,\sigma}\right), \qquad \sigma \text{ in cycles}, \tag{250}$$

so a single rounding and the n-dimensional search share the same failure-rate semantics.

The mechanism is subject to the standing rule that a stiff, time-linked quantity needs a wired event response. The fixed row writes a λ_{EWL}-quantised value into satellite bias states with very small process noise, so an integer that changes between epochs would inject a step that those states

take hours to relax out of. The wired response is withhold-on-disagreement. When this epoch's integer differs from the previous one on a continuous arc, the fix for that arc is withheld rather than repaired, because on a disagreement epoch neither value is known to be right. An arc that restarted (slip, gap, re-acquisition) replaces the stored integer without counting as a disagreement, because its integer has legitimately changed. The stored integer is used only to veto, never to supply a value; this keeps the mechanism short of a carried zero-process-noise hold, which is deliberately not built.

Status. Both mechanisms are default-off, and neither has a graded, datum-invariant field result. The loader also disarms `network.sd_ewl.fix_enable`, with a warning, unless the float row layer (`network.sd_ewl.enable`) is armed. That layer is itself inert without `network.phase_leveling` and an MW-class slip detector, because it reads the arc accumulator they build. The emitted network product is float.

16.8 Configuration summary

All tunable thresholds live under the JSON path `estimation.ambiguity_resolution.*`, parsed in `src/core/config/GnssConfigLoad.cpp` with defaults in `include/filter/AmbiguityResolver.hpp`. The estimator theory of Sections 16.1 to 16.6 has no settings, since changing it would change the mathematics rather than a policy. In the mode row of the table, modes 1 and 2 behave identically in the code (Section 16.6). The table omits three WL acceptance keys described in Section 16.3 (wl_only_ffr, default on; wl_only_min_arc_count, default 10; wl_only_max_innov_cyc, default 0.5) and the search-row option nl_iono_free_row (default off). The two network-producer mechanisms of Section 16.7 are configured separately, under `network.ambiguity_resolution.*` and `network.sd_ewl.*` (parsed in `src/core/config/GnssConfigLoadNetwork.cpp`); both master switches default to `false` and are omitted from the table.

Symbol	Config key (`...ambiguity_resolution.`)	Default	Role
	`mode`	`0`	0 float / 1 continuous / 2 instantaneous / 3 fix-and-hold
τ_{ratio}	`ratio_threshold`	`3.0`	ratio-test acceptance (242)
$\tau_{\text{ratio}}^{\max}$	`ratio_upper_bound`	`1e4`	degenerate-covariance reject (242)
$P_s^{\min}$	`min_success_rate`	`0.999`	FFR gate (243)
κ_{frac}	`nl_max_per_pair_frac`	`0.15`	per-pair residual cap (244)
θ_{AR}	`elevation_mask`	`20°`	candidate elevation mask
σ_{hold}^2	`hold_variance`	`0.001` m^2	pseudo-obs $\boldsymbol{R}$ (246)
$n_{\min}^{\text{ep}}$	`min_epochs`	`10`	convergence guard
$\sigma_{\text{pos}}^{\max}$	`max_position_sigma_m`	`0.05` m	convergence guard
$\sum \sin^2\theta_{\min}$	`min_arc_info`	`0.0` (off)	per-candidate arc-maturity gate, WL/NL path only (Section 16.4)
	`wl_only`	`false`	MW-WL-only, no NL search
σ_{wl}^2	`wl_only_pseudo_var`	`1e-4` cyc^2	WL-constraint pseudo-obs $\boldsymbol{R}$ (Section 16.3)
	`wl_max_frac`	`0.40`	per-arc WL fractional-residual gate (Section 16.3)
	`enable_partial_ar`	`false`	backward-elimination subset fix (Section 16.4)
	`active_systems`	`[]` (all)	constellation subset (Section 16.4)
	`state_ambiguity_basis` †	`per_band`	`per_band` vs `nl_wl` dispatch

† Path is `estimation.state_ambiguity_basis`, one level above the `ambiguity_resolution` block.

Part VI
Inertial aiding

17 Inertial aiding

With an IMU configured, a first-order strapdown mechanization in the Earth-centred Earth-fixed (ECEF) frame replaces the inter-epoch time update of Section 12.1. Two switches control it. `imu.enabled` enables the IMU. `estimation.aiding.imu` selects the IMU source in the `AidingSourceManager`. When `estimation.aiding.imu` is absent it defaults to the value of `imu.enabled`; an explicit value takes precedence (`src/core/config/GnssConfigLoad.cpp`).

The mechanization is implemented in `src/core/filter/ImuIntegrator.cpp`. The per-epoch prediction runs `Application::run` → `AidingSourceManager::collectPredict` → `src/core/aiding/ImuAidingSource.cpp`, and `KalmanFilter::predictAided` (`src/core/filter/KalmanFilter.cpp`) splices the resulting `PredictContribution` into the filter covariance. The real-time engine (`RealTimeEngine::processEpoch`) uses the same `predictAided` / `collectPredict` path. Only the sample ingest differs. Post-processing reads a CSV file with `loadData`, which sorts the samples by time. In real time each sensor sample reaches `ImuIntegrator::addSample`, which appends it to the same buffer in arrival order. For samples that arrive in time order the two paths therefore give the same `integrate` results.

The `IAidingSource` / `AidingSourceManager` abstraction (`include/aiding/`) is the interface through which every non-GNSS modality (IMU, ZUPT, NHC, baro, wheel odometry, vision) contributes up to three items: a `StateManager`-owned state block, a predict-time Φ/Q sub-block, and either independent measurement rows or extra Jacobian *columns* on existing GNSS rows. The IMU is its first client. `ImuAidingSource` reproduces the arithmetic of the older monolithic `KalmanFilter::predictImu`, so the strapdown equations below apply unchanged. `predictImu` remains in the source but has no callers; `predictAided` replaces it.

The formulation is an *error-state* extended Kalman filter. The mechanization propagates the nominal navigation state $(\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{C}_b^e)$ deterministically from the inertial samples, while a 15-element INS error vector

$$\delta\boldsymbol{x}_{\text{ins}} = \begin{bmatrix}\delta\boldsymbol{r} & \delta\boldsymbol{v} & \delta\boldsymbol{\psi} & \delta\boldsymbol{b}_a & \delta\boldsymbol{b}_g\end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^{15} \tag{251}$$

carries the linearised transition and process noise that propagate the covariance. Three attitude-error states $\delta\boldsymbol{\psi}$, three accelerometer-bias states $\boldsymbol{b}_a$ and three gyro-bias states $\boldsymbol{b}_g$ augment the position state (and the three velocity states when `estimation.estimate_velocity` is set). `StateManager` allocates the nine added states as one contiguous block (accessors `getAttitudeIndex`, `getAccelBiasIndex`, `getGyroBiasIndex`).

Remark 2 (Implemented scope). `ImuIntegrator` fixes the block order of Equation (251) ($[\delta\boldsymbol{r}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$). The filter carries no lever-arm state and no sensor scale-factor states, and the GNSS measurement model has no IMU-to-antenna lever-arm term (Section 17.4). These are planned extensions. The equations below omit the corresponding terms.

17.1 Strapdown mechanization

For each IMU sample, the raw specific force $\tilde{\boldsymbol{a}}$ and angular rate $\tilde{\boldsymbol{\omega}}$ (both in the body frame) are corrected with the current filter bias estimates,

$$\boldsymbol{f}_b = \tilde{\boldsymbol{a}} - \boldsymbol{b}_a, \qquad \boldsymbol{\omega}_b = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g, \tag{252}$$

and integrated over the sample interval Δt. A sample whose interval is not positive or exceeds 10 s is skipped. The attitude direction-cosine matrix $\boldsymbol{C}_b^e$ (body $\rightarrow$ ECEF) is updated to first order with Earth-rotation compensation,

$$\boxed{\boldsymbol{C}_b^e \;\leftarrow\; \boldsymbol{C}_b^e\big(\boldsymbol{I} + [\boldsymbol{\omega}_b]_\times\,\Delta t\big) \;-\; [\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{C}_b^e\,\Delta t,} \tag{253}$$

where $\boldsymbol{\omega}_{ie}^e = [\,0,\,0,\,\omega_e\,]^{\mathrm{T}}$ is the ECEF Earth-rotation vector ($\omega_e = 7.292\,115\,146\,7 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$, `common/Constants.hpp`) and $[\cdot]_\times$ is the skew-symmetric (cross-product) matrix. To limit numerical drift, $\boldsymbol{C}_b^e$ is re-orthonormalised by singular-value decomposition every 100 integrated samples, starting with the first. The velocity and position propagate as

$$\boxed{\begin{aligned} \boldsymbol{v}_{k+1} &= \boldsymbol{v}_k + \big(\boldsymbol{C}_b^e\,\boldsymbol{f}_b + \boldsymbol{g}^e(\boldsymbol{r}_k) - 2\,[\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{v}_k\big)\Delta t, && (254)\\ \boldsymbol{r}_{k+1} &= \boldsymbol{r}_k + \tfrac{1}{2}\big(\boldsymbol{v}_k + \boldsymbol{v}_{k+1}\big)\Delta t. && (255) \end{aligned}}$$

The specific force is resolved with the attitude already updated by Equation (253), and gravity is evaluated at the position of the start of the step. Equation (254) applies the Coriolis acceleration $-2\,[\boldsymbol{\omega}_{ie}^e]_\times \boldsymbol{v}$. The centrifugal acceleration is part of the gravity vector $\boldsymbol{g}^e$: $\boldsymbol{g}^e$ is a *plumb-bob* gravity (gravitational plus centrifugal). Equation (255) is a trapezoidal position update.

The gravity vector is a point-mass radial model plus the centrifugal term (`ImuIntegrator::computeGravityEcef`):

$$\boldsymbol{g}^e(\boldsymbol{r}) = -\,\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r} \;+\; \omega_e^2\,[\,p_x,\,p_y,\,0\,]^{\mathrm{T}}, \qquad \mu = 3.986\,005 \times 10^{14}\,\mathrm{m^3\,s^{-2}}. \tag{256}$$

The normal-gravity (Somigliana) flattening and the J_2 zonal term are not modelled. Over one GNSS epoch interval the resulting deterministic acceleration error is small, and the estimated accelerometer bias $\boldsymbol{b}_a$ absorbs it.

17.2 INS error-state dynamics

Linearising Equations (253) to (255) about the nominal trajectory yields the continuous error dynamics $\dot{\delta\boldsymbol{x}}_{\mathrm{ins}} = \boldsymbol{F}_{\mathrm{ins}}\,\delta\boldsymbol{x}_{\mathrm{ins}} + \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{w}$, with, in the block order of Equation (251),

$$\boldsymbol{F}_{\mathrm{ins}} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -2[\boldsymbol{\omega}_{ie}^e]_\times & -[\boldsymbol{f}^e]_\times & \boldsymbol{C}_b^e & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & -[\boldsymbol{\omega}_{ie}^e]_\times & \boldsymbol{0} & -\boldsymbol{C}_b^e \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}, \qquad \boldsymbol{f}^e := \boldsymbol{C}_b^e\,\boldsymbol{f}_b. \tag{257}$$

Both sensor biases are random walks (no deterministic dynamics: the null bottom rows of Equation (257)). Two terms of the standard ECEF error model are omitted by design: the position-error feedback row ($[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{r},\delta\boldsymbol{r}} = -[\boldsymbol{\omega}_{ie}^e]_\times$) and the gravity-gradient coupling $[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{v},\delta\boldsymbol{r}} = +\,\partial\boldsymbol{g}^e/\partial\boldsymbol{r}$ (the gradient of the *plumb* gravity of Equation (256), which already subsumes the centrifugal $-[\boldsymbol{\omega}_{ie}^e]_\times^2$; the leading sign follows from differentiating the velocity update Equation (254)). The resulting first-order model is accurate over the short epoch interval of a GNSS-aided run. The discrete transition is also first order,

$$[\boldsymbol{\Phi}_{\mathrm{ins}}]_k = \boldsymbol{I} + \boldsymbol{F}_{\mathrm{ins}}\,\Delta t. \tag{258}$$

The process noise maps the continuous accelerometer and gyro white noise and the bias random walks through

$$[\boldsymbol{Q}_{\text{ins}}]_k = \boldsymbol{G}_{\text{ins}}\,\boldsymbol{W}\,\boldsymbol{G}_{\text{ins}}^{\mathrm{T}}\,\Delta t, \quad \boldsymbol{G}_{\text{ins}} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \boldsymbol{C}_b^e & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\boldsymbol{C}_b^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \boldsymbol{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \boldsymbol{I} \end{bmatrix}, \quad \boldsymbol{W} = \operatorname{diag}\!\left(\sigma_a^2, \sigma_g^2, \sigma_{ba}^2, \sigma_{bg}^2\right) \otimes \boldsymbol{I}_3, \tag{259}$$

where the spectral densities $\sigma_a, \sigma_g, \sigma_{ba}, \sigma_{bg}$ are read from `imu.{accel_noise_density, gyro_noise_density, accel_bias_instability, gyro_bias_instability}`. The defaults in `config/GnssConfigTypes.hpp` are $0.05\,\mathrm{m\,s^{-2}\,Hz^{-0.5}}$, $5 \times 10^{-4}\,\mathrm{rad\,s^{-1}\,Hz^{-0.5}}$, $1 \times 10^{-4}\,\mathrm{m\,s^{-3}\,Hz^{-0.5}}$ and $1 \times 10^{-6}\,\mathrm{rad\,s^{-2}\,Hz^{-0.5}}$, in that order. Over one GNSS epoch the per-sample pairs $([\boldsymbol{\Phi}_{\text{ins}}]_k, [\boldsymbol{Q}_{\text{ins}}]_k)$ accumulate by the standard recursion

$$\boldsymbol{\Phi}_{\text{ins}} \leftarrow [\boldsymbol{\Phi}_{\text{ins}}]_k\,\boldsymbol{\Phi}_{\text{ins}}, \qquad \boldsymbol{Q}_{\text{ins}} \leftarrow [\boldsymbol{\Phi}_{\text{ins}}]_k\,\boldsymbol{Q}_{\text{ins}}\,[\boldsymbol{\Phi}_{\text{ins}}]_k^{\mathrm{T}} + [\boldsymbol{Q}_{\text{ins}}]_k. \tag{260}$$

17.3 Covariance splice and attitude feedback

The accumulated 15×15 pair $(\boldsymbol{\Phi}_{\text{ins}}, \boldsymbol{Q}_{\text{ins}})$ is written over the indices the INS owns in an identity-seeded full-state transition $\boldsymbol{F}$ and in the random-walk process-noise baseline $\boldsymbol{Q}$. The write is an assignment, so the baseline noise on those rows is not counted twice. The baseline process noise is evaluated over the interval the INS integrated. The non-INS states (clock, troposphere, ionosphere, ambiguities, ...) keep their random-walk model from Section 12.1; because the transition is the identity there, their means are not propagated. The covariance is then propagated once and symmetrised,

$$\boxed{\boldsymbol{P} \leftarrow \boldsymbol{F}\,\boldsymbol{P}\,\boldsymbol{F}^{\mathrm{T}} + \boldsymbol{Q}, \qquad \boldsymbol{P} \leftarrow \tfrac{1}{2}\left(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}}\right).} \tag{261}$$

The mechanised values of Equations (254) to (255) overwrite the nominal navigation states $(\boldsymbol{r}, \boldsymbol{v})$. The attitude error is reset, $\delta\boldsymbol{\psi} := \mathbf{0}$, because Equation (253) has already applied the rotation increment to $\boldsymbol{C}_b^e$: after each aided prediction the IMU aiding source writes the propagated attitude into the filter through its attitude setter. The filter holds the single nominal attitude. It is initialised to the identity (there is no initial alignment step), and on the live path only two operations write it: that setter call and the feedback step below. After the next GNSS measurement update, the corrected attitude error feeds back multiplicatively into the DCM and is reset again,

$$\boldsymbol{C}_b^e \leftarrow \left(\boldsymbol{I} + [\delta\boldsymbol{\psi}]_\times\right)\boldsymbol{C}_b^e, \qquad \delta\boldsymbol{\psi} := \mathbf{0}, \tag{262}$$

followed by SVD re-orthonormalisation (`KalmanFilter::update`). This closed-loop reset keeps $\delta\boldsymbol{\psi}$ a small angle, so the linearisation in Equation (257) remains valid.

17.4 Coupling modes and the GNSS update

The coupling mode (`imu.coupling_mode`) determines how the GNSS observations correct the inertial errors. In both modes the filter processes the raw GNSS observations; the modes differ in the Doppler Jacobian and in the relinearisation path (see the remark at the end of this section).

Loosely coupled (default). Among the INS states, the GNSS Jacobians of Section 4.3 touch only the position and velocity columns. The INS error states $\delta\boldsymbol{\psi}, \boldsymbol{b}_a, \boldsymbol{b}_g$ have no direct observation partials. They are corrected indirectly, through the cross-covariances that Equation (261) builds between the position/velocity block and the attitude/bias block. The feedback of Equation (262) then applies the attitude correction.

Tightly coupled. Tight coupling requires the velocity block, so the loader enables it regardless of `estimation.estimate_velocity`. A `TightCoupled` `coupling_mode` with the flag false sets the flag true and logs `"forcing estimate_velocity=true"` (`GnssConfigLoad.cpp`). The three velocity states, and with them the Doppler rows that carry the Jacobian below, are therefore always present. In this mode the Doppler observation, and only the Doppler observation, gains direct attitude and accelerometer-bias columns. It is the one aiding contribution that adds columns to existing GNSS rows instead of appending rows of its own. Each relinearisation pass calls `AidingSourceManager::applyTcJacobianInputs` → `ImuAidingSource::applyTcJacobianInputs`, which passes the current attitude and specific force to `MeasurementModel::setImuState` (`src/core/measurement/MeasurementModel.cpp`). With $\boldsymbol{f}^e = \boldsymbol{C}_b^e \boldsymbol{f}_b$ the specific force in ECEF and $\boldsymbol{e}_r^i$ the unit line-of-sight of Equation (7), the Doppler measurement function h_{D} contributes

$$\boxed{\frac{\partial h_{\mathrm{D}}}{\partial \delta\boldsymbol{\psi}} = -\left[\boldsymbol{f}^e\right]_\times^{\mathrm{T}} \boldsymbol{e}_r^i, \qquad \frac{\partial h_{\mathrm{D}}}{\partial \boldsymbol{b}_a} = -\left(\boldsymbol{C}_b^e\right)^{\mathrm{T}} \boldsymbol{e}_r^i.} \tag{263}$$

The specific force used in Equation (263) is the static-rest approximation $\boldsymbol{f}_b \approx (\boldsymbol{C}_b^e)^{\mathrm{T}}(-\boldsymbol{g}^e) - \boldsymbol{b}_a$ (`ImuAidingSource.cpp`), evaluated at the current position estimate rather than from the IMU samples. It is exact only when the true platform acceleration is negligible compared with gravity. In either mode the code and carrier-phase rows carry no attitude or bias partials; only the Doppler rows couple directly to the inertial errors.

Remark 3 (Relinearisation under tight coupling). The tightly coupled path keeps a single *naive* second-pass relinearisation instead of the prior-anchored iterated EKF of Section 12.3. Re-deriving the gain from a fixed prior is inconsistent with the multiplicative attitude feedback of Equation (262), which has already changed $\boldsymbol{C}_b^e$ and zeroed the attitude error between passes. The aiding layer exposes the requirement as a capability query (`AidingSourceManager::anySourceRequiresNaiveRelinearization`, true for the IMU source under tight coupling), so any later source with nominal-state feedback selects the same path. Direct `tightlyCoupledImu()` checks remain in `Application.cpp` / `KalmanFilter.cpp` (the IEKF gating and the prior capture), so the migration to the capability query is incomplete.

Part VII
Real-time SSR corrections

18 Real-time SSR corrections

The real-time engine consumes network corrections in *State Space Representation* (SSR): per-satellite, per-signal refinements to the broadcast orbit, clock and hardware biases. They arrive as RTCM 3.x messages (legacy 1057–1068 and 1240–1270) or in the IGS-SSR `4076` repackaging [9, 21]. Each SSR quantity refines one term already present in Equation (4) and Equation (5) of the Foundations document, and none introduces a new state. SSR is what distinguishes the precise real-time mode from broadcast-only operation: with SSR applied, $c\,\mathrm{d}t^i$ and $\boldsymbol{r}^i$ are centimetre-grade, and the satellite biases B_s^i, b_s^i are *known* instead of absorbed into the ambiguity. This document describes the rover-side *consumer*. The same store is the sink for a third-party IGS-RTS stream and for the engine's own network estimator (Section 20), so the application mathematics below does not depend on the producer.

Decoded records are cached per satellite in `products/SsrCorrectionStore`: a process-wide singleton for the live stream, or a caller-owned instance for the network producer's closed-loop

test. The consumers (`model/ModelGeometry`, `corrections/GnssCorrections`, `measurement/MeasurementModel`) query the store and fall back to broadcast-only when it is empty. In post-processing the store is empty, so every equation below reduces to the broadcast-only observation model. The streaming and decoding layer (RTCM frame parsing, the per-satellite mutex-guarded cache, the real-time push loop) is described in Section 19. This document covers only the store-driven correction mathematics, which does not depend on how the records arrived. There is no `enable_ssr` flag: SSR is applied whenever the store holds a usable record.

18.1 Orbit correction: refines ρ_r^i

The orbit message carries a delta in the satellite's radial, along-track and cross-track (RAC) frame, referenced to the broadcast ephemeris of issue IODE. The frame is built from the broadcast position $\boldsymbol{r}^i_{\mathrm{brdc}}$ and velocity $\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}$ (`products/SsrCorrectionStore.cpp`):

$$\hat{\boldsymbol{e}}_a = \frac{\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_c = \frac{\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_r = \hat{\boldsymbol{e}}_a \times \hat{\boldsymbol{e}}_c. \tag{264}$$

The RAC delta is propagated to the query epoch with its rate term and rotated into ECEF. With $\delta t = t - t_{\mathrm{SSR}}$ the age of the correction, $\boldsymbol{\delta}_{\mathrm{rac}} = (\delta_r, \delta_a, \delta_c)^{\mathrm{T}}$ its components and $\dot{\boldsymbol{\delta}}_{\mathrm{rac}}$ their rate,

$$\boxed{\boldsymbol{r}^i \;=\; \boldsymbol{r}^i_{\mathrm{brdc}} \;-\; \big[\,\hat{\boldsymbol{e}}_r \;\; \hat{\boldsymbol{e}}_a \;\; \hat{\boldsymbol{e}}_c\,\big]\big(\boldsymbol{\delta}_{\mathrm{rac}} + \dot{\boldsymbol{\delta}}_{\mathrm{rac}}\,\delta t\big).} \tag{265}$$

The subtraction follows the RTCM sign convention $\boldsymbol{\delta}$ = broadcast − precise (`products/SsrCorrectionStore.cpp`). If the broadcast velocity or the position-velocity cross product is degenerate, the frame is undefined and the broadcast position is returned unchanged. The corrected $\boldsymbol{r}^i$ enters $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line-of-sight $\boldsymbol{e}^i_r$ of the Foundations document unchanged. The orbit correction therefore acts on no observation equation directly, only on the geometry (Section 4) that feeds it.

The delta is applied only when its IODE matches that of the broadcast ephemeris from which the baseline orbit was evaluated (Equation (269)). On a live ephemeris cutover, the baseline position is re-referenced to the ephemeris *version* against which the SSR delta was issued, so the satellite does not drop to broadcast grade (`model/ModelGeometry.cpp`, via `LiveEphemerisStore::computeSatellitePV` with the SSR IODE). The clock is queried against the same broadcast ephemeris, so the broadcast clock's *radial* component cancels consistently against Equation (267). The decoded RAC scale factors are those of RTCM 10403.3: the radial δ_r is a 22-bit two's-complement field at 0.1 mm, along-track and cross-track are at 0.4 mm, and the rates are at 1 and 4 μm/s respectively (`rtcm/RtcmSsrDecoder.cpp`).

The orbit record also carries a *reference point* for the corrected $\boldsymbol{r}^i$. Legacy RTCM SSR corrects to the satellite antenna phase centre (APC); IGS-SSR `4076` and the network product correct to the centre of mass (CoM) (`rtcm/RtcmSsrDecoder.cpp`). Under `realtime.honor_ssr_reference_point` (RTB-5; default OFF, byte-identical) the consumer records whether the applied overlay was APC-referenced (`ssr_orbit_apc_applied_` in `model/ModelGeometry.cpp`). `PppModel` then omits its own satellite-PCO term on that geometry, so the phase-centre offset already contained in the orbit correction is not counted twice.

18.2 Clock correction: refines $-c\,\mathrm{d}t^i$

The clock message carries a quadratic polynomial in the time since the reference epoch, optionally augmented by an additive high-rate delta δC_{hr}. The total satellite-clock correction, in metres, is

$$\Delta C^i(t) \;=\; C_0 + C_1\,\delta t + C_2\,\delta t^2 + \delta C_{\mathrm{hr}}, \qquad \delta t = t - t_{\mathrm{SSR}}, \tag{266}$$

converted to seconds and *added* to the baseline clock (`products/SsrCorrectionStore.cpp`):

$$\boxed{c\,\mathrm{d}t^i \;=\; c\,\mathrm{d}t^i_{\mathrm{base}} \;+\; \Delta C^i(t),} \tag{267}$$

where $\mathrm{d}t^i_{\mathrm{base}}$ is the precise CLK sample (post-processing) or the broadcast clock polynomial (live rooftop mode, `products/LiveEphemerisStore`). The term enters both undifferenced observation equations through $-c\,\mathrm{d}t^i$, identically on code and phase. The high-rate clock is added to the polynomial and does not replace it (`corrections/GnssCorrections.cpp`); it is subject to its own freshness test in the store. The C_2 field is a 27-bit two's-complement quantity at $2 \times 10^{-8}\,\mathrm{m/s^2}$. The time base of the polynomial is the record's content epoch (the provider's reference epoch), not its arrival time.

Split-cadence fast clocks. The stored clock can also move by a *fast tick* (`SsrCorrectionStore::pushFastClock`), which the network producer can run between full solves on decimated-out epochs. A fast tick rewrites three fields of an *existing* clock record: C_0 and the two time stamps (receive time and content epoch). Equation (266) is then evaluated at the fast δt with the C_1 and C_2 of the last full solve:

$$\Delta C^i(t) = C_0^{\mathrm{fast}} + C_1\,(t - t_{\mathrm{fast}}) + C_2\,(t - t_{\mathrm{fast}})^2. \tag{268}$$

Two fields are left untouched by contract, and both protect the consumer. The discontinuity counter is not advanced: a fast row continues the same clock arc, so it must not trigger the ambiguity reset of the discontinuity paragraph below at the fast cadence. The shared per-satellite SSR header is not restamped, so the code-bias, phase-bias, URA and slant-iono channels do not take the fast clock's age from a single refresh. The method only refreshes: when no clock record exists it returns false and creates nothing, so a consumer never receives a satellite that exists on the fast channel alone, without a bias set. It is called in-process (`filter/NetworkFastClock`) and on replay of an on-disk product (`products/NetworkProductIO`). It is default OFF (`network.fast_clock.enable`); the loader also disarms it when the run has no decimated-out epoch on which a tick could run.

18.3 IOD gate at correction-set boundaries

Orbit and clock corrections are deltas against a specific broadcast ephemeris, so they are applied only when the SSR record's IODE matches that of the broadcast ephemeris from which the baseline orbit/clock was evaluated:

$$\text{apply} \iff \big(\mathrm{IODE}_{\mathrm{brdc}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{SSR}} < 0\big) \;\vee\; \big(\mathrm{IODE}_{\mathrm{brdc}} = \mathrm{IODE}_{\mathrm{SSR}}\big), \tag{269}$$

The gate is permissive when either side is unknown (`SsrCorrectionStore::iodeMatches`, `products/SsrCorrectionStore.hpp`). A combined orbit+clock record (RTCM 1060/1066, IGS-SSR sub-type 3) takes precedence over separate messages because it guarantees a single shared IODE (`products/SsrCorrectionStore.cpp`). The two overlays gate on separate anchors (RTF-2): the orbit delta on the record's orbit `iode` (`model/ModelGeometry.cpp`), the clock polynomial on a distinct `clock_iode` field (`corrections/GnssCorrections.cpp`). For a standard stream, `clock_iode` inherits the orbit anchor in `getOrbitClock`, so the two coincide and (269) is a single gate. Only a `brdc_rts` producer, which stamps its own clock IODE, makes them differ. At a correction-set boundary (the provider switches to a new ephemeris reference, hence a new IODE), the gate (269) declines the mismatched record until the matching broadcast ephemeris is available. This avoids a discontinuous jump in $\boldsymbol{r}^i$ or $c\,\mathrm{d}t^i$.

Records older than `max_age_s` (`processing.ssr_max_age_s`, default 30 s) are treated as absent. The age is that of the record's receive time relative to the query epoch, and the test is symmetric: a record dated more than the same limit ahead of the observation (a desynchronised live stream) is also rejected. The RTA-9(b) stale-hold ladder extends this binary clock gate to three rungs (`SsrCorrectionStore::classifyClockAge`). With `processing.ssr_clock_stale_hold_factor` $f > 1$, a clock aged in (`max_age_s`, $f \cdot$ `max_age_s`] is *StaleHeld*. It is still applied, extrapolated over the full age: the code evaluates the whole polynomial (266), of which $C_0 + C_1\,\delta t$ is the dominant part. The consuming rows receive a matching URA-staleness R inflation (`corrections/GnssCorrections.cpp`, `measurement/MeasurementModel.cpp`). A record beyond the hold window is *Rejected*. The default $f = 0$ reduces the ladder to the plain fresh-or-absent gate (byte-identical).

18.4 Satellite biases: refine $-B^i_s$ and $-b^i_s$

SSR delivers *observable-specific biases* (OSB): one value per (satellite, RINEX-3 signal) pair, in metres. They are applied as a known additive correction to the modelled observable, with no state and no Jacobian column (`measurement/MeasurementModel.cpp`). The code OSB refines B^i_s in the undifferenced code equation:

$$\boxed{P^i_{r,s} = \rho^i_r + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t^{(g)}_r + T^i_r + \mu_s I^i_{r,1} + B_{r,s} \underbrace{-\,B^i_s}_{\text{SSR OSB}} + \varepsilon^P,} \tag{270}$$

and the phase OSB refines b^i_s in the undifferenced phase equation:

$$\boxed{\Phi^i_{r,s} = \rho^i_r + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t^{(g)}_r + T^i_r - \mu_s I^i_{r,1} + \lambda_s N^i_{r,s} + b_{r,s} \underbrace{-\,b^i_s}_{\text{SSR OSB}} + \varepsilon^\Phi.} \tag{271}$$

The decoded value is the satellite bias as it is present in the observation (the underbraced term), so it is added to the clean model prediction with sign +1 (`kSsrCodeBiasSign` = `kSsrPhaseBiasSign` = +1, `measurement/MeasurementModel.cpp`). The sign can be confirmed only empirically against a provider's stream: float ambiguity fractions that cluster near one half instead of zero indicate an inverted constant. The value enters the predicted observable as `ssr_bias_term` (`MeasurementModel.cpp`), summed with the receiver clock, troposphere, ionosphere, estimated code and phase biases and the ambiguity. Biases are queried per *raw* signal only: an ionosphere-free composite key such as `C1W_C5X` returns zero (code) or `nullopt` (phase) and is skipped, consistent with the uncombined parameterisation. The code OSB is a 14-bit field at 0.01 m, the phase OSB a 20-bit field at 0.1 mm (`rtcm/RtcmSsrDecoder.cpp`).

By default the application path has no age gate: `ssr_bias_term` is loaded whenever the store carries the exact RINEX signal key (or, for code only and with the optional band fallback of RTA-13 armed, a same-band sibling key; phase never falls back, to preserve the integer property). The optional A5 staleness gate (`processing.enable_ssr_bias_max_age`, default OFF) measures the age of the bias record against the observation epoch. Its limit is `ssr_bias_stale_hold_factor` × `processing.ssr_bias_max_age_s`; the factor defaults to 2 and the maximum age defaults to the clock maximum age, hence 30 s. Below that limit the last value stays applied at full weight across staleness gaps, so a provider update interval near the maximum age produces no model step; a bias older than the limit is dropped (`measurement/MeasurementModel.cpp`). The network-consumer profile arms the gate under bias consumption unless the configuration sets the key explicitly. The consequence of the application path for the state vector, avoiding a redundant estimated bias on the same row, is the subject of the next subsection.

18.5 Satellite-bias state gate and SSR code biases

The uncombined model can also *estimate* a per-(satellite, signal) code-bias state (`estimation.estimate_sat_biases`; see Section 3). The estimated state and a known SSR code OSB both enter the code row with unit coefficient, so allowing both for the same (i, s) is redundant: the stream already fixes the satellite code bias, and a free state collinear with it is nearly unobservable and absorbs noise. State allocation therefore consults both correction sources before creating a sat-bias state (`MeasurementModel::ensureRequiredStates`):

$$\text{create sat-bias}(i,s) \iff \neg\,\text{hasBIA}(i,s) \;\wedge\; \neg\,\text{hasSsrCB}(i,s) \;\wedge\; \text{pivotCovers}(i,s), \tag{272}$$

where hasBIA tests the static `.BIA` product (`PppModel::hasBiasEntry`) and hasSsrCB tests the live stream (`SsrCorrectionStore::hasCodeBias`, surfaced as `ssrCoversCodeSignal`). On a GLONASS FDMA signal that carries a per-slot inter-frequency-bias state, no sat-bias state is created either, because the two would be collinear. The gate rests on two design points:

- **Presence, not magnitude.** `hasCodeBias` tests whether the key is *present* in the per-signal map, which differs from `getCodeBias` $\neq 0$: a provider's present-but-zero entry still defines that signal's datum, so the estimator must not re-estimate it. The two queries agree by construction, including the optional same-band fallback, so a covered signal is never both estimated and SSR-corrected on one row. With the A5 staleness gate armed, the presence test uses the same age limit as the application path; otherwise it ignores age.
- **Per-signal, not global.** IGS streams are partial (for example, Galileo E1 covered and E5a/E5b not). The gate keys on the exact (i, s) pair. A signal the stream does not carry keeps its estimated state, its only available code correction, while a covered signal on the same satellite is left to the OSB. A strict per-epoch prune (`pruneSatBiasStatesExcept`) drops the state when SSR coverage arrives mid-arc and re-creates it (warm-restarted from the stashed cache) if coverage later disappears.

In post-processing the live store is empty, so hasSsrCB is always false and the gate reduces to the static-`.BIA` test.

Network-producer exemption. When the engine is the network estimator (`ProcessingMode::PPP_RTK_Network`), the roles are reversed: it *estimates* the satellite code and phase biases as states and emits them as a product (Section 20). Importing SSR biases there would count them twice and make those states unobservable. Both the `ssr_bias_term` application path (270) and the hasSsrCB branch of the state gate (272) are therefore disabled on the producer side (`ssrCoversCodeSignal` returns false). The producer's emitter (`products/NetworkProductEmitter.cpp`) pushes the estimated clocks and code and phase biases into a separate store instance, which closes the loop against a held-out user consumer.

Integer recoverability via consistent phase biases. Removing b_s^i restores the integer property of $N_{r,s}^i$ stated in the phase observation equation of the Foundations document. With the satellite phase bias known, the remaining non-integer part of the ambiguity is the receiver phase bias $b_{r,s}$ alone, which cancels under between-satellite single differencing $\Delta\cdot$ (Section 5). The result is an integer single-difference ambiguity admissible to AR. This defines a *consistent* phase-bias product in operational terms: the provider must generate b_s^i on the same network datum whose integer property the rover then uses. The phase-bias message therefore carries three metadata fields per signal that AR depends on (`rtcm/RtcmTypes.hpp`): an integer-preservation indicator, a two-bit wide-lane integer indicator, and a four-bit *discontinuity counter*.

Discontinuity handling. An incremented discontinuity counter signals that the provider's phase bias jumped between correction sets, which breaks the integer connection of that arc. The engine treats the change as an ambiguity-arc boundary: the affected $N^i_{r,s}$ is reset before the Kalman update (`realtime/RealTimeEngine.cpp`, via `StateManager::noteSsrDiscontinuity`), so the post-jump phase is re-initialised instead of being fixed against a stale integer offset. Under the A5 staleness gate, a bias that changes between applied and dropped triggers the same reset. Two optional reactions, both default OFF, replace the reset: a phase-bias step translation (card PBT), which takes precedence, and a declared-step carry (card CC). Each shifts the ambiguity by the bias step where that step is known and falls back to the reset otherwise. A product whose phase biases are held by the producer is handled first (card PBH). The reset is the phase-bias counterpart of the orbit and clock IOD gate (269): both protect state continuity across correction-set boundaries, one by declining a mismatched delta, the other by resetting the dependent ambiguity.

18.6 GLONASS

GLONASS SSR is decoded and stored in the same way as the CDMA constellations. The legacy block (orbit 1063, clock 1064, code bias 1065, combined 1066, URA 1067, high-rate clock 1068, phase bias 1266) and the IGS-SSR `4076` subtypes in the GLONASS message-indicator band (IM = 41–47) both map onto the same orbit, clock and bias structures (`rtcm/RtcmSsrDecoder.cpp`). The orbit, clock and bias equations above apply unchanged; nothing in Equations (265), (267), (270) and (271) is constellation-specific.

The GLONASS-specific difference lies in the time tag, not in the correction mathematics. GLONASS SSR carries its content epoch as a time-of-day field (DF389, seconds since the start of the GLONASS UTC(SU)+3 h day) instead of a GPS time of week. The decoder converts it to the GPS time scale through `TimeUtils::glonassTodToGpsTime` (`rtcm/RtcmSsrDecoder.cpp`), resolving the day from the parser's last known GPS stream time. Without that conversion the SSR epoch would be $\{0, 0\}$; the store would then drop every GLONASS record as stale or IOD-mismatched (the `max_age_s` gate that accompanies (269)), and the satellite would fall back to broadcast grade without any error. After conversion, the per-record content epoch drives $\delta t = t - t_{\mathrm{SSR}}$ in Equations (265) and (267) as for GPS. Whether the resulting GLONASS observable then *contributes* to a between-satellite-differenced solution is a separate estimator-side option (the FDMA per-channel receiver delay does not cancel in SD); see Section 5.

18.7 Ionosphere: $\mu_s I^i_{r,1}$ off-wire, via the slant channel

The SSR standard defines a global VTEC spherical-harmonic message (IGS-SSR IM = 201) and regional STEC products, but this engine decodes neither. The IGS-SSR classifier rejects IM = 201 as unsupported (`rtcm/RtcmSsrDecoder.cpp`), so nothing on the RTCM wire writes the slant-iono state $I^i_{r,1}$. For a consumer of an RTCM stream, the ionospheric term $\mu_s I^i_{r,1}$ is therefore informed only by (i) the per-satellite iono *state* estimated by the filter and (ii) the optional GIM/IONEX deterministic prior (`atmosphere/GlobalIonosphereMap`, an independent VTEC→STEC mapping unrelated to the SSR stream).

The atmosphere loop is closed *off* the RTCM wire, through the same `SsrCorrectionStore` that carries the clock and bias overlays. The store has a dedicated undifferenced slant-iono channel (`SsrSlantIono`, via `SsrCorrectionStore::pushSlantIono` / `getSlantIono`). The network producer's emit seam (`filter/NetworkEmitStore`) evaluates its co-estimated SH-VTEC field (Section 9.2.4) at the consumer's pierce points and pushes a per-LOS slant delay and 1σ. The consumer (`RealTimeEngine::processEpoch`, through its network ionosphere-prior step) folds each

fresh record into an ionosphere-weighted (BLUP) pseudo-observation on the matching slant-iono state (`KalmanFilter::applyPseudoObservation`). This is the ionosphere-weighted constraint that makes single-epoch AR feasible. The fold is gated by `closed_loop.consume_network_iono` (default OFF: no rows are built and the update is byte-identical). It admits a value only when the producer refreshed it in the current epoch (`max_age` $= \frac{1}{2}$ the epoch spacing, or 30 s when the spacing is unknown), so a satellite the producer did not re-supply floats instead of inheriting a stale delay.

Three properties of the channel follow from the field and are not chosen by the consumer. The supply covers GPS and Galileo only: GLONASS is excluded from the SH field, so a consumer's GLONASS slant states float. When a second shell is armed, the pushed value is the *sum* over shells, each evaluated at its own pierce geometry, so the consumer does not depend on the layer model. The per-LOS σ is the field's representativeness floor (Equation (96)). Under `network.iono_supply_formal_sigma` (default OFF, byte-identical) it is root-sum-squared with the SH coefficients' *formal* σ, propagated through the same weighted basis, and the supply is skipped for a satellite that has no estimated satellite code bias (OSB) in the current producer solution.

On the consumer side the rows are not independent. They are folded as a per-constellation common-mode block with the noise structure of Equation (92). Frozen BSSD pivots, and any slant state with zero variance, are excluded for the reason given there: the common-mode block absorbs the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

A *standardised* on-wire STEC product remains open: the in-process channel is a closed-loop store hand-off, not an IGS-SSR or RTCM message that a consumer of a third-party stream can decode.

18.8 Provenance and code pointers

Both transports map onto the same internal correction structures (`rtcm/RtcmTypes.hpp`) through a single decoder (`rtcm/RtcmSsrDecoder.cpp`). Legacy RTCM uses per-constellation six-message blocks (orbit, clock, code bias, combined orbit+clock, URA, high-rate clock) and a separate phase-bias block (1265–1270). IGS-SSR `4076` repackages the same payloads under a message indicator (IM). Its *tens* place ($\lfloor \mathrm{IM}/10 \rfloor$) selects the constellation (GPS $2x$, GLONASS $4x$, Galileo $6x$, QZSS $8x$, BeiDou $10x$, SBAS $12x$), and its *ones* place (IM mod 10) selects the subtype (1=orbit, 2=clock, 3=combined, 4=high-rate, 5=code bias, 6=phase bias, 7=URA). This subtype ordering differs from the legacy SSR block ordering, so the legacy mapping is not reused (`RtcmSsrDecoder.cpp`). VTEC (IM 201) and NavIC (IM 141–147) are rejected by the message-indicator classifier before the header is parsed, so they return `ok = false` with an empty header. For this reason the ionospheric term receives no *on-wire* SSR contribution (the off-wire `SsrSlantIono` store channel of the ionosphere subsection is unaffected).

The correction definitions Equations (265), (267) and (269) are implemented in the static, deterministic `applyOrbitToPosition` / `applyClockToBias` / `iodeMatches` members of `products/SsrCorrectionStore`, the interface this document describes. The stateful streaming layer (the mutex-guarded per-satellite cache with freshness gating on receive time, the `RealTimeEngine::feedRtcmFrame` push loop, and the obs↔SSR synchronisation that bounds how far the obs stream may run ahead of lagging corrections) is described in Section 19. It is kept separate so that the correction mathematics can be analysed and tested in isolation. The estimator-side warm-up gate `estimation.defer_until_corrections_ready` (`config/GnssConfigLoad.cpp`; default OFF, real-time only) defers estimation until a quorum of satellites is centimetre-grade from broadcast plus SSR (by default at least 5 in total and 4 per constellation). The filter then seeds code and phase together, instead of cold-starting on metre-grade broadcast clocks and fixing a contaminated ionosphere/bias/ambiguity partition.

The store also carries a per-epoch producer *heartbeat* (`NetworkProducerStatus`). It lets a consumer distinguish "the producer withheld this epoch" from "the producer stopped": a withheld epoch still writes its status line, although it publishes no corrections. The wire token set is forward-compatible: an unrecognised token from a newer producer maps to `State::Unknown` and is reported, never fatal (`NetworkProducerStatus::parseState` / `stateName`, declared in the store's header). One of the tokens reports a withhold driven by the health of the producer's own SH-VTEC field. The *policy* behind that withhold (what engages it, what it suppresses and what it leaves alone) is described in Section 20 and is not restated here.

Part VIII
Real-time engine

19 Real-time engine

The real-time path reuses the estimator, measurement model and state manager of post-processing and reorganises the control flow around a *streaming* contract. Complete RTCM 3 (or SBF) frames arrive one at a time from a file or a network socket and are decoded into observations, corrections and broadcast ephemeris. A buffered epoch passes through the Section 12 predict/update cycle as soon as its boundary is detected. Post-processing iterates a closed list of epochs in time order with the full orbit and clock product set on disk; the engine sees each epoch once, receives corrections that may lead or lag the observations, and cannot look ahead. The entry point is `Application::runRealTime` (`src/core/app/ApplicationRealtime.cpp`), which constructs a `RealTimeEngine` (`src/core/realtime/RealTimeEngine.cpp`) and drives it from a single-threaded multi-source poll loop. The engine takes its input through the frame feeds `feedRtcmFrame` and `feedSbfFrame`; at stream end `flush` processes whatever epochs are still buffered. This section covers the parts specific to that path. The estimation mathematics, the observation equations and the SSR correction model are documented elsewhere and are only cross-referenced here.

19.1 Streaming control flow

The poll loop holds one `RealtimeStream` per logical source and services the sources in turn without blocking on any one of them:

- a *primary* stream carrying rover observations (RTCM 3 MSM or Septentrio SBF, from a file replay, an NTRIP mountpoint, or a raw TCP socket);
- an optional *corrections* stream (typically an IGS-RTS NTRIP mountpoint carrying SSR);
- an optional *ephemeris* stream (broadcast nav for an obs-only station that does not co-broadcast 1019/1020/...).

Each iteration drains a bounded burst from every active source through its framer and dispatches the complete frames to the engine (`pumpFrames`). The primary drain is capped at 1 MiB per iteration, and the corrections and ephemeris streams are read one 4 KiB chunk per iteration, so a high-rate observation stream cannot starve the sparse SSR and navigation streams. When a live network-product tail is configured, the loop also polls it at about 5 Hz and wakes the obs/SSR hold buffer of Section 19.7 when new products arrive. The loop exits when every source is inactive (file EOF or client stopped), when a stop is requested (Ctrl+C or the console-close handler), or when

the `processing.max_duration_s` data-time budget latches. On exit the engine flushes the final buffered epoch before teardown, so no observations or trace rows are lost.

A socket source counts as *active* while its client is running, whether or not it is connected at that moment. A transient drop or the pre-connect window at startup therefore does not end the session; the client reconnects in the background.

19.2 NTRIP transport

Caster connection. `NtripClient` (`src/core/realtime/NtripClient.cpp`) is an NTRIP v2 client built on libcurl. A background thread holds the HTTP GET open and appends the received bytes to a mutex-guarded buffer with a hard cap of 4 MiB. When the cap is reached the buffer drops its oldest bytes. Consumers read through a blocking `read` or a timeout-bounded `tryRead`; the multi-stream loop uses the latter, so one stream cannot block the others. The request carries the NTRIP v2 headers (`Ntrip-Version:Ntrip/2.0` and a `User-Agent`), and the mountpoint is the URL path. HTTP Basic authentication (`CURLAUTH_BASIC`) is supplied when a username is configured. An optional NMEA GGA sentence for VRS mountpoints is sent as the NTRIP v2 `Ntrip-GGA` request header on the GET; the request is never turned into a POST.

Transport security. The transport defaults to plain HTTP, which serves plain-HTTP casters such as IGS-IP (`igs-ip.net`) unchanged. A `use_tls` flag selects the `https` URL scheme and sets `CURLUSESSL_ALL`, as required for port-443 casters such as CDDIS that offer no plain-HTTP endpoint. Certificate verification (`SSL_VERIFYPEER`/`VERIFYHOST`) stays on unless a testing-only `tls_insecure` override disables it for self-signed or mis-chained casters. Both flags default off, so a plain-HTTP run is unaffected.

Reconnection. The receive loop reconnects automatically. After each transfer ends it classifies and logs the outcome: a transport error, a non-200 HTTP status (with hints for 401, 403 and 404), a connection that delivered zero bytes (an idle mountpoint, or a VRS mountpoint that needs a GGA position), or a clean stream end. It then always waits `reconnect_delay_s` plus a uniform 0–2000 ms jitter before retrying, so a misconfigured mountpoint appears in the log as an error and several streams dropped by one caster outage do not reconnect in lockstep. An NTRIP v1 caster answers "ICY 200 OK", which libcurl reports as status code 0; the loop treats code 0 as success.

19.3 RTCM 3 framing and CRC

`RtcmStreamReader` (`src/core/realtime/RtcmStreamReader.cpp`) turns an arbitrary byte stream into discrete RTCM 3 frames and so separates transport from parsing. An RTCM 3 frame is

$$\underbrace{\texttt{0xD3}}_{\text{preamble}}\ \underbrace{[\,6\text{ reserved}\,|\,10\text{-bit length }L\,]}_{2\text{ bytes}}\ \underbrace{[\,\text{payload, }L\text{ bytes}\,]}\ \underbrace{[\,\text{CRC-24Q}\,]}_{3\text{ bytes}}, \tag{273}$$

so the total frame length is $3 + L + 3$. `nextFrame` scans for the `0xD3` preamble (discarding other bytes), reads the 10-bit payload length (a length above 1023 bytes is rejected as a false preamble), waits until the full $3 + L + 3$ bytes are buffered, and verifies the trailing 24-bit Qualcomm CRC (CRC-24Q, table-driven) computed over header+payload. On a CRC mismatch the reader advances past the candidate preamble byte and resumes scanning, so a corrupt or misframed byte cannot stall it. Only CRC-valid frames are emitted, and the downstream parser (`RtcmParser`) relies on frame integrity. The reader counts extracted frames and discarded bytes for stream-health diagnostics.

Septentrio SBF. A parallel binary path decodes Septentrio SBF frames into the same observation and ephemeris outputs (`src/core/sbf/`). `SbfStreamReader` performs the `$@` (`0x24 0x40`) sync and the CRC-16/XMODEM check, and `SbfParser` dispatches blocks by ID and decodes the `MeasEpoch` observations. `Meas3Decoder` unpacks the compact, differentially encoded `Meas3Ranges` observations; GLONASS FDMA slots are resolved through `SbfSignalMap`. `SubframeAssembler` rebuilds broadcast ephemeris from raw-nav subframes. This section covers only the framing and the feed seam (Sections 19.1 and 19.9). The `Meas3Ranges` differential range reconstruction is the direct analogue of (274) and (275) below and the one part of the SBF path that is algebra rather than ICD bookkeeping. It is *not* yet derived in this reference, and no other document derives it: this is a known gap. The remaining SBF detail (`SubframeAssembler` ICD scaling, the `SbfSignalMap` tables, the framing constants) belongs in the Doxygen headers and is out of scope here.

The top-level `RtcmParser` (`src/core/rtcm/RtcmParser.cpp`) reads the 12-bit message type DF002 from the first 12 bits of the payload (the frame header carries only the length) and dispatches on it: 1005/1006 station coordinates; 1033 receiver and antenna descriptors and 1230 GLONASS code-phase biases (both logged, neither consumed); 4076 IGS-SSR; legacy SSR (1057–1068, 1240–1270); broadcast ephemeris (1019/1020/1042/1044/1046); and MSM4, MSM5 and MSM7 observations. The week that promotes a truncated epoch field to a full `GnssTime` comes from the parser's per-stream time tracker rather than from the host clock at each frame (Section 19.6). One parser instance serves the primary, corrections and ephemeris streams, so all three share one GPS timeline.

19.4 MSM observation decode

A single body (`decodeMsmImpl`, `src/core/rtcm/RtcmMsmDecoder.cpp`) decodes every MSM observation message. It is parameterised over an `MsmVariant` descriptor that supplies the per-field bit widths, scale factors, sentinels and capability flags. Three variants are instantiated: MSM4, MSM5 and MSM7. The variant mechanism can describe MSM6, but no MSM6 entry exists, so MSM6 is not decoded. The header carries a 64-bit satellite mask, a 32-bit signal mask and an $N_{\text{sat}} \times N_{\text{sig}}$ cell mask; the number of cells is $N_{\text{cell}} = \text{popcount(cell mask)}$. Each field block is packed contiguously across all satellites or all cells, in mask order, and is decoded into parallel vectors before recombination.

Range reconstruction. For each satellite the rough range in milliseconds is the sum of the 8-bit integer part (DF397) and the 10-bit modulo part (2^{-10} ms, DF398). Each cell adds a signed fine term with a variant-specific scale. With c the speed of light, the pseudorange and carrier phase of a cell are

$$P = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{P}_{\text{fine}}\, \sigma_P\Big), \tag{274}$$

$$\Phi_{[\text{m}]} = c \cdot 10^{-3}\Big(n_{\text{int}} + n_{\text{mod}}\, 2^{-10} + n^{\Phi}_{\text{fine}}\, \sigma_\Phi\Big), \qquad \Phi_{[\text{cyc}]} = \Phi_{[\text{m}]}/\lambda, \tag{275}$$

with fine scales $\sigma_P = 2^{-24}$, $\sigma_\Phi = 2^{-29}$ for MSM4 (and MSM5) and $\sigma_P = 2^{-29}$, $\sigma_\Phi = 2^{-31}$ for MSM7 (all in ms). Cells holding the RTCM-defined invalid sentinel are skipped. Carrier phase is emitted in cycles, the RINEX-3 convention the observation model consumes (Section 4); when the wavelength λ cannot be resolved, the phase is stored in metres under the same observation code. The decoder also produces CNR (DF403 in whole dB-Hz for MSM4/5, DF408 in 1/16 dB-Hz for MSM7) and, for MSM5/7, a Doppler value from the rough+fine phase-range-rate (the negated rate over the wavelength). The half-cycle-ambiguity bit (DF420) sets the LLI half-cycle flag.

DF419 extended-satellite-info gating. The 4-bit DF419 "extended satellite information" field determines the framing of the per-satellite block. In MSM5 and MSM7 it is present in the bitstream for every constellation, but only GLONASS assigns it a meaning (the FDMA frequency-channel number). The decoder therefore gates two operations separately:

- *bitstream consumption* is gated on the MSM variant (`honour_ext_sat_info`, true for MSM5/MSM7, false for MSM4/MSM6): when set, $4\,N_{\text{sat}}$ bits are read for all systems;
- *interpretation* is gated separately on the system descriptor; only GLONASS maps the value to a channel.

Reading DF419 only for GLONASS (or skipping it in MSM7) under-consumes the per-satellite block by $4\,N_{\text{sat}}$ bits, so the per-cell fine pseudoranges are read from the wrong bit offset. The rough range survives but the fine terms are biased, which produces an L1/L2 split of order hundreds of metres with the wrong sign on the geometry-free combination. These observations look plausible and pass an over-read check. Gating on the variant is the fix for the GPS L1/L2 geometry-free sign anomaly, and it matches RTCM 10403.3 and the cssrlib reference. As a second guard, the parser's `warnIfUnderConsumed` warns whenever a decode leaves ≥ 8 payload bits unread, which detects this class of misalignment where the bit reader's over-read flag does not.

19.5 Broadcast nav, SSR and station coordinates

Broadcast ephemeris. Messages 1019 (GPS LNAV), 1044 (QZSS), 1046 (Galileo I/NAV), 1042 (BeiDou D1) and 1020 (GLONASS) are decoded by `RtcmNavDecoder` into a common `BroadcastEphemeris` and pushed into the process-wide `LiveEphemerisStore`, from which the geometry layer resolves satellite orbits when no SP3/CLK file is loaded. The field order differs by message (QZSS, for example, leads with its clock block). GLONASS carries a sign-magnitude PZ-90 state vector, which is integrated separately.

SSR corrections. `RtcmSsrDecoder` decodes both the legacy SSR messages and the IGS-SSR 4076 container and pushes the records into the shared `SsrCorrectionStore` with their SSR header (provider epoch and IOD-SSR), so staleness and IOD diagnostics refer to the provider solution rather than to the time of receipt. The store holds orbit, clock, combined, code-bias, phase-bias, URA and high-rate-clock records per satellite. The orbit and clock query paths read the store directly, so no per-epoch wiring is needed. The correction mathematics (the radial/along/cross orbit projection, the clock and bias application, the ionosphere model) is documented in Section 18 and is not repeated here. Two reactions to the stream are specific to the real-time path:

- an incremented IGS-SSR phase-bias *discontinuity counter* for a signal breaks the integer property of that signal's ambiguity. By default the engine resets the affected ambiguity before the same epoch's measurement update (`StateManager::noteSsrDiscontinuity` detects the counter change). Two opt-in knobs replace the reset: the declared-step carry `processing.ssr_phase_bias_disc_carry` (card CC) and the step translation `processing.ssr_phase_bias_step_translate` (card PBT, which also covers undeclared steps and takes precedence). Both shift the ambiguity by the phase-bias step instead of re-floating it;
- every SSR push re-attempts a held observation epoch (Section 19.7), because the correction horizon has just advanced.

Station coordinates (1005/1006). `decodeStationCoords` reads the antenna reference point (ARP) as three signed 38-bit ECEF components in 0.1 mm units; 1006 adds a 16-bit antenna height in the same unit. A reference station that streams its own coordinates thereby supplies an a-priori seed (Section 19.9).

19.6 Time-system handling

Streaming messages carry truncated, system-specific epoch fields, which are promoted to a common GPS-time `GnssTime`. Each parser instance keeps a tracker, in continuous GPS seconds, of the last decoded stream epoch. It promotes every mod-week or time-of-day field by *nearest continuation* to that tracker: the week (or day) that minimises $|\Delta t|$, as in RTKLIB's `adjweek` (`RtcmParser::anchorGpsSeconds`). On this path the host clock (`TimeUtils::nowContinuousGpsSeconds`, POSIX UTC plus the leap offset) is read only at cold start, on the first time-bearing frame after construction, so a skewed host clock or a race at a week rollover cannot move an established stream to another week. Each successful MSM or SSR epoch advances the tracker (`updateTimeAnchor`). The Keplerian broadcast-nav decoders read the tracker for their week but do not advance it. The anchor below is the tracker's current value. One exception: the 1020 GLONASS ephemeris TOE is anchored on the host clock at every message (see the GLONASS item below).

- **Flat ms-of-week systems** (GPS/GAL/QZS/SBAS/BDS MSM): the 30-bit epoch is milliseconds of week. BeiDou adds the constant 14 s BDT→GPS offset and re-wraps the result into the week.
- **GLONASS** carries its 30-bit MSM epoch as a 3-bit day-of-week followed by a 27-bit time-of-day in ms. Reading only 27 bits misaligns the satellite mask and corrupts the whole GLONASS decode. `TimeUtils::glonassTodToGpsTime` promotes the ms-of-day: it places the reported time-of-day in the GLONASS = UTC(SU)+3 h ("Moscow") day nearest the stream-time tracker and converts back to GPS time, re-adding the leap seconds:

$$t_{\mathrm{GPS}} = \left(t^{0}_{\mathrm{day}} + \tilde{t}_{\mathrm{TOD}} \right) - \Delta_{\mathrm{Moscow}} + \Delta_{\mathrm{leap}}, \tag{276}$$

  where t^{0}_{day} is the Moscow-day midnight nearest the anchor and $\tilde{t}_{\mathrm{TOD}}$ is the reported time-of-day placed into that day by resolving the midnight wrap toward the nearest match (± 12 h). The 3-bit day-of-week is read but not used, since the anchor resolves the day. The same conversion promotes any GLONASS SSR epoch against the tracker. It also promotes the 1020 ephemeris `toe`, but there the anchor is the host clock, read at each message, rather than the stream tracker. On a live stream the two agree to well within 12 h; on a file replay of historical data they do not.
- **GLONASS FDMA wavelength.** GLONASS L1/L2 are frequency-division: each slot transmits on $f = f_{\mathrm{base}} + k\,\Delta f$ with channel $k \in [-7, +6]$, so the carrier wavelength is per satellite. The decoder maps DF419 to the channel as $k = v - 7$ for the valid range and uses it in Equation (275). The channel is also stored on the observation, so the slot map and the MW combinations can resolve frequency. When DF419 is absent (MSM4/MSM6) or holds a reserved value, the channel is marked unknown and the phase is stored in metres; a default of zero would decode to a false $k = -7$.

Each decoded record carries its own content epoch (`epoch_time`). The synchronisation logic of Section 19.7 compares against that epoch, not against the time of arrival.

19.7 Observation/SSR synchronisation

PPP-RTK needs observations time-*aligned* with the corrections rather than with the host clock. In a live session the observation stream commonly runs ahead of a lagging SSR stream. An observation

epoch that no satellite's SSR can cover is processed entirely at broadcast grade (and, with `drop_broadcast_grade`, loses every satellite). The opt-in `processing.enable_obs_ssr_sync` hold buffer prevents this. With t_{obs} the observation epoch, the SSR coverage horizon is the freshest stored SSR content epoch (`SsrCorrectionStore::newestSsrEpoch`) plus the maximum SSR age:

$$t_{\mathrm{horizon}} = t_{\mathrm{SSR}}^{\mathrm{newest}} + \tau_{\max}, \qquad \tau_{\max} = \texttt{ssr_max_age_s}. \tag{277}$$

The decision is:

- $t_{\mathrm{obs}} \le t_{\mathrm{horizon}}$: SSR covers the epoch. The epoch is processed normally and any hold is cleared;
- $t_{\mathrm{obs}} > t_{\mathrm{horizon}}$: the epoch is *deferred*. The hold buffer keeps only the latest such epoch (skip-to-latest semantics), and `drainHeldObs` re-attempts it as soon as an SSR push (or a new network-product epoch, through `notifyProductsAdvanced`) advances the horizon.

A liveness backstop keeps a dead SSR feed from starving the filter. If a hold persists past `obs_ssr_sync_max_hold_s` of host-clock time, the engine latches a degraded "outage" state and passes every epoch through (at broadcast grade, or dropped by the existing path) until SSR covers the observations again. The held epoch is drained with skip-to-latest *bypassed*, so the SSR-aligned epoch is not discarded. The final `flush` bypasses both the hold and skip-to-latest, so the last epoch is always processed. The feature is inert when disabled, before the first epoch, and in any run without a live SSR orbit and clock source.

A separate, coarser `enable_obs_skip_to_latest` drops observation epochs whose lag behind the *host-clock* GPS time exceeds `obs_skip_max_lag_s` (default 5 s), so a backlogged stream fast-forwards to the freshest epoch. The skip returns before any continuity state is touched, so the next processed epoch measures its time step from the last accepted one and trips the gap path (Section 19.10) when the skipped span exceeds the gap tolerance.

19.8 Ephemeris management

`LiveEphemerisStore` keeps, per satellite, the current record and one previous record (the current record is demoted to previous when the IODE changes), and evaluates the orbit and clock against the appropriate one. Two policies govern it in real time.

- **Freshest-toe selection.** An incoming record is rejected when its `toe`, expressed in continuous GPS seconds (`toeGpsContinuous`, which reconciles the BDT and GLONASS time scales), is older than the held record's by more than 1 s. A stale or re-ordered broadcast therefore never overwrites a newer one. A record from the IGS stream is also protected from a receiver-decoded record at equal `toe`: the IGS-RTS SSR deltas are computed against the IGS broadcast, and a receiver ephemeris with different fit parameters would make the pairing inconsistent. Receiver nav replaces an IGS-stream record only when its reference epoch is at least 1800 s newer (the GLONASS nominal `toe` step), which indicates that the IGS stream has stalled on that satellite.
- **toe-aware and IOD-aware selection.** For orbit and clock evaluation the store picks the record whose `toe` is nearest the query epoch (`pick`). For an SSR-corrected satellite it first selects the record the SSR delta was computed for (`pickForSsr`, which matches the SSR IODE against the current and the retained previous record). Across an ephemeris cutover the receiver can be one IOD ahead of the SSR stream; without this re-reference the IOD mismatch would drop the satellite to broadcast grade.

19.9 A-priori seeding from RTCM 1005/1006

A *live* SBF/RTCM observation stream carries no RINEX `APPROX POSITION` record (a file-fed source does; see the third item below). Without an a-priori position the engine falls back to a coarse Earth-centred seed at the geographic pole. `seedInitialState` calls `PppModel::initializeState` and then applies the fallback, since a state write before that call would be overwritten. The model it calls is built by `initProcessingComponents`, so `seedInitialState` must follow it in the construction order. From there almost every satellite sits below the elevation mask, the first `computeModels` yields no usable satellite, WLS never runs and the filter cannot bootstrap. The engine therefore seeds the a-priori position from the stream itself until the Kalman filter owns the position:

- from a *PVTGeodetic* solution in an SBF stream (`feedSbfFrame`);
- from an RTCM *1005/1006* station ARP (`feedRtcmFrame`), the RTCM analogue of the PVT seed. This makes `receiver.reference_position` optional for any station that streams 1005/1006. An optional blunder gate (`realtime.arp_blunder_gate_m`, RTA-12(3); 0, the default, disables it) rejects a broadcast ARP that disagrees with an existing a-priori by more than that distance, so a wrong mountpoint or stale 1005 coordinates cannot move the seed;
- from the RINEX header of a *file-fed* source, reduced from the marker to the antenna reference point through `ANTENNA:DELTA H/E/N` (`RinexParser::getArpPositionXyz`, which calls `Coordinates::markerToArp`). The position state, and the ATX PCO/PCV applied to it, refer to the ARP rather than the monument, so seeding on `APPROX POSITION XYZ` unreduced would start the filter one eccentricity away. The reduction is an exact no-op for a zero or absent eccentricity, so a live SBF/RTCM stream is unaffected; it matters only for the file-fed closed-loop consumer, whose header carries a real eccentricity. The reduction is not applied twice: `PppModel::initializeState` reduces from the *raw* marker, and only while the state position is still zero or non-finite.

A configured `reference_position` always takes precedence. The seed is applied only while `first_epoch_` is set. The startup gate keeps that flag set through warmup, so a 1005/1006 message that arrives a few seconds into the stream (these messages are sent less often than the 1 Hz observations) still seeds before the first estimation epoch. The flag clears in the first epoch that passes the startup gate, after measurement conditioning and before the predict (Section 19.10). From then on the filter owns the position, and a decoded ARP or PVT is only logged for comparison.

19.10 The per-epoch real-time update

`processEpoch` runs the predict/update pipeline of post-processing, with streaming-specific guards in front and the post-processing-only stages removed. It is a thin ordered driver. Each step is a private member function (a *seam*) in one of the files `src/core/realtime/RealTimeEngine*.cpp`, grouped by concern: admission (`RealTimeEngineAdmit.cpp`), measurement-set preparation (`RealTimeEnginePrepare.cpp`), slip QC (`RealTimeEngineSlipQc.cpp`), SSR datum-break reactions (`RealTimeEngineSsrBreaks.cpp`) and estimation (`RealTimeEngineEstimate.cpp`). Seams exchange per-epoch values through `RtEpochContext`, which holds nothing beyond the epoch; a value that must survive to the next epoch is a member of the engine. The four admission gates are the only seams that may end an epoch early. The driver itself also ends an epoch when no observation of an active system remains or when `computeModels` fails; after the startup gate every step runs unconditionally. An optional staged filter-state snapshot (warm resume) is restored before the gates, so the first time step spans the actual outage. In order:

1. **Lag, sync and duration gates.** The host-clock backlog warning, the obs↔SSR hold of Section 19.7 and the skip-to-latest drop form the first gate (`admitBacklogAndSync`); the `max_duration_s` latch is the second (`gateMaxDuration`). Both run before any continuity state is touched.
2. **Step and gap detection** (the third gate, `stepTimeAndDetectGap`). $\Delta t = t_k - t_{k-1}$, with a week-rollover correction; the first epoch assumes 1 s. A non-advancing $\Delta t \leq 0$ is a duplicate (a second co-timestamped SBF block or a retransmit) and is skipped with the state untouched. Folding it in would double-count the measurements and pass $\Delta t = 0$ to the process-noise build, which throws. A Δt above the gap tolerance (`processing.data_gap_tolerance_s`, 1 s when unset) flags a gap.
3. **Models and startup gate** (the fourth gate, `gateCorrectionsReady`). Observations are filtered to the active systems and run through `computeModels`. When `defer_until_corrections_ready` is armed, estimation (no predict, no update) is deferred until enough cm-grade broadcast+SSR satellites are present: 5 in total and 4 per active constellation by default, the per-constellation floor reduced to what the constellation currently fields but never below two. A timeout forces a degraded start. The filter then initialises code and phase together on a clean state instead of cold-starting on metre-grade broadcast clocks.
4. **Measurement-set conditioning.** After two observability sweeps (the SSR coverage log and the SSR consumer trace), `dropBroadcastGrade` removes broadcast-grade satellites when the startup gate's `drop_broadcast_grade` is set. In modes with a receiver-clock state, `seedClockFromWls` replaces the filter clock with a WLS clock estimate when the two differ by at least 5×10^{-5} s (`kJumpThresh`, 0.05 ms, about 15 km), after a gap, or on the first epoch. Under BSSD (no receiver-clock state) `regaugeBssd` subtracts the median of the epoch's code innovations, one receiver clock common to all satellites, from every code and phase observable. This common-mode shift leaves the single differences exactly unchanged and keeps the per-satellite bias states at DCB scale instead of absorbing the full unsteered receiver clock. The clock seed and the BSSD re-gauge are mutually exclusive: the first requires a receiver-clock state, the second its absence. The first-epoch flag (`first_epoch_`) clears at the end of this step.
5. **Predict.** `predictAided` over $(t_{k-1}, t_k]$, the aiding path shared with post-processing; for a GNSS-only stream it delegates byte-identically to `predict`(Δt) of Section 12.
6. **Cycle-slip detection and repair** (`detectAndRepairSlips`). The MW detector, the optional TurboEdit MW+GF joint repair and the soft-LLI gap bridge run before the update, with the detectors and repair machinery of Section 15. They are enabled under `estimation.cycle_slip`. TurboEdit is for high-rate data only; its GF-arc gap tolerance is 1.5 times the configured stream rate (1 Hz if none is set), because a live stream has no RINEX interval header.
7. **SSR datum-break reactions.** These react to the producer moving its datum and do nothing without SSR content. The order is fixed. A change of the SSR solution-generation id (`reactToGenerationChange`) resets every ambiguity and re-inflates the receiver code-bias states; it runs first because it subsumes the per-signal reactions. `logProducerStatus` logs a producer status transition. `resetOnPhaseBiasDisc` resets (or, with cards CC or PBT, shifts) the ambiguity of a signal whose phase bias shows a discontinuity or a freshness flip, as described for the SSR corrections above. `reinflateOnCodeBiasDisc` re-inflates the receiver code-bias states on a code-bias datum re-reference, and `reinflateOnClockDisc` re-inflates the affected inter-system bias states on a clock datum re-reference. Last, the opt-in declared-level-step reaction `reactToDeclaredLevelSteps_` (card CLV2, `processing.ssr_level_step_inflate`, default off) compares each satellite's producer-declared clock and code-bias levels with their previous values.

When the resulting step in a code signal's level exceeds a 0.1 mm dead band, it inflates the covariance along the direction that moves the modelled code but not the modelled phase (the satellite's slant ionosphere together with its ambiguities, each ambiguity weighted by the squared ratio of the GPS L1 frequency to its signal frequency). The added standard deviation is the step size times a configured scale. A satellite without an ionosphere state or an ambiguity is refused.

8. **Measurement update** (`runMeasurementUpdate`). The outlier and forced-pivot-handover rebuild loop of the post-processing driver (shared via `EpochUpdateSupport`): rebuild on `NeedsMeasurementRebuild`, forward each constellation's forced pivot handover, re-inflate reset-satellite states, clear the MW/TurboEdit arc history of KF-reset signals, and remove one outlier per pass. Phase signals held by the soft-LLI bridge are excluded from the update.
9. **Pseudo-observation constraints.** After the update and before AR, so the reduced covariance reaches the resolver. An optional soft a-priori ZWD pseudo-observation (`applyZwdConstraint`, `estimation.zwd_constraint`) anchors the BSSD [up, ZWD] near-null subspace. When the engine consumes a network ionosphere product, `applyNetworkIonoPrior` constrains each satellite's ionosphere state with a slant-ionosphere prior from the same epoch, with an error block common to each constellation.
10. **Ambiguity resolution** (`resolveAmbiguities`). After a successful float update, MLAMBDA AR runs on the committed state with the configuration surface of post-processing and is fed the MW-resolved wide-lane integers (Section 16). The closed-loop network-bias validation suppresses the wide-lane integers, so AR runs on the float state alone.
11. **State extraction and trace** (`writeEpochTraces`). The posterior is synced back to the PPP state and the trace rows are written. The optional ambiguity-level check (card ALC), the per-satellite consumer readout (card CST) and the periodic filter-state snapshot follow.

Post-processing stages skipped in real time. Non-causal and batch stages do not fit a streaming pipeline without look-ahead and are absent:

- the backward RTS Section 12 is disabled under `AppMode::RealTime`, because smoothing requires the full forward trajectory and cannot run causally;
- geometry relinearisation (the IEKF second pass) is a post-processing-only refinement and is not replicated;
- precise SP3/CLK products are not loaded when a live broadcast+SSR source replaces them (a real-time *file* replay without corrections still loads them). Orbit and clock are gated separately, so a configuration with precise orbits and network clocks loads the SP3 file and skips the CLK file;
- the stages that need a GIM have no source in the stream: higher-order ionosphere, GIM ionosphere weighting and adaptive ionospheric process noise are skipped with a warning when enabled.

19.11 Assessing a real-time run

A streaming run has no answer key, unlike a simulated one. The self-referenced statistic of the ordinary state viewer (scatter about the run's own median) cannot see an error *common* to every epoch, and a correction-driven pipeline is most likely to make that kind of error. A static receiver on a surveyed mark supplies the missing datum. Let $\boldsymbol{r}_{\text{ref}}$ be the surveyed antenna position (`receiver.reference_position_ecef`) with geodetic latitude and longitude (φ, λ), and $\hat{\boldsymbol{r}}_k$ the filter's posterior position at epoch k. The error is the ECEF difference rotated into the local horizon frame at the

reference point,

$$\begin{bmatrix} e_k \\ n_k \\ u_k \end{bmatrix} = \boldsymbol{R}(\varphi, \lambda)\left(\hat{\boldsymbol{r}}_k - \boldsymbol{r}_{\mathrm{ref}}\right), \qquad \boldsymbol{R} = \begin{bmatrix} -\sin\lambda & \cos\lambda & 0 \\ -\sin\varphi\cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi \\ \cos\varphi\cos\lambda & \cos\varphi\sin\lambda & \sin\varphi \end{bmatrix}, \tag{278}$$

with $\varepsilon_k^{\mathrm{3D}} = \|(e_k, n_k, u_k)\|$ and $\varepsilon_k^{\mathrm{H}} = \|(e_k, n_k)\|$. The difference is taken in ECEF before the rotation, so the rotation origin cancels and (278) is exact rather than a local-tangent approximation.

Convergence must be suffix-stable. The first epoch whose error falls below a threshold measures the noise of a solution, not the solution. A float PPP-RTK trajectory dips under any loose bound long before it *holds* it, and a run that later diverges scores the same as one that does not. The convergence epoch for threshold θ is therefore defined as

$$k^\star(\theta) = \min\Big\{\, k : \max_{j \geq k} \ \varepsilon_j^{\mathrm{3D}} \leq \theta \,\Big\}, \tag{279}$$

the first epoch from which the error stays within θ for the *entire remainder* of the run ($k^\star = \varnothing$ when no such epoch exists, the correct verdict for a run that never settles). The suffix maximum is computed in one reverse cumulative pass. The suffix function counts a non-finite error as a violation, but the scorecard first drops every trace row whose position is non-finite, so an epoch without a position is skipped rather than breaking the hold. The metric is reported as the ladder $\theta \in \{1.0, 0.5, 0.2, 0.1\}$ m rather than a single number, which keeps it informative for runs of very different quality. The steady-state statistics are taken over $k \geq k^\star$ of the coarsest achieved rung, so the convergence transient is excluded from the accuracy claim by construction; when no rung is held, they fall back to the second half of the run and the summary flags the fallback. The window is stated alongside the number it produced.

Covariance credibility. Grading the solution against the surveyed point also tests whether the filter's own uncertainty is credible. With $\boldsymbol{P}_k^{xx}$ the position block of the posterior covariance, the rotation-invariant scalar

$$\sigma_k^{\mathrm{3D}} = \sqrt{\operatorname{tr} \boldsymbol{P}_k^{xx}} = \sqrt{\sigma_x^2 + \sigma_y^2 + \sigma_z^2} \tag{280}$$

is compared directly with $\varepsilon_k^{\mathrm{3D}}$. The trace is invariant under the rotation (278), so no per-axis horizon σ has to be constructed from a trace that carries only the diagonal. A float solution whose error persistently exceeds $3\sigma^{\mathrm{3D}}$ is *overconfident* as well as inaccurate. This is the usual signature of unmodelled correction error absorbed by ambiguity and bias states whose formal variances do not reflect it. The simulator dashboard applies the same covariance-optimism diagnostic against its answer key; here the surveyed mark takes the place of the truth trajectory.

What the reference does not certify. The surveyed point grades *position* only. A static run cannot separate a receiver-clock error from a common-mode range error, and under BSSD (Section 19.10) there is no receiver-clock state to grade. The per-satellite bias states are gauge-fixed, so their absolute values cannot be checked against an external reference either. A position that grades well therefore certifies the geometry and the correction stack *jointly*, not any single component. The real-time scorecard viewer (`pyviz/gnssviz/viewers/rt_scorecard.py`) of the report built by `pyviz/run_report.py --run-config` implements (278)–(280). An explicit `--ref-ecef` overrides the configured reference position.

Part IX
Network mode

20 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`), which *produces* the satellite-side corrections that the single-receiver user modes consume. The other modes in this reference either difference away the satellite clock and hardware biases (Section 5) or import them as known SSR quantities (Section 18). The network producer inverts that relationship. It estimates the satellite clock and the satellite *phase* and *code* biases as explicit states, jointly across a network of stations with known coordinates, and emits them in IGS-SSR convention for a downstream consumer. The loop is closed by design: the single-receiver PPP-RTK engine documented elsewhere is the product test, run on a station *held out* of the network solve.

Scope and status. The numerical core consists of the arrowhead normal-equation kernel (`filter/NetworkNormalEquations`) and the semantic estimator that drives it (`filter/NetworkEstimator`). It is validated on a synthetic noise-free network, where it recovers the truth up to the declared S-transform. The surrounding pipeline is complete: the `PPP_RTK_Network` enum, its forced-uncombined `DifferencingMode::None` (`measurement/MeasurementBuilder`), the explicit satellite phase-bias state in `filter/StateManager`, and the mode dispatch in `Application::run`. `Application::runNetwork` is a thin epoch-loop orchestrator that calls a fixed, ordered sequence of single-concern seams (Section 20.1). Every step derived in this chapter runs inside one of those seams: per-station design-row assembly from each station's `MeasurementModel` (`networkPassAHead_` / `networkPassB_`), the S-basis datum with its null-space rank gate (Section 20.4, in `networkSolveGlue`), the between-epoch time update, and the product emitter (`NetworkProductEmitter`, driven from `networkEmitStore`). The mathematics below describes that estimator core.

This chapter treats the single-epoch solve. The between-epoch carry is layered. The base layer is the *diagonal* D2 shared carry: each shared parameter's posterior value and variance are re-injected as a diagonal prior on the next epoch's shared border, inflated by $q\,\Delta t$ and dropped on a datum change (`networkInjectD2CarryPrior`, `app/NetworkRunContext`). This layer has no enable flag, so the producer is always a filtered solve, never a snapshot solve. The layers built on it are each a large subsystem, default off in the loader, and are referenced here without derivation: the C1 full-covariance shared carry (`network.shared_carry_full_cov`, folded through `addSharedPriorDense`), the Card-4 invariant DD carry (`network.carry_invariant`, `setDdPriors`), the station-local phase-leveling, ZWD and receiver-code-bias carries folded through `addStationPrior`, and the MW-WL, EWL and lane-coordinate widelane layers. The golden configurations (`config/golden/golden_field_refound.json`, `config/golden/golden_sim_refound.json`) arm the C1 carry and phase leveling (`network.phase_leveling`) with the float ambiguity vehicle (`network.ambiguity_vehicle` set to `float_kf`; the loader default is `level`). They leave the invariant DD carry and every widelane layer off.

20.1 Producer architecture: the decomposed epoch loop

The rest of this chapter is mathematics. This subsection maps each piece of it to the code that executes it, which is spread over several translation units.

Decomposition. The producer was a single driver function of roughly 14,700 lines. At that size it could not be reasoned about as a whole, exercised below the granularity of a full run, or reviewed: a search for the datum build could not bound where it was *not*. It was decomposed into single-concern *seams* under the strongest available safety property. Relocation is *verbatim*: bodies move without rewriting, and each increment was gated on a matrix of golden runs required to be *byte-identical* to the pre-cut binary's output. That gate supports the claim that the equations below still describe the code. The driver shrank by roughly a factor of eight, to under 1,800 lines of epoch-loop orchestration, with no change in the numbers. Later cards have added seams to the call sequence: the file (`src/core/app/ApplicationNetwork.cpp`) exceeded 2,250 lines by 2026-08-15 and measures 3,071 lines on 2026-09-28. The placement rules of this section exist to bound that growth. The gate's coverage is incomplete. `docs/network_decomposition_plan.md` records per-increment byte gates through increment 8 (with increment 7b) only. Increment 8b has its own specification, and the later cuts (solve glue, carry install, DD-prior build, producer AR, MW/WL glue, re-level, teardown) have no gate record in that document.

Four stages. A network run has four stages: three translation units and the seam sequence between them.

- **Setup** (`app/ApplicationNetworkSetup`). `networkValidateInputs_` rejects an unrunnable configuration before anything is allocated. `networkSetup_` then builds the station roster and its synchroniser, the product sink and its declared S-basis contract, and, when the closed loop is armed, the consumer engine (Section 20.7). Its output is one `NetworkSetupResult`, filled in place and neither copyable nor movable: three run-long references pin its member addresses, so the member declaration order in that header is normative.
- **Assembly** (`app/ApplicationNetworkAssembly`), two passes per epoch. PASS A is *layout*. It runs each present station's geometry models and pre-allocates the shared satellite states, taking the datum (pivot) station *first*, so that the phase-bias columns admitted to the shared block are those backed by a pivot-receiver ambiguity pin. The local half of the datum (Section 20.4) therefore constrains the column layout as well as the solve. PASS B is *assembly*. Each station's undifferenced design block is computed with no receiver-side differencing and then *split*: the satellite clock and phase-bias columns scatter into the shared border $\boldsymbol{A}_{s,r}$, and the remainder becomes the private block $\boldsymbol{A}_{r,r}$ of Equation (281). The three receiver-position columns are dropped, because network station coordinates are held fixed. Equation (284) is formed from these two pieces.
- **The epoch seams**: the ordered sequence tabulated below, which is the body of the loop.
- **Teardown** (`filter/NetworkTeardown`). The run-end readout, the service counters and the summary writers live in `networkTeardown`, which also returns the process exit code. The driver ends with a `return` of that call.

The seam contract. A seam is a free function over the run context, the configuration, the logger and the trace bundle. It opens with an alias preamble that reconstructs the driver's former local names, so the relocated body reads as it did before the cut. Each seam header states that the seam *owns no state*: every carried object is a reference into the context. Pure relocation required two structural adaptations. A relocated block that used to `continue` the epoch loop returns an `EpochAction` of `Proceed`, `SkipEpoch` or `StopRun` (`filter/NetworkRunTypes`) instead. The per-skip bookkeeping stays verbatim inside the seam, and the caller's `continue` preserves the

original bypass of the timing bracket. A block whose locals are read further down the loop returns a result struct (`NetworkSolveGlueResult` and its siblings) that the call site re-aliases.

The run context. Carried state, configuration snapshots and counters live in one object, `NetworkRunContext` (`app/NetworkRunContext`). It holds a `const NetworkKnobs` snapshot of the configuration, taken in the original getter read order, the cross-epoch `NetworkCarryState`, and `NetworkServiceCounters`. A seam introduces no function-local carry: a relocated block that leaves a local behind *forks* the carry with no diagnostic, and increment 12 deleted 191 dead aliases that this pattern had left. Persistence follows the *three-place rule*: a carry field that is a snapshot member appears in the warm-resume fingerprint, in the restore and in the snapshot write, all three in `NetworkRunContext`. A field present in only two of the three yields a warm resume that disagrees with the run that wrote it, with no error raised.

The ordered epoch seams. The table below lists the per-epoch call sequence in driver order. The driver's section timers bracket the calls. Rows marked *default off* are inert unless armed; most of the newer machinery in this producer is in that state. The table omits smaller calls in the same loop, among them the signal roster (`networkBuildSignalRoster`), the float-vehicle drop step (`networkAmbFloatApplyDrops`), the gross line-of-sight screen (`networkGrossLosScreen`, loader default off, armed by the goldens), and the code-only clock-level re-base of card CLV (`networkClockLevelRebase`, after `networkStashStatus`; inert unless `network.clock_level_carry`, default off). The gross screen runs its response half after post-fit RAIM; under `network.gross_los_emit_exclude` its decision half runs earlier, before `networkSolPrep`, so that a declaration reaches the epoch it declares. The prefit census row runs only when one of its instruments is armed: `network.trace_prefit_census` (armed by the goldens), `network.trace_topology`, or a positive `network.seed_birth_check_sigma`.

seam call (driver order)	concern (translation unit)
`networkFastClockTick`	sub-epoch fast-clock tick between solution epochs; *default off* (`filter/NetworkFastClock`)
`networkPassAHead_`	PASS A head: per-station geometry and shared-state pre-allocation, datum station first (`app/ApplicationNetworkAssembly`)
`networkPassASlipScan_`	PASS A tail and the slip scans that fill the slip funnel (`app/ApplicationNetworkAssembly`, `slip/NetworkSlipQc`)
`networkAssemblyPrefilter_`	raw slip observables; cross-station observability prune (`app/ApplicationNetworkAssembly`)
`networkSignalIntegrity`	per-LOS band-state classifier: degraded/unvetted phase demotion and regain refloat; *default off* (`filter/NetworkSignalIntegrity`)
`networkMeasurementAdmission`	product-referenced corrupt-channel screen, the one admission test a differential screen cannot see; *default off* (`filter/NetworkMeasurementAdmission`)
`networkBuildPhaseDatumCandidates`, then `networkAnchorHygiene` and `networkElectPhaseAnchors` / `networkElectEnsembleDatum` / `networkElectChainDatum`	election of the *local* half of the datum: the candidate census and coverage gate, the eq34 reattach hygiene that governs anchor-host eligibility and the fresh-pin deweight ramp, then whichever realisation is armed (`filter/NetworkPhaseDatum`, `filter/NetworkAnchorHygiene`, `filter/NetworkChainDatum`)
`networkAssemblyRegisterColumns_`	clear the layout, register this epoch's shared columns, open the arrowhead system (`app/ApplicationNetworkAssembly`)
`networkSdEwlAr`	single-difference extra-widelane integer decision; *default off* (`filter/NetworkSdEwlAr`)
`networkRxCbDemeanPrepass`	receiver code-bias carry de-mean before assembly reads the map; *default off* (`app/NetworkRunContext`)
`networkPassB_`	PASS B produce/consume: the split design blocks handed to `addStation` (`app/ApplicationNetworkAssembly`, `filter/NetworkCoupling`)
`networkBuildEnsembleGauges`, `networkBuildCarryDdPriors`	sum-to-zero phase gauges; Card-4 DD carry priors (`filter/NetworkDdPriorBuild`)
`networkSolveGlue`	shared datum build, rank gate and drop-and-heal, D2 carry injection, the constrained solve, within-epoch RAIM re-solve, withhold gates (`filter/NetworkSolveGlue`, calling `NetworkSbasis`, `NetworkPhaseDatum`, `NetworkRelevel`)
`networkInstallCarry`	DD posterior read-back into the invariant carry; *default off* (`filter/NetworkInstallCarry`)
`networkProducerAr`	producer-side AR diagnostic and normal-equation export; read-only and log-only, *default off* (`filter/NetworkProducerAr`)
`networkSolPrep`	map to the solution epoch; per-satellite discontinuity signalling (`filter/NetworkSolPrep`)
`networkEmitStore`	emit re-base, phase-bias continuity filter, store push, product write, closed-loop consume (`filter/NetworkEmitStore`)
`networkFastClockHarvest`	fast-clock harvest off the accepted solution; *default off* (`filter/NetworkFastClock`)
`networkIonoDiag`, `networkDdFrac`, `networkShDevVtec`, `networkShHealth`, `networkIonoDwRelease`, `networkLeveling`	diagnostic sweeps, the iono-deweight release hygiene that reseeds slant states a zero-weight era left unvalidated, and the SH-deviation and phase-leveling carry harvests (`filter/NetworkIonoDiag`, `NetworkDdFrac`, `NetworkShDevVtec`, `NetworkShHealth`, `NetworkIonoDwRelease`, `NetworkLeveling`)
`networkPostFitRaim`	post-fit network RAIM and its attribution pass (`filter/NetworkRaim`)
`networkClockQc`	two-state clock update and clock fault QC (`filter/NetworkClockQc`)
`networkPostfitSigma0`, `networkShSpectrum`	post-fit variance factor and reduced-normal spectrum; *default off* (`filter/NetworkFrameDiag`)
`networkPrefitCensus`	the only always-on *pre*-solve screen: prefit innovation census, structural fuse, seed-birth tripwire and estimation-topology ledger (`filter/NetworkPrefitCensus`)
`networkRedundancyCensus`	row redundancy and testability, the statistical parked-garbage monitor; measure-only, *default off* (`filter/NetworkRedundancyCensus`)
`networkBlockBaardaEmit`	(station, satellite) LOS block outlier census; measure-only, *default off* (`filter/NetworkBlockBaarda`)
`networkCongruenceEmit`	station-congruence screen: coordinate-fault detection in position space; measure-only, *default off* (`filter/NetworkCongruence`)
`networkMdbFlush`	Baarda reliability numbers and minimal detectable bias on the

Reading key. The three derivations that follow map to three places in that sequence. The rank deficiency and its minimum-constraint datum (Section 20.4) are built and gated inside `networkSolveGlue`. The arrowhead Schur reduction (Section 20.5) runs in the kernel that the same seam invokes, on normals accumulated station by station during PASS B. The closed loop (Section 20.7), meaning emission, product write and, when armed, the held-out consumer step, runs in `networkEmitStore`. The rest of the table is quality control, diagnostics and carry bookkeeping around those three.

20.2 Joint multi-station observation model

The producer uses the same *undifferenced, uncombined* observations as single-receiver PPP (Section 4), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The unknowns partition into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias $\bar{b}^i_s$. These columns are identical for every station, since a satellite's clock and hardware delays do not depend on the observer. Orbits are held fixed to a precise product; orbit determination is outside the producer's scope. When `network.estimate_iono_sh` is set, the shared block carries a fourth family, network-wide rather than per-satellite: the coefficients a_{nm}, b_{nm} of a global single-layer spherical-harmonic VTEC expansion (`SharedParamKind::VtecSh`, registered through `NetworkEstimator::registerVtecCoeff`). An optional second, plasmasphere-side shell is set by `estimation.global_vtec_sh.layer2_max_degree` (default 0, i.e. single-layer); the coefficients of both shells coexist on the shared border, and the pin row sums them. This co-estimated ionosphere model replaces the external GIM. It also forces satellite code-bias estimation on, since the two are jointly required for the per-satellite slant-iono columns below, unless `network.external_sat_code_bias` takes the satellite code biases from the external `.BIA`. Its columns Schur-reduce onto the shared border in the same way as the satellite-indexed families. The goldens leave `network.estimate_iono_sh` off and estimate a free per-station, per-satellite slant ionosphere instead.
- **Private (station-side).** For each station r: its receiver clock $c\,\mathrm{d}t_r$, the inter-system time offsets $c\,\delta t^{(g)}_r$, the zenith wet troposphere ZWD_r (and gradients), the per-satellite slant ionosphere $I^i_{r,1}$, the receiver biases, and one float ambiguity $N^i_{r,s}$ per tracked carrier. Under the chain datum only (`network.phase_datum_eq34`, default off), the block also holds a per-(receiver, system, signal) receiver *phase*-bias split $\delta_{r,s}$, re-derived each epoch (`StateKeys::makeReceiverPhaseBiasKey`). Under that realisation it keeps the receiver's fractional phase bias out of the *emitted* satellite phase bias. Station coordinates are known and held fixed, so position is not a free parameter.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise (Section 4) to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{281}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances. There is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. The reduction of Section 20.5 rests on that sparsity.

No receiver-side differencing. Unlike the PPP-RTK user mode, the producer applies *no* between-satellite or between-receiver differencing. The differencing strategy is forced to `DifferencingMode::`

`None` (`MeasurementBuilder.cpp`, `differencingModeFor`), overriding any `between_satellite_sd` toggle in a network config. Differencing would cancel the satellite-side terms the producer *estimates*. The network carries them as explicit columns instead and resolves the resulting rank deficiency with a minimum-constraint datum (Section 20.4).

20.3 The explicit satellite phase-bias state

The main contrast with the user side is the treatment of the satellite phase bias b_s^i (written $\bar{b}_s^i$ once promoted to a state). On the user side this term is never an explicit unknown: between-satellite single differencing (Section 5) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The producer *generates* that known product, so it carries the satellite phase bias as an estimated state. The state vector gains a per-(satellite, signal) phase bias column,

$$\bar{b}_s^i \in \boldsymbol{x}_s, \qquad \text{keyed by } \texttt{makeSatPhaseBiasKey}(i, s) = \texttt{"}i\texttt{|}s\texttt{|PB"}, \tag{282}$$

held in `StateManager::sat_phase_bias_indices_`, a map kept disjoint from the satellite *code* bias map (`sat_bias_indices_`); the trailing `|PB` discriminator keeps the two key namespaces apart (`include/filter/StateKeySchema.hpp`). Its process noise is a small random walk (`network.process_noise.sat_phase_bias`, legacy `filter.process_noise.sat_phase_bias_q`; default $1 \times 10^{-12}\,\mathrm{m^2\,s^{-1}}$, two orders of magnitude below the `sat_code_bias_q` default of $1 \times 10^{-10}\,\mathrm{m^2\,s^{-1}}$). Satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column insensitive to real slow drift (`src/core/filter/StateManager.cpp`).

Remark 4. The estimated $\bar{b}_s^i$ is not the raw satellite phase hardware delay, which is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [17]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. Because of this lumping the phase-bias product must be *consistent* (Section 18): the producer's datum and the consumer's integer recovery must use the same S-system.

20.4 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (281) is rank deficient. The linear dependencies that the user side removes by differencing appear here as exact *gauge freedoms* of the joint normal matrix. No observation distinguishes a common shift of all satellite clocks from an opposite shift of all receiver clocks; none fixes the split of an absolute phase bias between the satellite column and the per-station ambiguity datum; none fixes an inter-system bias (ISB) common to a constellation. These are the S-system ambiguities of [25]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space.

Exact gauge, soft elimination. The datum is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a structural elimination of columns. Each gauge is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution moves to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c =$ value holds exactly (`NetworkEstimator::solve`, `src/core/filter/NetworkEstimator.cpp`). The pin is applied to a *copy* of the reduced

normals (the kernel's `solveSharedConstrained` seam). The estimator's own reduced system stays unmodified, so per-station back-substitution (Section 20.5) stays valid. The pinned diagonal is set to the reduced normals' own scale (the largest diagonal magnitude) rather than to unity. An *under*-specified datum then still appears as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude.

In S-transform language, let $\boldsymbol{x}$ collect the rank-deficient undifferenced parameters and $\boldsymbol{S}$ be the S-basis selection. The emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$. Choosing the datum is choosing $\boldsymbol{S}$, and the hard pins above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The two halves of the datum live in two places:

- **Shared-block gauges** touch the shared satellite columns: the overall clock datum (one per constellation), the per-constellation phase-bias↔ambiguity datum, and the ISB datum. They go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements** live on one station's private block: a network *pivot receiver*'s own ambiguity/clock pin, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9). The *caller* realises them as tight prior rows folded into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$, not through the shared `Datum` (`NetworkEstimator.hpp`). Station priors also make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 20.5).

Selectable datum realisations. The pins just described are the loader defaults. Each half of the datum admits alternatives, selected by config and recorded in the `SbasisDeclaration`. All produce the same estimable $\boldsymbol{S}\,\boldsymbol{x}$ and differ in how they distribute the gauge. The clock/ISB datum has three modes (`SbasisDeclaration::IsbDatum`): the hard `PerConstellationSatClockPin` above (default); a `SumToZeroPerConstellation` constraint (`network.clock_datum_sum_to_zero`) that spreads the pin across all satellites of a constellation; and a `BroadcastAnchoredPerSatellite` soft anchor (`network.clock_datum_broadcast_anchored`) that ties each satellite clock to its a-priori value (the broadcast clock under `network.apriori_source brdc_rts`, the precise clock under `sp3`) at a declared 1σ. The ambiguity↔phase-bias datum also has three modes (`AmbDatum`). The first is the local `PivotReceiverZeroAmbiguity` pin above (default). Its anchor may be elected *per* (satellite, signal) at the highest-coverage tracking receiver instead of at the single clock pivot (`network.phase_datum_prioritized`, default off; `networkElectPhaseAnchors`, `filter/NetworkPhaseDatum`). This is the same one-pin-per-freedom constraint realised at a different station, and it keeps a clean gauge for a satellite that is low or unseen at the clock pivot. The other two modes are a `SumToZeroPerReceiver` ensemble (`network.phase_datum_ensemble`) and a `SingleReceiverChain` realisation (`network.phase_datum_eq34`). When satellite code biases are estimated (the W6 slice, `network.estimate_sat_code_bias`), their gauge, a per-constellation sum-to-zero (`CodeBiasDatum::SumToZeroPerConstellation`), is folded as explicit constraint rows through `NetworkEstimator::addSharedConstraintRows` rather than as column pins. With no ionosphere constraint, `network.sat_gf_datum` (`none`, `zero` or `broadcast_tgd`; default `none`) adds per-satellite geometry-free code-bias datum rows. Whichever realisations are active, the rank gate below is declared against the matching S-basis dimension. The goldens keep the default clock pin (at `network.clock_datum_station`), arm the prioritized phase datum with `network.phase_datum_held_basis`, estimate the satellite code biases, and pin the geometry-free code bias at the broadcast T_{GD} (`broadcast_tgd`).

Remark 5. A null-space rank gate eigen-decomposes the pre-constraint reduced normals and asserts that the count of small eigenvalues equals the *declared* S-basis dimension. It is implemented as `NetworkSbasis::assertNullSpaceRank` and is invoked on `reducedSharedNormals()` *before* the

constrained solve, against the `SbasisDeclaration`'s `declared_dimension`. The call sits in the solve seam `networkSolveGlue` (`filter/NetworkSolveGlue`), which also builds the datum through `NetworkSbasis::buildSharedDatum` (Section 20.1). The gate complements the positive-definiteness guard. The guard catches the exact-deficiency (under-specified datum) case on the pinned pivot; the rank gate checks the reduced deficiency against the declared gauge dimension and withholds any epoch whose null-space count disagrees. Under the prioritized phase datum the gate first attempts a *drop-and-heal* pass that soft-anchors the near-null phase-bias columns (`healNearNullPhaseBias`, `filter/NetworkPhaseDatum`) and then re-gates once. On the legacy pivot datum there is nothing to heal: the heal returns zero and the epoch is withheld immediately.

20.5 Per-station Schur reduction (arrowhead normals)

Stacking Equation (281) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system. The station–station off-diagonal blocks are exactly zero, because stations couple only through $\boldsymbol{x}_s$:

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{283}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \tag{284}$$

and $\boldsymbol{u}_s^r = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$, $\boldsymbol{u}_r^r = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `NetworkNormalEquations.hpp`).

Lossless marginalisation. The arrowhead system is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^{r}$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added, and `reduce` and `reconstructStation` reuse the factor. The elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r\in\mathcal{R}} \left(\boldsymbol{N}_{ss}^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{N}_{sr}^{r\mathrm{T}} \right) \;+\; \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{285}$$

$$\tilde{\boldsymbol{u}}_s = \sum_{r\in\mathcal{R}} \left(\boldsymbol{u}_s^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{u}_r^{r} \right) \;+\; \boldsymbol{u}_s^{\mathrm{prior}}, \tag{286}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\,\boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation. Solving Equations (285) and (286) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = (\boldsymbol{N}_{rr}^{r})^{-1}\big(\boldsymbol{u}_r^{r} - \boldsymbol{N}_{sr}^{r\mathrm{T}}\,\boldsymbol{x}_s\big), \tag{287}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost is linear in the station count and the per-station eliminations run in parallel (`reconstructStation`, `NetworkNormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (285) (soft physical priors and any additive exact-gauge contributions) are added before the reduction through `addSharedInfo` and pass through the Schur complement unchanged.

Remark 6. This arrowhead topology with exact reduction mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver. In the specialisation, the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. A two-stage covariance-form alternative was rejected because the normal-equation Schur form is exact and cheaper here.

20.6 Constellation scope

The producer supports GPS, Galileo and BeiDou on the same footing, selected by `processing.active_systems`. The goldens are GPS-only (L1/L2). The Galileo extension of the golden posture (card GAL, `docs/refound/GAL_gps_galileo_dual_2026-09-22.md`) is built, default off and not armed. GLONASS phase biases are excluded. The reason is structural: GLONASS is FDMA, so each satellite transmits on a slightly different frequency, and the receiver inter-channel (hardware) bias does *not* cancel across satellites as it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system (Sections 20.3 and 20.4) assumes a per-signal bias common to a constellation's satellites. The per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the phase datum. GLONASS is therefore excluded from the network *phase*-bias product by design (`docs/attic/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. A *code-only* GLONASS mode is available behind `network.estimate_glonass` (`enableGlonassNetwork`, default off). It admits GLONASS to the solve with a satellite clock and a per-slot receiver inter-frequency bias but *no* FDMA phase bias, so the phase-datum bookkeeping above is unaffected. The mode is mutually exclusive with the W6 satellite code-bias slice: GLONASS admission skips the GPS/Galileo receiver DCB that code-bias separation needs, so `network.estimate_sat_code_bias` together with `network.estimate_glonass` is a hard load error. The same exclusion applies to the co-estimated SH-VTEC ionosphere, which forces the code-bias slice on (unless `network.external_sat_code_bias` is set), and to the goldens, which estimate satellite code biases. This mode is distinct from the float-only GLONASS support that the user engine carries for positioning. With the code-only mode off, the network *producer* generates no GLONASS bias products. Broadcast satellite health enters the network path through `network.sat_health` (card HLT; `off`, `pivot` or `exclude`, default `off`; built, not armed). Under `exclude`, setup removes a satellite's observations from the synchronised epochs at which the broadcast flags it; under `pivot`, only the clock-pivot election reads the health series.

20.7 The closed loop: producer → held-out user

The emitted product is one `NetworkSolutionEpoch` per content epoch: per-satellite clock C_0 plus per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). That product is the SSR stream the single-receiver engine already consumes: the satellite clock refines $-c\,\mathrm{d}t^i$, and the observable-specific biases refine $-B_s^i$ and $-b_s^i$ as *known* corrections, restoring the integer ambiguity property as described in Section 18.

The validation strategy follows from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but consumes the network's own bias/atmosphere product. A station held *out* of the network solve and positioned as a user against the produced corrections is a closed-loop product test: its recovered position and fixed ambiguities show whether the producer's S-system datum and the consumer's integer recovery are mutually consistent. The `PPP_RTK_User` consumer runs through the standard PPP path and is not a separate estimator; only the producer takes the joint multi-station pipeline of this section.

The test has two mechanised forms. The *in-process* harness (`network.closed_loop.enable`) builds a consumer engine from a derived configuration during setup and gives it an *owned* SSR correction store instead of the process-wide one, so the producer's products cannot leak into a store that another component reads. The emit seam `networkEmitStore` steps the consumer after that epoch's products have been pushed, which closes the loop *within* the epoch. The consumer runs a between-satellite single-difference user model by default, or an undifferenced one under `closed_loop.consumer_undifferenced`. The goldens arm the harness with the undifferenced consumer on a held-out station (WTZZ00DEU on the field bed, SZZ000SIM on the sim bed). The *offline replay* driver (`network.closed_loop.replay_products_path` → `Application::runConsumerReplay`, `app/ApplicationConsumerReplay`) short-circuits `runNetwork` before input validation and producer setup. It fills the same owned store from an already-written product file through the same reader path, so the recorded stream's discontinuity bookkeeping is reconstructed exactly, and a position-domain arm costs minutes instead of a producer run. One limit applies to what a replay arm proves: the in-process SH-VTEC slant-ionosphere supply has no product-wire record, so `closed_loop.consume_network_iono` is inert on the replay path (the driver logs a warning), and the consumer floats its own ionosphere there.

21 Worked example: estimability in a two-receiver network

This section works the rank deficiency and minimum-constraint datum of Section 20.4 through numerically on the smallest network that shows the structure: two receivers $\mathcal{R} = \{A, B\}$ and four satellites $i \in \{1, 2, 3, 4\}$. Everything not needed for the estimability argument is removed. Positions are known, so the geometric range drops out; a single frequency is used; the troposphere and ionosphere are absent; the data are noise-free. Every number below is therefore exact, and the gauge freedoms are visible by inspection. The subject of the example is the null space of the system, not its size.

21.1 Code only: clocks and one gauge freedom

With geometry removed, the single-frequency pseudorange of Section 4 collapses to a difference of two clock lumps,

$$P_r^i \;=\; a_r \;-\; g^i, \qquad a_r := c\,\mathrm{d}t_r + B_r, \quad g^i := c\,\mathrm{d}t^i + B^i, \tag{288}$$

where a_r is the receiver clock and g^i the satellite clock. On a single frequency the receiver and satellite *code biases* B_r, B^i are inseparable from the respective clocks, so each "clock" here is a clock + code-bias lump. With one band, the IF-vs-DCB split of Section 20.3 does not arise. The system has $|\mathcal{R}| + |\{i\}| = 2 + 4 = 6$ unknowns and $2 \times 4 = 8$ observations.

The ground truth (metres) is

$$a_A = 5, \quad a_B = 3, \qquad g^1 = 2, \quad g^2 = -1, \quad g^3 = 4, \quad g^4 = 0.5,$$

and Equation (288) maps it to the eight pseudoranges

P_r^i	$i=1$	$i=2$	$i=3$	$i=4$
$r=A$	3	6	1	4.5
$r=B$	1	4	−1	2.5

The deficiency from two directions. *(i) Invariance.* Adding a constant κ to all six unknowns, $a_r \mapsto a_r + \kappa$ and $g^i \mapsto g^i + \kappa$, leaves every difference $a_r - g^i$, and therefore every observation, unchanged. The truth above and the shifted set $\{15, 13, 12, 9, 14, 10.5\}$ fit the data identically. The data determine clock differences only, never absolute clocks.

(ii) Singular normals. With the unknowns ordered as $\boldsymbol{x} = [\,a_A, a_B, g^1, g^2, g^3, g^4\,]^{\mathrm{T}}$ and unit weights, the normal matrix $\boldsymbol{N} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$ is

$$\boldsymbol{N} = \begin{bmatrix} 4 & 0 & -1 & -1 & -1 & -1 \\ 0 & 4 & -1 & -1 & -1 & -1 \\ -1 & -1 & 2 & 0 & 0 & 0 \\ -1 & -1 & 0 & 2 & 0 & 0 \\ -1 & -1 & 0 & 0 & 2 & 0 \\ -1 & -1 & 0 & 0 & 0 & 2 \end{bmatrix}, \qquad \boldsymbol{N}\,\mathbf{1} = \mathbf{0}, \tag{289}$$

a 6×6 matrix of rank 5. The all-ones vector $\mathbf{1} = [1, 1, 1, 1, 1, 1]^{\mathrm{T}}$ is an exact null vector (every row sums to zero), and it is the common-shift direction of view (i). The system $\boldsymbol{N}\boldsymbol{x} = \boldsymbol{u}$ has no unique solution without one added constraint, a *datum*.

Two datum choices, same fit. Pinning one clock removes the gauge. Two natural choices give different parameter values, and both fit the eight observations exactly:

datum (S-basis)	a_A	a_B	g^1	g^2	g^3	g^4
$a_A \equiv 0$ (reference receiver)	0	-2	-3	-6	-1	-4.5
$g^1 \equiv 0$ (reference satellite)	3	1	0	-3	2	-1.5

Under $a_A \equiv 0$, every satellite-clock estimate comes out as $\hat{g}^i = g^i - a_A^{\mathrm{true}}$: the reference receiver's clock is absorbed into all of them. The estimated "satellite clock" is the physical clock shifted by the chosen datum. This offset is the *lumping*, and the user must account for it. For this reason a producer declares its datum (Section 20.7).

What is invariant. The estimable quantities (combinations annihilated by the null vector $\mathbf{1}$, i.e. differences) are identical in both bases:

estimable quantity	$a_A \equiv 0$	$g^1 \equiv 0$	(truth)
$a_A - a_B$	2	2	2
$g^2 - g^1$	-3	-3	-3
$g^3 - g^1$	2	2	2
$a_B - g^4$	2.5	2.5	$(= P_B^4)$

The absolute values are a convention; the differences are physical. This is the S-system of [25] in six parameters.

Remark 7 (Why the producer cannot difference). The double difference of this code-only model is $\nabla\Delta P_{AB}^{i,1} = (P_A^1 - P_A^i) - (P_B^1 - P_B^i)$. The between-satellite step gives $g^i - g^1$ for both receivers, so the between-receiver step yields exactly $\mathbf{0}$. Double differencing eliminates the entire model: DD removes clocks, and here the model contains nothing else (Section 5). A DD user estimates nothing. The network producer instead keeps the clocks and pins one datum. This degeneracy is, in miniature, the reason a network producer exists.

21.2 Adding carrier phase: the satellite phase bias needs the network

The single-frequency carrier phase, with the same clock lumps, is

$$\Phi_r^i \;=\; a_r - g^i \;+\; b^i \;+\; N_r^i, \tag{290}$$

where b^i is the satellite phase bias, the producer's target, and N_r^i is the float ambiguity (in metres). The receiver phase bias and the receiver code-minus-phase offset are constant per receiver and are absorbed into N_r^i. The satellite bias b^i stays explicit because the producer emits it. The truth is extended with

$$b^1 = 0.10, \qquad b^2 = -0.20, \qquad b^3 = 0.05, \qquad b^4 = 0.15,$$
$$N_A^i : 10,\ -5,\ 20,\ 8, \qquad N_B^i : 12,\ -3,\ 18,\ 9,$$

giving the eight carrier-phase observations

Φ_r^i	$i=1$	$i=2$	$i=3$	$i=4$
$r=A$	13.10	0.80	21.05	12.65
$r=B$	13.10	0.80	17.05	11.65

Counting: one receiver vs. two. The code fixes the clocks, so per satellite the phase contributes $b^i + N_r^i$. With one receiver this is a single equation $b^i + N_A^i$ in two unknowns. The satellite phase bias is then inseparable from that receiver's ambiguity, and the equation carries no redundancy. With two receivers there are two equations,

$$b^i + N_A^i, \qquad b^i + N_B^i, \tag{291}$$

in three unknowns (b^i, N_A^i, N_B^i): rank 2, one deficiency per satellite. The estimable two-dimensional space contains the *between-receiver single-difference ambiguity* $\nabla N_{AB}^i = N_A^i - N_B^i$. This quantity does not exist with one receiver, and it is the integer a user resolves (Section 5). The second receiver gives b^i an identity shared across stations.

Datum and solution. The joint code + phase system has a 1-dimensional clock gauge (as before) plus one phase-bias↔ambiguity gauge per satellite: $b^i \mapsto b^i + c_i$, $N_r^i \mapsto N_r^i - c_i$ for all r. The minimum-constraint datum used here has two parts. The first is the clock pin $a_A \equiv 0$. It is S-transform-equivalent to the reference-*satellite* clock pin $g^1 \equiv 0$ that the estimator's S-basis declares (Section 20.4): in its default pin mode the shared-block gauge pins one satellite clock per constellation, not a receiver clock (the optional `network.clock_datum_sum_to_zero` and `network.clock_datum_broadcast_anchored` modes replace the pin). The second part is the set of reference-receiver ambiguity pins $N_A^i \equiv 0$ for every i: the local datum on the pivot receiver, folded into station A's rows. The solution is

	$i=1$	$i=2$	$i=3$	$i=4$
emitted phase bias $\bar{b}^i = b^i + N_A^i$	10.10	−5.20	20.05	8.15
$\hat{N}_A^i$ (datum)	0	0	0	0
$\hat{N}_B^i = N_B^i - N_A^i$	2	2	−2	1

with the clocks unchanged from Section 21.1. The fit checks directly, e.g. $\hat{\Phi}_B^3 = a_B - g^3 + \bar{b}^3 + \hat{N}_B^3 = -2 - (-1) + 20.05 + (-2) = 17.05$, matching the observation.

Reading the result. The solution shows two properties.

- *The lump dominates the emitted bias.* The physical satellite phase bias of satellite 1 is $b^1 = 0.10$ m, while the emitted estimable bias is $\bar{b}^1 = 10.10$ m: it has absorbed receiver A's ambiguity (10 m) for that satellite. This is Odijk's $\bar{b}^i = b^i +$ (pivot-receiver ambiguity) (Section 20.3, Eq. 4/9) in numbers. A consumer recovers integers only by removing the same lumped quantity, so the producer datum and the consumer integer recovery must belong to one S-system.
- *The network makes the integer estimable.* The estimable $\hat{N}^i_B = N^i_B - N^i_A = \nabla N^i_{AB}$ is the between-receiver single-difference ambiguity, the quantity an RTK user fixes to an integer. It is absent from a single-receiver problem; the second station introduces it. The satellite phase bias becomes a definite, transferable number at the cost of pinning one reference ambiguity, and it carries that pin.

Up to the geometry and the second frequency it omits, this four-satellite, two-receiver instance has the structure of the synthetic network on which the estimator core is tested. The `RecoversSyntheticTruthUpToSTransform` test (`tests/test_network_estimator.cpp`) builds rows of this shape (four receivers and five satellites, from a seeded random truth), pins the reference-satellite clock datum $g^1 \equiv 0$ (S-transform-equivalent to the $a_A \equiv 0$ used above) through `NetworkEstimator::solve`, and pins the pivot receiver's ambiguities with tight-prior rows in that station's block. It asserts recovery of the satellite-side parameters up to the declared S-transform: the lumped $\bar{b}^i$ and the single-difference ambiguities above, not the unobservable physical b^i.

22 Between-epoch carry: arrowhead admissibility and the DD cycle basis

The Network-mode document establishes the arrowhead normal-equation topology and its exact Schur reduction (Section 20.5), and the worked estimability example establishes the minimum-constraint datum (Section 21). This document covers the step between them: *how information from outside the current epoch enters an arrowhead system.*

The structure of the problem sets the requirements. A network producer that solves each epoch independently discards what it has learned about the carrier-phase ambiguities, the only constant quantities in the problem. A producer that carries them without care destroys either the sparsity that keeps the solve affordable or the datum invariance that gives the product its meaning. The design has three parts, developed in order:

1. An arrowhead system admits exactly *three* kinds of information fold. The columns the new information touches determine which one applies; its physical meaning does not (Section 22.1).
2. The carried quantity must therefore be gauge-free. The natural gauge-free basis of the ambiguity block is the *cycle space* of the receiver–satellite observation graph: the double differences indexed by the spanning tree's *chords* (Section 22.2).
3. Carrying that basis costs one bounded Woodbury correction per connected component, with closed forms for the posterior and for an exact innovation test (Section 22.3).

Section 22.4 then carries a two-receiver, two-satellite network, the smallest network whose observation graph has a cycle, through every step with exact arithmetic.

Remark 8 (Theoretical footing). Time-linking a *selected* set of linear functions $\boldsymbol{z} = \boldsymbol{F}^{\mathrm{T}}\boldsymbol{x}$ while re-estimating the remainder fresh each epoch is the generalised Kalman filter of Teunissen, Khodabandeh

& Psychas (*J. Geod.* 95:108, 2021) and the generalised least-squares filter of Hou & Zhang (*J. Geod.* 99:3, 2025). Both leave open the construction of the transition $\boldsymbol{\Phi}^z$ of the linked functions across a *changing* linked set. Choosing $\boldsymbol{z}$ as the cycle basis closes that gap: every basis change the graph can force is then a unimodular integer matrix (Section 22.2). The spanning-tree S-basis itself is due to Jonge [10], who selects it once and never re-selects it. The construction here extends it to a basis that changes from epoch to epoch.

22.1 Three doors into an arrowhead system

The epoch's system has the structure of Section 20.5: a shared satellite border $\boldsymbol{x}_s$, one private block $\boldsymbol{x}_r$ per station, and zero station–station coupling. Consider a Gaussian prior $\boldsymbol{z} \sim N(\hat{\boldsymbol{z}}, \boldsymbol{P}^z)$ on a linear function $\boldsymbol{z} = \boldsymbol{C}\boldsymbol{x}$ of the epoch's parameters. In information form it is the increment $\boldsymbol{C}^{\mathrm{T}}(\boldsymbol{P}^z)^{-1}\boldsymbol{C}$ to the normal matrix and $\boldsymbol{C}^{\mathrm{T}}(\boldsymbol{P}^z)^{-1}\hat{\boldsymbol{z}}$ to the right-hand side. The block in which that increment lands determines how it can be folded.

Door 1: the shared border. If $\boldsymbol{C}$ touches only shared columns, the increment lands entirely in $\boldsymbol{N}_{ss}$. Such a fold is *reduction-preserving*:

> **Lemma.** For any symmetric $\Delta\boldsymbol{N}$ and any $\Delta\boldsymbol{u}$ on the shared border,
>
> $$\widetilde{(\boldsymbol{N}_{ss} + \Delta\boldsymbol{N})} = \tilde{\boldsymbol{N}}_{ss} + \Delta\boldsymbol{N}, \qquad \widetilde{(\boldsymbol{u}_s + \Delta\boldsymbol{u})} = \tilde{\boldsymbol{u}}_s + \Delta\boldsymbol{u}.$$
>
> *Proof.* The Schur subtrahend $\sum_r \boldsymbol{N}_{sr}^r (\boldsymbol{N}_{rr}^r)^{-1} \boldsymbol{N}_{sr}^{r\mathrm{T}}$ is a function of $\boldsymbol{N}_{sr}$ and $\boldsymbol{N}_{rr}$ only; $\boldsymbol{N}_{ss}$ enters Equation (285) additively. □

A border prior may therefore be folded *before or after* the reduction with identical results. The between-epoch carry of the satellite clock and biases costs no second Schur pass even when it is applied late, for example between the null-space rank gate and the solve. `addSharedInfo` (`filter/NetworkNormalEquations`) uses this: when a reduction is already cached, it folds the delta in place and leaves the cache current. The dense form of the same carry (`addSharedPriorDense`, `NetworkEstimator`) takes a *covariance* rather than an information matrix. A carried key absent from the current epoch's layout must be *marginalised*, a row and column deletion on $\boldsymbol{P}^z$; deleting the same rows of $(\boldsymbol{P}^z)^{-1}$ would instead *condition* on the departed key and bias every surviving parameter.

Door 2: one station's own columns, before the reduction. If $\boldsymbol{C}$ touches only station r's local columns, the increment lands in $\boldsymbol{N}_{rr}^r$. For a diagonal prior on columns $c \in \mathcal{C}$ with information weights $\Lambda_c \geq 0$ and information right-hand sides η_c,

$$\boldsymbol{N}_{rr}^r \mathrel{+}= \sum_{c\in\mathcal{C}} \Lambda_c\, \boldsymbol{e}_c \boldsymbol{e}_c^{\mathrm{T}}, \qquad \boldsymbol{u}_r^r \mathrel{+}= \sum_{c\in\mathcal{C}} \eta_c\, \boldsymbol{e}_c, \tag{292}$$

which the implementation applies as a sequence of rank-one updates of the *cached* $\mathrm{LDL}^{\mathrm{T}}$ factor (`addStationPrior`). The raw $\boldsymbol{N}_{rr}$ is never stored or re-factorised, and a positive Λ_c keeps the factor positive-definite by construction (a zero weight skips the factor update and still adds its right-hand-side increment). Unlike door 1, this fold is *not* reduction-preserving: it changes $\boldsymbol{N}_{rr}$, hence the Schur subtrahend, hence $\tilde{\boldsymbol{N}}_{ss}$. It must be applied *before* the reduction; there is no post-reduce fast path. It is cheap for two reasons: it is orthogonal to every other station (the arrowhead has no station–station block to disturb), and it conditions only $\boldsymbol{x}_r$.

The two doors have opposite ordering constraints: door 1 may be late, door 2 must be early. The asymmetry follows directly from Equation (285).

Door 3: local columns across several stations. If $\boldsymbol{C}$ touches local columns of two or more stations, the information increment $\boldsymbol{C}^{\mathrm{T}}(\boldsymbol{P}^{z})^{-1}\boldsymbol{C}$ has a nonzero (r, r') block. It is representable neither as a border increment nor as a per-station one. Folding it into the unreduced system *destroys the arrowhead*: it fills in the station–station zeros that make the marginalisation exact and cheap.

This is the case that matters in practice. The only ambiguity combinations worth carrying are between-station ones, because a single station's ambiguity is not estimable on its own (Section 21.2). Such a prior is folded *after* the reduction, by Woodbury through the block-diagonal local normals, using the per-station factors the reduction has already computed and left intact. Section 22.3 gives the algebra; the rest of this subsection records the two hazards it creates.

Remark 9 (Post-reduce folds do not compose). Two door-3 folds applied to the same reduced system are each derived against the *pristine* $\boldsymbol{N}_{rr}$. Composing them over coupled local columns is therefore inexact: neither fold sees the other's augmentation of $\boldsymbol{N}_{rr}$, so the composition drops the cross term $\boldsymbol{C}_L\boldsymbol{N}_{rr}^{-1}\boldsymbol{C}_{L,\mathrm{other}}^{\mathrm{T}}$ of the joint middle inverse without any signal. The implementation enforces the exclusion: the sum-to-zero ensemble phase gauge (`foldReducedSumZeroGauges`) and the carried DD priors (`foldReducedDdPriors`) are mutually exclusive, and `NetworkEstimator::solve` throws if both are set. Under the invariant carry the phase datum is realised instead through door 2, by the pre-reduce tree pins. These compose *exactly* with a post-reduce door 3 fold, because the fold then reads an already-pinned factor.

The same truncation applies between DD blocks. `foldReducedDdPriors` folds one block per connected component of one (constellation, signal) group, each against the unmodified per-station factors, so the result is exact only when no station is touched by two blocks. A station tracking two signals sits in two blocks, and the fold then makes the block-Jacobi approximation of dropping the cross terms between them.

Remark 10 (A door-3 fold cannot repair a rank deficiency). Let $\delta\boldsymbol{x}_s$ span a null direction of $\tilde{\boldsymbol{N}}_{ss}$, with the matching local response $\delta\boldsymbol{x}_r = -(\boldsymbol{N}_{rr}^{r})^{-1}\boldsymbol{N}_{sr}^{r\mathrm{T}}\delta\boldsymbol{x}_s$. If the carried function is estimable, $\boldsymbol{z} = \boldsymbol{C}_L\boldsymbol{x}_L$ is invariant along that direction, so $\boldsymbol{C}_L\,\delta\boldsymbol{x}_L = -\boldsymbol{G}^{\mathrm{T}}\delta\boldsymbol{x}_s = 0$ with $\boldsymbol{G}$ as defined in Equation (295). Every column of $\boldsymbol{G}$ is therefore orthogonal to the null space, the rank-K update $\boldsymbol{G}\boldsymbol{M}\boldsymbol{G}^{\mathrm{T}}$ adds nothing along it, and the declared S-basis dimension the rank gate tests is unchanged by the carry. A carry that changed it would be carrying a gauge rather than information.

22.2 The observation graph and its cycle space

Door 3 admits only gauge-free functions, and the carried basis is chosen accordingly. The graph structure determines that choice; no numerical criterion enters.

The graph. For a constellation g and a signal s, let $G_{g,s} = (V, \mathcal{E})$ be the graph whose vertices $V = \mathcal{R} \cup \mathcal{S}_g$ are the receivers and the satellites, and whose *arcs* are the receiver–satellite pairs carrying an ambiguity this epoch: $(r, i) \in \mathcal{E}$ iff station r tracks satellite i on signal s. There is at most one arc per pair, so an unordered vertex pair is a unique arc identity (`ArcKey`, `filter/NetworkZStore`). The graph is *bipartite* by construction: no arc joins two receivers or two satellites.

Trees and chords. A connected component with n_c receivers, m_c satellites and $|\mathcal{E}_c|$ arcs has a spanning tree of $n_c + m_c - 1$ edges. The remaining

$$K_c \;=\; |\mathcal{E}_c| \;-\; (n_c + m_c - 1) \tag{293}$$

arcs are the *chords*. Adding a chord to the tree closes exactly one cycle, its *fundamental cycle*: the chord plus the unique tree path joining its endpoints. The K_c fundamental cycles are a basis of

the component's cycle space. Equation (293) is the first Betti number of the component. It is the exact number of independent double differences the component supports, and it is a property of the *graph*, independent of the estimator.

The carried functions. The graph is bipartite, so every cycle alternates receiver–satellite–receiver–satellite, has even length, and admits an alternating signing of its arcs. For chord c with fundamental cycle $\mathcal{C}_c$, define

$$z_c \;=\; \sum_{(r,i)\,\in\,\mathcal{C}_c} \sigma_{(r,i)}\, N^i_{r,s}, \qquad \sigma_{(r,i)} = \pm 1 \text{ alternating}, \tag{294}$$

which for the shortest cycle (length four) is a plain double difference $N^{i_1}_{r_1} - N^{i_1}_{r_2} + N^{i_2}_{r_2} - N^{i_2}_{r_1}$, and for a longer fundamental cycle is a higher-order alternating difference on six or more columns.

> **Proposition (gauge-freeness).** z_c is invariant under both ambiguity gauges of the network model: the receiver shift $N^i_r \mapsto N^i_r - d_r$ for all i, and the satellite shift $N^i_r \mapsto N^i_r - c^i$ for all r.
>
> *Proof.* A cycle visits each of its vertices exactly once, and each visited vertex is incident to exactly *two* arcs of the cycle, which by the alternating signing carry opposite signs. A shift attached to that vertex (d_r at a receiver, c^i at a satellite) therefore contributes $(+1)\delta + (-1)\delta = 0$ to Equation (294). Summing over vertices gives invariance. □

Hence z_c touches no shared column: the receiver and satellite phase biases cancel analytically rather than numerically. Three consequences follow, and together they make the cycle basis the carried object:

- **Estimable in every S-system.** z_c is invariant under the datum, so it may be compared across epochs whose datum differs and graded without an S-transform (Section 20.4).
- **Integer, and constant between slips.** z_c is an integer combination of integer ambiguities, and its process noise is $q_z = 0$ in the absence of a cycle slip. It is the *only* part of the ambiguity block for which a zero-q time link is physically correct.
- **Basis changes are unimodular.** Two spanning trees of the same arc set give cycle bases related by an integer matrix $\boldsymbol{T}$ with $\det \boldsymbol{T} = \pm 1$, so $\boldsymbol{z}' = \boldsymbol{T}\boldsymbol{z}$, $\boldsymbol{P}^{z\prime} = \boldsymbol{T}\boldsymbol{P}^z\boldsymbol{T}^{\mathrm{T}}$ is an *exact* change of basis with no information loss. Basis churn defeats any carry of the raw ambiguities, whose individual values depend on the datum; in the cycle basis it has no effect by construction.

Graph events are exact operations. Every way the graph can change between epochs maps to an exact operation on the carried $(\hat{\boldsymbol{z}}, \boldsymbol{P}^z)$ (`ZComponent`, `filter/NetworkZStore`):

Graph event	Operation on the store
Chord dies (arc lost)	Drop that row and column of $\boldsymbol{P}^z$, pure marginalisation
Chord born (arc closes a new cycle)	Append an entry initialised by tap-off $\hat{z}_{\mathrm{new}} = \boldsymbol{f}^{\mathrm{T}}\hat{\boldsymbol{x}}$ with its conditional variance and cross-covariances
Leaf growth (arc brings in a new vertex)	Extends the tree, closes no cycle, touches no z
Tree edge dies	Pivot a covering chord into the tree ($\boldsymbol{z}' = \boldsymbol{T}\boldsymbol{z}$, $\boldsymbol{T}$ unimodular), then kill the arc as a chord. An uncovered tree edge is a cut edge: its death splits the component, which merely re-partitions existing blocks
Components merge	Union the two blocks block-diagonally (they were independent), promote one bridge arc to the tree, tap off any surplus bridges

The producer does not apply these as a sequence of events. The per-epoch driver (`filter/NetworkInvariantCarryDriver`) receives the current epoch's tree/chord partition from the election and computes the *composed* transform directly. A new chord's fundamental cycle lies in the previous epoch's cycle space if and only if every arc of that cycle was present last epoch; its carried mean is then $\hat{z}_c^{\text{new}} = \sum_j w_j \, \hat{z}_j^{\text{old}}$, with w_j the incidence of old chord j's arc in the new cycle. Assembling $\boldsymbol{W}$ over the contributing components gives $\hat{\boldsymbol{z}}^{\text{prior}} = \boldsymbol{W}\hat{\boldsymbol{z}}^{\text{old}}$ and $\boldsymbol{P}^{z,\text{prior}} = \boldsymbol{W}\boldsymbol{P}^{z}\boldsymbol{W}^{\mathrm{T}}$. A dead chord's column of $\boldsymbol{W}$ is zero (marginalisation). Merges and splits follow from the contributor gather with no special case.

A chord whose cycle uses a newborn arc is *fresh*. The driver gives it a finite diffuse prior (variance `fresh_prior_var`, default 1.0e6 in state units squared) rather than the exact zero-information tap-off of the table. Its posterior is therefore the data-only tap-off up to a term of the order of the ratio of the data variance to `fresh_prior_var`. An arc flagged as slipped this epoch counts as absent from the previous arc set, so every chord whose cycle uses it is also born fresh (for a two-band pair group under the lane shape, the driver takes the union of the band-1 and band-2 slip flags). The production caller passes only arcs that resolve to an estimator column; a carried chord whose cycle has an unresolvable arc this epoch is dropped from the carry and re-tapped when the arc returns.

Remark 11 (Why the tree must avoid slipped arcs). A slip on a *chord* costs exactly one carried DD. A slip on a *tree edge* corrupts every fundamental cycle routed through it. With `network.phase_datum_eq34_slip_aware_election` armed (default off), the election therefore keeps slipped arcs out of the tree wherever the graph allows it. The argument is structural and independent of any detector threshold.

22.3 Folding the carried double differences

Let one connected component carry K double differences with prior mean $\hat{\boldsymbol{z}}$ and covariance $\boldsymbol{P}^{z}$, expressed as $\boldsymbol{z} = \boldsymbol{C}_L \boldsymbol{x}_L$ where $\boldsymbol{C}_L$ is the $K \times n_L$ signed incidence matrix of Equation (294) over the epoch's local ambiguity columns, and $\boldsymbol{C}_{L,r}$ is its restriction to station r. This is a door-3 fold. Woodbury through the block-diagonal local normals gives, per component,

$$\boldsymbol{B} = \sum_r \boldsymbol{C}_{L,r}\boldsymbol{N}_{rr}^{-1}\boldsymbol{C}_{L,r}^{\mathrm{T}}, \quad \boldsymbol{G} = \sum_r \boldsymbol{N}_{sr}\boldsymbol{N}_{rr}^{-1}\boldsymbol{C}_{L,r}^{\mathrm{T}}, \quad \boldsymbol{z}_{\text{free}} = \sum_r \boldsymbol{C}_{L,r}\boldsymbol{N}_{rr}^{-1}\boldsymbol{u}_r, \quad \boldsymbol{M} = (\boldsymbol{P}^{z} + \boldsymbol{B})^{-1}, \tag{295}$$

and the reduced system takes the rank-K correction

$$\boxed{\begin{aligned} \tilde{\boldsymbol{N}}_{ss} &\mathrel{+}= \boldsymbol{G}\,\boldsymbol{M}\,\boldsymbol{G}^{\mathrm{T}}, && (296)\\ \tilde{\boldsymbol{u}}_s &\mathrel{+}= \boldsymbol{G}\,\boldsymbol{M}\,(\boldsymbol{z}_{\text{free}} - \hat{\boldsymbol{z}}). && (297) \end{aligned}}$$

Every ingredient of Equation (295) comes from the *cached* per-station $\mathrm{LDL}^{\mathrm{T}}$ factors. No re-reduction is needed and the factors are left unmodified. The only new inverse is the $K \times K$ middle matrix $\boldsymbol{M}$, which is per component and bounded in size. Back-substitution then returns the *constrained* locals,

$$\boldsymbol{x}_r \mathrel{-}= \boldsymbol{N}_{rr}^{-1}\boldsymbol{C}_{L,r}^{\mathrm{T}}\,\boldsymbol{M}\left(\boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_s - \hat{\boldsymbol{z}}\right), \tag{298}$$

so the ambiguities read by the quality-control layer and by the next epoch's tap-off are in the same frame as the fold.

Posterior. The value the store carries forward is the prior shrunk towards the data-implied DD,

$$\hat{\boldsymbol{z}}^{+} = \hat{\boldsymbol{z}} + \boldsymbol{P}^{z}\,\boldsymbol{M}\,\boldsymbol{v}, \qquad \boldsymbol{v} = \boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_s - \hat{\boldsymbol{z}}, \tag{299}$$

with covariance, in the datum-as-known shared frame $\boldsymbol{\Sigma}_{ss}$ (the solve's shared covariance with every datum-pinned and gauge column treated as exact),

$$\boldsymbol{P}^{z+} = \underbrace{(\boldsymbol{P}^{z} - \boldsymbol{P}^{z}\boldsymbol{M}\boldsymbol{P}^{z})}_{\text{conditional on } \boldsymbol{x}_s} + \left(\boldsymbol{P}^{z}\boldsymbol{M}\boldsymbol{G}^{\mathrm{T}}\right)\boldsymbol{\Sigma}_{ss}\left(\boldsymbol{G}\boldsymbol{M}\boldsymbol{P}^{z}\right). \tag{300}$$

The first term is the posterior given the border; the second restores the coupling through it. $\boldsymbol{\Sigma}_{ss}$ is datum-constrained, but $\boldsymbol{P}^{z+}$ is datum-invariant because $\boldsymbol{z}$ is estimable. `ddPriorPosteriorMean` and `ddPriorPosteriorCovariance` evaluate the two expressions per block from the cached fold factors.

Innovation. The same factors give an exact test of the carry against the epoch's data (`ddPriorInnovation`). Under the null hypothesis that the carried DD is still valid,

$$\boldsymbol{v} = \boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_s - \hat{\boldsymbol{z}}, \qquad \mathrm{Cov}(\boldsymbol{v}) = \boldsymbol{M}^{-1} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{\Sigma}_{ss}\boldsymbol{G}, \tag{301}$$

and $|v_i|/\sqrt{\mathrm{Cov}(\boldsymbol{v})_{ii}}$ is the standardised statistic that flags a slipped carried DD for re-tapping. With `network.carry_invariant_innov_gate` armed (default off), the driver flags a chord when the statistic exceeds `network.carry_invariant_innov_k` (default 4.0) times a scale factor: 1 by default, or the LS-VCE estimate of the empirical innovation scale when `network.carry_invariant_innov_calibrate` is armed. The *minus* sign in Equation (301) is exact rather than a conservative approximation. $\boldsymbol{x}_s$ is itself influenced by the prior through Equation (297), and subtracting $\boldsymbol{G}^{\mathrm{T}}\boldsymbol{\Sigma}_{ss}\boldsymbol{G}$ corrects for testing the carry against a solution the carry helped produce. Consistency with Equations (299) and (300) is direct: $\boldsymbol{P}^{z}\boldsymbol{M}\,\mathrm{Cov}(\boldsymbol{v})\,\boldsymbol{M}\boldsymbol{P}^{z} = \boldsymbol{P}^{z} - \boldsymbol{P}^{z+}$.

The hard-constraint limit. Setting $K = 1$, $\hat{z} = 0$ and $\boldsymbol{P}^{z} \to \boldsymbol{0}$ turns Equations (296) and (297) into the rank-one fold of an exact linear constraint $\sum_r z_r = 0$ on local columns, with $\boldsymbol{M} \to \boldsymbol{B}^{-1}$. That is the anchor-free sum-to-zero phase-bias gauge (`foldReducedSumZeroGauges`). The DD prior is its soft generalisation: the two are the same object, which is why they cannot be composed.

Positive-definiteness, and why it is counted. $\boldsymbol{P}^{z} + \boldsymbol{B}$ is positive-definite in exact arithmetic, but it can fail to be numerically. A stiff carry ($\boldsymbol{P}^{z}$ near zero) added to an ill-conditioned $\boldsymbol{B}$ subtracts operands of similar magnitude to reach a much smaller result. When the screen fails, the fold is applied *inertly*, $\boldsymbol{M} := \boldsymbol{0}$: the reduced system is untouched and the posterior equals the prior. The block keeps its slot, so the per-block readouts stay index-aligned with the carry's block list.

The inert branch raises no error, and that makes it hazardous. The component's carried sub-state stops taking data, while every downstream reading of it (its formal σ, the innovation test, any ambiguity-resolution probe) continues to report on a quantity that is no longer updated. A carry can therefore diffuse for hundreds of epochs behind clean logs. The implementation treats the PD screen as *always-on telemetry*: per-fold counts of degenerate blocks and their dimension, the worst relative pivot, and the count of repairs are exposed unconditionally (`DdFoldStats`, `filter/NetworkNormalEquations`). An optional PD-safe path (`network.carry_invariant_pd_safe`, default off) replaces the absolute screen with an equilibrated factorisation and a scale-relative pivot test, evaluates the posterior in a product form that avoids the subtraction, and applies a counted eigenvalue-clamp repair to round-off-level failures. This is standing invariant 4 of the project (run-end counters are the telemetry of an ON path): an inert path needs a counter, because the absence of an error is no evidence that the mechanism ran.

22.4 Worked example: the smallest network with a cycle

This subsection carries a complete network through every step above with exact arithmetic. The network is the smallest one whose observation graph has a cycle: two receivers, two satellites, one signal. As in Section 21, everything inessential is removed: known coordinates (so the geometric range drops out), one frequency, no atmosphere, noise-free data and unit weights, so every number below is exact. Ambiguities are in cycles.

Model and truth. With geometry and the receiver/satellite clocks removed by the code observations, the carrier phase reduces to

$$\Phi_r^i \;=\; \beta_r \;+\; b^i \;+\; N_r^i, \qquad r \in \{A, B\}, \quad i \in \{1, 2\}, \tag{302}$$

where β_r is the receiver phase bias, b^i the satellite phase bias (the producer's target), and $N_r^i \in \mathbb{Z}$ the ambiguity. The truth is

$$\beta_A = 0.30, \quad \beta_B = -0.20, \qquad b^1 = 0.10, \quad b^2 = -0.15,$$

$$N_A^1 = 3, \quad N_A^2 = -1, \quad N_B^1 = 5, \quad N_B^2 = 2,$$

which through Equation (302) gives the four observations

Φ_r^i	$i=1$	$i=2$
$r=A$	3.40	−0.85
$r=B$	4.90	1.65

The graph, its tree, and its one chord. The observation graph has $n_c = 2$ receivers, $m_c = 2$ satellites and $|\mathcal{E}_c| = 4$ arcs, so by Equation (293) its spanning tree has $2 + 2 - 1 = 3$ edges and it has exactly $K_c = 1$ chord (Figure 4). With the tree $\{(A,1),(A,2),(B,2)\}$, the chord is $(B,1)$, and its fundamental cycle $B \to 1 \to A \to 2 \to B$ gives the single carried function

$$z \;=\; N_B^1 - N_A^1 + N_A^2 - N_B^2. \tag{303}$$

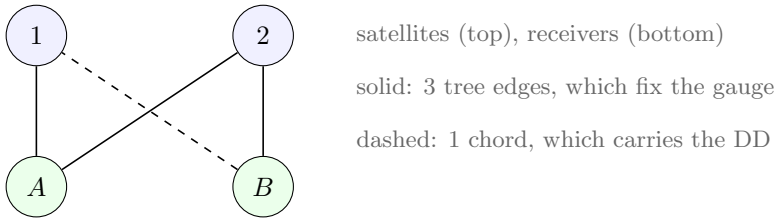


Figure 4: The smallest observation graph with a cycle: two receivers, two satellites, four arcs. Three arcs form a spanning tree; the fourth is a chord, and its fundamental cycle is the one estimable double difference Equation (303).

Two exact checks of the Proposition follow. From the truth, $z = 5 - 3 + (-1) - 2 = -1$, an integer as required. From the *observations* alone, with nothing estimated,

$$(\Phi_B^1 - \Phi_B^2) - (\Phi_A^1 - \Phi_A^2) = 3.25 - 4.25 = -1,$$

because β_A, β_B, b^1 and b^2 all cancel in the double difference. The carry transports this single number -1; every other parameter in this subsection depends on the datum.

The tree pins are the datum; the chord is left free. Model Equation (302) has $2+2+4=8$ unknowns and 4 observations, hence a four-dimensional null space (choose $\delta\beta_A, \delta\beta_B, \delta b^1, \delta b^2$ freely; the δN_r^i follow). Pinning the three *tree* ambiguities $N_A^1 = N_A^2 = N_B^2 = 0$ removes three of those four dimensions, leaving the single direction $(\beta_A, \beta_B, b^1, b^2) \mapsto (\beta_A + t,\ \beta_B + t,\ b^1 - t,\ b^2 - t)$, the one remaining receiver-versus-satellite gauge (the network's clock gauge in miniature). One *shared* pin, $b^1 \equiv 0$, removes it.

These four arcs contain the whole datum architecture. The tree edges are pinned through door 2 (locally, before the reduction), the last gauge is fixed by the shared minimum-constraint datum, and the chord is left free: it is the only ambiguity that carries information rather than convention. The pins are taken here as *exact* to keep the arithmetic simple. The implementation realises them as tight finite priors folded by `addStationPrior`, so the pinned columns remain present in $\boldsymbol{N}_{rr}$. The exact-pin idealisation has one visible consequence: with the tree ambiguities identically zero, $\boldsymbol{C}_L$ in Equation (303) collapses onto the chord's own column. With finite pins it retains all four entries, and the fold algebra below has the same form.

Arrowhead blocks. The shared block is ordered $\boldsymbol{x}_s = [\,b^1, b^2\,]^{\mathrm{T}}$ and the free locals $\boldsymbol{x}_A = [\,\beta_A\,]$, $\boldsymbol{x}_B = [\,\beta_B,\ N_B^1\,]^{\mathrm{T}}$. With unit weights the blocks of Equation (284) are

$$\boldsymbol{N}_{ss}^A = \boldsymbol{I}_2, \quad \boldsymbol{N}_{sr}^A = \begin{bmatrix}1\\1\end{bmatrix}, \quad \boldsymbol{N}_{rr}^A = \begin{bmatrix}2\end{bmatrix}, \quad \boldsymbol{u}_s^A = \begin{bmatrix}3.40\\-0.85\end{bmatrix}, \quad \boldsymbol{u}_r^A = \begin{bmatrix}2.55\end{bmatrix},$$

$$\boldsymbol{N}_{ss}^B = \boldsymbol{I}_2, \quad \boldsymbol{N}_{sr}^B = \begin{bmatrix}1 & 1\\1 & 0\end{bmatrix}, \quad \boldsymbol{N}_{rr}^B = \begin{bmatrix}2 & 1\\1 & 1\end{bmatrix}, \quad \boldsymbol{u}_s^B = \begin{bmatrix}4.90\\1.65\end{bmatrix}, \quad \boldsymbol{u}_r^B = \begin{bmatrix}6.55\\4.90\end{bmatrix},$$

with $(\boldsymbol{N}_{rr}^B)^{-1} = \left[\begin{smallmatrix}1 & -1\\-1 & 2\end{smallmatrix}\right]$. There is no A–B block: the stations share only b^1 and b^2.

Schur reduction. Eliminating each station's locals through Equation (285) gives

$$\boldsymbol{N}_{ss}^A - \boldsymbol{N}_{sr}^A(\boldsymbol{N}_{rr}^A)^{-1}\boldsymbol{N}_{sr}^{A\mathrm{T}} = \begin{bmatrix}0.5 & -0.5\\-0.5 & 0.5\end{bmatrix}, \qquad \boldsymbol{u}_s^A - \boldsymbol{N}_{sr}^A(\boldsymbol{N}_{rr}^A)^{-1}\boldsymbol{u}_r^A = \begin{bmatrix}2.125\\-2.125\end{bmatrix},$$

while station B contributes exactly zero to both: it has two observations and two free locals, so eliminating the locals absorbs its rows entirely. Summing,

$$\tilde{\boldsymbol{N}}_{ss} = \begin{bmatrix}0.5 & -0.5\\-0.5 & 0.5\end{bmatrix}, \qquad \tilde{\boldsymbol{u}}_s = \begin{bmatrix}2.125\\-2.125\end{bmatrix}, \qquad \mathrm{eig}(\tilde{\boldsymbol{N}}_{ss}) = \{0,\ 1\}. \tag{304}$$

The reduced system has rank 1 and one zero eigenvalue, with eigenvector $[1, 1]^{\mathrm{T}}$: the single gauge predicted above. A null-space rank gate that declares dimension 1 for this configuration passes; one that declares 0 or 2 throws (Section 20.4).

Datum, solve, back-substitute. Applying the shared pin $b^1 \equiv 0$ to Equation (304) and solving gives $\hat{b}^2 = -4.25$, and back-substituting through Equation (287),

$$\hat{\beta}_A = 3.40, \qquad \hat{\beta}_B = 5.90, \qquad \hat{N}_B^1 = -1.00.$$

The fit is exact (all four residuals vanish), and the result agrees to machine precision with a dense stacked solve of the same system: the lossless-marginalisation claim of Section 20.5 in numbers.

These estimates do not resemble the truth. The estimated satellite phase bias $\hat{b}^2 = -4.25$ differs from $b^2 = -0.15$, and $\hat{N}_B^1 = -1$ differs from $N_B^1 = 5$; each has absorbed the datum. The chord's double difference in this frame is

$$\hat{z} = \hat{N}_B^1 - \hat{N}_A^1 + \hat{N}_A^2 - \hat{N}_B^2 = -1 - 0 + 0 - 0 = -1,$$

identical to the truth and to the value read directly off the observations. Datum invariance here is an identity, not an asymptotic property.

Folding a carried DD. Suppose the previous epoch carried this chord. With $\boldsymbol{C}_L = [\,0 \;\; 1\,]$ on station B (one row, the exact-pin collapse noted above), Equation (295) gives

$$\boldsymbol{B} = 2, \qquad \boldsymbol{G} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \qquad \boldsymbol{z}_{\text{free}} = 3.25,$$

and the identity $z = \boldsymbol{z}_{\text{free}} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{x}_s = 3.25 - 4.25 = -1$ recovers the DD: $\boldsymbol{z}_{\text{free}}$ is a data-only quantity in the correction frame, not the DD itself. Also $\boldsymbol{G}^{\mathrm{T}}[1,1]^{\mathrm{T}} = 0$: the fold direction is orthogonal to the gauge, as the second Remark of Section 22.1 requires, so no choice of prior can make this carry fix the datum.

A consistent carry is a no-op. With $\hat{z} = -1$ and a rigid prior $\boldsymbol{P}^z = 0$, Equations (296) and (297) reproduce $\hat{b}^2 = -4.25$, $\hat{\beta}_A = 3.40$, $\hat{\beta}_B = 5.90$, $\hat{N}_B^1 = -1$ and zero residuals. The innovation is $v = 3.25 - 4.25 - (-1) = 0$. Carrying information the data already implies changes nothing.

A stale carry distorts the whole solution. Suppose instead that the chord slipped by one cycle undetected, so $\hat{z} = 0$ is folded rigidly. Re-solving gives

	$\hat{b}^2$	$\hat{\beta}_A$	$\hat{\beta}_B$
consistent carry ($\hat{z} = -1$)	-4.25	3.40	5.90
stale carry ($\hat{z} = 0$)	-3.75	3.15	5.15

and post-fit residuals of $+0.25$, -0.25, -0.25, $+0.25$ cycles on rows $(A,1)$, $(A,2)$, $(B,1)$, $(B,2)$ respectively. The one-cycle error in a stiff carried quantity does not stay in the ambiguity that slipped. It moves the *emitted satellite phase bias* by half a cycle and spreads the remainder evenly over every observation in the network. This is the general mechanism by which a stale zero-q carry damages a solution: the network cannot reject information asserted with infinite confidence, so it redistributes the contradiction into whatever states can absorb it. It is the reason for standing invariant 1 of the project: any zero-q time link requires a wired, tested slip response before it may be armed. Invariant 7 applies the same reasoning to the edges of the carry: the driver treats a slipped or unresolvable arc as a decoupling event (the chord is dropped or born fresh, and counted), and a returning arc re-couples only through a fresh tap-off.

The innovation test, and its limits. From Equation (301), with $\boldsymbol{\Sigma}_{ss} = \mathrm{diag}(0,\, 1)$ (the pinned column treated as exact),

$$v = 3.25 - 3.75 - 0 = -0.5, \qquad \mathrm{Cov}(v) = \boldsymbol{M}^{-1} - \boldsymbol{G}^{\mathrm{T}}\boldsymbol{\Sigma}_{ss}\boldsymbol{G} = 2 - 1 = 1, \qquad \frac{|v|}{\sqrt{\mathrm{Cov}(v)}} = 0.5.$$

The test detects the slip at only 0.5σ, far below any usable flag threshold (the default `network.carry_invariant_innov_k` is 4.0). The residuals show why: with four observations and almost no redundancy, the parameters absorbed half the error, so only half of it reached the innovation. The statistic is exact rather than conservative; its *power* is a property of the network's redundancy, and a two-by-two network has almost none. Any estimation-side slip test is diagnostic of the carry only in proportion to the part of the error the network cannot absorb.

A tree-edge death: the unimodular pivot. The two-by-two network is too small to show a pivot: with one chord, every tree edge whose death matters is a cut edge. A third satellite gives $n_c = 2$, $m_c = 3$, $|\mathcal{E}_c| = 6$, so the tree has four edges and there are $K_c = 2$ chords. The tree $\{(A,1),(A,2),(A,3),(B,1)\}$ with chords $(B,2)$ and $(B,3)$ gives

$$z_1 = N_B^2 - N_A^2 + N_A^1 - N_B^1, \qquad z_2 = N_B^3 - N_A^3 + N_A^1 - N_B^1.$$

Let the tree edge $(A,1)$ die. Both chords cover it, so one covering chord, $(B,2)$, is promoted into the tree and the dead arc is killed as a chord. Against the new tree $\{(A,2),(A,3),(B,1),(B,2)\}$ the cycle basis is

$$z_1' = N_A^1 - N_B^1 + N_B^2 - N_A^2 = z_1, \qquad z_2' = N_B^3 - N_A^3 + N_A^2 - N_B^2 = z_2 - z_1,$$

that is $\boldsymbol{z}' = \boldsymbol{T}\boldsymbol{z}$ with

$$\boldsymbol{T} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \qquad \det \boldsymbol{T} = 1.$$

$\boldsymbol{T}$ is integer with unit determinant, so $\boldsymbol{P}^{z\prime} = \boldsymbol{T}\boldsymbol{P}^{z}\boldsymbol{T}^{\mathrm{T}}$ is an exact change of basis: the carried covariance is transformed without approximation. Killing the dead arc's row of $\boldsymbol{z}'$ is then a marginalisation, and the surviving z_2' continues with no loss. A carry of the raw ambiguities would require an S-transformation of a datum-dependent state for the same graph event; the cycle basis removes that requirement.

22.5 Where each object lives

Object	Symbol	Home
Arrowhead accumulation, Schur reduction, back-substitution	`addStation`, `reduce`, `reconstructStation`	`filter/NetworkNormalEquations`
Door 1: shared border prior	`addSharedInfo`, `addSharedPriorDense`	`filter/NetworkNormalEquations`, `filter/NetworkEstimator`
Door 2: pre-reduce local prior	`addStationPrior`	`filter/NetworkNormalEquations`
Door 3: post-reduce Woodbury folds	`foldReducedSumZeroGauges`, `foldReducedDdPriors`	`filter/NetworkNormalEquations`
Cycle algebra, lifecycle operations	`ArcKey`, `ZComponent`	`filter/NetworkZStore`
Per-epoch reconcile $\boldsymbol{W}$, posterior install, innovation gate	`buildDdPriors`, `installPosterior`	`filter/NetworkInvariantCarryDriver`
Posterior and innovation readouts	`ddPriorPosteriorMean`, `ddPriorPosteriorCovariance`, `ddPriorInnovation`	`filter/NetworkNormalEquations`
PD screen telemetry	`DdFoldStats`, `ddPriorBlockDegenerate`	`filter/NetworkNormalEquations`
Datum and rank gate	`NetworkSbasis`, `assertNullSpaceRank`	`filter/NetworkSbasis`

Part X
Reference

23 Pipeline and architecture diagrams

This section shows the control and data flow behind the algebra of the other chapters: the per-epoch estimation loop (Figure 5), the shared geometry kernel behind the three differencing regimes (Figure 6), the real-time SSR correction pipeline (Figure 7), the real-time streaming front-end that feeds it (Figure 8), and the joint estimator of the PPP-RTK network producer (Figure 9). Each box names the function or class that implements the step, so the diagrams also serve as a map into `src/` . The diagrams are maintained by hand and simplify the code: they show the main sequence, not every gate or diagnostic.

A Generated reference tables

The tables in this appendix are generated from the source tree by `scripts/gen_report_tables.py`. They are not maintained by hand: a renamed flag, a new cycle-slip detector, or a changed state-key delimiter appears here on the next run of the generator. *Known gap:* the feature-flag table de-duplicates on the C++ member name rather than on the JSON path, so a section master switch named `enable` in more than one config sub-struct appears only once, so several `network.*` masters, including those governing the default-off subsystems this appendix exists to disclose, are consequently absent. Treat the flag table as indicative and `GnssConfigLoad.cpp` / `GnssConfigLoadNetwork.cpp` as authoritative until `gen_report_tables.py` keys on the path. The fragments under `generated/` are `.gitignore`d and rebuilt on demand (`build.ps1 -Regen` / `build.sh --regen`); to see whether anything moved, re-run the generator and check `git status` (`--check` is not read-only, since it rewrites the fragments before comparing them).

A.1 Processing modes

Table 3: `ProcessingMode` – the estimation algorithm selector.

Value	Meaning
`PPP`	Precise Point Positioning (single receiver, external products).
`PPP_RTK`	PPP with RTK enhancements (single receiver, uncombined).
`BaselineSingle`	Double-differenced single baseline: one rover + one reference station.
`BaselineMulti`	Double-differenced multi-receiver network (star topology: N rovers against one reference).
`PPP_RTK_Network`	Minimum-constraint S-system network PRODUCER: joint multi-station estimator emitting satellite clock + code/phase bias (+ atmosphere) under a declared S-basis.
`PPP_RTK_User`	PPP-RTK CONSUMER of network PPP_RTK_Network products (post-processing closed-loop counterpart): uncombined PPP-RTK + network SSR bias/atmosphere consume + S-basis cross-check.

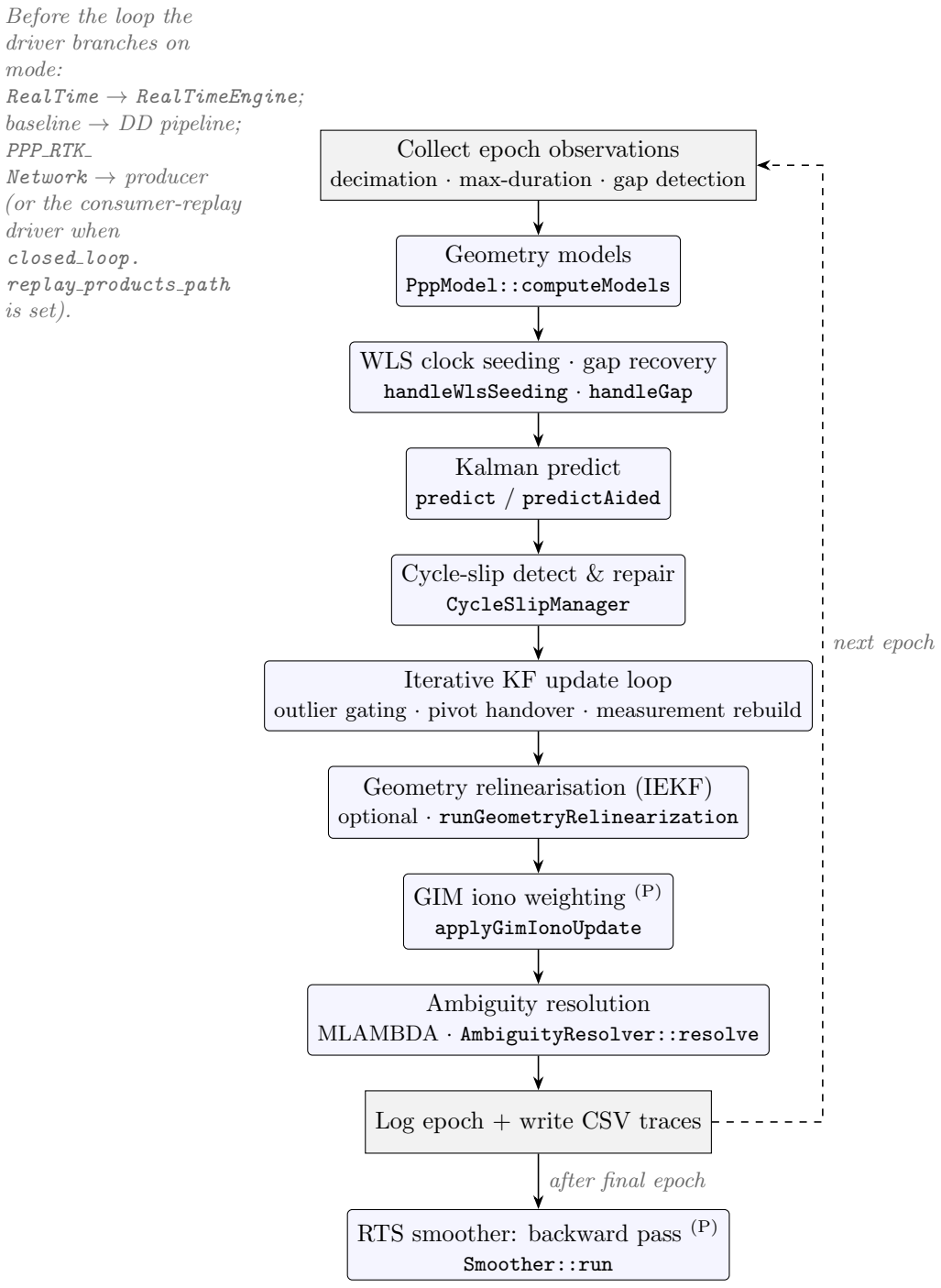


Figure 5: Per-epoch post-processing loop in `Application::run`. Solid arrows give the sequence within an epoch; the dashed arrow is the feedback from one epoch to the next. Steps tagged (P) run in post-processing only; the real-time path (`RealTimeEngine`) has no GIM source and no smoother. The smoother runs once, after the forward pass, and only when `run_smoother` is set.

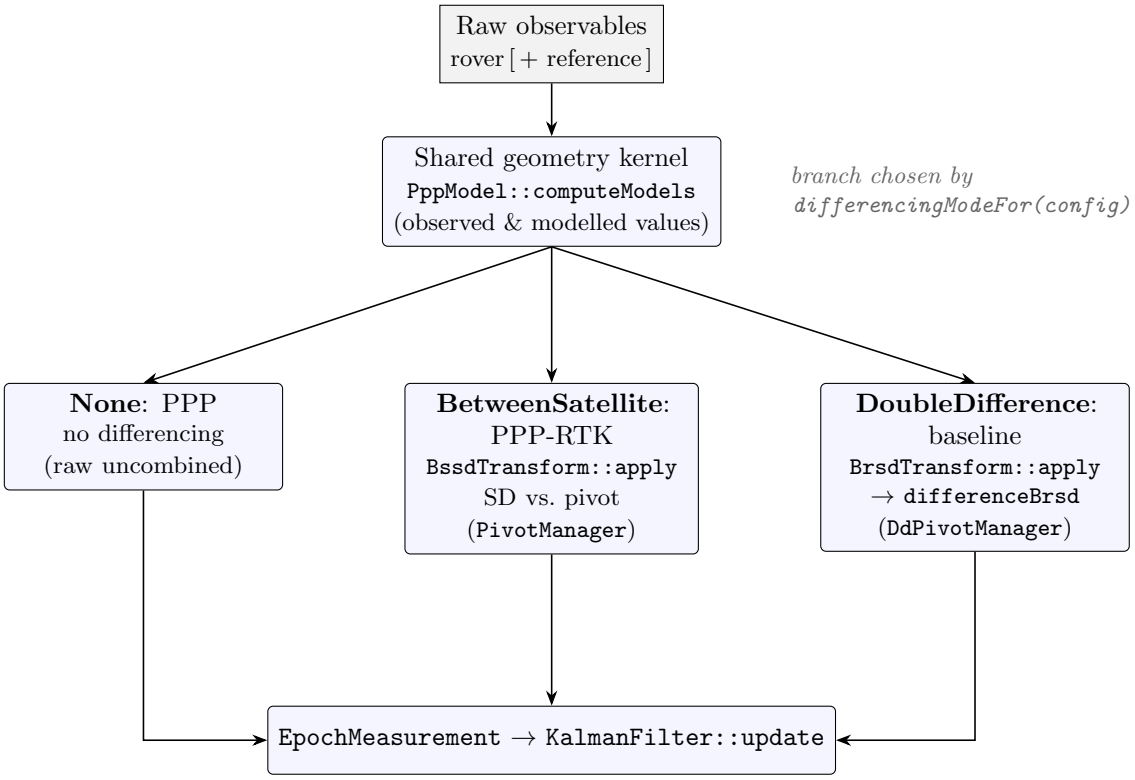


Figure 6: The three differencing regimes share one geometry kernel. The undifferenced observed and modelled values are computed once. The regime returned by `differencingModeFor` (selected once, when `MeasurementBuilderFactory` builds the measurement builder) then applies no transform (PPP), a between-satellite single difference against a pivot (PPP-RTK), or a between-receiver → between-satellite double difference (baseline). All three feed the same Kalman update.

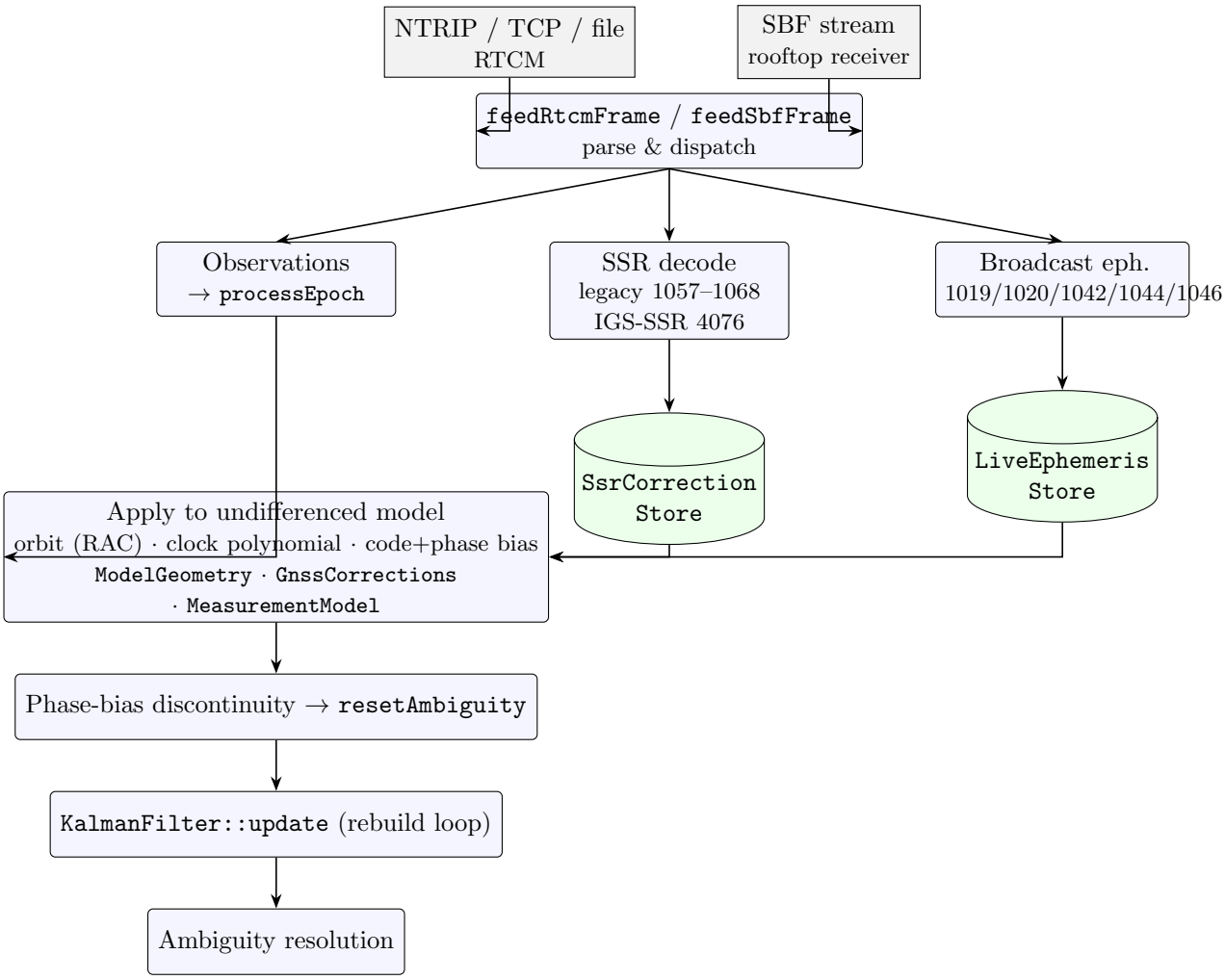


Figure 7: Real-time SSR correction pipeline. The decoder splits the byte stream three ways: observations go to `processEpoch`, SSR corrections to `SsrCorrectionStore`, and broadcast ephemerides to `LiveEphemerisStore`. The legacy SSR decoder accepts message types 1057–1068 and 1240–1270; the diagram lists the first range only. The undifferenced observation model then applies the precise orbit (RAC frame), the clock polynomial and the code and phase biases. Between the slip check and the Kalman update, `resetOnPhaseBiasDisc` reacts to a change of the phase-bias discontinuity counter, by default with an ambiguity reset; `reinflateOnCodeBiasDisc` and `reinflateOnClockDisc` follow for the code-bias and clock counters. See `RealTimeEngine::processEpoch`.

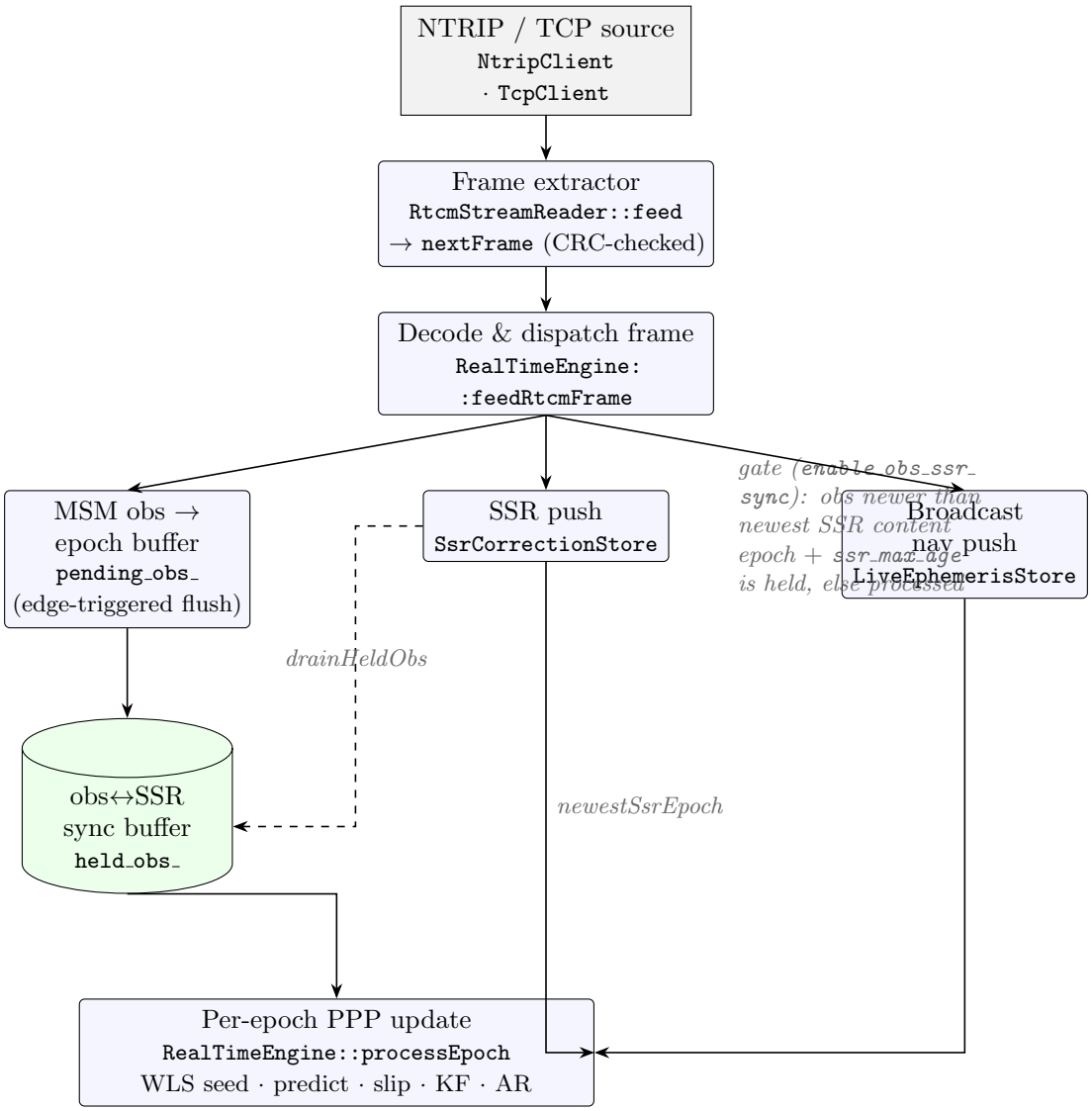


Figure 8: Real-time streaming front-end ahead of the SSR correction stage of Figure 7. `RtcmStreamReader` reassembles the raw bytes from `NtripClient` or `TcpClient` into CRC-validated RTCM3 frames, and `feedRtcmFrame` decodes and dispatches each frame. MSM observations accumulate in `pending_obs_`, which is submitted to `processEpoch` when a frame with a new epoch time arrives (the edge trigger). SSR corrections and broadcast ephemerides go to their stores. With `enable_obs_ssr_sync` set, the first admission gate of `processEpoch` (`admitBacklogAndSync`) holds an observation epoch that is newer than the SSR coverage horizon (newest SSR content epoch + `ssr_max_age`). The held epoch is kept in `held_obs_` (only the latest one) and resubmitted by `drainHeldObs` after each SSR push. An epoch inside the horizon is processed at once. A hold longer than `obs_ssr_sync_max_hold_s` (default 30 s) latches a degraded outage state in which epochs pass through without SSR until the stream covers them again. The gate applies only with a live SSR orbit and clock source and never before the first epoch. See Section 19.

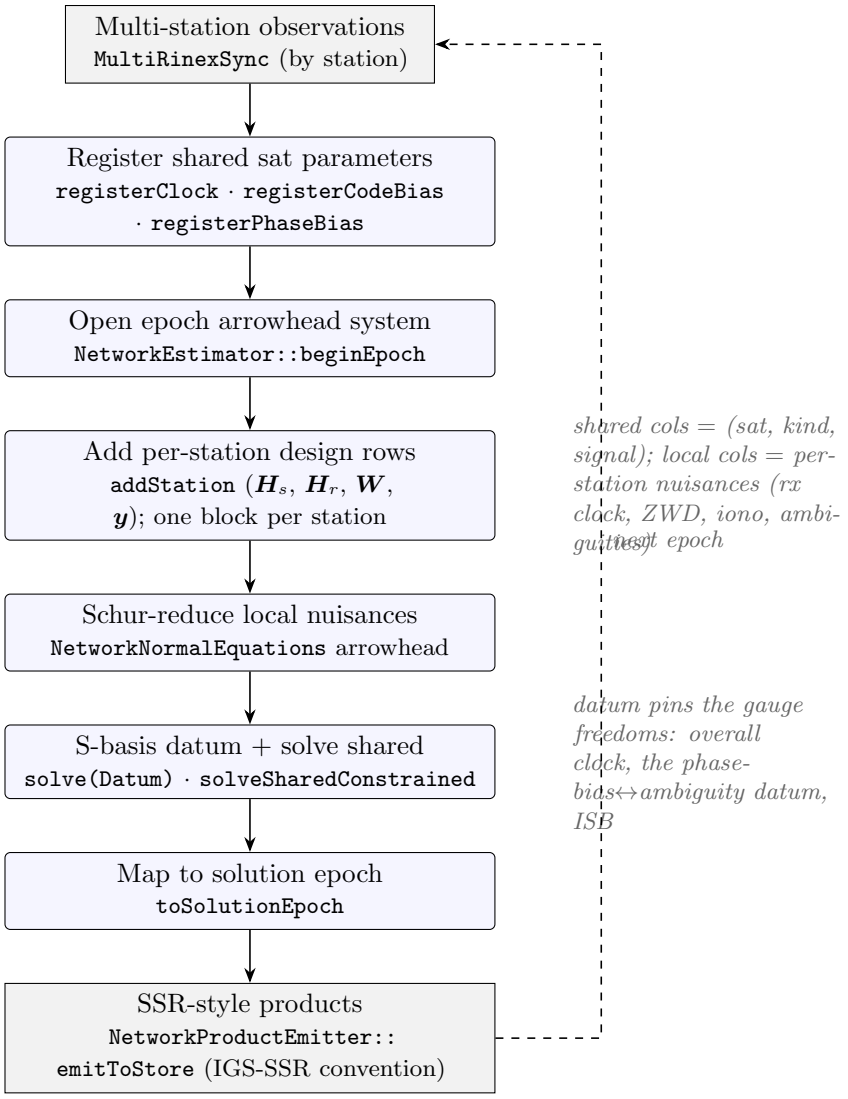


Figure 9: Joint estimator of the PPP-RTK network producer (`NetworkEstimator`). The shared satellite-side columns, one per (satellite, parameter kind, signal) for the estimable clock, code bias and phase bias, are registered afresh every epoch (`reset`, `registerClock`, `registerCodeBias`, `registerPhaseBias`, and `registerVtecCoeff` for the SH-VTEC coefficients when `network.estimate_iono_sh` is on); `beginEpoch` opens an arrowhead normal-equation system of that width. `addStation` adds one pre-formed block of local-versus-shared design rows per station, with no receiver-side differencing. Schur reduction removes the local nuisances; the reduced shared system is rank-deficient by the gauge freedoms. `solve` fixes them with an explicit minimum-constraint S-basis datum on a copy of the reduced system, and `toSolutionEpoch` maps the estimate to a record that `NetworkProductEmitter` writes in the IGS-SSR convention. The driver `Application::runNetwork` runs this core once per epoch over the `MultiRinexSync` stations and only orchestrates: `networkPassAHead_` models each station, `networkAssemblyRegisterColumns_` registers the columns and calls `beginEpoch`, `networkPassB_` calls `addStation` (all in `app/ApplicationNetworkAssembly`), `networkSolveGlue` applies the datum and solves, `networkSolPrep` maps, and `networkEmitStore` emits. With `network.closed_loop.enable` set, the emit step also drives a held-out PPP-RTK consumer on the new products: BSSD by default, undifferenced with `closed_loop.consumer_undifferenced`. See Section 20 and Section 20.1.

A.2 Execution mode and data source

Table 4: `AppMode` – top-level post-processing vs. real-time dispatch.

Value	Meaning
`PostProcessing`	Batch processing from RINEX / product files.
`RealTime`	Streaming / live processing from RTCM data.

Table 5: `DataSource` – bundled fixtures vs. auto-download.

Value	Meaning
`Internal`	Use files under internal_data_root only.
`Production`	Auto-download from CDDIS into data_root.

A.3 Differencing regime

Table 6: `DifferencingMode` – the measurement differencing regime.

Value	Meaning
`None`	Raw observables (PPP / PPP-RTK without SD).
`BetweenSatellite`	Per-constellation single difference vs. pivot (PPP-RTK SD).
`DoubleDifference`	Between-receiver + between-satellite (baseline processing).

A.4 Config-to-differencing mapping

The differencing regime is resolved once per run by the factory below (verbatim from `MeasurementBuilder.cpp`):

```
DifferencingMode differencingModeFor(const GnssConfig& config) {
    // The network producer runs on undifferenced + uncombined rows and estimates
    // the satellite-side bias lumps explicitly -- it must NEVER apply receiver-side
    // BSSD even if a between_satellite_sd toggle is present in its config.
    if (config.getProcessingMode() == ProcessingMode::PPP_RTK_Network)
                                        return DifferencingMode::None;
    if (config.isBaselineMode())        return DifferencingMode::DoubleDifference;
    if (config.enableBetweenSatelliteSd()) return DifferencingMode::BetweenSatellite;
    return DifferencingMode::None;
}
```

A.5 Feature toggles

These are the loop- and state-vector-level switches.

Table 7: Top-level boolean feature toggles, parsed from the jval<bool> calls in GnssConfigLoad.cpp and GnssConfigLoadNetwork.cpp. The legacy alias is the pre-refactor flat key still accepted as a fallback.

JSON path	Default	Legacy alias
estimation.accumulate_phase_windup	off	accumulate_phase_windup
estimation.adaptive_iono_process_noise.trace	off	adaptive_iono_q_trace
estimation.aiding.baro	off	aiding_baro
estimation.aiding.nhc	off	aiding_nhc
estimation.aiding.vision	off	aiding_vision
estimation.aiding.wheel	off	aiding_wheel
estimation.aiding.zupt	off	aiding_zupt
estimation.ambiguity_resolution.enable_partial_ar	off	ar_enable_partial_ar
estimation.ambiguity_resolution.nl_iono_free_row	off	ar_nl_iono_free_row
estimation.ambiguity_resolution.wl_only	off	ar_wl_only
estimation.between_satellite_sd.enable_glonass	off	enable_glonass_bssd
estimation.between_satellite_sd.enabled	off	enable_between_satellite_sd
estimation.cycle_slip.ewl	off	cycle_slip_ewl
estimation.cycle_slip.gf	off	cycle_slip_gf
estimation.cycle_slip.graphic	off	cycle_slip_graphic
estimation.cycle_slip.mw	on	cycle_slip_mw
estimation.cycle_slip.turbo_edit	off	cycle_slip_turbo_edit
estimation.cycle_slip.turbo_edit_bridge_lli	off	cycle_slip_turbo_edit_bridge_lli
estimation.cycle_slip.turbo_edit_mw_only_veto	on	cycle_slip_turbo_edit_mw_only_veto
estimation.defer_until_corrections_ready.drop_broadcast_grade	off	defer_until_corrections_ready_drop_broadcast_grade
estimation.defer_until_corrections_ready.require_all_above_mask	off	defer_until_corrections_ready_require_all_above_mask
estimation.enable_cio_frame	off	enable_cio_frame
estimation.enable_data_snooping	off	enable_data_snooping
estimation.enable_full_r_kernel	off	enable_full_r_kernel
estimation.enable_huber_innovation	off	enable_huber_innovation
estimation.enable_huber_r_inflation	on	enable_huber_r_inflation
estimation.enable_if_slip_detector	off	enable_if_slip_detector
estimation.enable_iono_rate	off	enable_iono_rate
estimation.enable_subcycle_slip_suppression	on	enable_subcycle_slip_suppression

JSON path	Default	Legacy alias
estimation.enable_variance_factor	off	enable_variance_factor
estimation.estimate_ifb	off	estimate_ifb
estimation.estimate_phase_clock	off	estimate_phase_clock
estimation.estimate_sat_biases	off	estimate_sat_biases
estimation.estimate_sat_clocks	off	estimate_sat_clocks
estimation.estimate_sat_phase_biases	off	estimate_sat_phase_biases
estimation.estimate_tropo_gradients	off	estimate_tropo_gradients
estimation.estimate_velocity	off	estimate_velocity
estimation.global_vtec_sh.rep_error_d5_keep	off	global_vtec_sh_rep_error_d5_keep
estimation.global_vtec_sh.rep_error_demean	off	global_vtec_sh_rep_error_demean
estimation.global_vtec_sh.sun_fixed	on	global_vtec_sh_sun_fixed
estimation.glonass_ifb.enable	off	enable_glonass_ifb
estimation.lock_signals_to_clock_ref	off	lock_signals_to_clock_ref
estimation.promote_sat_code_bias	off	promote_sat_code_bias
estimation.require_bia_for_signals	off	require_bia_for_signals
estimation.residual_history.enabled	off	residual_history_enabled
network.adaptive_iono_q	off	network_adaptive_iono_q
network.adm_revet	off	adm_revet
network.adm_revet_census	on	adm_revet_census
network.adm_revet_hold	off	adm_revet_hold
network.adm_screen	off	adm_screen
network.adm_trace	off	adm_trace
network.ambiguity_resolution.enable_partial_ar	on	network_ar_enable_partial_ar
network.ambiguity_resolution.wl_cascade	off	network_ar_wl_cascade
network.block_baarda	off	block_baarda
network.block_baarda_corroborate	off	block_baarda_corroborate
network.carry_aware_covariance	off	network_carry_aware_covariance
network.carry_conditional_coupling	off	carry_conditional_coupling
network.carry_coupling_joint_whiten	off	carry_coupling_joint_whiten
network.carry_coupling_sh_marginalize	off	carry_coupling_sh_marginalize
network.carry_coupling_time_update	off	carry_coupling_time_update
network.carry_invariant	off	network_carry_invariant
network.carry_invariant_emit_rebase	on	network_carry_invariant_emit_rebase
network.carry_invariant_honesty	off	network_carry_invariant_honesty
network.carry_invariant_innov_calibrate	off	network_carry_invariant_innov_calibrate

JSON path	Default	Legacy alias
network.carry_invariant_innov_carried_only	off	network_carry_invariant_innov_carried_only
network.carry_invariant_innov_corroborate	off	network_carry_invariant_innov_corroborate
network.carry_invariant_innov_gate	off	network_carry_invariant_innov_gate
network.carry_invariant_innov_refold	off	network_carry_invariant_innov_refold
network.carry_invariant_innov_retap_only	off	network_carry_invariant_innov_retap_only
network.carry_invariant_pd_safe	off	network_carry_invariant_pd_safe
network.carry_rx_code_bias	off	carry_rx_code_bias
network.carry_vce	off	carry_vce
network.carry_vce_apply	off	carry_vce_apply
network.carry_zwd	off	carry_zwd
network.cb_admission	off	cb_admission
network.cb_excursion_reinit	off	cb_excursion_reinit
network.cb_excursion_reinit_all_sats	off	cb_excursion_reinit_all_sats
network.cb_kaula_prior	off	network_cb_kaula_prior
network.clock_datum_broadcast_anchored	off	network_clock_datum_broadcast_anchored
network.clock_datum_sum_to_zero	off	network_clock_datum_sum_to_zero
network.clock_disc_on_set_change	off	network_clock_disc_on_set_change
network.clock_model	off	network_clock_model
network.clock_monitor	off	network_clock_monitor
network.clock_monitor_couple_raim	off	network_clock_monitor_couple_raim
network.clock_runaway_check	off	network_clock_runaway_check
network.clock_signal_basis_hold	off	network_clock_signal_basis_hold
network.code_bias_disc_jump_test	off	network_code_bias_disc_jump_test
network.code_datum_time_linked	off	network_code_datum_time_linked
network.cold_clock_seed	off	cold_clock_seed
network.congruence_screen	off	congruence_screen
network.consume_phase_bias_sigma	off	network_consume_phase_bias_sigma
network.consume_ura	off	network_consume_ura
network.cycle_slip.gap_instrument	off	network_cycle_slip_gap_instrument

JSON path	Default	Legacy alias
network.cycle_slip.if_phase_td	off	network_cycle_slip_if_phase_td
network.cycle_slip.if_phase_td_ref_gate	off	network_cycle_slip_if_phase_td_ref_gate
network.cycle_slip.mw_detect	off	network_cycle_slip_mw_detect
network.cycle_slip.mw_lli_corroborate	off	network_cycle_slip_mw_lli_corroborate
network.cycle_slip.turbo_edit	off	network_cycle_slip_turbo_edit
network.datum_graph_census	off	datum_graph_census
network.emit_clock_disc	off	network_emit_clock_disc
network.emit_clock_rate	off	network_emit_clock_rate
network.emit_combined_sigma	off	network_emit_combined_sigma
network.emit_epoch_status_record	off	network_emit_epoch_status_record
network.emit_generation_id	off	network_emit_generation_id
network.emit_generation_time	off	network_emit_generation_time
network.emit_pb_filter	off	network_emit_pb_filter
network.emit_pb_filter_emit_sigma	off	network_emit_pb_filter_emit_sigma
network.emit_pb_filter_gate_v2	off	network_emit_pb_filter_gate_v2
network.emit_phase_bias_sigma	off	network_emit_phase_bias_sigma
network.emit_ura	off	network_emit_ura
network.estimate_glonass	off	estimate_glonass
network.estimate_glonass	off	estimate_glonass
network.estimate_iono_sh	off	estimate_iono_sh
network.estimate_sat_code_bias	off	estimate_sat_code_bias
network.exclude_unmodeled_antenna_stations	off	network_exclude_unmodeled_antenna_stations
network.external_sat_code_bias	off	external_sat_code_bias
network.fast_clock.allow_single_band	off	network_fast_clock_allow_single_band
network.fast_clock.emit	off	network_fast_clock_emit
network.fast_clock.shadow	off	network_fast_clock_shadow
network.iono_constraint_deweight_adaptive	off	iono_constraint_deweight_adaptive
network.iono_dw_release_reseed	off	network_iono_dw_release_reseed
network.iono_seed_field_frame	off	iono_seed_field_frame
network.iono_supply_formal_sigma	off	iono_supply_formal_sigma

JSON path	Default	Legacy alias
network.lane_carry.bridge_meas	on	network_lane_carry_bridge_meas
network.lane_carry.fold_blocked	off	network_lane_carry_fold_blocked
network.lane_carry.lane_meas	on	network_lane_carry_lane_meas
network.lane_carry.parallel_components	off	network_lane_carry_parallel_components
network.lane_carry.skip_pinned_increments	off	network_lane_carry_skip_pinned_increments
network.mdb	off	mdb
network.mw_wl.bridge_sample_debias	off	network_mw_wl_bridge_sample_debias
network.mw_wl.d5_dump	off	network_mw_wl_d5_dump
network.mw_wl.ewl_enable	off	network_mw_wl_ewl_enable
network.mw_wl.ewl_fold_rows	off	network_mw_wl_ewl_fold_rows
network.mw_wl.f6_all_blocks	off	network_mw_wl_f6_all_blocks
network.mw_wl.f6_dump	off	network_mw_wl_f6_dump
network.mw_wl.fix_enable	off	network_mw_wl_fix_enable
network.mw_wl.lane_trace	off	network_mw_wl_lane_trace
network.mw_wl.uwl_cold_fix	off	network_mw_wl_uwl_cold_fix
network.mw_wl.uwl_dual_top	off	network_mw_wl_uwl_dual_top
network.mw_wl.uwl_enable	off	network_mw_wl_uwl_enable
network.mw_wl.uwl_family_pairs	off	network_mw_wl_uwl_family_pairs
network.mw_wl.uwl_mid_enable	off	network_mw_wl_uwl_mid_enable
network.mw_wl.uwl_sib_gate	off	network_mw_wl_uwl_sib_gate
network.mw_wl.wl_inject_enable	off	network_mw_wl_wl_inject_enable
network.parallel_carry	off	parallel_carry
network.parallel_reduce	off	parallel_reduce
network.parallel_seam	off	parallel_seam
network.parallel_stations	on	parallel_stations
network.per_const_withhold	off	network_per_const_withhold
network.phase_bias_exclude_unmatched_antenna	off	network_phase_bias_exclude_unmatched_antenna
network.phase_datum_ensemble	off	network_phase_datum_ensemble
network.phase_datum_eq34	off	network_phase_datum_eq34
network.phase_datum_eq34_carry_shift	off	network_phase_datum_eq34_carry_shift
network.phase_datum_eq34_extend_only	off	network_phase_datum_eq34_extend_only
network.phase_datum_eq34_longevity_election	off	network_phase_datum_eq34_longevity_election

JSON path	Default	Legacy alias
network.phase_datum_eq34_pin_round	on	network_phase_datum_eq34_pin_round
network.phase_datum_eq34_slip_aware_election	off	network_phase_datum_eq34_slip_aware_election
network.phase_datum_prioritized	off	network_phase_datum_prioritized
network.phase_datum_sticky	on	network_phase_datum_sticky
network.phase_datum_time_linked	off	network_phase_datum_time_linked
network.phase_leveling	off	phase_leveling
network.phase_leveling_refloat_at_convergence	off	phase_leveling_refloat_at_convergence
network.phase_leveling_warmup_on_convergence	off	phase_leveling_warmup_on_convergence
network.pivot_vanish_return_signal	off	network_pivot_vanish_return_signal
network.prefit_fuse_require_uncheckable	off	prefit_fuse_require_uncheckable
network.prior_residual_census	off	prior_residual_census
network.prior_residual_corroborate	off	prior_residual_corroborate
network.prior_residual_global_sigma0	off	prior_residual_global_sigma0
network.product_flush_per_epoch	off	product_flush_per_epoch
network.raim	off	network_raim
network.raim_attribution	off	network_raim_attribution
network.raim_redundancy_normalize	off	network_raim_redundancy_normalize
network.raim_refloat_maturity_gate	off	network_raim_refloat_maturity_gate
network.raim_surgical_station	on	network_raim_surgical_station
network.raim_we_code_iono	off	network_raim_we_code_iono
network.raim_within_epoch	off	network_raim_within_epoch
network.require_signal_integer_for_ar	off	network_require_signal_integer_for_ar
network.rhs_attrib_on_trip	off	rhs_attrib_on_trip
network.rhs_attrib_on_trip_prev	on	rhs_attrib_on_trip_prev
network.rx_cb_carry_demean	off	rx_cb_carry_demean
network.rx_cb_carry_demean_within	off	rx_cb_carry_demean_within
network.rx_cb_carry_step_gate	off	rx_cb_carry_step_gate
network.rx_code_bias_datum	off	rx_code_bias_datum
network.sbasis_complete	off	network_sbasis_complete
network.scoped_handover_response	off	network_scoped_handover_response
network.serialize_disc_counters	off	network_serialize_disc_counters

JSON path	Default	Legacy alias
network.serialize_sbasis_contract	off	serialize_sbasis_contract
network.service_mode	off	network_service_mode
network.sh_health_gate	off	sh_health_gate
network.shared_carry_full_cov	off	shared_carry_full_cov
network.signal_integrity.census	off	network_signal_integrity_census
network.signal_integrity.refloat_on_regain	on	network_signal_integrity_refloat_on_regain
network.signal_integrity.resume_warmup_vet	on	network_signal_integrity_resume_warmup_vet
network.signal_integrity.withhold_instead	off	network_signal_integrity_withhold_instead
network.state_snapshot_carry	off	state_snapshot_carry
network.state_snapshot_carry_diagonal	off	state_snapshot_carry_diagonal
network.state_snapshot_keep_history	off	state_snapshot_keep_history
network.state_snapshot_require	off	state_snapshot_require
network.station_pd_pin_aware	off	network_station_pd_pin_aware
network.sticky_pivot	off	network_sticky_pivot
network.streaming.msm_lli_from_locktime	off	network_streaming_msm_lli_from_locktime
network.streaming.require_datum_station	on	network_streaming_require_datum_station
network.trace_anchor_info	off	trace_anchor_info
network.trace_clock_full	off	trace_clock_full
network.trace_dd_fractional	off	trace_dd_fractional
network.trace_epoch_status	off	trace_epoch_status
network.trace_estimability	off	trace_estimability
network.trace_events	off	trace_events
network.trace_flag_rates	off	trace_flag_rates
network.trace_ionosphere	off	trace_ionosphere
network.trace_phase_anchor	off	trace_phase_anchor
network.trace_postfit_keyed	off	trace_postfit_keyed
network.trace_postfit_sigma0	off	trace_postfit_sigma0
network.trace_prefit_census	off	trace_prefit_census
network.trace_section_timers	off	trace_section_timers
network.trace_sh_spectrum	off	trace_sh_spectrum
network.trace_slip_events	off	trace_slip_events
network.trace_slip_raw	off	trace_slip_raw
network.trace_ssr_consumer	off	trace_ssr_consumer
network.trace_station_residuals	off	trace_station_residuals
network.trace_stations	on	trace_stations
network.trace_topology	off	trace_topology
network.trace_zwd	off	trace_zwd

JSON path	Default	Legacy alias
network.ura_retro_bump	off	network_ura_retro_bump
network.use_sif_coordinates	off	use_sif_coordinates
network.write_run_manifest	off	write_run_manifest
processing.ar_ignore_mw_wl	off	ar_ignore_mw_wl
processing.auto_report	off	auto_report
processing.enable_neq_export	off	enable_neq_export
processing.enable_noncausal_slip_bridge	off	enable_noncausal_slip_bridge
processing.enable_obs_skip_to_latest	off	enable_obs_skip_to_latest
processing.enable_obs_ssr_sync	off	enable_obs_ssr_sync
processing.enable_orbit_normal_attitude	off	enable_orbit_normal_attitude
processing.enable_ssr_bias_max_age	off	enable_ssr_bias_max_age
processing.enable_station_info	off	enable_station_info
processing.enable_trig_sp3_interp	off	enable_trig_sp3_interp
processing.exclude_eclipsing_satellites	off	exclude_eclipsing_satellites
processing.inhouse_bias_estimation	off	inhouse_bias_estimation
processing.network_consumer.bias	on	network_consumer_bias
processing.network_consumer.clock	on	network_consumer_clock
processing.network_consumer.enable	off	network_consumer_enable
processing.network_consumer.undifferenced	on	network_consumer_undifferenced
processing.relinearize_geometry	off	relinearize_geometry
processing.report_async	off	report_async
processing.rinex_skip_zero_values	off	rinex_skip_zero_values
processing.run_smoother	off	run_smoother
processing.ssr_bias_band_fallback	off	ssr_bias_band_fallback
processing.uncombined_enabled	off	uncombined_enabled
receiver.enable_antenna_orientation	off	enable_antenna_orientation
receiver.enable_antenna_substitution	off	enable_antenna_substitution
receiver.enable_itrf_velocity	off	enable_itrf_velocity

A.6 Model-correction switches

Table 8: Physical model on/off switches (ModelCorrections). These gate the correction terms applied to the undifferenced observation model.

Field	Default	Effect
enable_troposphere	on	Saastamoinen ZHD + GMF dry mapping (slant hydrostatic delay).

Field	Default	Effect
enable_klobuchar_ionosphere	off	Klobuchar ionospheric delay model (single-frequency fallback only). IF combinations and DD short-baselines cancel ionosphere geometrically; PPP-RTK uncombined estimates per-satellite slant iono states. In all three of those configs the Klobuchar model is a no-op, but constructing the Ionosphere instance still pays for an extra Model::Geometry load and the Klobuchar coefficient parse. Default off. Enable only for single-frequency PPP runs that need a-priori iono.
enable_higher_order_ionosphere	off	Apply second-order ($B \cdot \cos(\theta) \cdot STEC/f^3$) and third-order ($\eta \cdot STEC^2/f^4$) ionospheric delays as a per-signal observation-level correction, evaluated at the GIM-derived IPP using a centered tilted dipole geomagnetic field. Requires a loaded IONEX GIM as the absolute-STEC source; the engine auto-loads the GIM when this flag is on even if estimation.ionosphere_weighting.enabled is false. No-op when no GIM resolves (warning logged at startup). Magnitudes: I_2 ~1–2 cm at L1 during high TEC, I_3 ~mm; the IF combination only partially cancels these, so the largest visible benefit is in standard PPP (IF). For PPP-RTK uncombined and short-baseline DD the higher-order term mostly cancels in the SD / DD differencing, so this flag is closer to a consistency knob than a bias remover. Default off.
enable_relativistic_correction	on	Periodic relativistic clock correction (-2 rv / c).
enable_shapiro_delay	on	Shapiro (gravitational path) delay.
enable_solid_earth_tide	on	IERS 2010 solid Earth tide displacement.
enable_ocean_tide_loading	on	Ocean tidal loading displacement (BLQ).
enable_pole_tide	on	Pole tide displacement (from ERP): IERS 2010 Sec. 7.1.4 rotational deformation.
enable_ocean_pole_tide	off	Ocean pole tide load displacement (IERS 2010 Sec. 7.1.5 / Desai 2002), added on top of enable_pole_tide. Requires the opole load grid (files.opt_grid_path or ¡data¿/static/opole/). ~0.1-2 mm; default off keeps runs byte-identical.
enable_atmospheric_loading	off	Atmospheric pressure loading (S1/S2 thermal-tide site displacement). Requires per-station S1/S2 load coefficients (files.atm_tide_grid_path or ¡data¿/static/apl/). Sun-synchronous → does NOT average out in static daily PPP. Default off keeps runs byte-identical.
enable_satellite_pco	on	Satellite antenna phase centre offset.
enable_satellite_pcv	on	Satellite antenna phase centre variation.

Field	Default	Effect
enable_receiver_pco	on	Receiver antenna phase centre offset.
enable_receiver_pcv	on	Receiver antenna phase centre variation.
enable_receiver_cross_system_atx_fallback	off	RECEIVER-ONLY: when the receiver ATX entry lacks a non-GPS frequency, substitute the nearest GPS band (E1-¿G01, E5a-¿G05, B1C-¿G01, B2a-¿G05, ...). Valid: receiver phase pattern is frequency-dependent on shared hardware. NEVER applied to the satellite antenna (E01 and G01 are different spacecraft). Opt-in.
enable_atx_frequency_fallback	on	When the ATX entry doesn't have PCO/PCV for the requested frequency (common for L5 / E5* / B2a on older satellites or receiver antennas), look up the nearest available frequency on the same antenna and use its values instead of silently skipping the correction. Cross-frequency PCO drift on a single physical antenna is typically ¡1 cm, better than the few-cm vertical residual that comes from skipping a frequency entirely. Set false to revert to the legacy "skip if missing" behavior for A/B comparison.
require_atx_coverage	off	When true, drop observations whose (sat, signal) lacks complete ATX coverage on **both** the satellite and receiver entries, i.e., satellite PCO+PCV and receiver PCO+PCV must all resolve to non-empty entries for the row's exact frequency, *without* enable_atx_frequency_fallback being used to substitute a neighbouring frequency. Default false (legacy behaviour: observations are kept and any missing PCO/PCV is silently zeroed). Set true when you've explicitly chosen 'signal_mode: "all"' and want the ATX file to be the authoritative whitelist, dropping rows that would otherwise process without proper antenna calibration.
require_atx_coverage_for_ar	off	When true, AR (WL constraints, LAMBDA NL search) skips (sat, signal) pairs lacking strict ATX coverage even though the underlying observations are still used in the float Kalman update. Softer counterpart to require_atx_coverage: lets you keep "all"-signal float estimation alive while restricting integer fixing to the ATX-curated subset. Default false (no AR-side gating).

Field	Default	Effect
restrict_ar_for_substituted_atx	off	When true, AR additionally skips (sat, signal) pairs whose ATX correction was computed via the frequency-fallback substitution chain (rather than an exact entry). Targets specifically the cases where the antenna model is approximate; signals lacking ATX entirely (no substitution applied) are still allowed. Use alongside or instead of require_atx_coverage_for_ar depending on which kind of imperfection you want to gate on. Default false.
loosen_amb_q_for_substituted_atx	off	When true, ambiguity states whose underlying (sat, signal) had their ATX correction frequency-fallback-substituted use a looser process noise (ambiguity_q_substituted_atx) instead of the standard process_noise.ambiguity. Compensates for the small but non-zero modeling error introduced by the substituted PCV/PCO. Default false.
apply_phase_shifts	off	When true, RINEX phase observations have the receiver-applied "SYS / PHASE SHIFT" values subtracted at parse time, recovering the unshifted phase convention. Default false because most modern BIA products (CODE, CAS, GFZ) are calibrated against the receiver-shifted phases as recorded in RINEX, so leaving the shifts in place is the standard PPP convention. Set true only when your bias product is explicitly calibrated for the unshifted convention, or when running without external BIA and using in-house bias estimation that benefits from a uniform shift-free input.
enable_phase_windup	on	Carrier-phase wind-up correction.
enable_code_phase_bias	on	Code/phase bias corrections from BIA product files.
bias_coverage_requires_apply	off	When true, the "already corrected?" gates in
refine_emission_time	on	When true, Geometry::computeSatelliteGeometry runs a

A.7 Logging and trace channels

Table 9: Logging and CSV-trace channels (LoggingConfig). Each toggles one spdlog file logger or one per-epoch CSV trace.

Field	Default	Channel
enable_main_log	on	logs/main_log.log
enable_gnss_config	on	logs/gnss_config.log

Field	Default	Channel
enable_kalman_filter	on	logs/kalman_filter.log
enable_smoother	on	logs/smoother.log
enable_state_manager	on	logs/state_manager.log
enable_ppp_model	on	logs/ppp_model.log
enable_measurement_model	on	logs/measurement_model.log
enable_mw_slip_detector	on	logs/mw_slip_detector.log
enable_if_slip_detector	on	logs/if_slip_detector.log
enable_gf_slip_detector	on	logs/gf_slip_detector.log
enable_turbo_edit_resolver	on	logs/turbo_edit_resolver.log
enable_cycle_slip_manager	on	logs/cycle_slip_manager.log
enable_ambiguity_resolver	on	logs/ambiguity_resolver.log
enable_corrections	on	logs/corrections.log
enable_geometry	on	logs/geometry.log
enable_troposphere	on	logs/troposphere.log
enable_ionosphere	on	logs/ionosphere.log
enable_antenna	on	logs/antenna.log
enable_antenna_model	on	logs/antenna_model.log
enable_ocean_tide	on	logs/ocean_tide.log
enable_planetary_model	on	logs/planetary_model.log
enable_stochastic_model	on	logs/stochastic_model.log
enable_rinex_parser	on	logs/rinex_parser.log
enable_product_parser	on	logs/product_parser.log
enable_product_cache	on	logs/product_cache.log
enable_resolver	on	logs/resolver.log
enable_state_initializer	on	logs/state_initializer.log
enable_bssd_pivot_manager	on	logs/bssd_pivot_manager.log
enable_bssd_transform	on	logs/bssd_transform.log
enable_ppp_utils	on	logs/ppp_utils.log
enable_downloader	on	logs/downloader.log
enable_ntrip	on	logs/ntrip.log
enable_rt_engine	on	logs/rt_engine.log
enable_imu	on	logs/imu.log
enable_state_trace	on	logs/state_trace.csv
enable_ambiguity_trace	on	logs/ambiguity_trace.csv
enable_ionosphere_trace	on	logs/ionosphere_trace.csv
enable_sat_bias_trace	on	logs/sat_bias_trace.csv
enable_code_bias_trace	on	logs/code_bias_trace.csv (receiver DCB/IFB)
enable_sat_status_trace	on	logs/satellite_status_trace.csv

Field	Default	Channel
enable_dop_trace	on	logs/dop_trace.csv
enable_residual_trace	on	logs/measurement_residuals.csv
enable_wls_trace	on	logs/wls_trace.csv
enable_smoother_trace	on	logs/smoother_trace.csv
enable_mw_slip_trace	on	logs/mwwl_trace.csv
enable_if_slip_trace	on	logs/if_slip_trace.csv
enable_gf_slip_trace	on	logs/gf_slip_trace.csv
enable_turbo_edit_trace	on	logs/turbo_edit_trace.csv
enable_windup_trace	off	logs/windup_trace.csv (phase wind-up diagnostic; off by default)
kalman_filter_concise_log	off	

A.8 Cycle-slip detector registry

Table 10: Cycle-slip detectors known to SlipDetectorRegistry. *Active* detectors have a creator in the factory table and run when their flag is set; *staged* detectors are declared in the SlipDetectorKind enum but excluded from the build (the registry warns and skips them if requested). The $k\sigma$ column is the MAD gate multiplier from CycleSlipConfig.

Detector	Config flag	Default	$k\sigma$	Status	Combination
MelbourneWubbena	estimation.cycle_slip.mw	on	8.0	**active**	Wide-lane MW combination (primary, always available).
Graphic	estimation.cycle_slip.graphic	off	6.0	**active**	GRAPHIC (code+phase average) per-signal detector.
ExtraWideLane	estimation.cycle_slip.ewl	off	5.0	**active**	Extra-wide-lane MW for 3+ frequency satellites.
GeometryFree	estimation.cycle_slip.gf	off	–	*staged*	2nd-order time-differenced geometry-free (STAGED).
IonoFreePhase	estimation.cycle_slip.ifp	off	–	*staged*	Double-differenced ionosphere-free phase OMC (STAGED).

A.9 State-vector key schema

Table 11: Dynamic state-vector key formats, reconstructed from the `make*Key` bodies in `StateKeySchema.hpp`. Angle-bracketed fields are runtime values; literal characters are the delimiter / infix / prefix / suffix constants.

State	Key format	Builder
Ambiguity	`<satellite>_<signal>`	`makeAmbiguityKey`
NL Ambiguity	`<satellite>_NL_<band>`	`makeNlAmbiguityKey`
WL Ambiguity	`<satellite>_WL_<band_a>_<band_b>`	`makeWlAmbiguityKey`
DD Ambiguity	`DD|<cons>|<sat_i>|<code>|<pivot>`	`makeDdAmbiguityKey`
Ionosphere	`<satellite>`	`makeIonosphereKey`
IFB	`<system>_IFB<band>`	`makeIfbKey`
DCB	`<system>_DCB`	`makeDcbKey`
Glonass Per Slot IFB	`<system>_IFB<band>_k`	`makeGlonassPerSlotIfbKey`
Receiver Phase Bias	`<system>_RPB_<signal>`	`makeReceiverPhaseBiasKey`
Satellite Bias	`<satellite>|<signal>`	`makeSatBiasKey`
Satellite Phase Bias	`<satellite>|<signal>|PB`	`makeSatPhaseBiasKey`
Satellite Clock	`<satellite>|SC`	`makeSatClockKey`
System Time Offset	`<system>`	`makeSystemTimeOffsetKey`
Pivot Seed	`<cons>|<band>`	`makePivotSeedKey`
Vtec Sh	`|||<trig>`	`makeVtecShKey`

References

[1] Simon Banville and Richard B. Langley. "Mitigating the impact of ionospheric cycle slips in GNSS observations". In: *Journal of Geodesy* 87.2 (2013), pp. 179–193. DOI: 10.1007/s00190-012-0604-1.

[2] Jean-Paul Berrut and Lloyd N. Trefethen. "Barycentric Lagrange Interpolation". In: *SIAM Review* 46.3 (2004), pp. 501–517. DOI: 10.1137/S0036144502417715.

[3] Guang Chen and Thomas A. Herring. "Effects of atmospheric azimuthal asymmetry on the analysis of space geodetic data". In: *Journal of Geophysical Research: Solid Earth* 102.B9 (1997), pp. 20489–20502. DOI: 10.1029/97JB01739.

[4] Shailen D. Desai. "Observing the pole tide with satellite altimetry". In: *Journal of Geophysical Research: Oceans* 107.C11 (2002), p. 3186. DOI: 10.1029/2001JC001224.

[5] Jianghui Geng and Yehuda Bock. "Triple-frequency GPS precise point positioning with rapid ambiguity resolution". In: *Journal of Geodesy* 87.5 (2013), pp. 449–460. DOI: 10.1007/s00190-013-0619-2.

[6] *GLONASS Interface Control Document: Navigational Radiosignal in Bands L1, L2 (Edition 5.1)*. Russian Institute of Space Device Engineering. Moscow, 2008.

[7] M. Mainul Hoque and Norbert Jakowski. "Estimate of higher order ionospheric errors in GNSS positioning". In: *GPS Solutions* 12.2 (2008), pp. 87–97. DOI: 10.1007/s10291-007-0069-5.

[8] IAU SOFA Board. *SOFA Tools for Earth Attitude*. Software version 18; http://www.iausofa.org/. International Astronomical Union, Standards of Fundamental Astronomy (SOFA). 2021.

[9] International GNSS Service Real-Time Working Group. *IGS State Space Representation (SSR) Format, Version 1.00*. Format specification. RTCM message 4076 IGS-SSR sub-types. International GNSS Service (IGS), 2020.

[10] P. J. de Jonge. "A processing strategy for the application of the GPS in networks". PhD thesis. Delft, The Netherlands: Delft University of Technology, 1998.

[11] J. A. Klobuchar. "Ionospheric time-delay algorithm for single-frequency GPS users". In: *IEEE Transactions on Aerospace and Electronic Systems* AES-23.3 (1987), pp. 325–331. DOI: 10.1109/TAES.1987.310829.

[12] Daniel Landskron and Johannes Boehm. "VMF3/GPT3: refined discrete and empirical troposphere mapping functions". In: *Journal of Geodesy* 92.4 (2018), pp. 349–360. DOI: 10.1007/s00190-017-1066-2.

[13] D. Laurichesse et al. "Integer ambiguity resolution on undifferenced GPS phase measurements and its application to PPP and satellite precise orbit determination". In: *Navigation* 56.2 (2009), pp. 135–149. DOI: 10.1002/j.2161-4296.2009.tb01750.x.

[14] Jean Meeus. *Astronomical Algorithms*. Richmond, VA, USA: Willmann-Bell, 1991. ISBN: 0-943396-35-2.

[15] Oliver Montenbruck and Eberhard Gill. *Satellite Orbits: Models, Methods and Applications*. Berlin, Heidelberg: Springer, 2000. DOI: 10.1007/978-3-642-58351-3.

[16] A. E. Niell. "Global mapping functions for the atmosphere delay at radio wavelengths". In: *Journal of Geophysical Research: Solid Earth* 101.B2 (1996), pp. 3227–3246. DOI: 10.1029/95JB03048.

[17] D. Odijk. "Fast precise GPS positioning in the presence of ionospheric delays". In: *Publications on Geodesy* 52 (2003).

[18] Gérard Petit and Brian Luzum. *IERS Conventions (2010)*. IERS Technical Note 36. International Earth Rotation and Reference Systems Service (IERS). Frankfurt am Main: Verlag des Bundesamts für Kartographie und Geodäsie, 2010. ISBN: 3-89888-989-6.

[19] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd. Cambridge: Cambridge University Press, 2007.

[20] H. E. Rauch, F. Tung, and C. T. Striebel. "Maximum likelihood estimates of linear dynamic systems". In: *AIAA Journal* 3.8 (1965), pp. 1445–1450. DOI: 10.2514/3.3166.

[21] *RTCM Standard 10403.3: Differential GNSS Services – Version 3*. Special Committee No. 104; State Space Representation messages 1057–1068, 1240–1270, 4076. Radio Technical Commission for Maritime Services. Arlington, VA, USA, 2016.

[22] J. Saastamoinen. "Atmospheric correction for the troposphere and stratosphere in radio ranging of satellites". In: *The Use of Artificial Satellites for Geodesy*. Ed. by S. W. Henriksen, A. Mancini, and B. H. Chovitz. Vol. 15. Geophysical Monograph Series. Washington, D.C.: American Geophysical Union, 1972, pp. 247–251. DOI: 10.1029/GM015p0247.

[23] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[24] Tomoji Takasu and Akio Yasuda. "Development of the Low-Cost RTK-GPS Receiver with an Open Source Program Package RTKLIB". In: *International Symposium on GPS/GNSS*. Jeju, Korea, 2009, pp. 4–6.

[25] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[26] P. J. G. Teunissen. "The least-squares ambiguity decorrelation adjustment: a method for fast GPS integer ambiguity estimation". In: *Journal of Geodesy* 70.1–2 (1995), pp. 65–82. DOI: 10.1007/BF00863419.

[27] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.

[28] G. Wübbena. "Software developments for geodetic positioning with GPS using TI 4100 code and carrier measurements". In: *Proceedings of the First International Symposium on Precise Positioning with the Global Positioning System*. Rockville, MD, USA, 1985, pp. 403–412.

[29] Thomas P. Yunck. "Coping with the atmosphere and ionosphere in precise satellite and ground positioning". In: *Environmental Effects on Spacecraft Positioning and Trajectories*. Geophysical Monograph Series 73 (1993), pp. 1–16. DOI: 10.1029/GM073p0001.